

2026 COMPLETE EDITION · 500 PAGES

Full-Stack Web Development Handbook

From absolute zero to job-ready developer. HTML, CSS, JavaScript, Git, Node.js, Express, SQL, MongoDB, Authentication, Security and React — explained with simple words, real examples and 500+ practice tasks.

HTML5 · CSS3 + Flexbox + Grid · JavaScript ES6+ · DOM & Async · Node & Express · PostgreSQL & MongoDB · JWT Auth · React

Beginner-friendly · Junior-to-Senior reference · Full datasheets & glossary inside

WRITTEN BY

Mahmoud Mohamed Mahmoud Khattab

+20 114 122 5852 / +20 103 411 7684 · Full-Stack Web Development

Read it from page 1 to the end — **this book alone is enough** to learn modern web development. · Includes roadmap, projects, exercises with solutions and senior datasheets.

Copyright & About This Book

Title	Full-Stack Web Development Handbook — from Zero to Job-Ready (2026 Complete Edition)
Author	Mahmoud Mohamed Mahmoud Khattab
Contact	+20 114 122 5852 / +20 103 411 7684
Audience	Absolute beginners, juniors, working developers and seniors who need a fast reference.
Method	Easy → hard. Every idea is explained twice: first with simple words and an analogy, then with exact technical detail and runnable code.

What makes this book different

- **One book is enough.** If you read it carefully and do the exercises, you do not need another course to become a full-stack developer.
- **Built from the best courses.** The curriculum follows the lesson order and depth of Elzero Web School, Abdelrahman Gamal's tracks, Angela Yu's Web Development Bootcamp, plus MDN, W3Schools and Tutorialspoint references.
- **Reference for life.** Datasheets at the end of every part plus master appendices let you find any tag, property or method in seconds during real work.
- **Real code everywhere.** Every concept ships with a complete, runnable example — not fragments you cannot run.

How to read this book

- **New to everything?** Start at Chapter 1 and go in order. Do every Try it yourself box and every exercise before reading its solution.
- **Junior developer?** Skim the grey Key takeaway boxes, slow down on Flexbox, Grid, closures and async chapters.
- **Senior / working?** Jump with the Table of Contents straight to datasheets, the HTTP table, the security checklist and the glossary.

All code in this book is standard HTML5, CSS3 and JavaScript (ES6+). Server examples use Node.js with Express. Database examples use PostgreSQL and MongoDB. Web addresses and course names are referenced for study purposes.

Contents

Absolute Zero? Start Here (Day 0)	18
Your first web page in 3 steps	18
The 3 technologies in one picture	18
How to Use This Book	20
The promise of this book	20
How each chapter works	20
The visual language of this book	21
What you need before page 1	21
The 2026 Full-Stack Roadmap	22
The big picture	22
Tier details — what "done" looks like	22
About the Author	24
How to reach the author	24
Your 16-Week Study Plan	25

Web Foundations — How the Internet Really Works

28

Chapter 1 — What Happens When You Open a Website	29
The restaurant analogy (read this first)	29
Addresses of the web: DNS, IP and ports	30
The three handshakes: TCP, TLS, HTTP	31
Front-end vs back-end vs full-stack	32
The browser is not magic: how a page is drawn	32
Your first look at DevTools (2 minutes)	33
Chapter 2 — Files, URLs and How Websites Are Organized	35
Anatomy of a URL	35
The files of a website	36
Folders that make sense	37
Absolute vs relative paths (with zero confusion)	37
Chapter 3 — HTTP — The Conversation Between Browser and Server	39

Requests and responses	39
The seven methods (verbs)	40
Status codes you must memorize	40
Headers in plain words	41
JSON, the data language of the web	41

HTML — The Skeleton of Every Page

44

Chapter 4 — Your First HTML — Tags, Elements and the Page Skeleton 45

Tags, elements, attributes — finally clear	45
Rules of nesting (3 rules, zero exceptions)	46
The only skeleton you need (memorize it)	47
Headings and paragraphs done right	48

Chapter 5 — Text, Links and Lists 50

Text formatting that MEANS something	50
Links: the soul of the web	51
Lists: unordered, ordered, description	52
Building a real navigation menu	53

Chapter 6 — Images, Media and Tables 55

Images done professionally	55
Modern formats (stop shipping heavy JPGs)	56
Figures, audio and video	57
Tables: data, not layout	57

Chapter 7 — Forms — Collecting Data Like a Pro 60

The golden trio: form + label + input	60
Every input type (with when to use it)	61
Built-in validation (free, before JavaScript)	62
Select, radio, checkbox, fieldset	63

Chapter 8 — Semantic HTML, Accessibility and SEO 65

Div-soup vs landmarks	65
ARIA: when HTML needs a microphone	66
The accessibility checklist (WCAG in 1 page)	67
SEO basics that actually matter	68

Chapter 9 — Project — Personal Profile Page (HTML only)	70
Step 1 — Plan on paper (15 min)	70
Step 2 — Build the skeleton + header	71
Step 3 — Main sections	72
Step 4 — Self-review checklist	73
Chapter 10 — HTML Datasheet — Every Important Tag in One Place	75

CSS — Designing Beautiful Pages

82

Chapter 11 — CSS First Steps — Selectors and Ways to Style	83
A CSS rule in 10 seconds	83
The selectors you will use daily	84
Combining selectors (read right-to-left)	85
Three places CSS can live (use only one!)	86
Chapter 12 — The Cascade, Specificity and Inheritance	88
The cascade: three rounds of battle	88
Specificity scoring (count 0-0-0)	89
Inheritance: what children get for free	89
Debugging conflicts in DevTools	90
Chapter 13 — The Box Model — Every Element Is a Box	92
The 4 layers	92
content-box vs border-box (the fix of the century)	93
Margin collapsing (the surprise)	94
The box inspector + centering patterns	95
Chapter 14 — Sizing & Typography Deep-Dive — font-size vs font-weight, width vs max-width	96
DEEP-DIVE 1: font-size vs font-weight	96
Units: px vs rem vs em vs % vs vw	98
DEEP-DIVE 2: width vs max-width, height vs min-height	99
line-height: the unitless law	100
Building a type scale (copy this system)	101
Chapter 15 — Colors, Backgrounds, Borders and Shadows	102
Color formats (learn all three, prefer two)	102
The contrast law (non-negotiable)	103

Backgrounds and gradients	104
Borders, radius, shadows	105
Chapter 16 — Display and Positioning — Controlling the Flow	107
display: how elements occupy space	107
position: the five modes	108
z-index and stacking (why overlaps misbehave)	109
Centering cookbook (stop googling these)	110
Chapter 17 — Flexbox — One-Dimensional Layout Mastery	111
Mental model: container, items, two axes	111
Container properties (set on parent)	112
Item properties (set on children)	113
Five classic patterns (memorize these)	113
Chapter 18 — CSS Grid — Two-Dimensional Layout Mastery	116
Core vocabulary (5 words)	116
The one-line responsive grid (no media queries!)	117
Placement: lines, span, areas	118
Alignment: two axes at once	118
Chapter 19 — Responsive Design — Mobile First	120
Mobile-first philosophy	120
Fluid typography with clamp()	121
Responsive media (never overflow again)	122
Chapter 20 — Transitions, Transforms and Animations	124
transition: smooth A→B changes	124
transform: the free superpower	125
keyframes: full choreography	125
The 60fps rule + reduced motion	126
Chapter 21 — Variables, Architecture and the CSS Project	128
Custom properties: design tokens	128
BEM naming (stop class chaos)	129
Project: responsive landing page (90 min)	129
Chapter 22 — CSS Datasheet — Every Important Property in One Place	131

JavaScript — Bringing Pages to Life**138****Chapter 23 — JavaScript First Steps — Values and Your Console 139**

Run code in 60 seconds (two ways) 139

Values and types (only 8 exist) 140

Strings: quotes, escape, template literals 141

console: your X-ray (5 methods) 141

Chapter 24 — Variables and Memory — let, const, var and the Truth About Copying 143

let vs const vs var 143

Naming rules (machines + humans) 144

Primitives vs references (the big mental model) 144

The Temporal Dead Zone (TDZ) in 60 seconds 145

Chapter 25 — Operators, Conditions and Loops 147

Arithmetic and precedence 147

=== vs == (memorize one rule) 148

Conditions: if, ternary, switch 148

Logical operators: && || ?? ?. 149

Loops: for, while, break, continue 150

Chapter 26 — Functions, Scope and Closures — The Heart of JavaScript 152

Three ways to write functions 152

Parameters: defaults, rest, destructuring preview 153

Scope chain + closures (with REAL uses) 153

Hoisting truth (30 seconds, no myths) 154

`this` in 5 rules (the demystification) 155

Chapter 27 — Arrays — Ordered Data and the Methods That Rule Them 157

Create, read, and the index trap 157

Mutating vs non-mutating (the safety line) 158

The big six: map/filter/reduce/find/some/every 158

Chaining like a senior 159

Spread, destructure, copy right 160

Chapter 28 — Objects, ES6 Power and Classes 161

Objects: named data + behavior 161

Destructuring + spread (daily professional syntax) 162

JSON: objects as text	162
Prototypes in 5 minutes (then classes forever)	163
Classes + private fields + modules	163
Chapter 29 — The DOM — JavaScript Meets the Page	165
The DOM tree (HTML as live objects)	165
Selecting elements (the 4 you need)	166
textContent vs innerHTML (read before touching!)	166
Create, insert, remove, clone	167
Classes, styles, attributes	168
Chapter 30 — Events — Clicks, Keys and the Delegation Superpower	170
Listening (the modern way)	170
Bubbling: events float upward	171
Chapter 31 — Forms, Validation and Browser Storage	174
Reading forms with JavaScript	174
Regex survival kit (only what you'll use)	175
Storage: local vs session vs cookies	176
Project: theme toggle (Chapter 21 payoff!)	177
Chapter 32 — Async JavaScript — The Event Loop, Promises and async/await	178
The event loop (JS juggles on one thread)	178
Promises: receipts for future values	179
async/await (write async like sync)	180
Timers + debounce (search boxes done right)	181
Chapter 33 — Fetch and Real APIs — Talking to the World	183
fetch CRUD (the complete set)	183
The 4 UI states (seniors never ship 2)	184
Project: posts explorer (JSONPlaceholder)	185
Chapter 34 — Errors, Debugging and Regular Expressions	187
try/catch/finally + throwing well	187
Debug like a senior (breakpoints > console.log spam)	188
Regex: validate shapes, safely	189
Chapter 35 — JavaScript Projects — Quiz App and Expense Tracker	191
The architecture (memorize this triangle)	191
Project 1 — Timed quiz with highscores	192

Project 2 — Expense tracker (state + stats + bar chart by hand) **193**

Chapter 36 — JavaScript Datasheet — Methods and Syntax in One Place **195**

Tools of the Trade — Git, GitHub and Shipping

200

Chapter 37 — Git, GitHub, npm and Deployment **201**

The mental model (repos, commits, branches) **201**

Branching workflow (never break main) **203**

.gitignore (secrets NEVER ship) **203**

npm + package.json (the world's code, reused) **204**

Deploy: from laptop to link (static sites) **204**

Chapter 38 — CI/CD, Pull Requests and Team Workflow **207**

CI: robots check every push (the safety net) **207**

Pull requests: collaboration + review **209**

Back-End — Node.js and Express APIs

212

Chapter 39 — Node.js — JavaScript on the Server **213**

Node vs browser: same language, new powers **213**

Event loop + non-blocking (the rule behind everything) **214**

Chapter 40 — Express and REST — Professional APIs **216**

REST design (nouns, verbs, codes) **216**

Express setup + routing **217**

Middleware: the assembly line **217**

Chapter 41 — Validation, Files and Production Habits **220**

Validate everything (the validator pattern) **220**

Pagination (never return 100,000 rows) **221**

Uploads without disasters **221**

Production habits checklist **222**

Databases — Remembering Everything

223

Chapter 42 — SQL and PostgreSQL — Structured Data Done Right **224**

Tables, rows, keys (the spreadsheet that enforces rules) **224**

Queries: the daily 90%	225
JOINS visualized (the fear-killer)	226
Safe Node access (parameters, never strings!)	226

Chapter 43 — MongoDB and Mongoose — Flexible Document Data 228

Documents: JSON that persists	228
CRUD + queries	229
Embed vs reference (the design decision)	229
SQL vs NoSQL (the honest guide)	230

Authentication and Security — Trust, Done Right 232

Chapter 44 — Authentication — Passwords, Sessions and JWT 233

Authentication vs authorization (say it right)	233
Passwords with bcrypt (the only way)	234
Sessions vs JWT (the architecture choice)	234

Chapter 45 — Web Security — Attacks and Defenses 237

Think like an attacker (trust boundaries)	237
XSS: your site running attacker code	237
CSRF: hijacking the logged-in browser	238
Injection: queries built from strings	239
Hardening stack (copy this)	239

React — Modern Front-End 241

Chapter 46 — React — Components, State and Hooks 242

Thinking in components (Lego UI)	242
State: useState + events	244
Lists, keys, conditional UI	244
Effects: useEffect + data fetching	245
useRef + routing idea + cart project	246
Beyond useState: Zustand + Redux Toolkit (global state grows up)	247
React performance checklist (measure, then memo)	249

Capstone — Ship, Show, Get Hired**250**

Chapter 47 — Capstone — Portfolio, Deployment and Interviews	251
The 3 projects that get interviews	251
Full-stack deployment map	252
CV + portfolio that recruiters read	252
Technical interview playbook	252
Your next 12 months	253
Launch week: the shipping sprint (7 days, 1 hired-ready portfolio)	254
Open source: your first contribution (faster than you think)	255

Deep Dives I — HTML & CSS Beyond the Core**257**

Chapter 48 — SVG — Infinitely Sharp Graphics	258
Why SVG wins for UI graphics	258
currentColor icon system (the pro pattern)	259
Accessible + responsive SVG rules	260
Chapter 49 — Advanced Selectors — :has, :is, :where and Pseudo Power	261
:has() — style parents by children (the revolution)	261
:is() vs :where() (specificity weapons)	262
nth patterns + generated content	262
Chapter 50 — Backgrounds, Gradients and Masks Deep-Dive	264
Layering (paint order decoded)	264
Pattern gallery (zero image files!)	265
Masks, clips and blend reveals	265
Chapter 51 — Bootstrap and Tailwind Crash Course	267
Two philosophies	267

Deep Dives II — JavaScript Beyond the Core**270**

Chapter 52 — Modern JavaScript Deep — Map, Set, Iterators and More	271
Map vs Object, Set vs Array (choose right)	271
Generators: lazy power	272

Chapter 53 — Design Patterns — Module, Factory, Observer, Singleton	274
1. Module (ESM + closures)	274
2. Factory (one maker, many configs)	275
3. Observer (pub/sub at scale)	275
4. Singleton (exactly one)	275
Chapter 54 — Testing JavaScript — Vitest From Zero	277
The bug-cost curve (why bother)	277
Mocking: fetch, time, randomness	278
Chapter 55 — TypeScript Essentials — JavaScript That Checks Itself	280
The refactor test (why types win)	280
Interfaces vs type + gradual adoption	281
Deep Dives III — Backend & Data Beyond the Core	283
Chapter 56 — Command Line, Vite and Code Quality	284
Terminal survival (the daily 20)	284
Vite: instant dev, optimized build	285
ESLint + Prettier + hooks (robots enforce taste)	286
Chapter 57 — Templates, Static Files and Realtime Intro	288
EJS: dynamic HTML without a framework	288
Static files + caching that respects updates	289
Realtime: SSE vs WebSockets	289
Chapter 58 — Data Modeling, Indexes and Transactions	291
Relationships (say the sentence!)	291
Normalize vs denormalize (the eternal trade)	292
Indexes + EXPLAIN (speed with proof)	292
Transactions + N+1	293
Chapter 59 — Deploying Full-Stack Apps — API + DB + Domain	295
Topology + managed database	295
Env discipline + health/logs	295
Smoke tests + rollback	296

Deep Dives IV — React Pro & Performance**298****Chapter 60 — React Router, Context and Forms****299**

Router: URLs for every view (npm i react-router-dom)

299

Context: shared state without drilling

300

Forms: controlled + validated + accessible

301**Chapter 61 — Web Performance — Core Web Vitals and Speed Budgets****302**

The three Core Web Vitals

302

Measure like a lab + field scientist

303

Diets: images, fonts, JavaScript

303

Caching + budgets

304**Chapter 62 — Next.js First Look — React With Superpowers****306**

Folders ARE routes (App Router)

306

Images + decision matrix

307**Interview & Exam Prep — Prove Your Skills****309****Chapter 63 — Interview Bank I — HTML & CSS (30 Questions)****310**

Answering technique (read first!)

310

HTML (1-10)

310

CSS core (11-20)

312

Layout + responsive + motion (21-30)

314**Chapter 64 — Interview Bank II — JavaScript (35 Questions)****316**

Language core (1-12)

316

Functions/scope/this (13-20)

318

Async + APIs (21-28)

319

DOM + browser (29-35)

320**Chapter 65 — Interview Bank III — Backend, DB, Auth & React (25 Questions)****322**

Node/Express/REST (1-8)

322

SQL/NoSQL/modeling (9-14)

323

Auth/security (15-20)

324

React (21-25)

325

Exams & Code Vault — Test and Copy**327****Chapter 66 — Graded Exam I — HTML & CSS (25 Marks)****328**

Section A — quick answers (10 × 1 mark)

328

Section B — read the code (4 × 2 marks)

331

Section C — build (2 + 2 + 3 marks)

331

Marking guide

332**Chapter 67 — Graded Exam II — JavaScript (25 Marks)****334**

Section A — predict (10 × 1 mark)

334

Section B — debug (4 × 2 marks)

335

Section C — write (2 + 2 + 3 marks)

335**Chapter 68 — Graded Exam III — Backend, DB, Auth & React (25 Marks)****337**

Section A — decide (10 × 1 mark)

337

Section B — write (4 × 2 marks)

340

Section C — architecture (7 marks)

341

Speed drills — 10-minute blitzes (answers at the end of the drill)

342**Code Vault — Copy-Paste Production Snippets****345****Chapter 69 — Snippet Vault I — CSS (20 Snippets)****346****Chapter 70 — Snippet Vault II — JavaScript (20 Snippets)****350****Chapter 71 — Snippet Vault III — Backend, SQL & React (20 Snippets)****355**

Vault IV — full-stack mini-recipes (61-68, complete + runnable)

359**Projects & Career — Build and Earn****364****Chapter 72 — Skill Ladders — 40 Micro-Projects by Level****365****Chapter 73 — Project Gallery — 10 Portfolio Builds With Specs****368**

Frontend builds

368

Full-stack builds

369

Advanced builds

371**Chapter 74 — Ship & Earn — Freelancing and Your First Job****373**

Portfolio that converts (the 10-second test)

373

Freelance: profiles + pricing	373
Proposals + CV + interviews	374

Extended Datasheets — The Complete Lookup

376

Chapter 75 — HTML Tags A-M — Complete Reference	377
Chapter 76 — HTML Tags N-Z + Global Attributes	394
Global attributes (work on EVERY element)	407
Chapter 77 — CSS Properties A-M — Complete Reference	410
Chapter 78 — CSS Properties N-Z + Selectors Master List	426
Selectors master list	438
Modern CSS: nesting, layers, container queries	439
Chapter 79 — JavaScript Language A-Z — Complete Reference	440
Chapter 80 — Browser + Async APIs A-Z — Complete Reference	452

Extended Datasheets III — Backend & Tools Lookup

464

Chapter 81 — Node.js + Express A-Z — Complete Reference	465
Chapter 82 — SQL + Mongo + Git + React Hooks A-Z — Complete Reference	475

Capstone — Build BookHub End to End

486

Chapter 83 — Capstone Backend — API + Auth + Data (Full Code)	487
0. Shape + install (one minute)	487
1. Store (JSON-file repository — zero setup, honest pattern)	489
2. Auth (bcrypt + JWT + guards)	490
3. Routes (books + nested reviews)	491
4. App + boot + seed (wire it all)	493
Chapter 84 — Capstone Frontend — Pages + Style + App (Full Code)	495
1. index.html — semantic, accessible, complete	496
2. styles.css — tokens + gallery + dialogs + toasts	499
3. app.js — api client + auth + 4-state rendering	500
5. Stretch implementations (full code: refresh rotation + SSE feed)	503

Appendices — Datasheets, Glossary and Course Maps**505****Chapter 85 — Master Datasheet A — HTTP, Git, SQL and Express****506**

A1. HTTP master table

506

A2. Git command map

507

A3. SQL patterns

508

A4. Express essentials

509**Chapter 86 — Master Datasheet B — CSS, JavaScript, React and Regex****510**

B1. Flexbox + Grid one-screen recap

510

B2. JavaScript method matrix

511

B3. React hooks map

512

B4. Regex + JSON + storage recap

512**Chapter 87 — Master Datasheet C — Terminal, HTTP Deep-Dive, Cron and Meta****514**

C1. Terminal + npm command map

514

C2. HTTP headers + caching + CORS (server fluency)

515

C3. Status codes complete (the full map)

516

C4. Cron schedules (automate everything)

516

C5. Meta/OG/SEO head (paste-ready)

517

C6. Linux survival (deploy-day commands)

518

C7. package.json scripts cookbook

518**Chapter 88 — Glossary — 166 Terms in English and Arabic****520****Chapter 89 — Course Maps — Study With the Best Videos, Guided by This Book****527**

How to combine video + book (the method that works)

527

Absolute Zero? Start Here (Day 0)

Zero experience welcome

Never written a line of code? Perfect — this page is for you. In the next 20 minutes you will create your first web page and understand what HTML, CSS and JavaScript actually are.

Your first web page in 3 steps

Step 1. Open any text editor (Notepad on Windows, TextEdit on Mac — set to plain text) and type this exactly:

```
<!DOCTYPE html>
<html>
<head><title>My first page</title></head>
<body>
  <h1>Hello, I am learning the web!</h1>
  <p>This page was made by me on day zero.</p>
</body>
</html>
```

Step 2. Save the file as `hello.html` (make sure it ends with `.html`, not `.txt`).

Step 3. Double-click the file. It opens in your browser. You are now a web developer — everything after this is leveling up.

PRO TIP — YOU JUST DID REAL WEB DEVELOPMENT

That file IS a website: the same HTML professionals write, opened the same way (browsers read files and servers alike). Chapter 1 explains every line; for now, celebrate — day zero complete.

The 3 technologies in one picture

Table 1 — HTML vs CSS vs JavaScript (the 10-second version)

Technology	Job	Analogy
HTML	STRUCTURE (headings, text, images, buttons)	The skeleton and walls of a house
CSS	DESIGN (colors, sizes, layout, animation)	Paint, furniture and decoration
JavaScript	BEHAVIOR (clicks, logins, live data)	Electricity and plumbing that respond

Change one word in your file, save, and refresh the browser — you just edited structure (HTML). Chapters 11+ teach design (CSS); chapters 23+ teach behavior (JavaScript). The whole book is this page, grown up.

Quick check — What does HTML do?

1. Colors and layout
2. Page structure and content
3. Click behavior
4. Nothing

Answer: B · HTML = structure (skeleton); CSS = design (paint); JavaScript = behavior (electricity).

How to Use This Book

Read this first

This book teaches you **full-stack web development** — everything needed to build complete websites and web applications, from the page you see in the browser to the server and database behind it. It starts from absolute zero and ends at a professional level.

The promise of this book

If you read this book from the first chapter to the last, type every example with your own hands, and solve every exercise before looking at the solution — **this book alone is enough**. You will be able to build, publish and explain real full-stack projects.

How each chapter works

Every chapter follows the same rhythm so your brain always knows what is coming:

- 1. Simple explanation first.** Each idea starts with everyday words and a real-life analogy. No jargon before understanding.
- 2. Exact technical detail second.** After you "get it", you learn the precise professional version — the one used in documentation and job interviews.
- 3. Complete runnable example.** Not fragments. Every example is a full file you can save and run.
- 4. Quick checks and exercises.** Short questions inside the chapter, plus exercises with full solutions at the end of the chapter.
- 5. Key takeaway boxes.** Grey-gold boxes summarize what you must never forget.

PRO TIP — HOW TO GET 10X MORE FROM THIS BOOK

Type the code — do not copy-paste. Your fingers teach your brain. When an example works, **break it on purpose**: change one thing, predict what happens, then run it. That is how developers really learn.

The visual language of this book

Table 2 — What each box means

Box	Color	Meaning
Note	Blue	Extra explanation or background knowledge
Pro tip	Green	Professional habit that saves you time
Warning	Red	A mistake almost every beginner makes — avoid it
Try it yourself	Purple	Stop reading and do it now on your machine
Key takeaway	Gold	The one paragraph to remember from the section
Think of it like this	Pink	A real-life analogy for the idea
Interview question	Orange	A question employers actually ask, with the expected answer

What you need before page 1

Almost nothing. You need a computer with a web browser (Chrome or Edge is best) and a way to write text files. That is it. This book intentionally contains **no software installation tutorials** — they become outdated in months. All code here is standard HTML, CSS and JavaScript that runs in any modern browser, and the server chapters use plain Node.js code you can run anywhere Node exists.

Quick check — This book assumes you already know programming.

1. True
2. False, it starts from absolute zero
3. Only for seniors
4. Only for designers

Answer: B · The first chapters assume zero knowledge; later chapters grow to professional depth.

The 2026 Full-Stack Roadmap

This is the complete map of modern web development. The book follows it chapter by chapter. Keep this page open in your mind: whenever you feel lost, come back here and find where you are.

The big picture

TIER 1	· FOUNDATIONS		Ch 1-3		Internet, browsers, HTTP, DevTools
TIER 2	· HTML		Ch 4-10		Structure of every page
TIER 3	· CSS		Ch 11-22		Design, Flexbox, Grid, responsive
TIER 4	· JAVASCRIPT		Ch 23-36		Programming, DOM, async, APIs
TIER 5	· TOOLS		Ch 37-38	.	Git, GitHub, npm, deploy, CI/CD
TIER 6	· BACK-END		Ch 39-41		Node.js, Express, REST APIs
TIER 7	· DATABASES		Ch 42-43		PostgreSQL (SQL) + MongoDB (NoSQL)
TIER 8	· AUTH & SECURITY		Ch 44-45		Login systems, JWT, attacks & defense
TIER 9	· REACT		Ch 46		Modern front-end framework
TIER 10	· CAPSTONE		Ch 47		Real projects, portfolio, interviews
TIER 11	· DEEP DIVES		Ch 48-62	...	SVG, patterns, testing, TS, perf, Next.js
TIER 12	· PROOF		Ch 63-74	...	Interviews, exams, snippets, gallery, career
TIER 13	· DATASHEETS		Ch 75-82	...	HTML/CSS/JS/Backend A-Z references
TIER 14	· CAPSTONE + APPENDIX		Ch 83-89	...	Guided full build + datasheets + glossary

Figure 1 — The full-stack journey from zero to hired

Tier details — what "done" looks like

Table 3 — Roadmap tiers and exit skills

Tier	You can do	Chapters
1 · Foundations	Explain what happens when you open a website; inspect any page with DevTools	1–3
2 · HTML	Build any page structure: articles, galleries, tables, forms	4–10
3 · CSS	Design beautiful responsive pages with Flexbox and Grid	11–22
4 · JavaScript	Program logic, interactive pages, fetch data from APIs	23–36
5 · Tools	Version your code with Git and publish websites online	37–38
6 · Back-end	Build your own REST API with Node and Express	39–41
7 · Databases	Store data properly in SQL and NoSQL databases	42–43
8 · Auth & Security	Add login, protect routes, block common attacks	44–45
9 · React	Build component-based apps like the pros	46
10 · Capstone	Ship 3 portfolio projects and pass interviews	47
11 · Deep Dives	Master SVG, patterns, tests, TypeScript, performance, Next.js	48–62
12 · Proof	Pass interviews + exams; own snippets, gallery builds, career plan	63–74
13 · Datasheets	Look up ANY HTML/CSS/JS/backend API with explained examples	75–82
14 · Capstone + Appendix	Type a complete app; keep datasheets + glossary as work reference	83–89

KEY TAKEAWAY — THE GOLDEN RULE OF THE ROADMAP

Never skip a tier to chase a framework. Every React or back-end problem you will ever meet is secretly an HTML, CSS or JavaScript problem in disguise. Strong fundamentals beat framework-hopping every time.

About the Author

Mahmoud Mohamed Mahmoud Khattab

Mahmoud Mohamed Mahmoud Khattab is a full-stack web developer who believes great teaching beats great tooling: any motivated beginner can become job-ready with one honest book, a browser, and the discipline to type every example.

This handbook distills thousands of hours of learning and building — the lesson order of the best video courses (Elzero Web School, Abdelrahman Gamal's tracks, Angela Yu's Bootcamp), the precision of MDN and W3C documentation, and the hard-won habits of real client work — into a single path from absolute zero to hired developer.

How to reach the author

Table 4 — Contact

Channel	Details
Phone / WhatsApp	201034117684 · 201141225852
What to send	Your shipped project link, your hardest bug, or the chapter that changed everything for you
Response	Every shipped capstone gets a reply — proof of work earns proof of reading

KEY TAKEAWAY — KEY TAKEAWAY

This book's last page is your first day as a developer. Ship the capstone, send the link, and keep building — the author is rooting for you.

Your 16-Week Study Plan

Follow this plan and you will finish the book in about five to six months, studying roughly one hour per day. Each week ends with something you **built**, not just something you read.

Table 5 — Week-by-week plan

Week	Chapters	You build
1	1-4	Your first pages; you understand the Internet deeply
2	5-6	A rich article page with images, tables and links
3	7-8	A complete registration form + semantic blog layout
4	9-10	Personal profile page project + HTML drills
5	11-13	Styled cards; you master selectors and the box model
6	14-16	Typography system; positioned layouts
7	17-18	Flexbox navbar + Grid photo gallery
8	19-22	Fully responsive landing page + CSS drills
9	23-25	Console programs: calculator, guess game
10	26-28	Todo-list logic; object & array mastery drills
11	29-31	Interactive quiz app with localStorage
12	32-33	App that fetches real data from a public API
13	34-37	Debugged portfolio + Git + first deployment
14	39-41	Your own REST API for notes or products
15	42-45	Database-backed API with login and security
16	46-47	React version of your project + interview prep
17	48-51	SVG icons + advanced CSS art + framework duel

Week	Chapters	You build
18	52-55	Collections + patterns + tested code + TypeScript basics
19	56-59	Templates + realtime + data modeling + full-stack deploy
20	60-62	React pro (router/forms) + 90 Lighthouse + Next.js
21	63-65	All 90 interview questions answered aloud
22	66-68	All 3 graded exams passed (re-take weak ones!)
23	69-71	Snippet vault drilled + gallery build started
24	72-74	Gallery build shipped + earn plan live
25	75-82	Datasheet lookup drills (find any API in <60 seconds)
26	83-84	Capstone BookHub typed, tested and deployed
27+	85-89	Appendix datasheets + glossary = daily work reference

PRO TIP — IF YOU HAVE ONLY 30 MINUTES A DAY

Do half a chapter daily and double the weeks. Consistency beats speed: 30 focused minutes every day outperforms a 7-hour session once a month. Never let two days in a row pass without touching code.

WEEKLY RITUAL (REPEAT EVERY WEEK)

- - Read the week's chapters with a notebook beside you
- - Type every example and complete every Try-it box
- - Solve the chapter exercises without peeking at solutions
- - Build the week's mini-project from scratch, alone
- - Explain what you learned out loud, as if teaching a friend

Quick check — The study plan requires previous experience.

1. True
2. False — week 1 starts from zero
3. Only with a teacher
4. None of these

Answer: B · Week 1 assumes you know nothing at all.

PART

Web Foundations — How the Internet Really Works

Before writing a single tag, understand the machine you are building for: networks, browsers and servers.

- Ch 1** What Happens When You Open a Website
- Ch 2** Files, URLs and How Websites Are Organized
- Ch 3** HTTP — The Conversation Between Browser and Server

A developer who understands the platform debugs in minutes what others debug in days.

CHAPTER 1

What Happens When You Open a Website

Beginner

~ 90 min

The full journey of a web page — from your finger pressing Enter to pixels on the screen — explained with zero assumed knowledge.

IN THIS CHAPTER

- The client-server story
- DNS, IP and ports in plain words
- TCP, TLS and HTTP requests
- Front-end vs back-end vs full-stack
- Your first tour of browser DevTools

Angela \$1-\$3

Elzero fundamentals

MDN Web basics

The restaurant analogy (read this first)

THINK OF IT LIKE THIS — THE INTERNET IS A RESTAURANT

You (the **client**) sit at a table and read the menu. The waiter (the **browser**) takes your order to the kitchen (the **server**). The kitchen cooks (runs code, reads the **database**) and the waiter brings your dish (the **web page**). You never enter the kitchen — you only talk to the waiter. The entire web works exactly like this: clients ask, servers answer.

Computers that **ask** for pages are called **clients** (your phone, your laptop). Computers that **answer** with pages are called **servers** — powerful machines in datacenters that stay on day and night waiting for requests.

Addresses of the web: DNS, IP and ports

Every server has a numeric address called an **IP address**, like `142.250.190.46`. Humans hate numbers, so we invented names like `google.com`. Translating names into numbers is the job of the **DNS** (Domain Name System) — the phonebook of the Internet.

```
1. BROWSER ..... "What is the IP of google.com?" ----\
2. DNS SERVER ..... "It is 142.250.190.46" <-----/
3. BROWSER ..... opens a connection to 142.250.190.46
4. SERVER ..... receives the request, prepares the page
5. BROWSER ..... receives files (HTML+CSS+JS) and draws the page
```

Figure 2 — What happens when you type google.com and press Enter

A **port** is like an apartment number inside one building (one IP). Web servers normally listen on port **443** (secure) or **80** (old, insecure). A full address is therefore IP + port, for example `142.250.190.46:443`.

Table 6 — Words you must know from day one

Word	Simple meaning	Example
Client	The device asking for pages	Your browser
Server	The machine answering with pages	Google's computers
IP address	Numeric address of a machine	142.250.190.46
Domain	Human name for an IP	google.com
DNS	System translating domains to IPs	Phonebook of the web
Port	Which service inside the machine	443 = secure web
URL	Full address of one page	https://site.com/news

Quick check — DNS translates...

1. IP to domain
2. Domain to IP
3. HTML to CSS
4. JS to Python

Answer: B · DNS resolves human names like google.com into numeric IP addresses.

The three handshakes: TCP, TLS, HTTP

Three protocols (agreed languages) cooperate on every page load:

1. **TCP** opens a reliable connection — like dialing a phone number and waiting for "hello?". Your browser and the server agree they can hear each other.
2. **TLS** (the S in HTTPS) encrypts everything — like switching to a secret language nobody eavesdropping can understand. It also proves the server is really who it claims to be (certificates).
3. **HTTP** is the actual conversation: the browser sends a **request** ("give me /news") and the server returns a **response** ("here it is, status 200" + the files).

Table 7 — HTTP versions in 30 seconds

Version	Idea	Status
HTTP/1.1	One request at a time per connection; simple text	Still works everywhere
HTTP/2	Many requests over one connection; faster	Standard today
HTTP/3	Same idea over a faster transport (QUIC); best on mobile	Modern standard

KEY TAKEAWAY — KEY TAKEAWAY

Type URL → DNS finds IP → TCP connects → TLS encrypts → HTTP asks and receives → browser draws. Memorize this chain; half of all web debugging is finding which link broke.

Front-end vs back-end vs full-stack

Table 8 — The three jobs of web development

Job	Runs where	Built with	This book
Front-end	In the user's browser	HTML, CSS, JavaScript	Parts 2-4, 9
Back-end	On the server	Node.js, Express, databases	Parts 6-8
Full-stack	Both	Everything above	The whole book

- **Front-end** is everything the user sees and touches: layout, colors, buttons, animations.
- **Back-end** is everything hidden: saving accounts, checking passwords, storing orders.
- **Full-stack** means you can build both sides and connect them — the goal of this book.

The browser is not magic: how a page is drawn

When files arrive, the browser's engine performs the **Critical Rendering Path**:

```
HTML bytes ---> DOM tree (structure) ----\
                                         +---> Render tree ---> Layout ---> Paint ---> Screen
CSS bytes ----> CSSOM tree (styles) -----/
JavaScript ..... can modify both trees (interactivity)
```

Figure 3 — The Critical Rendering Path (simplified)

1. **DOM tree** — the browser turns HTML text into a tree of objects JavaScript can touch.
2. **CSSOM tree** — it turns CSS into a tree of style rules.
3. **Render tree** — structure + styles combined (invisible things like `display: none` are dropped).
4. **Layout** — the browser measures every box: x, y, width, height.
5. **Paint** — pixels are colored: text, backgrounds, borders, shadows.

PRO TIP — WHY THIS MATTERS ALREADY

Slow websites usually break at Layout or Paint. Later you will learn the golden performance rule: **animate transform and opacity only** — they skip Layout and Paint and run on the GPU at 60 frames per second.

Your first look at DevTools (2 minutes)

DevTools is the X-ray machine built into your browser. Press **F12** (or Ctrl+Shift+I) on any page right now.

Table 9 — The 4 DevTools panels you need first

Panel	What it shows	First trick to try
Elements	The live HTML + CSS of the page	Click the arrow icon, hover page parts, change text live
Console	JavaScript errors and your own test code	Type <code>2 + 2</code> and press Enter
Network	Every file the page downloaded	Reload and watch requests appear
Application	Stored data: cookies, localStorage	See what sites remember about you

TRY IT YOURSELF — OPEN DEVTOOLS NOW

Visit any website, press F12, find the Elements panel, double-click some headline text and rewrite it. Congratulations — you just "hacked" a page (only on your screen; reload restores it). Developers do this daily to experiment.

INTERVIEW QUESTION — WHAT HAPPENS WHEN YOU TYPE A URL AND PRESS ENTER?

Expected answer: "DNS resolves the domain to an IP, the browser opens a TCP connection, performs a TLS handshake for HTTPS, sends an HTTP request, receives the response, then parses HTML/CSS, builds DOM and CSSOM, and renders the page through layout and paint." Saying this chain calmly already puts you ahead of most juniors.

Exercise 1 — Explain it to a 10-year-old

Without looking back, explain DNS to a child in two sentences, then explain client vs server in two sentences.

SOLUTION

DNS is like the contacts app on a phone: you tap a name (google.com) and it finds the real number (the IP address) to call. A client is the person ordering food; a server is the kitchen cooking it — the client asks, the server answers.

Exercise 2 — Spot the tier

For each of these, say front-end or back-end: (a) a dropdown menu animation, (b) checking a login password, (c) page colors and fonts, (d) saving a new user in the database.

SOLUTION

(a) front-end (b) back-end (c) front-end (d) back-end. Rule of thumb: visible and interactive = front-end; hidden data and decisions = back-end.

CHAPTER 2

Files, URLs and How Websites Are Organized

Beginner

~ 60 min

Domains, paths, file types and project folders — the boring-looking knowledge that prevents 100 confusing errors later.

IN THIS CHAPTER

- Anatomy of a URL
- Files of a website
- Folders that make sense
- Absolute vs relative paths
- Case sensitivity trap

Gamal HTML #1-#5

W3Schools HTML

Anatomy of a URL

Look at this address piece by piece:

CODE · One URL, five parts

```
https://  shop.example.com:443  /products/shoes  ?color=red&size=42  #reviews
|         |                   |         |         |
protocol  domain              path      query string  fragment
```

Table 10 — URL parts

Part	Meaning	Example
Protocol	Language + security	<code>https://</code> (secure) vs <code>http://</code> (insecure)
Domain	Server name	<code>shop.example.com</code>

Part	Meaning	Example
Port	Service number (443 is default for HTTPS, usually hidden)	:443
Path	Which page/file	/products/shoes
Query string	Extra options after <code>?</code>	?color=red&size=42
Fragment	Jump inside the page after <code>#</code>	#reviews

The files of a website

Table 11 — The file types you will meet constantly

File	Contains	Runs where
<code>index.html</code>	Structure and text	Browser
<code>styles.css</code>	Design and layout	Browser
<code>app.js</code>	Behavior and logic	Browser (or server if Node.js)
<code>.jpg</code> / <code>.png</code> / <code>.webp</code> / <code>.svg</code>	Images	Browser
<code>.json</code>	Data in text format	Everywhere
<code>package.json</code>	Project info + dependencies (back-end)	Server tooling
<code>.env</code>	Secrets like passwords and keys (NEVER share)	Server only

WARNING — NEVER UPLOAD .ENV OR PASSWORDS TO GITHUB

The `.env` file holds secrets (database passwords, API keys). Real companies have been hacked because a developer pushed `.env` to a public repository. You will learn the safe way in the Git and Security chapters.

Folders that make sense

A clean front-end project looks like this from week one:

```
my-website/
|-- index.html      (home page)
|-- about.html      (second page)
|-- css/
|   |-- main.css    (your styles)
|   +-- reset.css   (optional: normalizes browser defaults)
|-- js/
|   +-- app.js       (your JavaScript)
+-- images/
|   |-- logo.svg
|   +-- hero.webp
```

Figure 4 — A professional starter structure

PRO TIP — NAMING RULES PROFESSIONALS FOLLOW

Lowercase only, no spaces (use `-`), English names, and extensions that match content (`.html`, `.css`, `.js`). `My Photo.JPG` breaks on real servers — `my-photo.jpg` never does.

Absolute vs relative paths (with zero confusion)

Imagine you are in the `my-website` folder above, editing `index.html`:

CODE · Linking files relatively

```
<!-- Same folder: just the name -->
<a href="about.html">About us</a>
<!-- Inside a subfolder: folder/name -->
<link rel="stylesheet" href="css/main.css">
<!-- One folder up: ../ means "go up" -->

<!-- Absolute web path: starts at site root -->
<a href="/products/shoes">Shoes</a>
```

Table 12 — Path cheat-sheet

Written	Meaning
<code>about.html</code>	File next to this one

Written	Meaning
<code>css/main.css</code>	Go into <code>css</code> , take <code>main.css</code>
<code>../logo.svg</code>	Go one folder up, take <code>logo.svg</code>
<code>/img/x.png</code>	From the website root, take <code>img/x.png</code>
<code>https://...</code>	Full address on another site (external)

WARNING — CASE SENSITIVITY: THE SILENT KILLER

On Windows `Logo.SVG` and `logo.svg` are the same file. On real web servers (Linux) they are **different files**. If your image works on your PC but not online, wrong uppercase/lowercase is suspect #1. Always use lowercase.

Quick check — `../img/a.png` from `/site/pages/about.html` points to...

1. `/site/pages/img/a.png`
2. `/site/img/a.png`
3. `/img/a.png`
4. Nowhere

Answer: B · `..` climbs from `pages/` to `site/`, then enters `img/`.

Exercise 3 — Fix the broken site

A beginner wrote `` and it fails online but works on his PC. List every problem and the fix.

SOLUTION

Three problems: (1) backslash `\` — web paths always use forward slashes `/`; (2) uppercase folder and extension — servers are case-sensitive; (3) vague name. Fix: rename to `images/photo.jpg` and write ``.

CHAPTER 3

HTTP — The Conversation Between Browser and Server

Beginner

~ 75 min

Requests, responses, methods and status codes: the shared language of the entire web, front and back.

IN THIS CHAPTER

- Requests and responses
- GET vs POST vs PUT/PATCH/DELETE
- Status codes you must memorize
- Headers in plain words
- JSON, the data language

Angela SAPI

MDN HTTP

Tutorialspoint HTTP

Requests and responses

Every interaction is two halves: the browser sends a **request**, the server returns a **response**.

BROWSER	SERVER
-- GET /products (request) ----->	
<----- 200 OK + JSON (response) --	
-- POST /orders {shoes} (request) --->	
<----- 201 Created (response) ----	

Figure 5 — An HTTP conversation

A request has: a **method** (what to do), a **path** (to what), **headers** (extra info) and sometimes a **body** (data). A response has: a **status code** (what happened), headers, and usually a body.

The seven methods (verbs)

Table 13 — HTTP methods

Method	Job	Example	Safe to repeat?
GET	Read data	Load products	Yes (never changes data)
POST	Create data	Submit registration	No (creates again!)
PUT	Replace whole item	Replace product #5	Yes
PATCH	Update part of item	Change only the price	Yes
DELETE	Remove item	Delete order #9	Yes
HEAD	GET without the body (check headers/size)	"Is file.big fresh?"	Yes
OPTIONS	Ask what is allowed (CORS preflight!)	Browser's permission check	Yes

THINK OF IT LIKE THIS — HTTP METHODS ARE RESTAURANT VERBS

GET = "show me the menu" (nothing changes). POST = "place a new order". PUT = "take my order back, here is a completely new one". PATCH = "same order, but no onions". DELETE = "cancel my order". HEAD = "just tell me the menu's size, not the food". OPTIONS = "waiter, what MAY I order here?" (CORS asks this before cross-origin requests!).

Status codes you must memorize

Table 14 — The 10 codes that cover 99% of real work

Code	Name	Meaning	Who caused it
200	OK	Success, here is your data	—
201	Created	New item saved	—
204	No Content	Success, nothing to return	—

Code	Name	Meaning	Who caused it
301	Moved	Page permanently moved (redirect)	Server config
304	Not Modified	Use your cached copy	—
400	Bad Request	Your data is invalid	Client
401	Unauthorized	Log in first	Client
403	Forbidden	Logged in, but not allowed	Client
404	Not Found	No such page/item	Client
429	Too Many Requests	Slow down (rate limit)	Client
500	Server Error	Bug in server code	Server

PRO TIP — READ CODES LIKE A PRO

2xx = success, **3xx** = redirect/cache, **4xx** = you (client) made a mistake, **5xx** = the server made a mistake. When debugging, this single digit tells you which side to fix.

Headers in plain words

Headers are small labeled notes attached to requests and responses:

CODE · Real headers, translated

```
Content-Type: application/json ..... "the body is JSON data"
Authorization: Bearer eyJhbG... ..... "this user is logged in, token attached"
Cache-Control: max-age=3600 ..... "you may reuse this for 1 hour"
Location: /login ..... "go here instead" (with 3xx)
```

You do not memorize headers now — just recognize the pattern `Name: value`.

JSON, the data language of the web

JSON (JavaScript Object Notation) is how front-end and back-end exchange data: text shaped like JavaScript objects.

CODE · A product in JSON

```
{
  "id": 5,
  "name": "Running shoes",
  "price": 1299,
  "inStock": true,
  "sizes": [40, 41, 42, 43]
}
```

Table 15 — JSON rules (only 6 matter)

Rule	Example
Keys always in double quotes	"name" not 'name'
Strings in double quotes	"Running shoes"
Numbers plain	1299 (no quotes)
Booleans lowercase	true / false
null for empty	" discount ": null
No trailing commas	[1, 2] not [1, 2,]

Quick check — A form submits but the server says 400. Who is wrong?

1. The server crashed
2. The client sent invalid data
3. The Internet is down
4. DNS failed

Answer: B · 4xx always means the request (client side) was wrong — here, validation failed.

Exercise 4 — Debug the status

For each case name the best status: (a) login with wrong password, (b) new article saved, (c) admin page visited by a normal user, (d) server code crashes.

SOLUTION

(a) 401 Unauthorized (b) 201 Created (c) 403 Forbidden (d) 500 Internal Server Error. Note (a) vs (c): unknown/at-login problems are 401, known-user-but-not-allowed is 403.

PART

HTML — The Skeleton of Every Page

Structure first: tags, text, links, images, tables and forms.

Ch 4 Your First HTML — Tags, Elements and the Page Skeleton

Ch 5 Text, Links and Lists

Ch 6 Images, Media and Tables

Ch 7 Forms — Collecting Data Like a Pro

Ch 8 Semantic HTML, Accessibility and SEO

Ch 9 Project — Personal Profile Page (HTML only)

Ch 10 HTML Datasheet — Every Important Tag in One Place

Learn HTML deeply once, and every framework becomes easy.

CHAPTER 4

Your First HTML — Tags, Elements and the Page Skeleton

Beginner

~ 90 min

What a tag really is, how elements nest, and the only page template you will ever need.

IN THIS CHAPTER

- Tags vs elements vs attributes
- The non-negotiable page skeleton
- Headings and paragraphs done right
- Comments and void elements
- Your first complete page

Elzero HTML #1-#14

Gamal HTML #1-#10

Angela's HTML

Tags, elements, attributes — finally clear

Table 16 — The three words beginners mix up

Word	What it is	Example
Tag	The angle-bracket marker	<code><p></code> and <code></p></code>
Element	Opening tag + content + closing tag	<code><p>Hello</p></code>
Attribute	Extra info inside the opening tag	<code>lang="en"</code> in <code><html lang="en"></code>

CODE · The anatomy of an element

```
<p class="intro" id="first">Hello, web!</p>
|   |           |   |   |
| attribute   attribute content |
opening tag                                closing tag
```

THINK OF IT LIKE THIS — TAGS ARE LABELED BOXES

An element is a labeled box: the opening tag writes the label (`<p>` = "this is a paragraph"), the content goes inside, the closing tag (`</p>`) seals the box. Attributes are sticky notes on the box ("class: intro").

Rules of nesting (3 rules, zero exceptions)

1. **Close in reverse order** (last opened = first closed).
2. **Never overlap** tags.
3. **Some elements can never contain anything** — they are void elements.

CODE · Nesting right vs wrong

```
<!-- RIGHT: perfect Russian dolls -->
<article><h2>Title</h2><p>Text with <strong>bold</strong> word.</p></article>

<!-- WRONG: overlapping (browsers guess, differently!) -->
<p>Text with <strong>bold</p></strong>

<!-- VOID elements: no closing tag, nothing inside -->

<br>
<hr>
<input type="text">
```

WARNING — THE #1 BEGINNER BUG

Forgetting that `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>` and `<link>` are **void**: writing `</img>` or putting text inside them. Validate mentally: "can this hold content?" A picture cannot hold a paragraph — so no closing tag.

The only skeleton you need (memorize it)

CODE · The professional HTML5 template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="My first professional web page.">
  <title>My First Page</title>
  <link rel="stylesheet" href="css/main.css">
</head>
<body>
  <h1>Hello, web!</h1>
  <p>This page follows every professional rule from day one.</p>
  <script src="js/app.js"></script>
</body>
</html>
```

Table 17 — Skeleton lines, translated

Line	Why it exists
<!DOCTYPE html>	Tells the browser "this is modern HTML5"
<html lang="en">	Root element + language for screen readers and Google
<meta charset="UTF-8">	Correct display of all languages and emojis
viewport meta	Makes mobile show the real responsive layout (never skip!)
description meta	The text Google shows under your link
<title>	Browser tab text + bookmark name
CSS in <head>	Styles load before content (no unstyled flash)
<script> at body end	Page shows first, code runs after

Quick check — Where does the page description for Google live?

1. In <title>
2. In meta description
3. In <h1>
4. In the URL

Answer: B · `<meta name="description">` feeds search-result snippets; `<title>` feeds the tab/link title.

Headings and paragraphs done right

CODE · All six levels — h1 to h6, logical order

```
<h1>Complete Guide to Coffee</h1>
<p>Coffee is a brewed drink prepared from roasted beans.</p>
<h2>History</h2>
<p>Centuries ago in Ethiopia...</p>
<h2>Brewing methods</h2>
<h3>Espresso</h3>
<p>Hot water forced through fine grounds...</p>
<h4>Grind size</h4>
<p>Fine, almost powdery...</p>
<h5>Dialing in</h5>
<p>Adjust dose and yield...</p>
<h6>Footnote: water first</h6>
<p>Always heat fresh water...</p>
```

KEY TAKEAWAY — KEY TAKEAWAY

Exactly **one** `<h1>` **per page** (the page topic), all six levels in order (`h1` → `h2` → `h3` → `h4` → `h5` → `h6` , never skipping like `h1` → `h4`), and paragraphs for text — never a heading chosen "because it looks big" (size is CSS's job).

TRY IT YOURSELF — YOUR FIRST PAGE (10 MINUTES)

Create `index.html` with the skeleton above, change the title and text to describe yourself, open it in the browser by double-clicking. Then press F12 and find your `<h1>` in the Elements panel. You are now officially a web developer.

Exercise 5 — Fix this skeleton

This page shows `????` for Arabic text and looks tiny on phones. Two lines are missing. Which?

SOLUTION

`<meta charset="UTF-8">` (fixes `????` encoding) and

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` (fixes mobile zoom).

These two lines belong in every page you ever write.

CHAPTER 5

Text, Links and Lists

Beginner

~ 75 min

The elements that carry 90% of web content: formatting text, linking pages, and building menus.

IN THIS CHAPTER

- Text formatting that means something
- Links: internal, external, email, phone, downloads
- Images vs buttons vs links
- Lists: unordered, ordered, description
- Building a real navigation menu

Elzero HTML #15-#28

Gamal HTML #11-#20

W3Schools HTML

Text formatting that MEANS something

HTML tags describe **meaning**, not looks. Screen readers and Google read the meaning.

Table 18 — Text elements by meaning

Element	Meaning	Use when
<code></code>	Important	Warnings, key terms (reads with emphasis)
<code></code>	Stressed emphasis	"You must save first"
<code><mark></code>	Highlighted	Search results
<code><small></code>	Fine print	Copyrights, disclaimers
<code></code> / <code><ins></code>	Deleted / inserted	Price changes, document edits
<code><code></code>	Code fragment	<code>let x = 1</code>

Element	Meaning	Use when
<code><time datetime="2026-09-07"></code>	Machine-readable date	Events, articles
<code>
</code>	Line break	Poems, addresses (rare!)
<code><hr></code>	Topic change	Between article sections

CODE · Meaningful text in action

```
<p>Sale ends <time datetime="2026-09-30">September 30</time>:
<del>$99</del> <ins>$59</ins>. <strong>Tickets are limited</strong>,
so <em>please</em> book early. Use code <code>SAVE40</code>.</p>
```

WARNING — `**<b>**` AND `**<i>**` ARE LAST-CENTURY STYLE

`<strong>` / `<em>` carry meaning; `<b>` / `<i>` are pure styling. In new code, default to `<strong>` and `<em>` — and do all visual styling in CSS anyway.

Links: the soul of the web

CODE · Every link type you need

```
<!-- Internal page -->
<a href="about.html">About us</a>
<!-- Section inside this page -->
<a href="#pricing">See pricing</a>
<!-- External site: ALWAYS add rel for security -->
<a href="https://developer.mozilla.org" target="_blank" rel="noopener">MDN Docs</a>
<!-- Email and phone -->
<a href="mailto:hello@example.com">Email us</a>
<a href="tel:+201141225852">Call us</a>
<!-- Download a file -->
<a href="files/menu.pdf" download>Download menu (PDF)</a>
```

Table 19 — Link rules professionals never break

Rule	Why
External + <code>target="_blank"</code> ⇒ add <code>rel="noopener"</code>	Without it the new page can hijack yours (tabnabbing attack)

Rule	Why
Link text describes the target	"Read the pricing guide" beats "click here" for Google and blind users
<code>href="#id"</code> jumps to <code>id="..."</code>	Powers tables of contents and "back to top"
<code>download</code> for files	Tells the browser to save, not open

Lists: unordered, ordered, description

CODE · All three list types

```

<!-- Bullets: order does not matter -->
<ul>
  <li>Milk</li>
  <li>Bread</li>
</ul>
<!-- Numbers: order matters (customize with reversed/start/type!) -->
<ol>
  <li>Boil water</li>
  <li>Add coffee</li>
  <li>Serve hot</li>
</ol>
<!-- Countdown: reversed + start -->
<ol reversed start="3">
  <li>Liftoff soon</li>
  <li>Ignition</li>
  <li>Liftoff!</li>
</ol>
<!-- Roman + letters, renumbered item -->
<ol type="I">
  <li>First</li>
  <li value="5">Fifth (jumped!)</li>
  <li>Sixth (continues 6)</li>
</ol>
<!-- Terms + definitions -->
<dl>
  <dt>HTML</dt><dd>Structure of the page.</dd>
  <dt>CSS</dt><dd>Design of the page.</dd>
</dl>

```

KEY TAKEAWAY — KEY TAKEAWAY

Navigation menus are `<ul>` lists of `<a>` links. Breadcrumbs, footers, dropdowns — all lists. If items look like "a set of similar things", reach for a list first.

Building a real navigation menu

CODE · A professional navbar (structure only; CSS comes later)

```
<nav aria-label="Main navigation">
  <ul>
    <li><a href="index.html" aria-current="page">Home</a></li>
    <li><a href="products.html">Products</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>
```

`aria-label` and `aria-current` are free accessibility wins: screen readers announce "Main navigation" and "current page: Home". You will master ARIA in Chapter 8.

Quick check — Which link is dangerous?

1. `<a href="/x">`
2. `<a target="_blank" href="https://evil.com">` without `rel`
3. `<a href="#top">`
4. `<a href="tel:+20">`

Answer: B · `target="_blank"` without `rel="noopener"` lets the new page control your tab. Always pair them.

Exercise 6 — Build a mini documentation page

Create a page with: an `<h1>`, a table of contents linking to 3 sections (`# part1 ...`), each section with `<h2 id="...">`, one external link (safe), and a "Back to top" link.

SOLUTION

Structure: `<a href="#part1">...` items at top; `<h2 id="part1">` etc. below; external link with `target="_blank" rel="noopener"`; `<a href="#top">` with `<body id="top">` (or an element `id="top"` at the start). Test every link by clicking.

CHAPTER 6

Images, Media and Tables

Beginner

~ 90 min

Showing pictures, video and audio correctly — plus tables that are accessible, not just grids.

IN THIS CHAPTER

- `img` done professionally (`alt`, `sizes`, `lazy`)
- Modern formats: WebP, AVIF, SVG
- `figure` + `figcaption`
- Audio and video
- Tables: `thead`, `tbody`, `th scope`, captions

Elzero HTML #29-#45

Gamal HTML #21-#32

Angela \$HTML

Images done professionally

CODE · The complete professional image

```

```

Table 20 — Every image attribute, explained

Attribute	Job	Rule
<code>src</code>	File path	Use <code>images/name.webp</code> , lowercase, no spaces
<code>alt</code>	Text replacement of the image	Informative images: describe. Decorative: <code>alt=""</code> (announces nothing)
<code>width</code> + <code>height</code>	Reserve space before loading	Prevents layout jump (Core Web Vital: CLS)

Attribute	Job	Rule
<code>loading="lazy"</code>	Load only when scrolled near	All below-the-fold images; NEVER on the top hero image

THINK OF IT LIKE THIS — ALT TEXT IS A PHONE CALL

Read your `alt` to a friend on the phone: can they picture the image? "IMG_2024" fails. "A barista pouring latte art shaped like a leaf" wins. If the image adds nothing (a decorative line), `alt=""` tells screen readers to skip it silently.

Modern formats (stop shipping heavy JPGs)

Table 21 — Which format when

Format	Best for	Why
SVG	Logos, icons, diagrams	Vector = infinitely sharp, tiny size, styleable with CSS
WebP	Photos and general images	~30% smaller than JPG, supported everywhere modern
AVIF	Large hero photos	~50% smaller than JPG; add JPG/WebP fallback
JPG	Fallback photos	Universal compatibility
PNG	Screenshots needing transparency	Bigger; use only when needed

CODE · Responsive art direction with picture

```
<picture>
  <source media="(min-width: 1024px)" srcset="images/hero-big.avif" type="image/avif">
  <source media="(min-width: 640px)" srcset="images/hero-mid.webp" type="image/webp">
  
</picture>
```

The browser picks the **first** `<source>` **it supports**; the `<img>` is the guaranteed fallback. `fetchpriority="high"` on the hero tells the browser "load this first".

Figures, audio and video

CODE · Media elements complete

```
<figure>
  
  <figcaption>Figure 1 – Company sales 2023–2026.</figcaption>
</figure>

<video controls width="640" preload="metadata" poster="images/talk-cover.webp">
  <source src="media/talk.webm" type="video/webm">
  <source src="media/talk.mp4" type="video/mp4">
  Sorry, your browser cannot play this video.
</video>

<audio controls preload="none">
  <source src="media/episode.mp3" type="audio/mpeg">
</audio>
```

PRO TIP — MEDIA PERFORMANCE RULES

`preload="none"` / `"metadata"` for below-fold media, always `controls` unless you build custom controls with JavaScript, always a `poster` for video, and prefer WebM/MP3 with fallbacks.

Tables: data, not layout

Tables are for **tabular data** (prices, schedules, specs) — never for page layout (that was a 1999 hack; CSS Grid replaced it).

CODE · An accessible professional table

```
<table>
  <caption>Course prices – 2026</caption>
  <thead>
    <tr><th scope="col">Course</th><th scope="col">Hours</th><th scope="col">Price</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">HTML Basics</th><td>10</td><td>$29</td></tr>
    <tr><th scope="row">CSS Mastery</th><td>25</td><td>$49</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Bundle</th><td>35</td><td>$59</td></tr>
  </tfoot>
</table>
```

Table 22 — Table elements

Element	Job
<code><caption></code>	Table title (announced first by screen readers)
<code><thead></code> / <code><tbody></code> / <code><tfoot></code>	Header / body / footer groups (also help printing + styling)
<code><th scope="col"></code>	Column header ("this labels the column below")
<code><th scope="row"></code>	Row header ("this labels the row beside")
<code><td></code>	Normal data cell
<code>colspan</code> / <code>rowspan</code>	Merge cells across columns / rows

Quick check — Why width+height on img?

1. Bigger images
2. Reserves space, prevents layout shift
3. Required by HTML
4. Faster download

Answer: B · Reserved aspect-ratio space stops content jumping when images load (fixes CLS).

Exercise 7 — Accessible schedule table

Build a weekly class schedule: caption, thead with Day/Topic/Time, 3 tbody rows with `scope="row"` day names, tfoot with total hours. Then view it with images off — it must still make full sense.

SOLUTION

Follow the course-price template: `<caption>Weekly schedule</caption>`, `<th scope="col">` for Day/Topic/Time, each row starting `<th scope="row">Saturday</th>`, tfoot summing hours. Screen-reader test: each data cell is announced with its headers.

CHAPTER 7

Forms — Collecting Data Like a Pro

Beginner+

~ 120 min

Labels, inputs, validation and accessible forms: the bridge between your page and the server.

IN THIS CHAPTER

- form, label, input trio
- Every input type with examples
- Built-in validation (required, pattern, min/max)
- Select, radio, checkbox, textarea
- Accessible, mobile-friendly forms

Elzero HTML #46-#62

Gamal HTML #33-#45

Angela \$Forms

The golden trio: form + label + input

CODE · The smallest correct form

```
<form action="/signup" method="post">
  <label for="email">Email address</label>
  <input type="email" id="email" name="email" required autocomplete="email">
  <button type="submit">Create account</button>
</form>
```

Table 23 — The trio, translated

Piece	Job	Deadly sin
<code><form action method></code>	Where data goes + how (<code>get</code> = in URL, <code>post</code> = in body)	Forgetting <code>action</code> (submits to same page randomly)
<code><label for="id"></code>	Clickable name of the field	Placeholder instead of label (vanishes, inaccessible)

Piece	Job	Deadly sin
<code><input name></code>	The field; <code>name</code> becomes the data key sent to the server	Missing <code>name</code> = field silently never sent
<code><button type="submit"></code>	Sends the form	<code><button></code> without type inside forms defaults to submit (surprise!)

THINK OF IT LIKE THIS — LABELS ARE NAME TAGS AT A PARTY

`for="email"` on the label must exactly match `id="email"` on the input — like calling someone by their exact name tag. Clicking the label then focuses the input (try it!). Screen readers also announce "Email address, edit text" only when this link exists.

Every input type (with when to use it)

CODE · The input zoo — each with its keyboard and validation

```
<label>Text <input type="text" name="city" maxlength="40" placeholder="Cairo"></label>
<label>Email <input type="email" name="email" required></label>
<label>Password <input type="password" name="pw" minlength="8" required autocomplete="new-password"></label>
<label>Age <input type="number" name="age" min="13" max="120"></label>
<label>Birthday <input type="date" name="dob"></label>
<label>Meeting <input type="datetime-local" name="meet"></label>
<label>Alarm <input type="time" name="alarm" value="07:00"></label>
<label>Card expiry <input type="month" name="exp"></label>
<label>Sprint week <input type="week" name="sprint"></label>
<label>Search <input type="search" name="q" placeholder="Beans..."></label>
<label>Favorite color <input type="color" name="color" value="#2563eb"></label>
<label>Volume <input type="range" name="vol" min="0" max="100" value="70"></label>
<label>Avatar <input type="file" name="avatar" accept="image/*"></label>
<label>Phone <input type="tel" name="phone" pattern="[0-9+ ]{8,15}"></label>
<label>Website <input type="url" name="site" placeholder="https://..."></label>
<input type="hidden" name="source" value="landing-page">
```

Table 24 — Choosing types like a senior

Need	Type	Bonus you get free
Email	<code>email</code>	Mobile shows @ keyboard + format check
Numbers	<code>number</code> + min/max	Spinners + range validation

Need	Type	Bonus you get free
Secret text	<code>password</code>	Hidden dots + manager support
Pick one of few	radio (same <code>name</code>)	Arrow-key navigation
Pick many	checkbox	Independent on/off values
Long text	<code><textarea rows></code>	Resizable multiline
Big option list	<code><select></code> + <code><option></code>	Native dropdown, searchable on mobile

Built-in validation (free, before JavaScript)

CODE · Validation without a single line of JS

```
<form action="/register" method="post">
  <label for="user">Username (3–12 letters)</label>
  <input id="user" name="user" required minlength="3" maxlength="12" pattern="[A-Za-z]+"
    title="Letters only, 3 to 12 characters">
  <label for="pw2">Password (8+ chars, 1 digit)</label>
  <input id="pw2" type="password" name="pw" required
    pattern="(?=.*\d){8,}" title="At least 8 characters including one digit">
  <button type="submit">Register</button>
</form>
```

Table 25 — Validation attributes

Attribute	Effect
<code>required</code>	Blocks submit when empty
<code>minlength</code> / <code>maxlength</code>	Text length limits
<code>min</code> / <code>max</code> / <code>step</code>	Number/date limits
<code>pattern</code> (regex)	Custom format rule + <code>title</code> explains it
<code>type=email/url/number</code>	Format validation included

WARNING — NEVER TRUST THE BROWSER ALONE

Built-in validation is UX sugar — attackers bypass it in seconds. The server **MUST** validate everything again (you will build that in Chapters 39–40). Rule: validate in browser for speed, on server for safety.

Select, radio, checkbox, fieldset

CODE · Grouped choices, accessibly

```
<fieldset>
  <legend>Delivery options</legend>
  <label><input type="radio" name="ship" value="fast" checked> Fast (2 days)</label>
  <label><input type="radio" name="ship" value="slow"> Slow (7 days, free)</label>
</fieldset>
<label for="pay">Payment</label>
<select id="pay" name="pay" required>
  <option value="">— Choose —</option>
  <option value="card">Credit card</option>
  <option value="cash">Cash on delivery</option>
</select>
<label><input type="checkbox" name="news" checked> Email me offers</label>
<label for="notes">Notes</label>
<textarea id="notes" name="notes" rows="4" maxlength="500"></textarea>
```

`<fieldset>` + `<legend>` groups related controls with a real announced title — the accessible way to say "these belong together".

Quick check — Radio buttons in one group must share...

1. id
2. name
3. value
4. type

Answer: B · Same `name` = one group (browser enforces single choice); each keeps its own `value` and unique `id`.

Exercise 8 — Complete registration form

Build: username (required, 3–12 letters), email (required), password (required, 8+ with digit), gender (radio: she/he, required), country (select with 4 options), bio (textarea, max 200), terms checkbox (required). Every control labeled, grouped with fieldset/legend.

SOLUTION

Follow the patterns above: labels with matching `for / id`, shared `name="gender"` radios, first `<option value="">` placeholder, `required` on terms checkbox. Submit empty to watch the browser block you with messages — that is built-in validation working.

CHAPTER 8

Semantic HTML, Accessibility and SEO

Beginner+

~ 90 min

Write pages that Google loves and everyone can use — including people who cannot see the screen.

IN THIS CHAPTER

- div-soup vs semantic landmarks
- header/nav/main/article/section/aside/footer
- ARIA when HTML is not enough
- Alt, labels, contrast, keyboard
- SEO basics that actually matter

Elzero HTML #63-#75

Gamal HTML #46-#55

MDN ARIA

Div-soup vs landmarks

CODE · Before — meaningless div soup

```
<div id="header"><div class="nav">...</div></div>
<div id="main"><div class="post">...</div></div>
<div id="footer">...</div>

<!-- After — a blind user can "jump to main content" -->
<header><nav aria-label="Main">...</nav></header>
<main>
  <article><h1>...</h1><p>...</p></article>
  <aside><h2>Related</h2>...</aside>
</main>
<footer>...</footer>
```

Table 26 — Landmark elements

Element	Meaning	Rule
<code><header></code>	Intro of page or section	Logo/nav inside; not <code><head></code> (head = metadata!)
<code><nav></code>	Major navigation block	Only main navs, not every link group
<code><main></code>	The page's unique core content	Exactly ONE per page
<code><article></code>	Self-contained piece (post, card, comment)	Must make sense alone / shareable
<code><section></code>	Thematic group, usually with heading	No heading? Probably a <code><div></code>
<code><aside></code>	Tangentially related	Sidebars, related links
<code><footer></code>	Closing info	Copyright, contacts, sitemap links
<code><div></code>	No meaning — styling/layout box	The honest fallback
<code></code>	No meaning — inline styling hook	Inside text only

KEY TAKEAWAY — KEY TAKEAWAY

`<article>` vs `<section>` vs `<div>`: could it be shared alone (a post)? → `<article>`. Is it a themed chunk with a heading (Pricing, Reviews)? → `<section>`. Just a box for styling? → `<div>`. This single test ends 90% of semantic debates.

ARIA: when HTML needs a microphone

Native elements come with free accessibility (buttons are clickable by keyboard, inputs announce labels). Only when building custom widgets do you add **ARIA** attributes:

CODE · ARIA essentials

```

<!-- Custom widget: tell assistive tech what it is and its state -->
<div role="alert">Saved successfully.</div>
<button aria-expanded="false" aria-controls="menu">Menu</button>
<nav id="menu" aria-label="Main navigation">...</nav>


```

Table 27 — ARIA survival kit

Attribute	Meaning
<code>role="alert/dialog/button"</code>	"Treat this div as..." (last resort — prefer real elements!)
<code>aria-label="Close"</code>	Invisible name for icon-only buttons
<code>aria-current="page"</code>	"This nav link is the current page"
<code>aria-expanded</code> / <code>aria-controls</code>	Widget open/closed state + what it controls
<code>aria-hidden="true"</code> + <code>alt=""</code>	"Skip this decoration entirely"

WARNING — FIRST RULE OF ARIA: DON'T USE ARIA

A real `<button>` beats `<div role="button">` (free keyboard, focus, clicks). A real `<input type="checkbox">` beats a custom one. Reach for ARIA only when HTML truly has no element for the job.

The accessibility checklist (WCAG in 1 page)

SHIP NOTHING WITHOUT THESE

- - Every `img` has correct `alt` (or `alt=""` for decoration)
- - Every input has a matching `<label for>`
- - Color contrast $\geq 4.5:1$ for text (check with DevTools → CSS overview)
- - Everything clickable works with keyboard alone (Tab + Enter)
- - Focus is always visible (never `outline: none` without a replacement)
- - Page has one `<h1>`, logical heading order, `<html lang>`
- - `aria-label` on icon-only buttons; `aria-current` in nav
- - Videos have captions; audio has transcripts

SEO basics that actually matter

CODE · SEO head starter

```
<title>Fresh Coffee Beans in Cairo – Roastly</title>
<meta name="description" content="Order freshly roasted coffee beans in Cairo. Same-day delivery, 12 origins.">
<link rel="canonical" href="https://roastly.example/">
<meta property="og:title" content="Roastly – Fresh Coffee Beans">
<meta property="og:description" content="Same-day delivery in Cairo.">
<meta property="og:image" content="https://roastly.example/images/og.jpg">
```

Table 28 — SEO that works in 2026

Do	Why
One clear <code><title></code> + honest description per page	This is your Google listing
Semantic headings with real keywords	Google reads structure, not font sizes
Fast + mobile-friendly (viewport, WebP, lazy)	Speed is a ranking factor
Descriptive <code>alt</code> + filenames	Image search traffic is free traffic
Open Graph tags	Pretty previews when shared on social apps
<code>lang</code> , sitemap, HTTPS	Basics Google expects from serious sites

Quick check — Best page title?

1. Home
2. Roastly
3. Fresh Coffee Beans in Cairo — Roastly
4. CLICK HERE!!!

Answer: C · Specific + keyword + brand. "Home" tells Google nothing; keyword-stuffing hurts.

Exercise 9 — Accessibility audit

Take your Chapter 6 page: unplug the mouse and use only Tab/Enter — can you reach everything? Then run Lighthouse in DevTools (Audits tab) and fix every Accessibility item until the score is 95+.

SOLUTION

Typical fixes: add missing `alt` s, pair every label, raise grey-on-white contrast to 4.5:1, add `:focus-visible` styles, give the nav `aria-label`. Re-run Lighthouse to verify — green scores are the proof.

CHAPTER 9

Project — Personal Profile Page (HTML only)

Beginner+

~ 120 min

Assemble everything from Part 2 into one complete, semantic, accessible page. No CSS yet — structure must shine alone.

IN THIS CHAPTER

- Planning with pen and paper
- Header + nav + main + footer
- About, skills table, gallery, contact form
- Accessibility self-review
- What CSS will change next

Project

Elzero practice

Angela §Project

Step 1 — Plan on paper (15 min)

```
+-----+
| HEADER: photo + name + tagline | NAV: about/ |
| skills/gallery/contact         | work/contact |
+-----+
| MAIN:                           |
| SECTION#about .... short bio + 3 facts list |
| SECTION#skills ... table: skill | level | years |
| SECTION#gallery .. 3 figures with captions   |
| SECTION#contact .. form: name/email/message  |
+-----+
| FOOTER: copyright + social links + back to top |
+-----+
```

Figure 6 — Profile page wireframe

PRO TIP — PROFESSIONALS SKETCH BEFORE CODING

Ten minutes with a pencil saves one hour of rewriting tags. Draw boxes, name them (header , section#skills), then translate boxes to elements one by one.

Step 2 — Build the skeleton + header

CODE · Profile page part 1 — head, header, nav

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Mahmoud Khattab — junior front-end developer
portfolio.">
  <title>Mahmoud Khattab — Front-End Developer</title>
</head>
<body id="top">
<header>
  
  <h1>Mahmoud Khattab</h1>
  <p>Junior front-end developer — HTML, CSS, JavaScript.</p>
  <nav aria-label="Page sections">
    <ul>
      <li><a href="#about">About</a></li>
      <li><a href="#skills">Skills</a></li>
      <li><a href="#gallery">Work</a></li>
      <li><a href="#contact">Contact</a></li>
    </ul>
  </nav>
</header>
<main>
```

Step 3 — Main sections

CODE · Profile page part 2 — about + skills table

```
<section id="about" aria-labelledby="about-h">
  <h2 id="about-h">About me</h2>
  <p>I build fast, accessible websites. I love <strong>semantic HTML</strong>
  and clean, honest code.</p>
  <ul>
    <li>Based in Giza, Egypt</li>
    <li>Open to junior roles and freelance</li>
    <li>Learning in public every week</li>
  </ul>
</section>
<section id="skills" aria-labelledby="skills-h">
  <h2 id="skills-h">Skills</h2>
  <table>
    <caption>Technical skills and experience</caption>
    <thead><tr><th scope="col">Skill</th><th scope="col">Level</th><th
scope="col">Years</th></tr></thead>
    <tbody>
      <tr><th scope="row">HTML</th><td>Advanced</td><td>2</td></tr>
      <tr><th scope="row">CSS</th><td>Intermediate</td><td>1</td></tr>
      <tr><th scope="row">JavaScript</th><td>Beginner</td><td>1</td></tr>
    </tbody>
    <tfoot>
      <tr><th scope="row">Total</th><td colspan="2">3 skills · 4 years combined</td></
tr>
    </tfoot>
  </table>
</section>
```


CODE · Profile page part 3 — gallery + contact + footer

```

<section id="gallery" aria-labelledby="gal-h">
  <h2 id="gal-h">Selected work</h2>
  <figure>
    
    <figcaption>Coffee shop homepage – semantic HTML practice.</figcaption>
  </figure>
  <figure>
    
    <figcaption>Registration form with built-in validation.</figcaption>
  </figure>
</section>
<section id="contact" aria-labelledby="con-h">
  <h2 id="con-h">Contact me</h2>
  <form action="/contact" method="post">
    <label for="c-name">Your name</label>
    <input id="c-name" name="name" required autocomplete="name">
    <label for="c-email">Your email</label>
    <input id="c-email" type="email" name="email" required autocomplete="email">
    <label for="c-msg">Message</label>
    <textarea id="c-msg" name="message" rows="5" required></textarea>
    <button type="submit">Send message</button>
  </form>
</section>
</main>
<footer>
  <p>© 2026 Mahmoud Khattab. Built with semantic HTML.</p>
  <p><a href="https://github.com/">GitHub</a> · <a href="mailto:hello@example.com">Email</a> · <a href="#top">Back to top ↑</a></p>
</footer>
</body>
</html>

```

Step 4 — Self-review checklist

PROFILE PAGE REVIEW

- - Exactly one `<h1>`; headings in order; landmarks `header/nav/main/footer` present
- - All images have `alt` + width/height; below-fold ones `loading="lazy"`
- - Form: labels match ids, `name`s set, `required` where needed
- - Keyboard-only navigation works; try unplugging the mouse
- - No `<div>` where `<section>` / `<article>` / `<nav>` fits; no `<br>` for spacing

KEY TAKEAWAY — KEY TAKEAWAY

If your page makes full sense with CSS turned off (DevTools → Rendering → disable CSS), your HTML is excellent. Structure first, beauty second — always.

CHAPTER 10

HTML Datasheet — Every Important Tag in One Place

Reference

~ 60 min

The quick-lookup chapter: find any tag in seconds, see what it does and copy a correct example.

IN THIS CHAPTER

- Document tags
- Text tags
- Links & media
- Lists & tables
- Forms & semantic — all with examples

Datasheet

W3Schools ref

MDN ref

Use this chapter during real work: each card shows the tag, what it is for, a correct example, and the one thing to remember.

!DOCTYPE **DOCUMENT**

Declaration (not a tag) that switches the browser to standards mode. Must be the very first line.

```
<!DOCTYPE html>
```

key: Without it browsers enter "quirks mode" and CSS behaves like 1999.

html DOCUMENT

Root element wrapping the whole page. Always set lang for screen readers and search engines.

```
<html lang="en">...</html>
```

key: lang="en" (or "ar" for Arabic) changes pronunciation of screen readers.

head DOCUMENT

Invisible metadata container: title, meta tags, CSS links. Nothing here is displayed.

```
<head><meta charset="UTF-8"><title>Shop</title></head>
```

key: CSS links go here; page JS usually goes at the end of body.

meta charset METADATA

Declares UTF-8 encoding so all languages and emojis display correctly.

```
<meta charset="UTF-8">
```

key: First tag inside head, always.

meta viewport METADATA

Tells mobile browsers to use the real device width — the foundation of responsive design.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

key: Missing viewport = broken mobile layout. Never skip.

title METADATA

Browser tab text, bookmark name and the clickable Google headline.

```
<title>Fresh Coffee in Cairo – Roastly</title>  
key: Unique per page, under ~60 characters.
```

h1–h6 TEXT

Headings level 1 (page topic) to 6 (deep sub-topic). Structure, not font size.

```
<h1>Guide</h1><h2>History</h2><h3>Early years</h3>  
key: One h1 per page; never skip levels.
```

p TEXT

A paragraph of text. Browsers add spacing automatically.

```
<p>Shipping is free over $50.</p>  
key: Never use <br><br> to fake paragraphs.
```

a LINKS

Hyperlink to pages, sections (#id), files, email (mailto:) or phone (tel:).

```
<a href="about.html">About</a>  
<a href="https://x.com" target="_blank" rel="noopener">X</a>  
key: External + new tab ⇒ always rel="noopener".
```

strong / em TEXT MEANING

Important (strong) and stressed (em) text — announced with emphasis by screen readers.

```
<p><strong>Warning:</strong> save <em>before</em> closing.</p>  
key: Meaning first; style with CSS.
```

ul / ol / li LISTS

Unordered (bullets), ordered (numbered) lists and their items. Menus are ul of links.

```
<ul><li>Milk</li><li>Bread</li></ul>  
<ol><li>Boil</li><li>Brew</li></ol>  
key: li must be a direct child of ul/ol.
```

img MEDIA

Embeds an image. src = path, alt = replacement text, width/height reserve space.

```
  
key: Informative ⇒ describe; decorative ⇒ alt="".
```

picture / source MEDIA

Art direction: serve different/better formats per screen, with img fallback.

```
<picture><source srcset="big.avif" type="image/avif"></picture>  
key: Browser takes the first source it supports.
```

figure / figcaption MEDIA

Self-contained media with a caption (diagrams, photos, code screenshots).

```
<figure><figcaption>Fig 1 – Sales.</figcaption></figure>
```

key: Caption belongs to the figure, moves with it.

video / audio MEDIA

Native players. Multiple sources = format fallbacks; text inside = last-resort fallback.

```
<video controls width="640" poster="p.webp"><source src="v.webm" type="video/webm"></video>
```

key: Always controls unless you build custom JS controls.

table / thead / tbody / tfoot TABLES

Table with header/body/footer groups. caption first; th for headers, td for data.

```
<table><caption>Prices</caption><thead><tr><th scope="col">Item</th></tr></thead><tbody><tr><td>X</td></tr></tbody></table>
```

key: Tables are for data – never for layout.

form FORMS

Collects inputs and submits to action via method get (URL) or post (body).

```
<form action="/signup" method="post">...</form>
```

key: Every submittable control needs a name.

label FORMS

Accessible name of a control. for must equal the control's id.

```
<label for="e">Email</label><input id="e" type="email" name="email">  
key: Clicking the label focuses the input – test it.
```

input FORMS

One-line field; type changes behavior: text email password number date file checkbox radio range color tel url.

```
<input type="email" name="email" required>  
key: Missing name ⇒ value never sent to the server.
```

textarea / select / option FORMS

Multiline text; dropdown list with options. First empty option = placeholder.

```
<textarea name="msg" rows="4"></textarea>  
<select name="pay"><option value="">– Choose –</option><option value="card">Card</option></select>  
key: select needs required + empty first option for "must choose".
```

button FORMS

Clickable button. Types: submit (send), reset (clear), button (JS only).

```
<button type="submit">Save</button><button type="button" id="more">Load more</button>  
key: Inside forms, typeless button = submit (common bug).
```


header / nav / main / footer LANDMARKS

Page intro, navigation, unique content (one only), closing info.

```
<header><nav>...</nav></header><main>...</main><footer>...</footer>
```

key: Landmarks let screen-reader users jump around the page.

article / section / aside SEMANTIC

Shareable piece (article), themed group with heading (section), tangential content (aside).

```
<article><h2>Post</h2></article><section><h2>Pricing</h2></section><aside>Related</aside>
```

key: No heading and no independent meaning ⇒ plain div.

div / span GENERIC

Meaningless block (div) and inline (span) containers for styling and JS hooks.

```
<div class="card"><span class="price">$9</span></div>
```

key: Honest fallback – but check semantic options first.

script / link RESOURCES

link loads CSS in head; script loads JS (end of body, or head with defer).

```
<link rel="stylesheet" href="css/main.css">
<script src="js/app.js" defer></script>
```

key: defer = download now, run after HTML parsed. Best default.

PART

CSS — Designing Beautiful Pages

From selectors to Flexbox and Grid: the complete visual system of the web.

- Ch 11** CSS First Steps — Selectors and Ways to Style
- Ch 12** The Cascade, Specificity and Inheritance
- Ch 13** The Box Model — Every Element Is a Box
- Ch 14** Sizing & Typography Deep-Dive — font-size vs font-weight, width vs max-width
- Ch 15** Colors, Backgrounds, Borders and Shadows
- Ch 16** Display and Positioning — Controlling the Flow
- Ch 17** Flexbox — One-Dimensional Layout Mastery
- Ch 18** CSS Grid — Two-Dimensional Layout Mastery
- Ch 19** Responsive Design — Mobile First
- Ch 20** Transitions, Transforms and Animations
- Ch 21** Variables, Architecture and the CSS Project
- Ch 22** CSS Datasheet — Every Important Property in One Place

CSS is easy to start and glorious to master — this part takes you all the way.

CHAPTER 11

CSS First Steps — Selectors and Ways to Style

Beginner

~ 90 min

How CSS talks to HTML: rules, selectors, and the three places styles can live.

IN THIS CHAPTER

- Rules, declarations, selectors
- Element / class / id / universal selectors
- Combining selectors
- Inline vs internal vs external CSS
- Your first styled page

Elzero CSS #1-#15

Gamal CSS #1-#12

Angela §CSS

A CSS rule in 10 seconds

CODE · The anatomy of a rule

```
h1 {  
  color: navy;  
  font-size: 32px;  
}  
  
|   |   |  
|   |   value  
|   property  
| selector (who to style)
```

A **selector** picks elements; **declarations** (`property: value;`) describe the styling. Forget a semicolon or brace and the whole rule dies silently — CSS never shows errors, it just ignores mistakes.

WARNING — CSS FAILS SILENTLY

A typo like `colour: red` or a missing `}` breaks styling with ZERO error messages. When "CSS doesn't work", suspect #1 is always a typo. DevTools → Elements → Styles shows crossed-out invalid lines.

The selectors you will use daily

Table 29 — Core selectors

Selector	Picks	Example
<code>p</code>	All paragraphs	<code>p { ... }</code>
<code>.card</code>	All elements with <code>class="card"</code>	The workhorse — reuse everywhere
<code>#menu</code>	The single element with <code>id="menu"</code>	Unique things only
<code>*</code>	Everything	Resets only (never for real styling)
<code>h1, h2, p</code>	Groups (any listed)	Shared rules in one place
<code>.card p</code>	<code><p></code> inside <code>.card</code> (any depth)	Descendant
<code>.menu > li</code>	Direct children only	Top-level items, not nested ones
<code>input[type="email"]</code>	By attribute	Email fields specifically
<code>a:hover</code>	On mouse hover	Interactive states
<code>li:first-child</code>	First child of its parent	First items, no classes needed

CODE · Selectors in action

```
/* All cards share the box look */
.card { border: 1px solid #ddd; padding: 16px; }
/* Only paragraphs INSIDE cards get this color */
.card p { color: #333; }
/* The one hero title */
#hero-title { font-size: 48px; }
/* Email fields glow on focus */
input[type="email"]:focus { outline: 2px solid navy; }
```

KEY TAKEAWAY — KEY TAKEAWAY

Classes for styling (reusable), IDs for JS hooks and anchors (unique). If you styled with `#id` everywhere, you cannot reuse the style — seniors style with classes ~95% of the time.

Combining selectors (read right-to-left)

CODE · Reading selectors like a senior

```
/* "Links inside list items inside .menu" */
.menu li a { text-decoration: none; }
/* "Direct li children of .menu" (excludes nested dropdown items) */
.menu > li { display: inline-block; }
/* "Paragraphs that come right after an h2" (intro paragraphs) */
h2 + p { font-size: 18px; }
/* "Any p after an h2 until the next heading" */
h2 ~ p { color: #444; }
```

Table 30 — Combinators

Symbol	Name	Meaning
(space)	Descendant	Anywhere inside
<code>></code>	Child	Direct child only
<code>+</code>	Adjacent sibling	The very next sibling
<code>~</code>	General sibling	Any later sibling

Three places CSS can live (use only one!)

CODE · The three ways — and the winner

```
<!-- 1. EXTERNAL (the professional way): one file styles the whole site -->
<link rel="stylesheet" href="css/main.css">

<!-- 2. INTERNAL: <style> in head — only for single-page experiments -->
<style>h1 { color: navy; }</style>

<!-- 3. INLINE: style="" — emergencies and JS only, never for real styling -->
<h1 style="color: navy;">Hi</h1>
```

Table 31 — Where to put CSS

Way	Use when	Problem
External <code>.css</code> file	Always (real projects)	None — cached, shared, clean
<code><style></code> in head	Quick single-page tests	Cannot share across pages
<code>style=""</code> inline	JS dynamic styles, HTML emails	Unreusable, beats all specificity, unmaintainable

Quick check — Best selector for "all submit buttons site-wide"?

1. `#submit`
2. `button.submit-btn` class
3. `div > div > button`
4. `*`

Answer: B · A class is reusable and readable; IDs can't repeat; long chains are brittle.

Exercise 10 — Selector gym

Given `<nav class="menu"><ul><li><a>Home</a></li></ul></nav>`: write selectors for (a) the nav, (b) all links in it, (c) links on hover turning gold.

SOLUTION

(a) `.menu` (b) `.menu a` (c) `.menu a:hover { color: gold; }`. Note (b) uses a descendant selector — short, robust, reusable.

CHAPTER 12

The Cascade, Specificity and Inheritance

Beginner+

~ 90 min

Why one rule wins over another: the scoring system browsers use on every element.

IN THIS CHAPTER

- Cascade order (origin + position)
- Specificity scoring 0-0-0
- !important and why to avoid it
- Inheritance: what passes to children
- Debugging conflicts in DevTools

Elzero CSS #16-#25

Gamal CSS #13-#20

W3Schools CSS

The cascade: three rounds of battle

When rules collide, the browser decides in this order:

- 1. Origin & importance:** your styles beat browser defaults; `!important` beats everything normal.
- 2. Specificity:** the most specific selector wins (scored below).
- 3. Position:** if still tied, the LAST rule in the file wins.

CODE · Position decides ties

```
p { color: red; }  
p { color: navy; } /* wins: same specificity, comes later */
```


Specificity scoring (count 0-0-0)

Selector	IDs	Classes/attrs/pseudo	Elements	Score
p	0	0	1	... 0-0-1
.card	0	1	0	... 0-1-0
.menu li a	0	1	2	... 0-1-2
#nav .item	1	1	0	... 1-1-0 (winner)
style=""	beats everything (except !important)			

Figure 7 — Specificity scoreboard: ID - CLASS - ELEMENT

CODE · Specificity in action

```
/* 0-0-1 */
a { color: blue; }
/* 0-1-1 — wins over the rule above */
.menu a { color: white; }
/* 1-0-0 — wins over both */
#special-link { color: gold; }
```

THINK OF IT LIKE THIS — SPECIFICITY IS A MEDAL TABLE

Countries rank by gold first, then silver, then bronze: 1 gold beats 100 silvers. Likewise **1 ID beats any number of classes**, and 1 class beats any number of elements. Compare left column first — only move right on ties.

WARNING — THE !IMPORTANT TRAP

`color: red !important` bullies past everything — so the next developer adds a bigger `!important`, then another... Soon nothing can be overridden and the stylesheet is unfixable. Allowed only for: utility classes (`.hidden`) and overriding third-party code you cannot edit. If you use it weekly, your specificity structure is sick.

Inheritance: what children get for free

Some properties **inherit** (children copy the parent's value), others don't:

Table 32 — Inherits vs doesn't

Inherits (set once on body)	Never inherits (set per element)
<code>color</code> , <code>font-family</code> , <code>font-size</code> , <code>line-height</code> , <code>text-align</code> , <code>letter-spacing</code>	<code>margin</code> , <code>padding</code> , <code>border</code> , <code>background</code> , <code>width</code> , <code>display</code> , <code>position</code>

CODE · Set typography once, use everywhere

```
body {  
  font-family: Arial, sans-serif;  
  color: #222;  
  line-height: 1.6;  
}  
/* Every p, li, td now inherits these – zero repetition */
```

Special values: `inherit` (force copy parent), `initial` (browser default), `unset` (inherit if inheritable else initial).

Debugging conflicts in DevTools

TRY IT YOURSELF — SEE THE BATTLE LIVE (3 MINUTES)

Open any page → F12 → Elements → select a heading → look at the Styles panel. Winning rules are on top; losing rules are **crossed out** with the winner's selector visible. This panel alone solves most "why is my CSS ignored?" mysteries.

INTERVIEW QUESTION — HOW DO YOU HANDLE CSS SPECIFICITY WARS?

Expected: "I keep specificity low and flat — mostly single classes — so order decides; I avoid IDs in CSS and never use `!important` except for utilities; I debug in DevTools Styles panel where losers are crossed out."

Quick check — `.a .b` (0-2-0) vs `#x` (1-0-0): who wins?

1. `.a .b` (more selectors)
2. `#x` (an ID beats any classes)
3. Tie → later wins
4. Neither

Answer: B · IDs outrank everything in the classes column, no matter how many classes there are.

Exercise 11 — Score these

Rank weakest→strongest: `h1`, `.title`, `#main .title`, `#main`, `body h1.title`.

SOLUTION

`h1` (0-0-1) < `body h1.title` (0-1-2)... wait, compare properly: `h1` = 0-0-1; `body h1.title` = 0-1-2; `.title` = 0-1-0; `#main` = 1-0-0; `#main .title` = 1-1-0. Order: `h1` < `.title` < `body h1.title` < `#main` < `#main .title`. Remember: classes column beats elements column entirely.

CHAPTER 13

The Box Model — Every Element Is a Box

Beginner+

~ 100 min

Content, padding, border, margin — plus the box-sizing fix every professional applies on day one.

IN THIS CHAPTER

- The 4 layers with a diagram
- content-box vs border-box (the big fix)
- Margin collapsing
- Developer box inspector
- Centering and spacing patterns

Elzero CSS #26-#35

Gamal CSS #21-#30

Angela §CSS

The 4 layers

```
+-- MARGIN (transparent gap OUTSIDE, separates from neighbors) --+
| +-- BORDER (visible edge line) -----+ |
| | +-- PADDING (space INSIDE, around content) -----+ | | | | | |
| | | +-- CONTENT (text/images: width x height) ---+ | | |
| | | | Hello, I am the content.                | | | |
| | | +-----+ | | |
| | +-----+ | |
| +-----+ |
+-----+
```

Figure 8 — Every element is 4 nested boxes

CODE · The 4 layers in code

```
.card {
  width: 300px;           /* content width */
  padding: 20px;          /* inner breathing room */
  border: 2px solid navy; /* visible edge */
  margin: 24px auto;      /* outside gap + horizontal centering */
}
```

Table 33 — Shorthand clock (1-4 values)

Written	Top	Right	Bottom	Left
margin: 10px	10	10	10	10
margin: 10px 20px	10	20	10	20
margin: 10px 20px 30px	10	20	30	20
margin: 10px 20px 30px 40px	10 ↑	20 →	30 ↓	40 ← (clockwise from top)

content-box vs border-box (the fix of the century)

The browser default (`content-box`) makes `width` count **content only**: padding and border are added ON TOP, so `width: 300px` + `padding: 20px` + `border: 2px` = **344px** of real width. Layouts break mysteriously.

CODE · The universal fix — first lines of every pro stylesheet

```
*, *::before, *::after { box-sizing: border-box; }
```

Table 34 — The difference that breaks layouts

Rule	content-box (default)	border-box (professional)
<code>width: 300px; padding: 20px; border: 2px</code>	Real width = 344px (300+40+4)	Real width = 300px exactly
Two 50% columns + padding	Overflow to next line (broken!)	Fit perfectly

Rule	content-box (default)	border-box (professional)
Mental model	"Width plus extras"	"Width means width" (sane)

THINK OF IT LIKE THIS — PIZZA BOX VS PIZZA

`content-box`: you order a 30cm pizza and get a 36cm box (pizza + walls + padding).

`border-box`: you order a 30cm box and the pizza shrinks to fit inside. Professionals always think in boxes — so `border-box` everywhere.

Margin collapsing (the surprise)

Vertical margins of neighbors **merge instead of adding**: `margin-bottom: 20px` meeting `margin-top: 30px` = **30px gap, not 50px**.

Table 35 — Collapsing rules

Case	Result
Two siblings (20px meets 30px)	30px (bigger wins)
Parent + first child top margins	Merge through the parent (child "pulls" parent down!)
Empty element's own top/bottom	Collapse into one
Horizontal margins (left/right)	NEVER collapse — always add
Flex/Grid children (gap)	NEVER collapse — use <code>gap</code> instead

CODE · Fixes for collapsing headaches

```
/* Option 1: use padding on parent instead of margin on child */
.parent { padding-top: 30px; } /* instead of child margin-top */
/* Option 2: modern layouts use gap (Ch 17-18), margins never meet */
/* Option 3: one-direction margins: only margin-bottom everywhere */
```

The box inspector + centering patterns

TRY IT YOURSELF — INSPECT THE BOX (2 MINUTES)

F12 → Elements → select any element → look at the Computed/Box panel: a colored diagram shows exact content/padding/border/margin in pixels. Hover it to highlight each layer on the page. This is how pros measure "why is there a gap here?".

CODE · Patterns you will reuse forever

```
/* Center a fixed-width block */
.container { width: 100%; max-width: 1100px; margin: 0 auto; }
/* One-direction spacing system */
.stack > * + * { margin-top: 16px; } /* every child except first */
/* Reset killing browser default margins */
h1, h2, h3, p, ul { margin: 0 0 12px 0; }
```

Quick check — `width:200px; padding:20px` **with border-box = ?**

1. 240px wide
2. 200px wide, content 160px
3. 200px content
4. Error

Answer: B · Border-box keeps total 200px; content shrinks to 200–40=160px.

Exercise 12 — Box math

`.box { width: 400px; padding: 25px; border: 5px solid; margin: 10px; }` — real rendered width in content-box vs border-box (excluding margin)?

SOLUTION

content-box: $400 + 50 + 10 = 460\text{px}$. border-box: 400px exactly (content becomes 340px). Margin (10px each side) sits outside both — never part of width.

CHAPTER 14

Sizing & Typography Deep-Dive — font-size vs font-weight, width vs max-width

Intermediate

~ 120 min

The most confused topics in CSS, explained once and forever: text size vs text weight, fixed vs fluid dimensions, units and line-height.

IN THIS CHAPTER

- font-size vs font-weight (the full truth)
- px vs rem vs em vs %
- width vs max-width, height vs min-height
- line-height unitless rule
- Building a type scale

Gamal CSS typography

Elzero CSS units

Deep-dive

DEEP-DIVE 1: font-size vs font-weight

Juniors constantly mix these two. They control **completely different things**:

Table 36 — font-size vs font-weight — the full truth

	font-size	font-weight
Controls	Spatial scale : how tall/wide letters are	Stroke thickness : how heavy/black the strokes are
Analogy	Zoom level of the text	Thickness of the pen
Units	px, rem, em, %, vw, clamp()	Numbers 100–900 or normal/bold
Layout effect	Changes element height, line boxes, wrapping	(Almost) no layout change — same box, thicker ink

	font-size	font-weight
Example pair	<code>font-size: 12px</code> vs <code>48px</code>	<code>font-weight: 400</code> vs <code>800</code>
Accessibility	Use <code>rem</code> so users can scale text	Thin weights (100–300) can fail contrast on cheap screens

CODE · Same sentence, four personalities

```
.tiny-bold { font-size: 12px; font-weight: 800; } /* small but shouts */
.huge-light { font-size: 48px; font-weight: 200; } /* big but whispers */
.body { font-size: 1rem; font-weight: 400; } /* normal reading */
.display { font-size: 3rem; font-weight: 700; } /* hero headline */
```

Table 37 — The 100–900 weight scale

Value	Name	Used for
100–200	Thin / Extra Light	Huge elegant headlines only
300	Light	Large subtitles (careful with contrast)
400	Normal / Regular	ALL body text default
500	Medium	Slightly emphasized UI text
600	Semi Bold	Sub-headings
700	Bold	Headings (<code></code> default)
800–900	Extra Bold / Black	Display headlines, badges, numbers

WARNING — FAUX BOLD: THE BLURRY LIE

If you request `font-weight: 800` but loaded only weight 400, the browser **fakes** thickness by smearing pixels (faux bold) — ugly and blurry. Fix: load the real weights you use (Google Fonts → select 400;600;800), or use variable fonts.

TRY IT YOURSELF — FEEL THE DIFFERENCE (1 MINUTE)

In DevTools, select a headline and toggle `font-weight` 300→900 (box barely changes), then toggle `font-size` 12px→48px (layout explodes). Your eyes now permanently know the difference.

Units: px vs rem vs em vs % vs vw

Table 38 — Units decision table

Unit	Means	Use for	Avoid for
px	Screen pixels (fixed)	Borders, shadows, tiny details	Body font-size (blocks user scaling)
rem	× root (<code><html></code>) font-size	ALL font sizes, spacing, widths	—
em	× parent's font-size	Padding that scales with its own text (buttons, badges)	Nested font sizes (compounds confusingly!)
%	× parent's same property	Fluid widths (<code>width: 50%</code>)	Font sizes (same compounding trap as em)
vw/vh	× viewport width/height	Full-screen heroes (<code>min-height: 100vh</code>)	Body text (too extreme on huge/small screens)
ch	Width of "0" glyph	Readable line length (<code>max-width: 65ch</code>)	—

CODE · The professional unit system

```
html { font-size: 16px; }           /* 1rem = 16px baseline */
body { font-size: 1rem; }          /* 16px, user-scalable ✓ */
h1 { font-size: 2.5rem; }          /* 40px, scales with user setting ✓ */
.card { padding: 1.5rem; max-width: 65ch; }
.btn { font-size: 0.9rem; padding: 0.6em 1.2em; } /* em: padding hugs text */
.hero { min-height: 90vh; }
```

KEY TAKEAWAY — KEY TAKEAWAY

`rem` for sizes, `em` for text-hugging padding, `%` for fluid widths, `px` for hairlines.

And the accessibility law: a user who sets their browser to 24px default must get a fully working site — only `rem`-based typography guarantees that.

Quick check — Which respects browser font-size settings?

1. `px`
2. `rem`
3. Both equally
4. Neither

Answer: B · `rem` multiplies the user's root size; `px` ignores it. That's why body text uses `rem`.

DEEP-DIVE 2: width vs max-width, height vs min-height

Table 39 — Fixed vs fluid — the responsive truth

Rule	Behavior	Verdict
<code>width: 600px</code>	Always 600px — overflows phones (375px!) with horizontal scroll	Avoid for layout containers
<code>width: 100%; max-width: 1100px</code>	Fills phones, stops growing on wide screens	THE container pattern — memorize it
<code>height: 250px</code>	Rigid box — long text overflows and collides below	Anti-pattern for text containers
<code>min-height: 250px</code>	At least 250px, grows with content	Correct for heroes, cards, sections
<code>aspect-ratio: 16/9</code>	Height auto-follows width	Modern way for media boxes

CODE • The two patterns behind 90% of professional layouts

```
/* Pattern 1: centered fluid container */
.container { width: 100%; max-width: 1100px; margin: 0 auto; padding: 0 16px; }
/* Pattern 2: elastic card that never overflows */
.card { min-height: 220px; } /* grows if text grows ✓ */
.media { aspect-ratio: 16 / 9; } /* video box keeps shape ✓ */
```

THINK OF IT LIKE THIS — FIXED VS ELASTIC

`width/height` are **glass boxes**: exact size, contents spill if bigger. `max-width/min-height` are **balloons with limits**: they fit their space but never exceed (or shrink below) your rule. Responsive design = balloons everywhere, glass almost nowhere.

WARNING — HEIGHT + TEXT = OVERFLOW BUG

The #1 responsive bug in junior portfolios: a card with `height: 200px` whose Arabic/English text wraps to 3 lines on mobile and spills over the next card. Fix: `min-height` instead — always.

line-height: the unitless law

CODE • Unitless (correct) vs px (dangerous)

```
body { line-height: 1.6; } /* ✓ each element: own font-size × 1.6 */
/* h1 at 40px gets 64px lines; p at 16px gets 25.6px lines – perfect */

body { line-height: 24px; } /* ✗ every child inherits rigid 24px */
/* h1 at 40px squeezed into 24px lines – letters collide! */
```

Table 40 — Line-height recipes

Text	line-height	Why
Body paragraphs	1.5–1.7	Reading comfort
Headlines	1.1–1.25	Tight display look
Buttons/small UI	1 + padding	Padding controls the height, not lines

Building a type scale (copy this system)

CODE · A complete professional type system

```
:root {
  --font-body: "Segoe UI", Arial, sans-serif;
  --text-xs: 0.75rem; /* 12px badges */
  --text-sm: 0.875rem; /* 14px captions */
  --text-base: 1rem; /* 16px body */
  --text-lg: 1.25rem; /* 20px leads */
  --text-xl: 1.5rem; /* 24px h3 */
  --text-2xl: 2rem; /* 32px h2 */
  --text-3xl: 3rem; /* 48px h1 */
}
body { font-family: var(--font-body); font-size: var(--text-base); line-height: 1.6; }
h1 { font-size: var(--text-3xl); line-height: 1.1; font-weight: 800; }
h2 { font-size: var(--text-2xl); line-height: 1.2; font-weight: 700; }
.lead { font-size: var(--text-lg); font-weight: 300; }
```

Exercise 13 — Type detective

A site uses `body { font-size: 14px; line-height: 20px; }` and `.card { height: 180px; width: 500px; }`. List 4 problems and fixes.

SOLUTION

(1) px font-size ignores user settings → `1rem`-based sizes. (2) px line-height squashes big headings → unitless `1.6`. (3) fixed height overflows wrapped text → `min-height`. (4) fixed 500px width breaks 375px phones → `width: 100%; max-width: 500px`.

CHAPTER 15

Colors, Backgrounds, Borders and Shadows

Beginner+

~ 75 min

The painter's toolkit: color formats, gradients, backgrounds, rounded corners and depth.

IN THIS CHAPTER

- hex vs rgb vs hsl (and when each wins)
- Text color + contrast law
- Backgrounds and gradients
- Borders and border-radius
- Shadows and opacity

Elzero CSS #36-#48

Gamal CSS #31-#40

W3Schools CSS

Color formats (learn all three, prefer two)

Table 41 — Color formats

Format	Example	Best when
HEX	<code>#2563eb</code> , <code>#fff</code>	Copying exact brand colors (short, universal)
RGB(A)	<code>rgb(37 99 235 / 0.8)</code>	Needing transparency (alpha 0-1)
HSL(A)	<code>hsl(221 83% 53%)</code>	Designing: tweak hue/saturation/lightness intuitively
Named	<code>navy</code> , <code>red</code>	Never in real code (only 140, unpredictable)

CODE · Colors in practice

```
:root {  
  --brand: #2563eb;           /* hex: the brand anchor */  
  --brand-soft: rgb(37 99 235 / 0.1); /* 10% tint for backgrounds */  
  --brand-dark: hsl(221 83% 40%); /* darker shade by lowering lightness */  
}  
.btn { background: var(--brand); color: #fff; }  
.btn:hover { background: var(--brand-dark); }
```

PRO TIP — DESIGN WITH HSL

Need a hover shade? Keep hue/saturation, lower lightness by 10%. Need a palette? Keep saturation/lightness, rotate hue. HEX can't think like that — HSL is a designer's format.

The contrast law (non-negotiable)

Text must contrast with its background: **4.5:1** for normal text, **3:1** for large text (18px+). Grey-on-white (`#999` on `#fff` = 2.8:1) fails and excludes millions of users.

TRY IT YOURSELF — CHECK CONTRAST IN 30 SECONDS

DevTools → select text → Styles → click the color square → the contrast ratio + pass/fail appears. Fix failures by darkening text or the background until AA passes.

Backgrounds and gradients

CODE · Background mastery

```
.card {
  background-color: #f8fafc;
  background-image: url("../images/pattern.svg");
  background-repeat: no-repeat;
  background-position: right top;
  background-size: 200px;
}
/* Gradients are images too – stack them */
.hero {
  background:
    linear-gradient(rgb(11 16 38 / 0.7), rgb(11 16 38 / 0.7)),
    url("../images/hero.webp") center / cover no-repeat;
  color: #fff;
}
.gold-text {
  background: linear-gradient(90deg, #c9a227, #f6e27a);
  -webkit-background-clip: text; background-clip: text; color: transparent;
}
```

Table 42 — background-size values

Value	Effect	
cover	Fills box, crops overflow (heroes)	
contain	Whole image visible, may letterbox (logos)	
200px / 50%	Exact size	

Borders, radius, shadows

CODE · Edges and depth

```
.box { border: 2px solid #1e3a8a; border-radius: 12px; }
.pill { border-radius: 999px; } /* perfect capsule */
.avatar { border-radius: 50%; aspect-ratio: 1; } /* perfect circle */
.card-soft { border: 1px solid #e2e8f0; border-radius: 12px;
  box-shadow: 0 10px 30px rgb(2 6 23 / 0.08); } /* x y blur color */
.btn-press { box-shadow: 0 4px 0 #1e3a8a; } /* hard offset = 3D press */
.btn-press:active { transform: translateY(3px); box-shadow: 0 1px 0 #1e3a8a; }
```

Table 43 — box-shadow anatomy: `0 10px 30px rgb(2 6 23 / 0.08)`

Value	Meaning
0	Horizontal offset
10px	Vertical offset (down)
30px	Blur radius (softness)
color/alpha	Shadow color + transparency (keep alpha ≤ 0.15 for elegance)

KEY TAKEAWAY — KEY TAKEAWAY

Elegant UI = subtle shadows (low alpha, big blur), generous radius (8–16px cards, 999px pills), and ONE brand color with HSL-derived shades. Loud shadows and 6 random colors scream "beginner".

Quick check — Best hover-darkening method?

1. New random hex
2. Lower HSL lightness ~10%
3. Add black text
4. opacity: 0.5

Answer: B · Same hue/saturation with less lightness keeps brand harmony; opacity affects children too.

Exercise 14 — Style a pricing card

White card, 16px radius, soft shadow, brand-blue price text (3rem/800), grey features list, gold "Popular" pill badge on top. Then add hover: lift 4px + deeper shadow.

SOLUTION

```
.card{border-radius:16px;box-shadow:0 10px 30px rgb(2 6 23/.08);transition:.25s} .card:hover{transform:tr
```

— transitions/transform fully explained in Ch 20.

CHAPTER 16

Display and Positioning — Controlling the Flow

Intermediate

~ 100 min

How elements occupy space and how to place anything exactly where you want it.

IN THIS CHAPTER

- block vs inline vs inline-block
- display: none vs visibility vs opacity
- static/relative/absolute/fixed/sticky
- z-index and stacking
- Centering cookbook

Elzero CSS #49-#62

Gamal CSS #41-#55

Angela §CSS

display: how elements occupy space

Table 44 — display values

Value	Behavior	Width/height?	Example
<code>block</code>	Full-width row, stacks vertically	Yes	div, p, h1, section
<code>inline</code>	Flows inside text, no breaks	NO (ignored!)	span, a, strong
<code>inline-block</code>	Flows inline BUT accepts sizes	Yes	Buttons, badges, nav items
<code>none</code>	Removed from page AND from screen readers	—	Hidden panels, JS toggles
<code>flex</code> / <code>grid</code>	Layout engines (next chapters)	—	Modern layouts

CODE · display fixes juniors google daily

```
/* Center an image (images are inline by default!) */
img.center { display: block; margin: 0 auto; }
/* Nav items in a row */
.menu li { display: inline-block; }
/* Truly hide vs invisibly keep space vs fade */
.gone { display: none; } /* removed: layout collapses, readers skip */
.invisible { visibility: hidden; } /* gap stays, clicks dead, readers vary */
.faded { opacity: 0; } /* gap stays, STILL clickable! */
```

WARNING — OPACITY: 0 IS NOT HIDDEN

An `opacity: 0` button still receives clicks and Tab focus — users click "nothing". For real hiding use `display: none` (or `hidden` attribute); for fade animations combine opacity + `visibility` / `pointer-events`.

position: the five modes

Table 45 — position modes

Mode	Positioned relative to	Scrolls?	Use for
<code>static</code>	Normal flow (default)	Yes	95% of elements — leave it
<code>relative</code>	Its own normal spot	Yes	Nudging + becoming an anchor for absolute children
<code>absolute</code>	Nearest positioned ancestor	With ancestor	Badges, tooltips, overlays inside cards
<code>fixed</code>	The viewport (window)	NO (stays)	Sticky headers, chat buttons, cookie bars
<code>sticky</code>	Scrolls until threshold, then sticks	Hybrid	Section headers, table of contents

CODE · Positioning patterns

```
/* Badge pinned to a card corner */
.card { position: relative; }
.card .badge { position: absolute; top: 12px; right: 12px; }
/* Always-visible chat button */
.chat { position: fixed; bottom: 20px; right: 20px; z-index: 50; }
/* Section title that sticks while reading */
.toc { position: sticky; top: 0; background: #fff; }
```

KEY TAKEAWAY — KEY TAKEAWAY

Absolute positions against the nearest ancestor with `position` $\neq$ `static`. No positioned ancestor? It flies to the page root — the classic "my badge is in the wrong corner" bug. Fix: `position: relative` on the intended parent (no offsets needed).

z-index and stacking (why overlaps misbehave)

`z-index` only works on **positioned** elements (or flex/grid children). Higher number = closer to viewer. But **stacking contexts** trap children: a child with `z-index: 9999` can NEVER escape above an uncle if its parent created a lower context (via `position+z-index`, `opacity<1`, `transform`, `filter`...).

TRY IT YOURSELF — DEBUG OVERLAPS (2 MINUTES)

Overlapping wrong? In DevTools inspect both elements: check who is positioned, compare z-indexes, then walk UP parents looking for `opacity`, `transform` or `z-index` creating a trapping context. Fix by restructuring, not by adding more 9s.

CODE · Sane z-index scale (define once)

```
:root { --z-base: 1; --z-nav: 50; --z-modal: 100; --z-toast: 200; }
.site-nav { position: sticky; top: 0; z-index: var(--z-nav); }
```

Centering cookbook (stop googling these)

CODE · Every centering recipe

```
/* Inline content inside a block */
.center-text { text-align: center; }
/* A block with known width */
.box { width: 600px; margin: 0 auto; } /* max-width version: Ch 14 */
/* Absolute dead-center (unknown size) */
.overlay { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }
/* Flexbox one-liner (Ch 17): the modern answer to everything */
.stage { display: flex; justify-content: center; align-items: center; min-height: 60vh; }
```

Quick check — Badge with position:absolute lands at page corner. Cause?

1. Wrong z-index
2. No positioned ancestor — parent needs position:relative
3. Missing display
4. Browser bug

Answer: B · Absolute anchors to the nearest positioned ancestor; static parents don't count.

Exercise 15 — Build a product card

Card with image, discount badge top-right (-20%), title, price, "Add" button. Badge must stay glued to the card on all screen sizes.

SOLUTION

```
.card{position:relative} .badge{position:absolute;top:12px;right:12px} + image
display:block;width:100%. Test: resize the window — the badge rides with the card because
its anchor IS the card.
```

CHAPTER 17

Flexbox — One-Dimensional Layout Mastery

Intermediate

~ 120 min

Rows and columns that align, space and wrap beautifully: navbars, cards, forms, centering.

IN THIS CHAPTER

- Container vs items, main vs cross axis
- justify-content / align-items / gap
- flex-grow/shrink/basis
- Wrapping and alignment per item
- 5 classic Flexbox patterns

Elzero CSS Flexbox

Gamal Flexbox

Angela \$Flex

Mental model: container, items, two axes

```
main axis (justify-content) ----->
+-----+
| [Item 1]   [Item 2]   [Item 3]   |
+-----+
cross axis (align-items) runs vertically (top to bottom)
flex-direction: column swaps them: main = vertical, cross = horizontal
```

Figure 9 — Flexbox axes (flex-direction: row)

The parent (container) gets `display: flex` and controls **alignment**; children (items) control **their own growth**.

CODE · Your first flexbox (perfect centering + navbar)

```
.center-stage { display: flex; justify-content: center; align-items: center; min-height: 50vh; }
.navbar { display: flex; justify-content: space-between; align-items: center; gap: 16px; }
```

Container properties (set on parent)

Table 46 — Flex container cheat-sheet

Property	Values	Job
<code>display: flex</code>	—	Activates flexbox, direct children
<code>flex-direction</code>	<code>row</code> (default) / <code>column</code> / <code>row-reverse</code>	Main-axis direction
<code>justify-content</code>	<code>flex-start</code> , <code>center</code> , <code>flex-end</code> , <code>space-between</code> , <code>space-around</code> , <code>space-evenly</code> , <code>gap</code>	Spacing along MAIN axis
<code>align-items</code>	<code>stretch</code> , <code>flex-start</code> , <code>center</code> , <code>flex-end</code> , <code>baseline</code>	Alignment along CROSS axis
<code>flex-wrap</code>	<code>nowrap</code> / <code>wrap</code>	Allow multiple lines (responsive rows!)
<code>gap</code>	<code>16px</code> (or <code>row-gap</code> <code>column-gap</code>)	Gutters WITHOUT margins/hacks

WARNING — SPACE-BETWEEN VS SPACE-EVENLY

`space-between` : first/last glued to edges, gaps only inside. `space-around` : half-gaps at edges. `space-evenly` : all gaps (including edges) perfectly equal. Pick deliberately — screenshots reveal the wrong one instantly.

Item properties (set on children)

Table 47 — Flex item cheat-sheet

Property	Example	Job
<code>flex-grow</code>	<code>1</code>	Share of EXTRA space (0 = none)
<code>flex-shrink</code>	<code>0</code>	Refuse to shrink (1 = shrink normally)
<code>flex-basis</code>	<code>250px</code>	Starting size before growing/shrinking
<code>flex</code>	<code>1 1 250px</code>	Shorthand: grow shrink basis (<code>flex: 1</code> = grow fully)
<code>align-self</code>	<code>center</code>	Override cross-axis alignment for THIS item only
<code>order</code>	<code>-1</code>	Visual order (DOM order unchanged — careful with screen readers!)

CODE · The holy-grail sidebar layout in 6 lines

```
.layout { display: flex; gap: 24px; }
.sidebar { flex: 0 0 260px; } /* fixed 260px, never grows/shrinks */
.content { flex: 1; } /* eats all remaining space */
```

THINK OF IT LIKE THIS — FLEX-GROW IS SHARING PIZZA

Three items with `flex-grow: 1, 2, 1` split leftover space like 4 pizza slices: 1, 2, 1. `flex-shrink: 0` is saying "never take MY slice" when space runs out. `flex-basis` is the plate size you start with.

Five classic patterns (memorize these)

CODE · Pattern 1 — Navbar: logo left, links right

```
.nav { display: flex; justify-content: space-between; align-items: center; }
```

CODE · Pattern 2 — Card row that wraps on mobile

```
.cards { display: flex; flex-wrap: wrap; gap: 20px; }  
.cards .card { flex: 1 1 260px; } /* min 260px, grow, wrap as needed */
```

CODE · Pattern 3 — Media object: thumbnail + text

```
.media { display: flex; gap: 16px; align-items: flex-start; }  
.media img { flex: 0 0 120px; }
```

CODE · Pattern 4 — Form row: input grows, button fixed

```
.form-row { display: flex; gap: 8px; }  
.form-row input { flex: 1; }
```

CODE · Pattern 5 — Footer: columns that stack on mobile

```
.footer { display: flex; flex-wrap: wrap; gap: 32px; }  
.footer > * { flex: 1 1 200px; }
```

INTERVIEW QUESTION — FLEXBOX VS GRID — WHEN WHICH?

Expected: "Flexbox for one-dimensional layouts — a row OR a column: navs, card rows, form lines, centering. Grid for two-dimensional — rows AND columns together: page scaffolding, galleries with alignment in both directions. They compose: Grid page + Flexbox components inside."

Quick check — `justify-content` aligns along...

1. Always horizontal
2. The MAIN axis (row→horizontal, column→vertical)
3. The cross axis
4. Diagonal

Answer: B · Justify = main axis; the axis direction depends on flex-direction. This answers half of all Flexbox confusion.

Exercise 16 — Pricing table with Flexbox

Three pricing cards in a row (wrap on mobile), middle card highlighted and slightly taller, features list inside each, buttons bottom-aligned across cards.

SOLUTION

```
.plans{display:flex;flex-wrap:wrap;gap:24px;align-items:stretch} .plan{flex:1 1 260px;display:flex;flex-direction:column;align-items:center;flex-grow:1} .plan .list{flex-grow:1} .plan .button{margin-top:auto}
```

— inner column flex + `flex:1` list pushes buttons to equal bottoms.

CHAPTER 18

CSS Grid — Two-Dimensional Layout Mastery

Intermediate

~ 120 min

Rows AND columns together: page scaffolding, galleries, dashboards, and the legendary one-line responsive grid.

IN THIS CHAPTER

- Tracks, lines, cells, areas
- repeat/minmax/auto-fit magic
- Placement: line numbers, span, areas
- Alignment in both axes
- Real page scaffolding

Elzero CSS Grid

Gamal Grid

Angela \$Grid

Core vocabulary (5 words)

Table 48 — Grid vocabulary

Word	Meaning
Track	One row or one column
Line	The boundary between tracks (numbered: 1, 2, 3...)
Cell	Intersection of one row + one column
Area	Named rectangle of cells ("header header")
fr	Fraction of FREE space (1fr 2fr = 1/3 + 2/3)

CODE · Your first grid (3 equal columns, responsive gap)

```
.gallery { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 20px; }
```

The one-line responsive grid (no media queries!)

CODE · The most famous line in modern CSS

```
.auto-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));  
  gap: 20px;  
}
```

THINK OF IT LIKE THIS — REPEAT(AUTO-FIT, MINMAX(260PX, 1FR)) IS SMART SHELVING

"Fit as many 260px-wide boxes as the shelf allows; stretch them to share leftover space; drop whole columns when the shelf shrinks." Desktop shows 4 across, tablet 2, phone 1 — zero breakpoints written.

Table 49 — repeat() decoded

Piece	Job
<code>repeat(n, ...)</code>	Repeat a track pattern n times
<code>auto-fit</code>	Fit as many tracks as possible (collapse empty ones)
<code>minmax(260px, 1fr)</code>	Each track: minimum 260px, maximum equal share
<code>1fr</code>	One fraction of leftover space

Placement: lines, span, areas

CODE · Method 1 — line numbers (dashboards)

```
.dash { display: grid; grid-template-columns: 200px 1fr 1fr; grid-template-rows: auto 1fr; gap: 16px; }  
.widget-big { grid-column: 2 / 4; grid-row: 1 / 3; } /* from line 2 to 4, rows 1-2 */  
/* span shortcut: "take 2 columns from wherever I am" */  
.widget-wide { grid-column: span 2; }
```

CODE · Method 2 — named areas (page scaffolding, most readable)

```
.page {  
  display: grid; min-height: 100vh;  
  grid-template-columns: 240px 1fr;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "side main"  
    "footer footer";  
}  
.site-header { grid-area: header; } .site-side { grid-area: side; }  
.site-main { grid-area: main; } .site-footer { grid-area: footer; }  
@media (max-width: 720px) {  
  .page { grid-template-columns: 1fr;  
    grid-template-areas: "header" "main" "side" "footer"; }  
}
```

Alignment: two axes at once

Table 50 — Grid alignment (justify = row axis, align = column axis)

Scope	Row axis	Column axis
Whole tracks in container	<code>justify-content</code>	<code>align-content</code>
Default for all items	<code>justify-items</code>	<code>align-items</code>
One item override	<code>justify-self</code>	<code>align-self</code>

CODE · Center anything in a grid cell (or the whole grid)

```
.cell-center { display: grid; place-items: center; } /* align + justify shorthand */
```

KEY TAKEAWAY — KEY TAKEAWAY

Layout decision tree: **page scaffolding or 2D gallery** → **Grid** (areas/lines). **Row/column of components** → **Flexbox**. **Unknown-size centering** → **either** (`place-items: center` vs flex centering — pick the one already in use nearby).

Quick check — `grid-column: 2 / 4` means...

1. Column 2 only
2. From line 2 to line 4 (spans 2 tracks)
3. 2 divided by 4
4. Rows 2-4

Answer: B · Grid placement uses LINE numbers: start line 2, end line 4 = occupies tracks 2 and 3.

Exercise 17 — Dashboard layout

Header spanning full width, sidebar 240px, 4 stat cards in responsive auto-fit grid, footer full width. Stack to one column under 720px.

SOLUTION

Page scaffolding via areas (header/side/main/footer as above); stats inside main with `.stats{display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));gap:16px}`; media query collapsing areas to one column. Grid-inside-Grid: each level does one job.

CHAPTER 19

Responsive Design — Mobile First

Intermediate

~ 100 min

One codebase, every screen: fluid units, media queries, responsive images and testing.

IN THIS CHAPTER

- Mobile-first philosophy
- Media query breakpoints that make sense
- Fluid type with clamp()
- Responsive images + videos
- Testing checklist

Gamal responsive

Elzero media queries

Angela §Responsive

Mobile-first philosophy

Write CSS for the **phone first**, then ADD rules as screens grow (`min-width`). Desktops get enhancements; phones get the fast core. The opposite (desktop-first with `max-width`) ships desktop weight to phones — backwards.

CODE · Mobile-first in action

```
/* Base = phone (everyone gets this) */
.cards { display: grid; gap: 16px; }
/* ≥640px: two columns */
@media (min-width: 640px) { .cards { grid-template-columns: 1fr 1fr; } }
/* ≥1024px: sidebar layout appears */
@media (min-width: 1024px) {
  .layout { display: grid; grid-template-columns: 260px 1fr; }
}
```

Table 51 — Breakpoints that make sense (content-based, not device-based)

Breakpoint	Name	Typical change
<code>≥640px</code>	Small tablet	2-column grids, bigger type
<code>≥768px</code>	Tablet	Nav expands, side-by-side sections
<code>≥1024px</code>	Laptop	Sidebars, multi-column dashboards
<code>≥1280px</code>	Desktop	Max-width container caps growth (Ch 14)

WARNING — NEVER TARGET SPECIFIC DEVICES

`@media (max-width: 375px)` for "iPhone X" breaks on the next phone. Instead: shrink your browser until the layout looks bad — THAT pixel value is your breakpoint. Design responds to content, not to Apple.

Fluid typography with clamp()

CODE · Text that scales smoothly — no breakpoints needed

```
h1 { font-size: clamp(1.75rem, 1rem + 4vw, 3.5rem); }
/*      minimum   preferred (fluid)   maximum   */
/* Phone: 28px → grows with screen → caps at 56px */
body { font-size: clamp(1rem, 0.9rem + 0.5vw, 1.125rem); }
```

Table 52 — clamp() decoded: `clamp(MIN, FLUID, MAX)`

Part	Job
MIN	Never smaller (readability floor)
FLUID (<code>vw</code> -based)	Grows with the viewport
MAX	Never bigger (design ceiling)

Responsive media (never overflow again)

CODE · Media that behaves on every screen

```
img, video { max-width: 100%; height: auto; display: block; }  
.hero-video { width: 100%; aspect-ratio: 16 / 9; }  
/* Tables on phones: scroll inside, page never breaks */  
.table-scroll { overflow-x: auto; }
```

TRY IT YOURSELF — THE 375PX TEST (5 MINUTES)

DevTools → Toggle device toolbar (Ctrl+Shift+M) → 375×667. Scroll your whole page: no horizontal scrollbar? Tap targets ≥44px? Text readable without zoom? Tables scroll inside their box? Fix failures with fluid widths, wrapping flex, and smaller clamp() minimums.

RESPONSIVE SHIP CHECKLIST

- - viewport meta present; no horizontal scroll at 360px
- - Touch targets ≥ 44×44px; no hover-only features (phones have no hover!)
- - Type fluid via clamp(); line length ≤ 70ch
- - Images WebP/AVIF + lazy below fold; hero not lazy
- - Tables wrapped in scroll containers; test 360px + 768px + 1440px

Quick check — Mobile-first means...

1. Design phones, then enhance with min-width
2. Design desktop, then max-width fixes
3. Two separate sites
4. Phones only

Answer: A · Base styles = phone; min-width queries add complexity upward. One codebase, layered.

Exercise 18 — Make it responsive

Take your Ch 9 profile page (unstyled): add container, clamp() headings, auto-fit gallery, stacked→side nav at 768px, scroll-safe table. Test at 360/768/1440.

SOLUTION

```
.container{width:100%;max-width:1000px;margin:auto;padding:0 16px} ,  
h1{font-size:clamp(2rem,1.2rem+4vw,3.5rem)} , .gallery{display:grid;grid-template-columns:repeat(auto-fit,  
nav column→row at 768px. Verify zero horizontal scroll everywhere.
```

CHAPTER 20

Transitions, Transforms and Animations

Intermediate

~ 90 min

Motion that feels expensive: hover effects, loaders, reveals — all on the GPU fast-path.

IN THIS CHAPTER

- transition (A→B)
- transform (move/scale/rotate for free)
- keyframes animations
- The 60fps rule (transform+opacity)
- prefers-reduced-motion

Elzero animations

Gamal effects

W3Schools CSS

transition: smooth A→B changes

CODE · Every hover effect is this pattern

```
.btn { background: navy; transition: background 0.25s ease, transform 0.25s ease; }
.btn:hover { background: royalblue; transform: translateY(-2px); }
.card { transition: box-shadow 0.3s ease; }
.card:hover { box-shadow: 0 18px 40px rgb(2 6 23 / 0.14); }
```

Table 53 — transition anatomy: `background 0.25s ease 0s`

Part	Meaning	Pro choice
property	What animates (or <code>all</code> — lazy, avoid)	List exact properties
duration	How long	150–300ms UI; up to 600ms big reveals
timing	Speed curve	<code>ease</code> default; <code>ease-out</code> for entrances

Part	Meaning	Pro choice
delay	Wait before starting	Stagger effects (0.05s steps)

transform: the free superpower

`transform` moves/scales/rotates WITHOUT triggering layout — the GPU handles it at 60fps.

CODE · Transform cookbook

```
.lift:hover { transform: translateY(-4px); }      /* hover lift */
.zoom:hover img { transform: scale(1.05); }      /* image zoom (parent overflow:hidden)
*/
.spin:hover { transform: rotate(180deg); }        /* icon flip */
.center-abs { transform: translate(-50%, -50%); } /* dead-center trick */
.flip { transform: scaleX(-1); }                  /* mirror (RTL icons!) */
.tilt { transform: perspective(600px) rotateX(8deg); } /* 3D card tilt */
```

keyframes: full choreography

CODE · A professional loader + entrance reveal

```
.loader { width: 40px; height: 40px; border-radius: 50%;
border: 4px solid #e2e8f0; border-top-color: navy;
animation: spin 0.8s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }

.reveal { animation: rise 0.6s ease-out both; }
@keyframes rise {
  from { opacity: 0; transform: translateY(24px); }
  to   { opacity: 1; transform: translateY(0); }
}
.stagger > *:nth-child(2) { animation-delay: 0.1s; }
.stagger > *:nth-child(3) { animation-delay: 0.2s; }
```

Table 54 — animation shorthand: `rise 0.6s ease-out 0s 1 both`

Part	Meaning
name	Which @keyframes

Part	Meaning
duration/timing/delay	Same as transition
iteration	1, 3, infinite (loaders)
fill-mode	both = keep first frame before + last frame after (prevents flashes)

The 60fps rule + reduced motion

KEY TAKEAWAY — KEY TAKEAWAY

Animate ONLY `transform` and `opacity`. Animating `width/margin/top/left` forces layout+paint every frame = janky 15fps on phones. When in doubt:
`transform: translate()` instead of `top/left`, `scale()` instead of `width/height`.

CODE · Respect users who disable motion (vestibular disorders)

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; }
}
```

Quick check — Smoothest hover lift?

1. margin-top:-4px
2. top:-4px
3. transform:translateY(-4px)
4. padding change

Answer: C · transform runs on the GPU compositor; the rest trigger layout/paint every frame.

Exercise 19 — Animated card gallery

Cards fade-rise on load (staggered), images zoom on hover, button lifts. All 60fps-safe + reduced-motion fallback.

SOLUTION

```
.card{animation:rise .5s ease-out both} img{transition:transform .4s} .card:hover img{transform:scale(1.0
```

+ stagger delays + the reduced-motion block above. Only transform/opacity animated — DevTools Performance shows steady 60fps.

CHAPTER 21

Variables, Architecture and the CSS Project

Intermediate+

~ 120 min

Write CSS that survives real projects: custom properties, theming, BEM naming — then build a complete landing page.

IN THIS CHAPTER

- Custom properties (variables) + theming
- BEM naming that scales
- Organizing stylesheets
- Project: responsive landing page
- CSS review checklist

Project

Elzero advanced

Gamal project

Custom properties: design tokens

CODE · One source of truth for the whole design

```
:root {
  --brand: #2563eb; --brand-dark: #1e40af; --gold: #c9a227;
  --ink: #1c2333; --muted: #64748b; --bg: #ffffff; --card: #f8fafc;
  --radius: 12px; --space: 1rem; --shadow: 0 10px 30px rgb(2 6 23 / 0.08);
  --font: "Segoe UI", Arial, sans-serif;
}
body { font-family: var(--font); color: var(--ink); background: var(--bg); }
.btn { background: var(--brand); border-radius: var(--radius); }
/* Dark theme = reassign variables, zero rewritten rules */
[data-theme="dark"] {
  --bg: #0b1026; --card: #131a3a; --ink: #e2e8f0; --muted: #94a3b8;
}
```


PRO TIP — VAR() SUPERPOWERS

Fallbacks: `color: var(--brand, navy)`. Scoping: redefine inside any selector (`.card { --brand: green; }` themes one card). Runtime: JavaScript can change them live (`el.style.setProperty('--brand', 'red')`) — the backbone of theme toggles.

BEM naming (stop class chaos)

Table 55 — BEM: Block__Element--Modifier

Part	Meaning	Example
Block	Standalone component	<code>.card</code> , <code>.nav</code>
Element (<code>__</code>)	Part inside block	<code>.card__title</code> , <code>.nav__link</code>
Modifier (<code>--</code>)	Variant/state	<code>.card--featured</code> , <code>.btn--large</code>

CODE · BEM in action

```
.card { ... } .card__img { ... } .card__title { ... } .card__price { ... }  
.card--featured { border: 2px solid var(--gold); }  
.btn--large { padding: 1rem 2rem; font-size: 1.1rem; }
```

KEY TAKEAWAY — KEY TAKEAWAY

BEM kills specificity wars: every selector is ONE flat class (0-1-0), so order decides ties and nothing fights. Readable in HTML, greppable in code, scalable to 10,000 lines.

Project: responsive landing page (90 min)

NAV (sticky): logo + links + CTA button
HERO: headline + sub + 2 buttons + hero image (grid 2col → 1col)
FEATURES: 3 cards (auto-fit grid) with icons
PRICING: 3 plans, middle featured (flex row → wrap)
TESTIMONIALS: quotes slider-look (grid) — static, no JS yet
CTA + FOOTER: big call-to-action + link columns

Figure 10 — Landing page blueprint

CODE · Landing core (theme + layout + components)

```

:root { --brand: #2563eb; --ink: #16225c; --bg: #fff; --radius: 14px; }
.container { width: 100%; max-width: 1120px; margin: 0 auto; padding: 0 20px; }
.site-nav { position: sticky; top: 0; z-index: 50; background: rgb(255 255 255 / 0.9);
backdrop-filter: blur(8px); }
.site-nav .container { display: flex; justify-content: space-between; align-items:
center; height: 64px; }
.hero .container { display: grid; grid-template-columns: 1.1fr 1fr; gap: 40px; align-
items: center; padding: 64px 20px; }
.hero h1 { font-size: clamp(2.2rem, 1.4rem + 4vw, 3.8rem); line-height: 1.05; }
.features { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
gap: 24px; }
.plans { display: flex; flex-wrap: wrap; gap: 24px; }
.plan { flex: 1 1 260px; border: 1px solid #e2e8f0; border-radius: var(--radius);
padding: 32px; }
.plan--featured { border: 2px solid var(--brand); transform: scale(1.03); }
@media (max-width: 860px) { .hero .container { grid-template-columns: 1fr; } }

```

CSS PROJECT REVIEW

- - border-box reset; rem type; unitless line-height; container pattern everywhere
- - One BEM class per component; zero !important; zero IDs in CSS
- - Flexbox for rows, Grid for 2D, clamp() for headings, auto-fit for galleries
- - 60fps motion only; reduced-motion block; contrast AA; 360px/768px/1440px tested

Exercise 20 — Theme toggle (CSS only, JS in Part 4)

Add `data-theme="dark"` styles for your landing page: dark bg, light ink, softer shadows. Toggle by editing the `<html>` tag manually for now — Chapter 29 wires it to a button with JavaScript.

SOLUTION

Define ALL colors as variables in `:root`, then override them under `[data-theme="dark"]`. If any color was hardcoded, theming breaks there — this exercise finds every hardcoded color in your code.

CHAPTER 22

CSS Datasheet — Every Important Property in One Place

Reference

~ 60 min

The working reference: find any property in seconds with syntax, values and a correct example.

IN THIS CHAPTER

- Selector & cascade quick table
- Box & sizing
- Typography
- Layout (flex/grid/position)
- Color, background, effects

Datasheet

W3Schools ref

MDN ref

Table 56 — Specificity & cascade (memorize)

Selector	Score	Notes
<code>p</code>	0-0-1	Elements weakest
<code>.card</code>	0-1-0	Classes: your daily tool
<code>#nav</code>	1-0-0	Beats any number of classes
<code>style=""</code>	Top	Inline (avoid)
<code>!important</code>	Nuclear	Utilities/third-party only

box-sizing BOX

border-box makes width include padding+border (sane). Apply universally.

```
*, *::before, *::after { box-sizing: border-box; }  
key: First rule of every professional stylesheet.
```

margin / padding BOX

Outside gap / inside space. Clock shorthand: top right bottom left.

```
.card { margin: 0 auto 24px; padding: 20px 24px; }  
key: Vertical margins collapse; use gap in flex/grid instead.
```

border / border-radius BOX

Edge line (width style color) and corner rounding.

```
.btn { border: 2px solid navy; border-radius: 999px; }  
key: 999px = capsule; 50% + square = circle.
```

width / max-width SIZING

Fixed width vs fluid-width-ceiling. The container pattern uses both.

```
.container { width: 100%; max-width: 1100px; margin: 0 auto; }  
key: Fixed widths break phones; max-width never does.
```

height / min-height SIZING

Rigid height (overflows text!) vs elastic minimum that grows.

```
.hero { min-height: 80vh; } .card { min-height: 200px; }  
key: Never fix height on text containers.
```

font-size / font-weight TYPOGRAPHY

Letter scale (rem!) vs stroke thickness (100-900).

```
body { font-size: 1rem; font-weight: 400; }  
h1 { font-size: 3rem; font-weight: 800; }  
key: Size = zoom; weight = pen thickness. Different universes.
```

line-height TYPOGRAPHY

Line spacing. Unitless multiplies each element's own font-size.

```
body { line-height: 1.6; } h1 { line-height: 1.1; }  
key: Unitless always; px line-height squashes headings.
```

color / opacity COLOR

Text color; opacity fades whole element (children included).

```
.muted { color: #64748b; } .ghost { opacity: 0.5; }  
key: Prefer rgb alpha for backgrounds (opacity hits children).
```

background BACKGROUND

Shorthand: color image position/size repeat. Gradients stack over images.

```
.hero { background: linear-gradient(rgb(0 0 0 / .5), rgb(0 0 0 / .5)), url(h.jpg)
center/cover; }
key: background is really background-image + color + friends.
```

display LAYOUT

block (row) / inline (text flow) / inline-block / none / flex / grid.

```
img.c { display: block; margin: auto; } .x { display: none; }
key: display:none removes from layout AND screen readers.
```

position LAYOUT

static / relative (anchor) / absolute (to anchor) / fixed (viewport) / sticky (hybrid).

```
.card { position: relative; } .badge { position: absolute; top: 8px; right: 8px; }
key: Absolute needs a positioned ancestor or it flies to the page.
```

flex container FLEXBOX

display:flex + direction + main-axis (justify) + cross-axis (align) + gap + wrap.

```
.row { display: flex; justify-content: space-between; align-items: center; gap: 16px;
flex-wrap: wrap; }
key: justify = main axis; align = cross axis.
```

flex item FLEXBOX

flex: grow shrink basis. flex:1 eats space; 0 0 260px stays fixed.

```
.side { flex: 0 0 260px; } .main { flex: 1; }  
key: Basis = starting size; grow = share of leftovers.
```

grid container GRID

display:grid + column/row tracks + gap. auto-fit+minmax = responsive with zero queries.

```
.g { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap:  
20px; }  
key: fr = fraction of free space.
```

grid placement GRID

Place by line numbers (2/4) or named areas. span N takes N tracks.

```
.big { grid-column: 1 / 3; } .tall { grid-row: span 2; }  
key: Lines are boundaries BETWEEN tracks, starting at 1.
```

media queries RESPONSIVE

min-width layers enhancements upward (mobile-first).

```
@media (min-width: 768px) { .layout { grid-template-columns: 240px 1fr; } }  
key: Breakpoints follow content pain, not device names.
```

clamp() **RESPONSIVE**

Fluid value with floor and ceiling. Perfect for type.

```
h1 { font-size: clamp(1.75rem, 1rem + 4vw, 3.5rem); }  
key: MIN, preferred-fluid, MAX — never unbounded vw.
```

transition **MOTION**

Smooth A→B: property duration timing delay.

```
.btn { transition: background .25s ease, transform .25s ease; }  
key: 150–300ms; list properties (never bare "all" in hot UI).
```

transform **MOTION**

GPU move/scale/rotate without layout cost.

```
.card:hover { transform: translateY(-4px) scale(1.01); }  
key: The only layout-safe motion (with opacity).
```

animation **MOTION**

keyframes choreography: name duration timing delay count fill.

```
.pop { animation: rise .5s ease-out both; }  
@keyframes rise { from { opacity: 0; transform: translateY(20px); } }  
key: fill "both" prevents pre/post flashes.
```


CSS variables **ARCHITECTURE**

Custom properties: single source of truth + runtime theming.

```
:root { --brand: #2563eb; } .btn { background: var(--brand, navy); }  
key: var() accepts a fallback second argument.
```

PART

JavaScript — Bringing Pages to Life

Programming from zero: logic, data, interactivity and servers' language.

- Ch 23** JavaScript First Steps — Values and Your Console
- Ch 24** Variables and Memory — let, const, var and the Truth About Copying
- Ch 25** Operators, Conditions and Loops
- Ch 26** Functions, Scope and Closures — The Heart of JavaScript
- Ch 27** Arrays — Ordered Data and the Methods That Rule Them
- Ch 28** Objects, ES6 Power and Classes
- Ch 29** The DOM — JavaScript Meets the Page
- Ch 30** Events — Clicks, Keys and the Delegation Superpower
- Ch 31** Forms, Validation and Browser Storage
- Ch 32** Async JavaScript — The Event Loop, Promises and async/await
- Ch 33** Fetch and Real APIs — Talking to the World
- Ch 34** Errors, Debugging and Regular Expressions
- Ch 35** JavaScript Projects — Quiz App and Expense Tracker
- Ch 36** JavaScript Datasheet — Methods and Syntax in One Place

| JavaScript rewards patience: master the core here and frameworks become dialects.

CHAPTER 23

JavaScript First Steps — Values and Your Console

Beginner

~ 90 min

Run your first code in 60 seconds; understand values, types and the console.

IN THIS CHAPTER

- Running JS (console + script tags)
- Numbers, strings, booleans
- Comments and semicolons
- `console.log` debugging
- Your first interactive page

Elzero JS #1-#15

Gamal JS #1-#8

Angela JS

Run code in 60 seconds (two ways)

CODE · Way 1 — the console (experiments): F12 → Console, type and Enter

```
2 + 3 * 4
"Hello" + " " + "web!"
```

CODE · Way 2 — a real file (save as talk.html, double-click)

```
<!DOCTYPE html><html><body>
<h1 id="msg">...</h1>
<script>
  document.getElementById("msg").textContent = "JavaScript is running!";
  console.log("Page loaded at", new Date().toLocaleTimeString());
</script>
</body></html>
```

Table 57 — script placement

Way	When
End of <code><body></code>	Default for learning (page shows, then code runs)
<code><head></code> + <code>defer</code>	Professional multi-file sites (parallel download, ordered run)
Console only	Quick experiments and debugging

Values and types (only 8 exist)

Table 58 — The 8 JavaScript types

Type	Example	Used for
number	<code>42</code> , <code>3.14</code> , <code>-7</code>	All math (integers + decimals, one type!)
string	<code>"Sara"</code> , <code>'Cairo'</code>	Text (single or double quotes)
boolean	<code>true</code> , <code>false</code>	Yes/no decisions
undefined	<code>let x;</code>	Declared but empty (accidental emptiness)
null	<code>let user = null</code>	Deliberately empty (chosen emptiness)
object	<code>{name: "Sara"}</code>	Grouped data (Ch 28)
array	<code>[10, 20]</code>	Ordered lists (Ch 27) — technically an object
function	<code>() => {}</code>	Reusable action (Ch 26)

CODE · Types in the console — try each line

```

typeof 42           // "number"
typeof "Cairo"      // "string"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof null         // "object" (a famous 1995 bug — just memorize it!)
typeof [1, 2]       // "object" (arrays are objects; use Array.isArray)

```

THINK OF IT LIKE THIS — TYPES ARE LABELED JARS

A variable is a jar; its type is the label. JavaScript jars are magic: the same jar can hold a number now and a string later (dynamic typing). Powerful — but you must always know WHAT is inside before operating (hence `typeof` and careful checks).

Strings: quotes, escape, template literals

CODE · String essentials

```
const single = 'It\'s Cairo';           // escape with backslash
const double = "Say \"hi\"!";          // or mix quote styles
const name = "Sara", city = "Cairo";
const msg = `Hello ${name} from ${city}!`; // template literal: ${} injects values
const multi = `Line one
Line two`;                             // backticks span lines
```

KEY TAKEAWAY — KEY TAKEAWAY

Default to **template literals** (backticks) for any string with variables: ``Hello ${name}`` beats `"Hello " + name` in readability and safety. Use quotes only for short fixed text.

console: your X-ray (5 methods)

CODE · Console beyond log

```
console.log("value:", 42);              // general output
console.table([a:1], {a:2});             // BEAUTIFUL tables for arrays/objects
console.warn("Password is weak");        // yellow warning
console.error("Payment failed");         // red error + stack trace
console.time("loop"); /* ... */ console.timeEnd("loop"); // timing blocks
```

TRY IT YOURSELF — BREAK THINGS IN THE CONSOLE (5 MINUTES)

Type `2 + "2"` (what?!), `"5" - 3`, `true + 1`, `alert("hi")`, `prompt("name?")`. Surprises here teach Chapter 25's coercion lesson early: JS converts types automatically, sometimes weirdly. Observe, don't memorize yet.

Quick check — `typeof null` **is...**

1. null
2. undefined
3. object
4. number

Answer: C · A legendary 1995 bug frozen forever for compatibility. Test null with `=== null`, never `typeof`.

Exercise 21 — Console calculator

In the console: compute a 15% tip on 240, build the string "Total: 276" with a template literal, and print both with ONE `console.log`. Then check each value's `typeof`.

SOLUTION

```
const bill = 240, tip = bill * 0.15, total = bill + tip; console.log( Tip: ${tip}, Total: ${total} );
```

→ "Tip: 36, Total: 276". `typeof total` = "number". You just did variables + math + strings in 2 lines.

CHAPTER 24

Variables and Memory — let, const, var and the Truth About Copying

Beginner+

~ 100 min

Declaring variables correctly, understanding where values live, and why objects "copy weird".

IN THIS CHAPTER

- let vs const vs var (and why var is retired)
- Naming rules
- Primitives vs references (stack vs heap)
- Copying without surprises
- const objects can still change inside!

Elzero JS #16-#30

Gamal JS #9-#15

MDN JS

let vs const vs var

Table 59 — The only variable table you need

Keyword	Scope	Reassign?	Redeclare?	Verdict
const	Block {}	NO	NO	DEFAULT for everything
let	Block {}	Yes	NO	Only when reassignment is required
var	Function (leaks!)	Yes	Yes (silently!)	RETIRED — never write in new code

CODE · const-first thinking

```
const PI = 3.14159;
const user = { name: "Sara" }; // const = the BINDING never changes...
user.name = "Omar";           // ...but the object's insides CAN change ✓
let score = 0;
score = 95;                    // let = reassignment expected ✓
// var x = 1;                  // ✗ retired: function scope + hoisting weirdness
```

KEY TAKEAWAY — KEY TAKEAWAY

const by default, **let** when you must reassign, **var** never. This single habit prevents an entire class of bugs (accidental overwrites) and signals intent to every reader: "this name always means this thing."

Naming rules (machines + humans)

CODE · Names that professionals respect

```
const userName = "sara"; // camelCase for variables/functions ✓
const MAX_RETRIES = 3;   // SCREAMING_SNAKE for true constants ✓
let isLoggedIn = false;  // booleans ask questions: is/has/can ✓
const userCount = 5;     // nouns + units, never x1, tmp, data2 ✓
// const 2fast = 1;       // ✗ cannot start with digit
// const user-name = 1;   // ✗ dash = minus! use camelCase
```

Primitives vs references (the big mental model)

PRIMITIVES copy the VALUE (independent twins)

```
let a = 5; let b = a; b = 99; --> a is STILL 5 ✓
```

REFERENCES copy the POINTER (two names, ONE object)

```
const o1 = {n: 5}; const o2 = o1; o2.n = 99; --> o1.n is 99!
```

Figure 11 — Stack (primitives, copied by VALUE) vs Heap (objects, shared by REFERENCE)

CODE · See it happen

```
let a = 5, b = a; b = 99;
console.log(a);           // 5 — independent ✓

const o1 = { n: 5 }, o2 = o1;
o2.n = 99;
console.log(o1.n);        // 99 — SHARED! o2 is a shortcut, not a copy

const arr1 = [1, 2], arr2 = arr1;
arr2.push(3);
console.log(arr1);        // [1, 2, 3] — shared again!
```

THINK OF IT LIKE THIS — PRIMITIVES ARE PHOTOCOPIES, OBJECTS ARE SHARED DOCS

Copying a number hands you an independent photocopy. Copying an object sends you a Google-Docs LINK — edits appear for everyone holding the link. Real copying needs explicit tools (spread, structuredClone — Chapter 28).

The Temporal Dead Zone (TDZ) in 60 seconds

`let` / `const` exist in their block but are **unusable before their declaration line** — touching them throws `ReferenceError`. This forbidden window is the TDZ. (`var` instead silently gives `undefined` — one more reason it retired.)

CODE · TDZ vs var weirdness

```
console.log(x); // undefined (var hoists the declaration, not the value)
var x = 1;

console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 2;
```

Quick check — After `const a=[1]; const b=a; b.push(2)` , **a is...**

1. [1]
2. [1,2]
3. Error (const!)
4. undefined

Answer: B · **const locks the binding (b always = that array), not the contents; a and b share one array.**

Exercise 22 — Freeze the bug

A function receives `settings` object and accidentally changes the caller's copy. Explain why, and name TWO ways to protect the original (details in Ch 28).

SOLUTION

Objects pass by reference — the parameter points at the caller's object. Protect via (1) spread copy `{...settings}` / `[...arr]` for shallow copies, (2) `structuredClone(obj)` for deep copies. Rule: never mutate inputs you don't own.

CHAPTER 25

Operators, Conditions and Loops

Beginner+

~ 120 min

Decisions and repetition: the two superpowers that turn values into programs.

IN THIS CHAPTER

- Arithmetic + precedence
- === vs == (the coercion trap)
- if/else, ternary, switch
- Logical &&
- ?? ?.
- for, while, break/continue

Elzero JS #31-#55

Gamal JS #16-#28

Angela JS Logic

Arithmetic and precedence

CODE · Math that bites

```
2 + 3 * 4      // 14 - * before + (school rules apply)
(2 + 3) * 4     // 20 - parentheses win
10 % 3         // 1 - remainder (odd/even, cycling, pagination!)
2 ** 3         // 8 - power (2³)
let n = 5;
n++            // 6 (postfix: use THEN add) vs ++n (add THEN use)
"5" + 3       // "53" - + with a string CONCATENATES!
"5" - 3       // 2 - -, *, / coerce strings to numbers
```

WARNING — + IS TWO-FACED

`+` means ADD for numbers but GLUE for strings — and ONE string converts the whole chain: `1 + 2 + "3" = "33"`, but `"1" + 2 + 3 = "123"`. Mixed-type `+` is the #1 source of "my total is 11050 instead of 60" bugs. Convert first: `Number(x) + y`.

=== vs == (memorize one rule)

Table 60 — Loose == (coercion chaos) vs strict === (sanity)

Expression	==	===	Lesson
<code>0 == "0"</code>	true	false	== converts types silently
<code>0 == false</code>	true	false	Falsy cross-talk
<code>"" == 0</code>	true	false	Empty string = zero?!
<code>null == undefined</code>	true	false	The ONLY sane == case

CODE · The professional rule: === everywhere, always

```
if (role === "admin") { ... } // type AND value must match ✓
// if (role == "admin")      // ✗ coercion roulette – banned in this book
```

The 8 falsy values (everything else is truthy): `false, 0, -0, 0n, "", null, undefined, NaN`.
Memorize via: "false, zero, empty string, null, undefined, NaN".

Conditions: if, ternary, switch

CODE · Choosing cleanly

```
const age = 20;
if (age >= 18) { console.log("Adult"); }
else if (age >= 13) { console.log("Teen"); }
else { console.log("Child"); }

// Ternary: value-IF in one line (assigning or returning only!)
const fee = age >= 18 ? 100 : 50;

// switch: MANY exact matches on one value
const day = "sat";
switch (day) {
  case "sat": case "sun": console.log("Weekend"); break;
  case "fri": console.log("Almost!"); break;
  default: console.log("Workday");
}
```

PRO TIP — TERNARY DISCIPLINE

Ternaries are for VALUES (`const x = c ? a : b`), never for actions (`c ? doA() : doB()`) — use if/else). Nested ternaries (`a ? b : c ? d : e`) are unreadable — banned in professional code.

Logical operators: && || ?? ?.

CODE · The four guards

```
const name = input || "Guest";      // || falls back on ANY falsy (" 0 null...)
const qty = input ?? 1;             // ?? falls back ONLY on null/undefined ✓
user && user.logout();               // && runs right side only if left truthy
const city = user?.address?.city;    // ?. stops safely (undefined, no crash)

// Real-world combo: safe + default
const cityName = user?.address?.city ?? "Unknown";
```

Table 61 — || vs ?? (the interview favorite)

Input	<code>`input`</code>	<code>"G"</code>	<code>input ?? "G"</code>	Why it matters
"Sara"	Sara	Sara	—	
"" (cleared field)	G ✗	"" ✓	""	Empty string is VALID input!
0 (quantity)	G ✗	0 ✓	0	Zero is VALID data!
null/undefined	G	G	G	Both fall back

Loops: for, while, break, continue

CODE · Loops complete

```
for (let i = 1; i <= 5; i++) { console.log("Round", i); }

const cart = ["milk", "bread", "eggs"];
for (const item of cart) { console.log(item); } // of = VALUES (arrays ✓)

const user = { name: "Sara", age: 30 };
for (const key in user) { console.log(key, user[key]); } // in = KEYS (objects)

let tries = 0;
while (tries < 3) { tries++; } // unknown iterations (retry, read lines)

for (let n = 1; n <= 10; n++) {
  if (n % 2 === 0) continue; // skip evens
  if (n > 7) break; // stop entirely
  console.log(n); // 1 3 5 7
}
```

KEY TAKEAWAY — KEY TAKEAWAY

`for...of` = array VALUES. `for...in` = object KEYS (never for arrays — it yields string indexes + surprises). Classic `for` = counting/index control. `while` = "until something happens".

Quick check — `0 ?? "backup"` gives...

1. backup
2. 0
3. Error
4. undefined

Answer: B · `??` only replaces null/undefined; 0 is real data and survives. (

Exercise 23 — FizzBuzz + guards (the classic interview)

Print 1–30: "Fizz" for multiples of 3, "Buzz" for 5, "FizzBuzz" for both. Then write `displayName(user)` returning `user?.name ?? "Guest"`.

SOLUTION

Loop 1..30; check `i%15===0` FIRST (both), then 3, then 5: order matters because 15 passes all three tests. `const displayName = u => u?.name ?? "Guest";` — safe navigation + nullish default in one line.

CHAPTER 26

Functions, Scope and Closures — The Heart of JavaScript

Intermediate

~ 150 min

Reusable actions, who-sees-what rules, functions that remember, and the `this` keyword finally demystified.

IN THIS CHAPTER

- Declarations vs expressions vs arrows
- Parameters, defaults, rest
- Scope chain + closures with real uses
- Hoisting truth
- `this` in 5 rules

Elzero JS #56-#90

Gamal JS #29-#45

Angela JS Functions

Three ways to write functions

CODE · Declaration vs expression vs arrow

```
// 1. Declaration: hoisted, named, the classic
function add(a, b) { return a + b; }

// 2. Expression: stored in a variable, NOT hoisted
const multiply = function (a, b) { return a * b; };

// 3. Arrow: short + inherits `this` (details below)
const square = n => n * n;
const greet = name => `Hello, ${name}!`;
const getUser = id => ({ id, name: "Sara" }); // ({} ) = return an object!

console.log(add(2, 3));      // 5
console.log(square(4));     // 16
```

Table 62 — Which function when

Form	Use for
<code>function name()</code>	Top-level reusable logic, event handlers needing <code>this</code>
<code>const f = () =></code>	Callbacks (map/filter), short utilities, methods not using <code>this</code>
Methods <code>obj.m() {}</code>	Object behaviors (keep <code>this</code> = the object)

Parameters: defaults, rest, destructuring preview

CODE · Professional parameters

```
function price(qty = 1, rate = 100) { return qty * rate; } // defaults
price();           // 100 – defaults kick in for undefined
price(3);          // 300

function sum(...nums) { return nums.reduce((a, n) => a + n, 0); } // rest: all into array
sum(1, 2, 3, 4);    // 10

// arguments object exists but is retired – rest params replaced it.
```

WARNING — DEFAULTS = UNDEFINED ONLY

`price(0)` uses 0 (NOT the default) — defaults trigger only on `undefined`. Same spirit as `??`. Passing `null` also keeps null. Design APIs accordingly.

Scope chain + closures (with REAL uses)

CODE · Scope: inner sees outer, never reverse

```
const shop = "Roastly";
function order(item) {
  const id = Math.random(); // order() locals...
  function receipt() {
    return `${shop}: ${item} #${id}`; // ...visible here (chain upward ✓)
  }
  return receipt();
}
// console.log(id); // ✗ ReferenceError – outside cannot see inside
```

A **closure** = a function + the variables it captured from its birthplace. It REMEMBERS them forever:

CODE · Closures doing real jobs

```
// Job 1: private state (a counter nobody can corrupt)
function createCounter() {
  let count = 0; // PRIVATE – unreachable from outside
  return {
    up: () => ++count,
    down: () => --count,
    value: () => count
  };
}
const c = createCounter();
c.up(); c.up(); console.log(c.value()); // 2 (c.count is undefined – truly private!)

// Job 2: configuration factories (one maker, many behaviors)
const greeter = greeting => name => `${greeting}, ${name}!`;
const sayHi = greeter("Hi"), sayWelcome = greeter("Welcome");
sayHi("Sara"); // "Hi, Sara!"

// Job 3: event handlers remembering their data (Ch 30 uses this daily)
function handleDelete(id) {
  return () => fetch(`/api/items/${id}`, { method: "DELETE" });
}
button.onclick = handleDelete(42); // remembers 42 until clicked
```

THINK OF IT LIKE THIS — CLOSURE = FUNCTION WITH A BACKPACK

When a function is born inside another scope, it packs a backpack with the variables around it and carries it everywhere — even after the outer function finished and left.

`createCounter`'s `count` lives in the backpack, invisible to the world, usable by the returned functions.

Hoisting truth (30 seconds, no myths)

Table 63 — What hoists

Declaration	Hoisted?	Usable before?
<code>function f() {}</code>	Whole function	Yes (entire body available)
<code>var x</code>	Declaration only (= undefined)	Silently undefined (bug factory)

Declaration	Hoisted?	Usable before?
<code>let/const</code>	Into TDZ (locked)	NO — ReferenceError

CODE · Only declarations enjoy hoisting

```

sayHi(); // works! (declaration hoisted whole)
function sayHi() { return "hi"; }
// sayBye(); // ✗ TypeError – const arrow sits in TDZ
const sayBye = () => "bye";

```

this in 5 rules (the demystification)**Table 64 — `this` = HOW the function was CALLED (4 rules + arrow exception)**

Call style	<code>this</code> is	Example
Method: <code>obj.m()</code>	The object before the dot	<code>user.greet()</code> → user
Alone: <code>f()</code>	undefined (strict) / window (sloppy)	Never rely on this!
<code>new F()</code>	The fresh instance	<code>new Cart()</code> → the new cart
<code>.call/.apply/.bind</code>	Whatever you pass	<code>f.call(timer)</code> → timer
Arrow <code>=></code>	NO own this — inherits enclosing	Callbacks keep outer this ✓

CODE · this, proven

```

const user = {
  name: "Sara",
  hi() { return `Hi, I'm ${this.name}`; }, // method: this = user ✓
  late() { setTimeout(function () {
    console.log(this.name); // ✗ undefined! plain call inside
  }, 0); },
  fixed() { setTimeout(() => {
    console.log(this.name); // ✓ "Sara" – arrow inherits fixed()'s
  }, 0); }
};
user.hi(); user.fixed();

```

KEY TAKEAWAY — KEY TAKEAWAY

Default to arrows for callbacks (they preserve your `this`), methods for objects (they receive it), and NEVER use `this` in standalone functions. 95% of `this` bugs vanish with these three habits.

INTERVIEW QUESTION — WHAT IS A CLOSURE? GIVE A REAL USE.

Expected: "A function bundled with its lexical scope, so it remembers outer variables after the outer function returns. Real uses: private state like counters, factory functions, and event handlers that remember their data." Then write `createCounter` from memory.

Quick check — Arrow functions and `this`:

1. Own `this` per call
2. Inherit enclosing `this`, never their own
3. Always window
4. Always undefined

Answer: B · Arrows capture `this` lexically at creation — that's why callbacks love them.

Exercise 24 — Secure vault

Write `createVault(pin)` returning `{deposit(n), withdraw(n, pin), balance(pin)}` where money is unreachable except with the right PIN, and wrong PIN throws.

SOLUTION

```
function createVault(pin){let m=0;const ok=p=>{if(p!==pin)throw new Error("Wrong PIN")};return{deposit:n=  
— closure privacy + guard clause, the professional combo.
```

CHAPTER 27

Arrays — Ordered Data and the Methods That Rule Them

Intermediate

~ 120 min

The list type behind carts, feeds and tables — plus map/filter/reduce fluency.

IN THIS CHAPTER

- Creating and indexing
- push/pop/shift/unshift/splice/slice
- map/filter/reduce/find/some/every
- Chaining like a senior
- Spread, destructure, copy correctly

Elzero JS #91-#120

Gamal JS #46-#60

Angela JS Arrays

Create, read, and the index trap

CODE · Array basics

```
const cart = ["milk", "bread", "eggs"];
cart[0];           // "milk" — indexes start at 0!
cart[cart.length - 1]; // "eggs" — last item pattern
cart[99];          // undefined — NO error (silent! validate indexes)
cart.length;       // 3
```

WARNING — OFF-BY-ONE + SILENT UNDEFINED

`arr[arr.length]` is ALWAYS undefined (last index = `length-1`), and JS won't warn you. Every "cannot read property of undefined" crash downstream starts here. Guard: check `i < arr.length` or use methods that can't overrun (`map/for...of`).

Mutating vs non-mutating (the safety line)

Table 65 — Array methods by danger

Mutates original (think twice)	Returns NEW (safe default)
<code>push/pop</code> (end), <code>shift/unshift</code> (start)	<code>map</code> (transform each)
<code>splice</code> (cut/paste anywhere)	<code>filter</code> (keep some)
<code>sort</code> , <code>reverse</code> (in place!)	<code>slice</code> (copy part), <code>concat</code> , <code>toSorted</code> (new)

CODE · Mutation vs purity, side by side

```
const nums = [3, 1, 2];
nums.push(4);           // nums = [3,1,2,4] (changed!)
const doubled = nums.map(n => n * 2); // NEW [6,2,4,8], nums untouched ✓
const top2 = [...nums].sort((a, b) => b - a).slice(0, 2); // copy-then-sort!
```

KEY TAKEAWAY — KEY TAKEAWAY

Default to non-mutating methods (`map/filter/slice`) — functions that don't surprise their callers. Mutate only local arrays you own (inside one function), and modern `toSorted()/toReversed()/toSpliced()` give sorted copies without touching originals.

The big six: map/filter/reduce/find/some/every

CODE · One dataset, six questions answered

```
const products = [
  { id: 1, name: "Mug",   price: 120, stock: 5 },
  { id: 2, name: "Beans", price: 350, stock: 0 },
  { id: 3, name: "Press", price: 900, stock: 2 },
];
products.map(p => `${p.name} (${p.price})`); // transform → string list
products.filter(p => p.stock > 0);           // keep → in-stock only
products.reduce((sum, p) => sum + p.price * p.stock, 0); // combine → 2400 (stock value)
products.find(p => p.id === 2);              // first match → Beans (or undefined!)
products.some(p => p.stock === 0);           // any? → true
products.every(p => p.price > 0);            // all? → true
```

Table 66 — Return-value cheat-sheet

Method	Returns	Empty-array gotcha
<code>map</code>	NEW array, same length	<code>[]</code> (safe)
<code>filter</code>	NEW array, 0..n items	<code>[]</code> (safe)
<code>reduce</code>	ONE value (you choose)	THROWS without initial value — always pass <code>, 0</code> !
<code>find</code>	First match or <code>undefined</code>	<code>undefined</code> (check before use!)
<code>some/every</code>	boolean	false / true respectively
<code>forEach</code>	<code>undefined</code> (actions only)	— (use for effects, never for building data)

Chaining like a senior

CODE · Read it as a sentence: "in-stock → names → top 3 → display"

```
const topNames = products
  .filter(p => p.stock > 0)
  .sort((a, b) => b.price - a.price)
  .slice(0, 3)
  .map(p => p.name.toUpperCase());
// ["PRESS", "MUG"]

// Numeric sort needs a comparator! Default sort is ALPHABETIC:
[10, 9, 80].sort();           // [10, 80, 9] ✗ strings!
[10, 9, 80].sort((a, b) => a - b); // [9, 10, 80] ✓ numbers
```

PRO TIP — FOREACH VS MAP DISCIPLINE

Building data → `map`. Doing effects (log, save, draw) → `forEach`. Using `forEach` + outer `push` to build arrays is junior style — `map` / `filter` say the intent in their names.

Spread, destructure, copy right

CODE · Modern array handling

```
const a = [1, 2], b = [3, 4];
const all = [...a, ...b, 5];      // [1,2,3,4,5] – merge + append
const [first, second] = a;       // first=1, second=2
const copy = [...a];             // shallow copy (top level only!)
copy.push(99);                   // a untouched ✓
// Nested objects still shared – structuredClone for deep copies (Ch 28).
```

Quick check — `[3,1,2].sort()` gives...

1. [1,2,3]
2. [1,3,2]
3. [3,1,2]
4. Error

Answer: A · Single digits sort fine either way — the trap needs 2-digit numbers ([10,9,80]→[10,80,9]). Always pass comparators for numbers regardless.

Exercise 25 — Cart total + receipt

From `cart = [{name, price, qty}...]`: (1) filter `qty>0`, (2) map to line strings "Mug ×2 = 240", (3) reduce to grand total. One chain + one reduce.

SOLUTION

```
const lines = cart.filter(i => i.qty > 0).map(i => `${i.name} ×${i.qty} = $
{i.price*i.qty} `); const total = cart.reduce((s, i) => s + i.price * i.qty, 0); — filter→map
builds display, reduce computes money. Note reduce's , 0 guard.
```


CHAPTER 28

Objects, ES6 Power and Classes

Intermediate

~ 130 min

Grouped data, modern syntax that professionals write daily, and object-oriented JS without the confusion.

IN THIS CHAPTER

- Literals, keys, methods
- Destructuring + spread + clone depths
- JSON stringify/parse
- Prototypes in 5 minutes
- Classes, private #, modules

Elzero JS #121-#150

Gamal JS #61-#75

Angela JS Objects

Objects: named data + behavior

CODE · An object is a labeled drawer set

```
const product = {
  name: "Mug",
  price: 120,
  stock: 5,
  sell(n = 1) { // method shorthand (this = product)
    if (n > this.stock) throw new Error("Not enough stock");
    this.stock -= n;
    return this.stock;
  }
};

product.name; // "Mug" (dot = known key)
product["pro" + "duct"] // dynamic keys via brackets
"name" in product; // true - existence check
delete product.stock; // remove a key (rare - prefer building new objects)
```

Destructuring + spread (daily professional syntax)

CODE · Unpack exactly what you need

```
const user = { id: 7, name: "Sara", role: "admin", city: "Cairo" };
const { name, role = "user" } = user;           // name="Sara", default if missing
const { city: town } = user;                   // rename: town="Cairo"
const { id, ...publicData } = user;            // rest: everything except id

function welcome({ name, role = "user" }) {    // destructure PARAMETERS
  return `${name} (${role})`;
}
```

CODE · Spread builds new objects (immutability pattern)

```
const updated = { ...user, city: "Alex" };      // copy + override city
const merged = { ...defaults, ...options };    // options win (order matters!)
// Shallow only: nested objects still shared. Deep? structuredClone(obj).
const deep = structuredClone(user);
```

Table 67 — Copy depths (the whole story)

Tool	Depth	Use
<code>{...o}</code> / <code>[...a]</code>	Shallow (top level)	Flat updates, React state
<code>structuredClone(o)</code>	Deep (everything)	Nested data, no functions inside
<code>JSON.parse(JSON.stringify(o))</code>	Deep-ish legacy	Only if structuredClone unavailable (loses dates/functions)

JSON: objects as text

CODE · Talking to servers and storage

```
const json = JSON.stringify(user);             // object → '{"id":7,...}' (send/store)
const back = JSON.parse(json);                 // text → object (receive/load)
JSON.stringify(user, null, 2);                 // pretty-print with 2-space indent
```

WARNING — JSON IS STRICTER THAN JS

Keys MUST be double-quoted, no trailing commas, no functions/undefined/comments.

`JSON.parse` throws on bad text — always wrap real-world parsing in try/catch (Chapter 34).

Prototypes in 5 minutes (then classes forever)

Every object has a hidden link to a **prototype** it inherits from. Arrays share `Array.prototype` (that's where `map` lives!); all objects share `Object.prototype` (`toString`, `hasOwnProperty`). Lookup climbs the chain until found or `null`.

CODE · The chain in action

```
const arr = [1, 2];
arr.map;           // found on Array.prototype (shared by ALL arrays ✓ memory-
                  // efficient)
arr.toString;      // found higher, on Object.prototype
// Classes below are CLEAN SYNTAX over this same prototype system.
```

Classes + private fields + modules

CODE · A professional class (bank account)

```
class Account {
  #balance = 0;           // truly private (outside CANNOT touch)
  constructor(owner) { this.owner = owner; }
  deposit(n) {
    if (n <= 0) throw new Error("Amount must be positive");
    this.#balance += n;
    return this.#balance;
  }
  get balance() { return this.#balance; } // read-only view
  static bankName() { return "Nile Bank"; } // on the class, not instances
}
const me = new Account("Sara");
me.deposit(500); me.balance;           // 500 (me.#balance = syntax error outside!)
```

CODE · Modules — split code across files (modern standard)

```
// math.js
export const add = (a, b) => a + b;
export default class Calc { /* ... */ }
// app.js
import Calc, { add } from "./math.js";
const lazy = await import("./heavy-charts.js"); // load ONLY when needed ✓
```

Table 68 — Module rules

Rule	Meaning
<code>export</code> / <code>export default</code>	Share from a file (many named, one default)
<code>import ... from</code>	Receive in another file (paths need <code>./</code> + <code>.js</code>)
<code><script type="module"></code>	Browser entry point (deferred + strict automatically)
Dynamic <code>import()</code>	Lazy-load heavy features on demand

INTERVIEW QUESTION — SHALLOW VS DEEP COPY?

Expected: "Spread copies top-level only — nested objects stay shared. `structuredClone` copies everything. I use spread for flat state updates and `structuredClone` for nested data." Bonus: mention JSON trick loses Dates/functions.

Exercise 26 — Safe settings updater

`update(settings, patch)` returns a NEW object with patch applied, original untouched — including one nested level (`theme: {...}`).

SOLUTION

`const update = (s, p) => ({ ...s, ...p, theme: { ...s.theme, ...p.theme } });` — top spread + manual nested spread. For unknown depth, `structuredClone(s)` then `Object.assign(clone, p)`.

CHAPTER 29

The DOM — JavaScript Meets the Page

Intermediate

~ 130 min

Select, read, create and change anything on the page — the foundation of all interactivity.

IN THIS CHAPTER

- DOM tree mental model
- querySelector mastery
- textContent vs innerHTML (XSS!)
- Creating/inserting/removing
- Classes, styles, attributes + DocumentFragment

Elzero JS DOM

Gamal JS DOM

Angela \$DOM

The DOM tree (HTML as live objects)

```
<body>                                body (Element)
  <h1 id="t">Hi</h1>                  --->  |-- h1#t (Element)
  <p>Text</p>                          :    |-- "Hi" (Text node)
                                       +-- p (Element)
                                           +-- "Text" (Text node)

document = the root. EVERYTHING hangs below it. JS edits objects → browser repaints.
```

Figure 12 — HTML text becomes a living tree

Selecting elements (the 4 you need)

CODE · Selection mastery

```
const title = document.querySelector("#t"); // FIRST match (any CSS selector!)
const cards = document.querySelectorAll(".card"); // ALL matches (static NodeList)
const byId = document.getElementById("t"); // fastest single lookup
const form = document.forms.signup; // named forms shortcut

cards.forEach(c => c.classList.add("seen")); // NodeList has forEach ✓
// [...cards].map(...) – spread to array for map/filter
```

Table 69 — Selector guide

Need	Use
One element, any selector	<code>querySelector("#x .y")</code>
Many elements	<code>querySelectorAll</code> + <code>forEach</code>
Inside another element	<code>card.querySelector("button")</code> (scoped search!)
Closest ancestor	<code>btn.closest(".card")</code> (event delegation gold — Ch 30)

textContent vs innerHTML (read before touching!)

CODE · The safety line of the DOM

```
name.textContent = userInput; // ✓ SAFE: inserts pure TEXT ("<b>" shows literally)
box.innerHTML = `<b>${total}</b>`; // ⚠ HTML parsed – NEVER with untrusted input (XSS!)

// Building safely with user data:
const li = document.createElement("li");
li.textContent = comment; // user text CANNOT become executable HTML ✓
list.append(li);
```

WARNING — INNERHTML + USER INPUT = HACKED SITE

`box.innerHTML = comment` where `comment = <img src=x onerror="steal()">` RUNS the attacker's code in every visitor's browser (XSS). Rule: `textContent` for user data; `innerHTML` only for YOUR OWN static templates — and even then prefer `createElement`.

Create, insert, remove, clone

CODE · DOM surgery complete

```
const li = document.createElement("li");
li.className = "item";
li.dataset.id = "42";           // → <li data-id="42"> (dataset API!)
li.textContent = "New item";

list.append(li);                // end ... prepend(start), before/after(siblings)
list.insertAdjacentHTML("beforeend", "<li>Fast static row</li>"); // fastest for YOUR
html
li.remove();                    // delete (modern — no parent needed)
const copy = card.cloneNode(true); // deep clone (listeners NOT copied)

// Zero-reflow batch rendering (500 rows, no jank):
const frag = document.createDocumentFragment();
items.forEach(t => { const el = document.createElement("li"); el.textContent = t;
frag.append(el); });
list.append(frag);              // ONE layout instead of 500 ✓
```

Table 70 — insertAdjacentHTML positions

Position	Meaning
<code>beforebegin</code>	Before the element itself (sibling)
<code>afterbegin</code>	First child
<code>beforeend</code>	Last child (append HTML)
<code>afterend</code>	After the element itself (sibling)

Classes, styles, attributes

CODE · The right tool per job

```
el.classList.add("active"); el.classList.remove("hidden"); el.classList.toggle("open");
el.classList.contains("card"); // check

el.style.color = "navy"; // inline style (JS-dynamic only!)
getComputedStyle(el).color; // READ final computed value

el.setAttribute("aria-expanded", "true"); // any attribute...
el.getAttribute("href"); el.removeAttribute("disabled");
el.dataset.userId; // data-* attributes as object
```

KEY TAKEAWAY — KEY TAKEAWAY

Style via CLASSES (CSS owns design), touch `el.style` only for truly dynamic values (drag positions, progress widths). Toggle UI states with `classList.toggle` + CSS doing the visuals.

Quick check — Rendering 1000 rows fastest?

1. innerHTML in a loop
2. append one-by-one
3. DocumentFragment batch
4. alert() each

Answer: C · Fragment builds off-screen with zero layouts, then commits once. innerHTML-once is runner-up for trusted static HTML.

Exercise 27 — Live search filter

Input filters a `<ul>` of 50 names as you type (hide non-matches). No innerHTML with the query!

SOLUTION

```
input.addEventListener("input", e => { const q = e.target.value.toLowerCase(); document.querySelectorAll(
— textContent reads safely; hidden toggles display. Case-insensitive via toLowerCase on
both sides.
```

CHAPTER 30

Events — Clicks, Keys and the Delegation Superpower

Intermediate

~ 120 min

React to everything users do — with ONE listener handling a thousand buttons.

IN THIS CHAPTER

- `addEventListener` + event object
- Mouse, keyboard, form events
- Bubbling vs capturing
- Delegation pattern
- `preventDefault` + custom behavior

Elzero JS events

Gamal JS events

Angela \$Events

Listening (the modern way)

CODE · Events done right

```
const btn = document.querySelector("#save");
btn.addEventListener("click", event => {
  console.log(event.type);           // "click"
  console.log(event.target);         // WHAT was clicked (deepest element)
  console.log(event.currentTarget); // WHERE the listener sits (btn)
});
// btn.onclick = ... // ✖ retired: only ONE handler allowed, overwritten silently
```

Table 71 — Everyday events

Category	Events	e.target gives you
Mouse	click dblclick mouseenter mouseleave	Clicked element
Keyboard	keydown keyup (+ e.key : "Enter", "Escape"...)	Focused element
Form	submit input change focus blur	The control

Category	Events	e.target gives you
Page	DOMContentLoaded load scroll resize	document/window

CODE · Keyboard + form patterns

```
input.addEventListener("keydown", e => {
  if (e.key === "Enter") search(e.target.value);
  if (e.key === "Escape") e.target.blur();
});
form.addEventListener("submit", e => {
  e.preventDefault(); // STOP page reload – handle with JS ✓
  save(new FormData(form)); // all fields as data object!
});
```

Bubbling: events float upward

<button>Delete</button> ← click starts HERE (target phase)
 ↑ bubbles: → <ul id="list"> → <main> → <body> → document → window
 A listener on HEARS clicks from every /<button> inside it!
 (Capturing = the reverse trip down; default listeners use bubbling.)

Figure 13 — Click ripples UP the tree (bubbling)

CODE · One listener, infinite buttons (THE delegation pattern)

```
document.querySelector("#list").addEventListener("click", e => {
  const btn = e.target.closest("button[data-act]"); // find the real button
  if (!btn) return; // clicked elsewhere – ignore
  const id = btn.closest("li").dataset.id;
  if (btn.dataset.act === "del") deleteItem(id);
  if (btn.dataset.act === "edit") editItem(id);
});
```

THINK OF IT LIKE THIS — DELEGATION = ONE DOORMAN

Instead of a guard at every office door (500 listeners = memory + re-binding new rows), post ONE doorman at the building entrance who asks "which office?" (`closest()`). New offices (dynamically added rows) work instantly — the doorman was already there.

Table 72 — When delegation wins

Case	Why
Lists/tables with many rows	1 listener vs N; new rows work free
Dynamically added content	No re-binding after every render
Memory + simplicity	Fewer closures, one place to debug

CODE · Full delegation example (todo list: toggle + delete)

```

<ul id="todos">
  <li data-id="1">Milk <button data-act="done">✓</button> <button data-act="del">✕</button></li>
</ul>
<script>
document.querySelector("#todos").addEventListener("click", e => {
  const btn = e.target.closest("button");
  if (!btn) return;
  const li = btn.closest("li");
  if (btn.dataset.act === "done") li.classList.toggle("done");
  if (btn.dataset.act === "del") li.remove();
});
</script>

```

INTERVIEW QUESTION — HOW WOULD YOU HANDLE CLICKS ON 1000 TABLE ROWS?

Expected: "Event delegation — one listener on the `<tbody>`, use `event.target.closest()` to find the row and its `data-id`. It uses constant memory and automatically covers rows added later." Mention bubbling as the mechanism.

Quick check — `e.target` vs `e.currentTarget` :

1. Same thing
2. target = clicked element, currentTarget = listener's element
3. Reversed
4. Both window

Answer: B · With delegation they differ constantly: target is the button, currentTarget the list. Confusing them breaks delegation.

Exercise 28 — Delegated gallery

Grid of 12 thumbnails; ONE listener opens the clicked full image in a modal; Escape or backdrop-click closes; focus returns to the thumbnail.

SOLUTION

Grid listener:
`const img = e.target.closest("img"); if (!img) return; openModal(img.dataset.full)`. Modal:
backdrop click (`e.target === backdrop`) or `keydown Escape` closes; store `lastFocus` and
`.focus()` it on close. Accessibility + performance in one pattern.

CHAPTER 31

Forms, Validation and Browser Storage

Intermediate

~ 110 min

Read forms with JS, validate like a pro, and remember data with localStorage.

IN THIS CHAPTER

- FormData + reading controls
- Custom validation UX
- Regex survival kit
- localStorage/sessionStorage/cookies
- Theme toggle project (Ch 21 payoff!)

Elzero JS forms

Gamal JS project

Angela \$Forms+JS

Reading forms with JavaScript

CODE · FormData reads everything at once

```
form.addEventListener("submit", e => {
  e.preventDefault();
  const data = new FormData(form);           // grabs ALL named controls
  const user = {
    name: data.get("name").trim(),
    email: data.get("email").trim(),
    plan: data.get("plan"),
    news: data.get("news") === "on"         // checkbox → "on"/null
  };
  console.table(user);
});
```

CODE · Custom validation with friendly messages

```
const email = form.elements.email;
email.addEventListener("input", () => {
  if (email.validity.valueMissing) email.setCustomValidity("Email is required.");
  else if (email.validity.typeMismatch)
    email.setCustomValidity("That doesn't look like an email.");
  else email.setCustomValidity(""); // "" = valid – REQUIRED to clear!
});
```

Table 73 — Constraint Validation API

Tool	Job
<code>input.checkValidity()</code>	boolean test (no UI)
<code>input.reportValidity()</code>	Test + show browser bubble
<code>setCustomValidity(msg)</code>	Custom error ("" clears)
<code>validity.*</code>	Exact flags: valueMissing, typeMismatch, tooShort, patternMismatch...

Regex survival kit (only what you'll use)

Table 74 — 90% of real-world regex

Pattern	Matches	Example use
<code>^\d+\$</code>	Only digits	<code>pattern="\d+"</code> quantity
<code>^[A-Za-z]{3,30}\$</code>	3-30 letters/spaces	Names
<code>^[^@\s]+@[^\s]+\.[^\s]+\$</code>	Basic email shape	(<code>type=email</code> does this free)
<code>(?=.*\d){8,}</code>	8+ chars with a digit	Passwords (with minlength too)

CODE · Test safely

```
const ok = /^[A-Za-z ]{3,30}$/.test(nameInput.value);
```

WARNING — DON'T WRITE THE "PERFECT EMAIL REGEX"

The RFC-valid email regex is 6,000+ characters. Use `type="email"` + server-side verification email instead. Regex validates SHAPE; only the server (and the inbox) validates TRUTH.

Storage: local vs session vs cookies

Table 75 — Browser storage

Store	Capacity	Lifetime	Sent to server?	Use for
localStorage	~5MB	Forever (until cleared)	NO	Theme, drafts, cart, UI prefs
sessionStorage	~5MB	Tab lifetime	NO	Multi-step wizards, temp flow state
cookies	~4KB	You set expiry	YES (every request!)	Auth tokens/sessions (HttpOnly!)

CODE · Storage done right (objects need JSON!)

```
// localStorage stores STRINGS ONLY – stringify objects:
const save = (k, v) => localStorage.setItem(k, JSON.stringify(v));
const load = (k, fb) => {
  try { return JSON.parse(localStorage.getItem(k)) ?? fb; }
  catch { return fb; } // corrupted data never crashes the app ✓
};
save("cart", [{ id: 1, qty: 2 }]);
const cart = load("cart", []);
```

WARNING — NEVER STORE SECRETS IN LOCALSTORAGE

Any XSS hole lets attackers read ALL localStorage — tokens included. Auth tokens belong in HttpOnly cookies (server-set, JS-invisible). localStorage = preferences and public data ONLY. (Full story: Chapters 43–44.)

Project: theme toggle (Chapter 21 payoff!)

CODE · Dark-mode toggle in 20 lines

```
<button id="themeBtn" aria-pressed="false">☐ Dark</button>
<script>
const root = document.documentElement;
const btn = document.querySelector("#themeBtn");
const apply = t => {
  root.dataset.theme = t;
  localStorage.setItem("theme", t);
  btn.textContent = t === "dark" ? "☒ Light" : "☐ Dark";
  btn.setAttribute("aria-pressed", t === "dark");
};
apply(localStorage.getItem("theme") || "light"); // restore saved choice
btn.addEventListener("click", () =>
  apply(root.dataset.theme === "dark" ? "light" : "dark"));
</script>
```

Pair with the `[data-theme="dark"]` variables from Chapter 21 — your landing page now remembers its theme forever.

Exercise 29 — Persistent todo (localStorage CRUD)

Todo app: add/toggle/delete with delegation (Ch 30) + autosave array to localStorage + restore on load + item count. Corrupted storage must not crash.

SOLUTION

State array → `render()` rebuilds list via `DocumentFragment`; every mutation calls `save()`; boot calls `load("todos", [])` in `try/catch`. Count = `todos.filter(t => !t.done).length`. This is a mini front-end architecture: state → render → persist.

CHAPTER 32

Async JavaScript — The Event Loop, Promises and async/await

Advanced

~ 140 min

How JS does many things on one thread: the engine's queuing system and clean async code.

IN THIS CHAPTER

- Sync vs async + the event loop
- Microtasks vs macrotasks
- Promises + combinators
- async/await + error handling
- Timeouts, intervals, debounce

Elzero async

Gamal async

Angela \$Async

The event loop (JS juggles on one thread)

```
CALL STACK ..... runs sync code NOW (one thing at a time)
  | stack empty?
  v
MICROTASK QUEUE .... ALL promise callbacks (.then/await) – VIP line, drains FULLY
  | microtasks empty?
  v
RENDER ..... paint the page (60fps chance)
  |
  v
MACROTASK QUEUE .... ONE task: setTimeout / setInterval / I/O / events – then loop!
```

Figure 14 — The loop that runs everything

CODE · Predict this (the classic interview demo)

```
console.log("1 sync");
setTimeout(() => console.log("2 timeout"), 0);
Promise.resolve().then(() => console.log("3 promise"));
console.log("4 sync");
// Output: 1, 4, 3, 2 – promises (micro) ALWAYS beat timeouts (macro)!
```

THINK OF IT LIKE THIS — THE EVENT LOOP IS A RESTAURANT KITCHEN

Sync code = the dish being cooked NOW. Microtasks = VIP orders the chef finishes completely before anything else. Macrotasks = the regular ticket rail (one per round). Render = serving the table between rounds. Starving VIPs (infinite promise chains) freeze the restaurant!

Promises: receipts for future values

CODE · Promise anatomy

```
const getUser = id => fetch(`/api/users/${id}`).then(r => {
  if (!r.ok) throw new Error(`HTTP ${r.status}`); // fetch only rejects on NETWORK
  failure!
  return r.json();
});
getUser(7)
  .then(user => console.log(user.name)) // fulfilled path
  .catch(err => console.error("Failed:", err.message)) // ANY failure above lands here
  .finally(() => hideSpinner()); // always runs (cleanup!)
```

WARNING — FETCH'S FAMOUS LIE

`fetch` RESOLVES on 404/500 (it got an answer!) and rejects ONLY on network failure (offline/DNS). Always check `response.ok` yourself — forgetting this ships bugs that "work" while showing error pages as data.

Table 76 — Promise combinators (many async jobs)

Tool	Waits for	Fails when	Use
<code>Promise.all</code>	ALL succeed	FIRST failure (rest ignored)	Dashboard needing every source

Tool	Waits for	Fails when	Use
<code>Promise.allSettled</code>	ALL finish	Never (reports each)	Bulk imports where partial OK
<code>Promise.race</code>	FIRST to finish	If winner fails	Timeouts racing requests
<code>Promise.any</code>	FIRST success	ALL fail	Mirror servers / fallbacks

CODE · Combinators in action

```
const [user, posts] = await Promise.all([getUser(7), getPosts(7)]); // parallel = fast ✓
// Sequential await in a loop = SLOW (each waits). Parallel array + all = FAST.
```

async/await (write async like sync)

CODE · The professional async function template

```
async function loadDashboard(id) {
  showSpinner();
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), 8000); // 8s timeout ✓
  try {
    const res = await fetch(`/api/dash/${id}`, { signal: ctrl.signal });
    if (!res.ok) throw new Error(`HTTP ${res.status}: ${await res.text()}`);
    return await res.json();
  } catch (err) {
    if (err.name === "AbortError") showToast("Request timed out – retry?");
    else showToast(`Failed: ${err.message}`);
    throw err; // let caller decide too
  } finally {
    clearTimeout(timer);
    hideSpinner(); // ALWAYS cleanup
  }
}
```

Table 77 — async/await rules

Rule	Why
<code>await</code> only inside <code>async</code> (or top-level modules)	Syntax law
Every <code>await</code> can throw → wrap in <code>try/catch</code>	Unhandled rejections crash logic
Independent jobs → <code>Promise.all</code> (parallel)	Sequential <code>await</code> multiplies latency
Always timeout network calls	Hanging spinners are UX death

Timers + debounce (search boxes done right)

CODE · Debounce: wait until the user STOPS typing

```
const debounce = (fn, ms = 300) => {
  let t;
  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), ms); };
};
searchInput.addEventListener("input", debounce(e => searchAPI(e.target.value)));
// 10 fast keystrokes = 1 API call instead of 10 ✓ (saves servers + money)
```

Quick check — Output order: sync A, promise P, timeout T?

1. A P T
2. A T P
3. P A T
4. T P A

Answer: A · Sync first, then microtasks (promises), then macrotasks (timeouts). The engine drains VIPs fully before one regular.

Exercise 30 — Parallel dashboard + timeout

Load user+posts+stats in parallel with 5s timeout; show partial data if one source fails (allSettled); render what arrived + retry button for failures.

SOLUTION

`Promise.allSettled` with `AbortController` timeout; filter fulfilled → render; rejected → collect names into retry panel. Parallel + resilient + honest UI — the senior async trifecta.

CHAPTER 33

Fetch and Real APIs — Talking to the World

Advanced

~ 110 min

GET/POST/PUT/DELETE with fetch, headers, auth tokens, loading/error states and a live project.

IN THIS CHAPTER

- fetch CRUD complete
- Headers + JSON bodies
- Auth headers (Bearer)
- Loading/error/empty states
- Project: posts explorer on a public API

Angela \$APIs

Elzero fetch

MDN fetch

fetch CRUD (the complete set)

CODE · One helper, every method (copy into projects)

```
async function api(path, { method = "GET", body, token } = {}) {
  const res = await fetch(`https://jsonplaceholder.typicode.com${path}`, {
    method,
    headers: {
      "Content-Type": "application/json",          // "my body is JSON"
      ...(token ? { Authorization: `Bearer ${token}` } : {})
    },
    ...(body ? { body: JSON.stringify(body) } : {})
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.status === 204 ? null : res.json();    // 204 = no body!
}

// CREATE      api("/posts", { method: "POST", body: { title: "Hi" } })
// READ        api("/posts/1")
// UPDATE      api("/posts/1", { method: "PATCH", body: { title: "New" } })
// DELETE      api("/posts/1", { method: "DELETE" })
```

Table 78 — Request anatomy

Piece	Example	Job
Method + path	PATCH /posts/1	What + which
Headers	Content-Type , Authorization	Meta-instructions about the call
Body	JSON.stringify(obj)	The data (never with GET!)
Response check	res.ok + status	fetch won't do it for you!

The 4 UI states (seniors never ship 2)

Every data screen has FOUR states — juniors build two (data + nothing) and users suffer:

CODE · State machine for any data view

```
async function renderPosts() {
  setState("loading"); // 1. skeleton/spinner
  try {
    const posts = await api("/posts?_limit=10");
    if (!posts.length) return setState("empty"); // 2. friendly zero-state
    paint(posts); setState("ready"); // 3. data ✓
  } catch (err) {
    setState("error", err.message); // 4. error + RETRY button
  }
}
```

Table 79 — The 4 states

State	Shows	Rookie mistake
Loading	Skeleton rows (not blank!)	Blank screen = "is it broken?"
Empty	"No posts yet — write the first!" + action	Blank = looks broken
Error	What failed + Retry button	console-only error = user stranded
Ready	The data	(the only state juniors build)

Project: posts explorer (JSONPlaceholder)

CODE · Full mini-app: list + search + details (public test API, no key needed)

```
<input id="q" placeholder="Search posts...">
<div id="status"></div>
<ul id="posts"></ul>
<article id="detail" hidden></article>
<script>
const postsEl = document.querySelector("#posts");
const render = async () => {
  status("Loading...");
  try {
    const posts = await api("/posts?_limit=20");
    const q = document.querySelector("#q").value.toLowerCase();
    const shown = posts.filter(p => (p.title + p.body).toLowerCase().includes(q));
    postsEl.innerHTML = ""; // own template below – safe...
    if (!shown.length) return status("No matches. Try another word.");
    const frag = document.createDocumentFragment();
    shown.forEach(p => {
      const li = document.createElement("li");
      const b = document.createElement("button");
      b.textContent = p.title; b.dataset.id = p.id; // textContent = XSS-safe ✓
      li.append(b); frag.append(li);
    });
    postsEl.append(frag); status("");
  } catch (e) { status("Failed to load. <button id='retry'>Retry</button>"); }
};
postsEl.addEventListener("click", async e => {
  const b = e.target.closest("button"); if (!b) return;
  const post = await api(`/posts/${b.dataset.id}`);
  const d = document.querySelector("#detail");
  d.hidden = false; d.querySelector("h2").textContent = post.title;
});
const status = msg => document.querySelector("#status").innerHTML = msg;
document.querySelector("#q").addEventListener("input", debounce(render, 300));
render();
</script>
```

KEY TAKEAWAY — KEY TAKEAWAY

Real API code = helper (method/headers/ok-check) + 4 UI states + XSS-safe rendering (textContent/createElement) + debounce on search. This pattern scales from homework to production unchanged.

Exercise 31 — Comments + create post

Extend the explorer: (1) show post comments under details (`/posts/:id/comments`), (2) a form POSTing a new post and prepending it optimistically, (3) rollback + error toast if POST fails.

SOLUTION

Comments: second delegated fetch into details section. Optimistic create: render immediately with "sending..." flag, then confirm/replace on success or remove + toast on failure. Optimistic UI + rollback = professional perceived speed.

CHAPTER 34

Errors, Debugging and Regular Expressions

Advanced

~ 90 min

Handle failure gracefully, debug in minutes, and wield regex safely.

IN THIS CHAPTER

- try/catch/finally + custom errors
- throw like a pro
- DevTools debugging (breakpoints!)
- console.table/timing
- Regex for validation

Elzero debug

MDN errors

W3Schools JS

try/catch/finally + throwing well

CODE · Errors as values, not crashes

```
function parseAge(input) {
  const n = Number(input);
  if (!Number.isInteger(n)) throw new TypeError(`Age must be an integer, got ${input!r}`);
  if (n < 0 || n > 130) throw new RangeError(`Age ${n} out of range 0–130`);
  return n;
}

try {
  const age = parseAge(form.age.value);
  save(age);
} catch (err) {
  console.error(err.name, err.message); // TypeError/RangeError + context
  showFieldError("age", err.message); // user sees FRIENDLY text
} finally {
  submitBtn.disabled = false; // ALWAYS re-enable (success or fail)
}
```

Table 80 — Error types to throw

Type	Meaning
Error	Generic (default)
TypeError	Wrong type/shape (<code>user.map</code> is not a function)
RangeError	Out of bounds (age 999)
SyntaxError	Bad text format (JSON.parse fails → catch it!)
Custom <code>class AuthError extends Error</code>	Domain failures (wrong PIN, expired token)

KEY TAKEAWAY — KEY TAKEAWAY

Throw EARLY with SPECIFIC types + context ("Age must be integer, got 'abc'"), catch at BOUNDARIES (event handlers, API calls), and NEVER show raw errors to users (log technical, display friendly).

Debug like a senior (breakpoints > console.log spam)

Table 81 — DevTools debugging workflow

Step	How
1. Pause on error	Sources → ☐ "Pause on exceptions" (stops AT the crash!)
2. Breakpoint	Click a line number → reload → execution FREEZES there
3. Inspect	Hover variables; Scope panel shows closures; Console runs live code
4. Step	F10 step-over, F11 step-into functions, F8 resume
5. Network	Failed request? Check status + response body + headers

TRY IT YOURSELF — CATCH A BUG RED-HANDED (5 MINUTES)

Paste buggy code (e.g. `users.map` where `users` is null) → Sources → enable Pause on exceptions → run → DevTools freezes ON the crash line with the full call stack. Fix in 1 minute what console.log-hunting takes 30.

Regex: validate shapes, safely

CODE · Practical regex (test + patterns from Ch 31, leveled up)

```
const patterns = {
  username: /^[a-z0-9_]{3,16}$/i,          // i = case-insensitive
  egyptPhone: /^(\+20|0)?1[0125]\d{8}$/,
  slug: /^[a-z0-9]+(?:-[a-z0-9]+)*$/,
  strongPw: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}$/
};
patterns.username.test("sara_dev");        // true
// Escape user text before embedding in regex:
const escapeRe = s => s.replace(/[\.\+\?\$\{\}\|\[\]\\\]/g, "\\$&");
```

WARNING — REDOS: EVIL REGEX CAN HANG SERVERS

Nested quantifiers like `(a+)+$` on long input can freeze Node for seconds (Regular-expression Denial of Service). Keep patterns simple, anchor with `^$`, prefer length checks + simple classes over clever one-liners.

Quick check — Best first debugging step?

1. Add 20 console.logs
2. Pause on exceptions + breakpoint
3. Rewrite the file
4. Ask AI immediately

Answer: B · Freezing AT the crash with live scope beats guessing. Logs confirm; breakpoints reveal.

Exercise 32 — Harden parseSettings()

`parseSettings(text)` JSON-parses user-pasted settings, validates `{theme: "light"|"dark", fontSize: 12-24}`, returns defaults + warnings for fixable issues, throws friendly errors for fatal ones. No crash possible.

SOLUTION

try/catch `JSON.parse` → `SyntaxError` becomes "Settings text is not valid JSON"; validate each field collecting warnings (clamp `fontSize`, default `theme`); throw only when root isn't an object. Return `{settings, warnings}`. Total robustness in ~25 lines.

CHAPTER 35

JavaScript Projects — Quiz App and Expense Tracker

Advanced

~ 150 min

Two complete apps assembling Parts 4: state, render, events, storage, async.

IN THIS CHAPTER

- Architecture: state → render → persist
- Project 1: timed quiz with highscores
- Project 2: expense tracker with charts-by-hand
- Review checklist

Project

Elzero practice

Angela's Project

The architecture (memorize this triangle)

```
STATE (plain JS object/array – the single truth)
  | mutate via functions only (addTodo, answerQuiz...)
  v
RENDER (state → DOM via template functions + delegation)
  |
  v
PERSIST (localStorage / API sync after each mutation)
Events flow BACK up: user → handler → mutate state → re-render.
```

Figure 15 — Every front-end app, from todo to trading dashboard

Project 1 — Timed quiz with highscores

CODE · Quiz: data + state machine (60 lines that teach architecture)

```
const QUESTIONS = [
  { q: "2 + '2' = ?", opts: ["4", "22", "NaN", "Error"], answer: 1 },
  { q: "Which copies deeply?", opts: ["{...o}", "slice()", "structuredClone", "assign"],
    answer: 2 },
];

const state = { i: 0, score: 0, time: 30, timer: null, lock: false };

function render() {
  const Q = QUESTIONS[state.i];
  qEl.textContent = `Q${state.i + 1}. ${Q.q}`;
  optsEl.innerHTML = "";
  Q.opts.forEach((t, idx) => {
    const li = document.createElement("li");
    const b = document.createElement("button");
    b.textContent = t; b.dataset.idx = idx; li.append(b); optsEl.append(li);
  });
  scoreEl.textContent = `Score: ${state.score} · Time: ${state.time}s`;
}

optsEl.addEventListener("click", e => {
  const b = e.target.closest("button"); if (!b || state.lock) return;
  state.lock = true;
  const correct = Number(b.dataset.idx) === QUESTIONS[state.i].answer;
  b.classList.add(correct ? "right" : "wrong");
  if (correct) state.score += 10;
  setTimeout(() => {
    state.i++; state.lock = false;
    state.i < QUESTIONS.length ? render() : finish();
  }, 600);
});

function finish() {
  clearInterval(state.timer);
  const best = Math.max(state.score, Number(localStorage.getItem("quizBest") || 0));
  localStorage.setItem("quizBest", best);
  app.innerHTML = `<h2>Done! ${state.score} pts (Best: ${best})</h2><button
onclick="location.reload()">Retry</button>`;
}

state.timer = setInterval(() => { if (--state.time <= 0) finish(); else render(); },
1000);

render();
```


Project 2 — Expense tracker (state + stats + bar chart by hand)

CODE · Tracker core: add/delete + totals + CSS-bar chart

```
let items = load("expenses", []); // [{id, label, amount, cat}]
const total = () => items.reduce((s, i) => s + i.amount, 0);
const byCat = () => items.reduce((m, i) => ((m[i.cat] ??= 0), m[i.cat] += i.amount, m), {});

function render() {
  list.innerHTML = "";
  const frag = document.createDocumentFragment();
  items.forEach(it => {
    const li = document.createElement("li");
    li.innerHTML = ""; // build safe nodes...
    const s = document.createElement("span");
    s.textContent = `${it.label} - ${it.amount} (${it.cat})`;
    const del = document.createElement("button");
    del.textContent = "x"; del.dataset.id = it.id; del.dataset.act = "del";
    li.append(s, del); frag.append(li);
  });
  list.append(frag);
  totalEl.textContent = `Total: ${total()}`;
  chart.innerHTML = "";
  const max = Math.max(1, ...Object.values(byCat()));
  for (const [cat, sum] of Object.entries(byCat())) {
    const bar = document.createElement("div");
    bar.className = "bar";
    bar.style.setProperty("--h", `${(sum / max) * 100}%`);
    bar.title = `${cat}: ${sum}`;
    chart.append(bar); // .bar{height:var(--h)} in CSS
  }
  save("expenses", items);
}

form.addEventListener("submit", e => {
  e.preventDefault();
  const d = new FormData(form);
  items.unshift({ id: crypto.randomUUID(), label: d.get("label").trim(),
    amount: Math.abs(Number(d.get("amount"))), cat: d.get("cat") });
  form.reset(); render();
});

list.addEventListener("click", e => {
  const b = e.target.closest("button[data-act='del']"); if (!b) return;
  items = items.filter(i => i.id !== b.dataset.id); render();
});

render();
```

PROJECT REVIEW (BOTH APPS)

- - Single state object; ALL DOM updates flow through render(); no stray DOM writes
- - Delegation for dynamic buttons; textContent for user data (XSS-safe)
- - Storage wrapped in try/catch with defaults; corrupted data can't crash boot
- - Empty states ("No expenses yet — add your first!"); counts and totals derived, never stored
- - Timer/interval cleaned up; no console errors; keyboard usable

Exercise 33 — Ship it

Deploy both apps (Chapter 37) and send the links to a friend. Watch them use it WITHOUT explaining — every confusion is a UX bug to fix. Real users are the final chapter of every project.

SOLUTION

Classic findings: no empty state ("is it broken?"), tiny tap targets, no feedback after Add ("did it save?"). Fix with states (Ch 33), 44px targets (Ch 19), and toasts/animations (Ch 20). Iteration > perfection.

CHAPTER 36

JavaScript Datasheet — Methods and Syntax in One Place

Reference

~ 60 min

The working reference: syntax, array/object methods, DOM, async and storage — find anything in seconds.

IN THIS CHAPTER

- Syntax quick tables
- Array methods matrix
- String/object helpers
- DOM + events + fetch
- Storage + JSON + regex

Datasheet

MDN ref

W3Schools ref

Table 82 — Equality + falsy (never google again)

Check	Rule		
Equality	<code>===</code> always (<code>==</code> banned)		
Falsy 8	false, 0, -0, 0n, "", null, undefined, NaN		
Default	<code>??</code> for null/undefined; <code>`</code>	<code>\</code>	<code>`</code> replaces ALL falsy
Safe dive	<code>user?.address?.city ?? "Unknown"</code>		

map / filter / reduce ARRAYS

Transform each / keep some / combine to one. Non-mutating; chain freely.

```
const names = users.filter(u => u.active).map(u => u.name.toUpperCase());
const total = cart.reduce((s, i) => s + i.price * i.qty, 0);
key: reduce ALWAYS takes an initial value (empty arrays throw without it).
```

find / some / every ARRAYS

First match (or undefined) / any true? / all true?

```
users.find(u => u.id === 7); users.some(u => u.banned); users.every(u => u.email);
key: find returns the ITEM; some/every return booleans.
```

slice / splice ARRAYS

slice copies a range (safe); splice cuts/pastes in place (mutates!).

```
arr.slice(0, 3); arr.toSpliced(1, 1); // modern non-mutating
arr.splice(1, 1); // mutates original!
key: slice = scissors with photocopy; splice = scissors on the original.
```

sort ARRAYS

In-place sort; DEFAULT IS ALPHABETIC — numbers need a comparator.

```
nums.sort((a, b) => a - b); users.sort((a, b) => a.name.localeCompare(b.name));
key: Copy first ([...a].sort / toSorted) unless mutation is intended.
```

Destructuring + spread **SYNTAX**

Unpack values; spread merges/copies (shallow).

```
const { name, role = "user" } = user; const next = { ...user, city: "Alex" };  
key: Shallow only – nested objects stay shared (structuredClone for deep).
```

Template literals **STRINGS**

Backtick strings with `\${}` injection + multiline.

```
`Hello ${name}, you have ${n} messages.`  
key: Default choice for any string containing values.
```

String helpers **STRINGS**

The daily set: case, trim, includes, slice, split/join, replace.

```
s.trim().toLowerCase().includes(q); s.slice(0, 10); "a,b".split(","); arr.join(" · ");  
key: Strings are IMMUTABLE – every method returns a NEW string.
```

Closures **FUNCTIONS**

Function + captured scope = private state and factories.

```
const createCounter = () => { let n = 0; return { up: () => ++n, value: () => n }; };  
key: Data privacy without classes.
```

this FUNCTIONS

Method call → object before dot; arrows inherit enclosing this.

```
user.hi(); // this = user ... setTimeout(() => this.save(), 0); // inherits  
key: Arrows for callbacks, methods for objects, never bare this.
```

querySelector DOM

First / all matches via CSS selectors; closest() climbs up.

```
document.querySelector("#app .btn"); el.closest(".card"); list.querySelectorAll("li");  
key: Scope searches (card.querySelector) and climb with closest().
```

DOM edit DOM

textContent (safe) vs innerHTML (trusted only); create/append/remove; dataset; classList.

```
li.textContent = name; box.classList.toggle("open"); el.dataset.id;  
key: User data → textContent/createElement. Always.
```

Events + delegation DOM

addEventListener; one parent listener + closest() handles infinite children.

```
list.addEventListener("click", e => { const b = e.target.closest("button"); if (!b)  
return; });  
key: target = clicked; currentTarget = listener's element.
```

fetch + async **ASYNC**

await fetch → check res.ok → parse. try/catch/finally + timeout.

```
const res = await fetch(url, { signal: AbortSignal.timeout(8000) });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
key: fetch resolves on 404/500 – check .ok yourself!
```

localStorage **STORAGE**

String-only forever-store. JSON + try/catch wrapper.

```
localStorage.setItem("k", JSON.stringify(v)); JSON.parse(localStorage.getItem("k")) ??
fb;
key: Preferences only – NEVER tokens/secrets (XSS-readable).
```

JSON **DATA**

stringify (send/store) / parse (receive/load, wrap in try/catch).

```
JSON.stringify(obj, null, 2); JSON.parse(text);
key: Double quotes, no trailing commas, no functions.
```

Modules **ORGANIZE**

export/import across files; dynamic import() lazy-loads.

```
export const add = (a, b) => a + b; import { add } from "./m.js";
key: type="module" scripts are deferred + strict automatically.
```

PART

Tools of the Trade — Git, GitHub and Shipping

Version control, packages and deployment: work like a professional team of one.

Ch 37 Git, GitHub, npm and Deployment

Ch 38 CI/CD, Pull Requests and Team Workflow

| Commit early, commit often — your future self will thank you.

CHAPTER 37

Git, GitHub, npm and Deployment

Intermediate

~ 120 min

Save every version, collaborate, reuse the world's code, and publish your work online.

IN THIS CHAPTER

- Repos, commits, branches
- The 12 commands that cover 95%
- GitHub flow (push/PR)
- npm + package.json
- Deploy static sites + .gitignore secrets

Angela \$Git

Elzero Git

MDN tooling

The mental model (repos, commits, branches)

Working files (your desk) --git add--> Staging (the box) --git commit--> REPO (history)

main branch: A -- B -- C -- D (stable timeline)

feature branch: E -- F (experiment universe, merge when perfect)

Remote (GitHub) = the same repo living online; push/pull sync them.

Figure 16 — Git = save-points + parallel universes

Table 83 — The 12 commands covering 95% of real work

Command	Job
<code>git init</code>	Start tracking this folder
<code>git status</code>	WHAT changed? (run constantly)
<code>git diff</code>	Show exact line changes
<code>git add .</code>	Stage everything for the snapshot

Command	Job
<code>git commit -m "Add pricing section"</code>	Save snapshot with a message
<code>git log --oneline</code>	History at a glance
<code>git branch cart</code> / <code>git switch cart</code>	Create / enter a universe
<code>git switch -c feature/x</code>	Create + enter in one move
<code>git merge feature/x</code>	Fold a universe back into main
<code>git push -u origin main</code>	Upload to GitHub (first time links them)
<code>git pull</code>	Download teammates' work
<code>git restore file</code>	Undo unstaged changes (careful!)

CODE · Your first repository (the exact sequence)

```
git init
git add .
git commit -m "Initial commit: profile page"
git branch -M main
git remote add origin https://github.com/YOU/profile.git
git push -u origin main
```

KEY TAKEAWAY — KEY TAKEAWAY

Commit MESSAGES describe the change like a newspaper headline: "Add dark-mode toggle", "Fix mobile nav overlap". Future-you (and employers reading your GitHub) judge your professionalism by these lines. Never "fix", "update", "final-final-2".

Branching workflow (never break main)

CODE · Feature workflow professionals use daily

```
git switch -c feature/quiz-timer    # new universe for the feature
# ... code, test, commit twice ...
git switch main && git pull          # fresh stable base
git merge feature/quiz-timer        # fold it in (resolve conflicts calmly)
git push                            # ship it
```

Table 84 — Merge conflicts (not scary!)

Step	Action
1	<code>git status</code> shows conflicted files; open them
2	Find <code><<<<<< / ===== / >>>>>></code> markers — keep the right code, delete markers
3	<code>git add .</code> then <code>git commit</code> (Git pre-writes the message)

TRY IT YOURSELF — BREAK AND RECOVER (10 MINUTES)

Create a branch, delete half your page, commit — then `git switch main`: everything is back! Branches make experiments free. Delete the experiment: `git branch -D experiment`. Fearless coding unlocked.

.gitignore (secrets NEVER ship)

CODE · .gitignore — the bodyguard file (project root)

```
node_modules/    # thousands of dependency files (reinstall via npm)
.env             # SECRETS: passwords, API keys — NEVER commit!
dist/           # build output (regenerate anytime)
.DS_Store        # OS junk
```

WARNING — PUSHING .ENV = LEAKING KEYS TO THE WORLD

Bots scan GitHub for keys within MINUTES. If you ever push a secret: revoke it immediately (new key), then clean history. Prevention (`.gitignore` from day one) beats surgery.

npm + package.json (the world's code, reused)

CODE · npm essentials

```
npm init -y           # create package.json (project ID card)
npm install express    # add a dependency (recorded + downloaded)
npm install -D vite     # dev-only tool (not shipped to users)
npx vite               # run a tool WITHOUT installing globally
npm run dev             # your custom scripts (defined in package.json)
```

CODE · package.json, translated

```
{
  "name": "my-api",
  "type": "module",           // modern import/export syntax
  "scripts": { "dev": "node server.js", "start": "node server.js" },
  "dependencies": { "express": "^4.19.0" } // what users need
}
```

Table 85 — node_modules wisdom

Fact	Meaning
Never commit it	<code>.gitignore</code> it; teammates run <code>npm install</code> to rebuild
<code>package-lock.json</code>	COMMIT this (locks exact versions = identical installs)
<code>npx</code>	Try tools without polluting your machine

Deploy: from laptop to link (static sites)

Table 86 — Free static hosting (front-end projects)

Platform	Deploy	Best for
GitHub Pages	Push → Settings → Pages	Portfolios, docs
Netlify / Vercel	Import repo → auto-deploy on push	Landing pages, React apps
Cloudflare Pages	Same flow	Speed-obsessed sites

CODE · Deploy checklist (works everywhere)

1. Push clean repo (no secrets, .gitignore correct)
2. index.html at root (or set build command for frameworks)
3. Connect repo in host dashboard → Deploy
4. Test the LIVE link on phone + desktop (paths/case bugs appear here!)
5. Custom domain later (optional) – content first, vanity second

INTERVIEW QUESTION — WALK ME THROUGH YOUR DEPLOYMENT PROCESS.

Expected: "I push feature branches, open pull requests, merge to main after review, and the host auto-deploys main. I verify the live URL on mobile and desktop, and roll back via the host dashboard or git revert if needed." Mentions .gitignore/secrets unprompted = senior signal.

Quick check — `.env` belongs in...

1. The repo (teammates need it)
2. .gitignore + secret manager
3. README
4. node_modules

Answer: B · Secrets live in environments/host dashboards — documented by NAME in README, valued nowhere public.

Exercise 34 — Ship your portfolio

Push your Ch 9 + Ch 21 work to GitHub (clean history, real messages, .gitignore), deploy to Netlify/Vercel/Pages, and verify on your phone. Add the live link to your README + CV.

SOLUTION

Done = public repo + live HTTPS link + README with screenshot, features, and tech list. This single exercise makes you hireable-adjacent: proof beats promises.

CHAPTER 38

CI/CD, Pull Requests and Team Workflow

Intermediate+

~ 100 min

Automate quality, collaborate via pull requests, and release with confidence.

IN THIS CHAPTER

- CI pipeline (test+lint on push)
- GitHub Actions first workflow
- PR etiquette + code review
- Conventional commits + releases
- Team branch strategy

Elzero Git

DevOps

Career

CI: robots check every push (the safety net)

```
push → [install: npm ci] → [lint: eslint] → [test: vitest run] → [build: vite build]
      → 🟢 green: merge allowed ... 🟠 red: merge BLOCKED (fix first!)
      No human merges red. Ever. The pipeline is the strictest reviewer.
```

Figure 17 — The CI pipeline (runs on EVERY push + PR)

CODE · .github/workflows/ci.yml — your first pipeline (copy-paste ready)

```

name: CI
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --run
      - run: npm run build

```

Table 87 — Pipeline anatomy (each line earns its place)

Step	Why
checkout	CI needs YOUR code (fresh clone per run!)
setup-node + cache	Right Node + cached deps (fast runs!)
npm ci	Lockfile-exact install (reproducible!)
lint → test → build	Fail FAST on style, then logic, then bundle
Badges	README !CI shows green = trust signal for employers

KEY TAKEAWAY — KEY TAKEAWAY

CI moves "does it work?" from hope to PROOF on every push. Red main = the team stops and fixes. Your portfolio repos with green badges + passing pipelines signal professional discipline louder than any CV line.

Pull requests: collaboration + review

CODE · The PR lifecycle (solo devs: still use it!)

1. Branch: `git switch -c feat/search` (never commit to main!)
2. Push + open PR: title + WHAT + WHY + screenshots + test plan
3. CI runs: green required ... Review: teammate (or future-you) comments
4. Address feedback → push more commits (PR updates automatically)
5. Squash-merge: one clean commit on main ... delete branch

Table 88 — PR description template (great PRs get fast reviews)

Section	Content
What/Why	"Adds debounced search (closes #12). Users complained results lag."
How to test	"1. Type 3+ chars 2. See results <300ms 3. Clear → full list"
Screenshots	Before/after (UI changes REQUIRE visuals!)
Risks	"Touches shared fetchJson — ran full suite, all green"
::: table Code review etiquette (both sides)	
Role	Rules
Author	Small PRs (<400 lines!), explain WHY not what, respond to every comment, never take it personally
Reviewer	Ask don't demand ("What if...?"), praise good code, approve-or-request-changes (no limbo), review within 24h

Section	Content	
Everyone	Argue about code, never people ... automate style (Prettier) so reviews discuss LOGIC	
## Conventional commits + releases		
::: table Commit messages that write your changelog		
Prefix	Use	Example
feat:	New feature	feat: add password reset flow
fix:	Bug fix	fix: cart total ignores discount
docs:	Docs only	docs: add API rate-limit notes
refactor:	No behavior change	refactor: extract useFetch hook
test:	Tests	test: checkout happy path + 401s
chore:	Tooling	chore: bump node to 20
``text:Releases without fear (the solo-dev flow)		
1. main is ALWAYS deployable (CI green + PR-reviewed)		
2. Tag releases: git tag v1.2.0 + GitHub Release notes (auto-generated from commits!)		

Section	Content
3. Deploy = push tag → CD pipeline builds + deploys + smoke-tests (Ch 59)	
4. Rollback = redeploy previous tag (one click, rehearsed!)	
...	
::: interview Walk me through your Git workflow.	
Expected: "Feature branches off main, small PRs with test plans, CI must be green, squash-merge, conventional commits, tagged releases with auto notes. Main is always deployable; I once caught a broken build via CI before it reached staging..." Process + story = hired.	

Quick check — CI fails on lint — you...

1. Merge, fix later
2. Fix, push, wait green
3. Disable lint
4. Force-push main

Answer: B · Red blocks merge. Fix-forward keeps main deployable; "later" becomes production incidents.

Exercise 35 — Green-pipeline portfolio repo

Take any ladder project (Ch 72): add the CI workflow + branch protection (require green PRs!) + README badge + conventional-commit history (rewrite 5 messages properly) + v1.0.0 release with notes. Screenshot the green badge — process proof for interviews.

SOLUTION

Settings → Branches → require status checks; push a deliberately-failing lint commit on a branch → watch CI block the merge → fix → green → squash-merge. You just practiced the exact workflow tech companies use daily.

PART

Back-End — Node.js and Express APIs

JavaScript on the server: build REST APIs that power any front-end.

Ch 39 Node.js — JavaScript on the Server

Ch 40 Express and REST — Professional APIs

Ch 41 Validation, Files and Production Habits

| Same language, new superpowers: files, databases, and thousands of users at once.

CHAPTER 39

Node.js — JavaScript on the Server

Advanced

~ 120 min

The runtime, the event loop, modules, files and npm servers in one chapter.

IN THIS CHAPTER

- Node vs browser JS
- Event loop + non-blocking I/O
- Modules (ESM)
- fs/path/crypto essentials
- Your first HTTP server

Angela \$Node

Elzero Node

MDN server

Node vs browser: same language, new powers

Table 89 — Browser JS vs Node.js

	Browser	Node.js
Globals	window, document, fetch, localStorage	process, Buffer, __dirname-less ESM
Superpowers	DOM + UI events	Files, OS, network servers, thousands of connections
Missing	(everything present for UI)	NO document/window/localStorage!
Modules	<code><script type="module"></code>	ESM in <code>.js</code> + <code>"type": "module"</code>

CODE · Your first server (save server.js, run: node server.js)

```
import http from "node:http";
const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ status: "alive", time: new Date().toISOString() }));
});
server.listen(5000, () => console.log("Listening on http://localhost:5000"));
```

THINK OF IT LIKE THIS — NODE IS A MASTER WAITER

One waiter (thread) takes ALL orders instantly and never stands at the kitchen waiting: chefs (libuv thread-pool) cook async and call back when ready. Blocking the waiter (slow sync loops, sync crypto) freezes the WHOLE restaurant — hence: never block the event loop.

Event loop + non-blocking (the rule behind everything)

CODE · Blocking (evil) vs non-blocking (Node way)

```
import fs from "node:fs";
// EVIL in servers: freezes ALL users while reading!
const data = fs.readFileSync("big.json", "utf8");
// NODE WAY: everyone else keeps being served meanwhile ✓
const text = await fs.promises.readFile("big.json", "utf8");
```

Table 90 — Node async toolkit

Tool	Use
<code>fs.promises</code>	Files without blocking (read/write/append)
<code>path.join/resolve</code>	Paths that work on Windows AND Linux
<code>crypto.randomUUID()</code>	IDs, tokens (never <code>Math.random</code> for security!)
<code>process.env.PORT</code>	Config/secrets from environment (never hardcoded!)

CODE · Everyday Node utilities

```
import fs from "node:fs/promises";
import path from "node:path";
import crypto from "node:crypto";
const file = path.join(process.cwd(), "data", "notes.json");
const notes = JSON.parse(await fs.readFile(file, "utf8").catch(() => "[]"));
notes.push({ id: crypto.randomUUID(), text: "Hello", at: Date.now() });
await fs.writeFile(file, JSON.stringify(notes, null, 2));
const PORT = Number(process.env.PORT || 5000); // env with fallback ✓
```

Quick check — `fs.readFileSync` in a request handler...

1. Fine, it's fast
2. Blocks ALL users until done — use `fs.promises`
3. Only blocks one user
4. Required for JSON

Answer: B · Sync I/O on the event loop freezes every concurrent request. Sync is for startup scripts, never request paths.

Exercise 36 — Uptime JSON API (no framework)

GET /health → `{status:"ok", uptime: seconds}`; GET /time → `{iso}`; 404 JSON for unknown paths. Test with browser + DevTools Network.

SOLUTION

Route on `req.url / req.method`, `writeHead` correct codes (200/404), JSON bodies, `PORT` from env. You just built routing by hand — Express (next chapter) makes this elegant.

CHAPTER 40

Express and REST — Professional APIs

Advanced

~ 140 min

Routing, middleware, validation and error handling: APIs you'd proudly ship.

IN THIS CHAPTER

- REST design (nouns + verbs + codes)
- Express setup + routing
- Middleware pipeline
- Validation + errors
- Notes API project

Angela \$Express

Elzero backend

MDN Express

REST design (nouns, verbs, codes)

Table 91 — REST in one table

Operation	Method + Path	Success	Body
List	GET /api/v1/notes	200	[{...}]
Get one	GET /api/v1/notes/:id	200 / 404	{...} / error
Create	POST /api/v1/notes	201	created {...}
Replace	PUT /api/v1/notes/:id	200 / 404	updated {...}
Patch	PATCH /api/v1/notes/:id	200 / 404	updated {...}
Delete	DELETE /api/v1/notes/:id	204 / 404	(empty!) / error

KEY TAKEAWAY — KEY TAKEAWAY

URLs are NOUNS (`/notes` , never `/getNotes`); methods are VERBS; status codes narrate the outcome. Version from day one (`/api/v1/...`) — v2 will thank you. Error bodies share ONE shape: `{error: {code, message}}` .

Express setup + routing

CODE · Minimal professional API (npm i express cors)

```
import express from "express";
import cors from "cors";
const app = express();
app.use(cors({ origin: "https://myapp.example" })); // front-end origin ONLY
app.use(express.json({ limit: "100kb" })); // parse JSON bodies (capped!)

let notes = [{ id: "1", text: "Learn REST", done: false }];

app.get("/api/v1/notes", (req, res) => res.json({ data: notes }));
app.get("/api/v1/notes/:id", (req, res, next) => {
  const n = notes.find(x => x.id === req.params.id);
  if (!n) return next({ status: 404, message: "Note not found" });
  res.json({ data: n });
});
app.post("/api/v1/notes", (req, res, next) => {
  const text = req.body?.text?.trim();
  if (!text) return next({ status: 400, message: "text is required" });
  const note = { id: crypto.randomUUID(), text, done: false };
  notes.unshift(note);
  res.status(201).json({ data: note });
});
```

Middleware: the assembly line

request → cors → express.json → logger → auth? → ROUTE → response

Any step may: pass (next()), answer early (res.json), or fail (next(err)).

ORDER MATTERS: app.use(auth) AFTER public routes protects only later ones!

Figure 18 — Request flows through middleware in order

CODE · Custom middleware (logger + auth guard + 404 + error handler)

```
// Logger (first, sees everything)
app.use((req, res, next) => {
  console.log(new Date().toISOString(), req.method, req.url);
  next();
});

// Auth guard (use per-route: app.post("/x", requireAuth, handler))
function requireAuth(req, res, next) {
  if (req.headers.authorization !== `Bearer ${process.env.API_TOKEN}`)
    return next({ status: 401, message: "Unauthorized" });
  next();
}

// 404 for unknown API paths (AFTER all routes!)
app.use("/api", (req, res) => res.status(404).json({ error: { message: "Unknown endpoint" } }));

// Central error handler (4 args = Express knows; LAST!)
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: { message: err.status ? err.message : "Server error" } });
});

const PORT = Number(process.env.PORT || 5000);
app.listen(PORT, () => console.log(`API on :${PORT}`));
```

WARNING — MIDDLEWARE ORDER BUGS

`express.json()` AFTER routes = `req.body` undefined everywhere. Error handler NOT last = crashes bypass it. CORS misconfigured = browser blocks your own front-end. When Express "ignores" things, read your `app.use` order top-to-bottom.

INTERVIEW QUESTION — HOW DO YOU STRUCTURE AN EXPRESS API?

Expected: "Thin routes, fat services: routes validate + call services; services hold logic; a data layer talks to the DB. Central error middleware with one error shape, request validation on every input, auth middleware on protected routes, and versioned paths."
Bonus: rate limiting + helmet (Ch 45).

Quick check — Central error middleware signature?

1. (req,res)
2. (err,req,res,next)
3. (res,err)
4. (next)

Answer: B · Four arguments mark it as error-handling; Express routes `next(err)` here.

Placement: after all routes.

Exercise 37 — Complete the Notes API

Add PATCH (toggle/edit with validation), DELETE (204), `?q=` search on list, and `GET /api/v1/notes/count`. Test every route + every error (400/404) with fetch from the Ch 33 helper.

SOLUTION

PATCH: find → validate fields → merge → 200/404. DELETE: filter → 204/404 (204 = no body!). Search: `notes.filter(t => t.text.toLowerCase().includes(q))`. Count: `{data: {count}}`. Front-end from Ch 33 can consume it unchanged — that's the API contract working.

CHAPTER 41

Validation, Files and Production Habits

Advanced

~ 100 min

Trust no input, handle uploads safely, and run servers that survive reality.

IN THIS CHAPTER

- Validate everything (validator pattern)
- Pagination + filtering
- File uploads without disasters
- Logging + env config
- Rate limiting basics

[Angela SBackend](#)[MDN forms-server](#)[Elzero backend](#)

Validate everything (the validator pattern)

CODE · Small validators, composed (no library needed to learn)

```
const isStr = (v, min = 1, max = 500) => typeof v === "string" && v.trim().length >= min
&& v.trim().length <= max;
const isId = v => typeof v === "string" && v.length >= 8;
function validateNote(body) {
  const errors = {};
  if (!isStr(body?.text, 1, 500)) errors.text = "text required (1-500 chars)";
  if (body.done !== undefined && typeof body.done !== "boolean") errors.done = "done must
be boolean";
  return errors;
}
app.post("/api/v1/notes", (req, res) => {
  const errors = validateNote(req.body);
  if (Object.keys(errors).length) return res.status(400).json({ error: { message:
"Invalid data", errors } });
  /* ...create... */
});
```

KEY TAKEAWAY — KEY TAKEAWAY

Validate SHAPE (types/lengths/ranges) on every input — params, query, body. Return ALL errors at once (`{errors}`), not one-by-one (users hate fix-one-discover-next). Sanitize for storage, escape for display (XSS defense, Ch 45).

Pagination (never return 100,000 rows)

CODE · Offset pagination with sane caps

```
app.get("/api/v1/notes", (req, res) => {
  const page = Math.max(1, Number(req.query.page) || 1);
  const limit = Math.min(100, Math.max(1, Number(req.query.limit) || 20));
  const q = (req.query.q || "").toLowerCase();
  const filtered = notes.filter(n => n.text.toLowerCase().includes(q));
  const start = (page - 1) * limit;
  res.json({ data: filtered.slice(start, start + limit),
    meta: { page, limit, total: filtered.length, pages: Math.ceil(filtered.length /
limit) } });
});
```

Uploads without disasters

Table 92 — File upload safety rules

Rule	Why
Cap size (<code>limits.fileSize</code>)	100MB "avatars" kill servers
Allowlist mime + extension	<code>image/*</code> only — never trust client headers alone
Rename files (uuid)	<code>../../../../etc/passwd</code> names = path traversal attacks
Store OUTSIDE web root / use object storage	Uploaded <code>.html/.svg</code> can run XSS
Scan/resize images server-side	Strips malicious metadata, saves bandwidth

CODE · Safe upload sketch (npm i multer)

```
import multer from "multer";
const upload = multer({ dest: "uploads/",
  limits: { fileSize: 2 * 1024 * 1024 }, // 2MB cap
  fileFilter: (req, f, cb) => cb(null, f.mimetype.startsWith("image/")) });
app.post("/api/v1/avatar", upload.single("avatar"), (req, res) => {
  res.status(201).json({ data: { file: req.file.filename } });
});
```

Production habits checklist

SHIP-READY SERVER REVIEW

- - `helmet` headers + `cors` locked to your domains + rate limit on auth routes
- - JSON body cap; ALL inputs validated; central error shape; no stack traces to clients
- - Secrets ONLY in env/host dashboard; `.env` gitignored; logs without passwords
- - Health route (`/health`); graceful PORT from env; process manager (PM2/systemd) + restarts
- - Backups for file/DB data; dependency audits (`npm audit`) before releases

Exercise 38 — Harden the Notes API

Add helmet + rate limit (100/15min) + validation errors object + pagination + `/health`. Attack your own API: huge bodies, bad JSON, wrong types — every abuse gets a clean 4xx, never a crash or stack trace.

SOLUTION

`npm i helmet express-rate-limit`; mount globally; verify oversized POST → 413/400 JSON, bad types → 400 `{errors}`, unknown route → 404 shape, `/health` → 200. Abuse-driven testing is how seniors prove readiness.

PART

Databases — Remembering Everything

SQL structure vs NoSQL freedom: store data correctly for each job.

Ch 42 SQL and PostgreSQL — Structured Data Done Right

Ch 43 MongoDB and Mongoose — Flexible Document Data

| Choose the database that fits the data — not the hype of the month.

CHAPTER 42

SQL and PostgreSQL — Structured Data Done Right

Advanced

~ 130 min

Tables, keys, queries, joins and safe access from Node.

IN THIS CHAPTER

- Tables, rows, keys
- CREATE + constraints
- SELECT/WHERE/ORDER/LIMIT
- JOINS visualized
- Parameterized queries (no injection!)

Angela \$SQL

Elzero database

Tutorialspoint SQL

Tables, rows, keys (the spreadsheet that enforces rules)

```
users: id (PK) | email (UNIQUE, NOT NULL) | name | created_at
      1      | sara@x.com          | Sara | 2026-01-02
      2      | omar@x.com          | Omar | 2026-03-11
PK = Primary Key: unique ID per row. FK = Foreign Key: points at another table's PK.
```

Figure 19 — A users table (columns = schema, rows = data)

CODE · Create tables with real constraints

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  total INT NOT NULL CHECK (total >= 0),
  status TEXT NOT NULL DEFAULT 'new'
);
CREATE INDEX idx_orders_user ON orders(user_id); -- JOINS stay fast
```

Table 93 — Constraints (rules the DB enforces FOR you)

Constraint	Job
PRIMARY KEY	Unique row ID (indexed automatically)
FOREIGN KEY ... REFERENCES	Must point at a real row (no orphans!)
UNIQUE	No duplicates (emails, usernames)
NOT NULL / DEFAULT / CHECK	Required values, fallbacks, valid ranges

Queries: the daily 90%

CODE · SELECT power tour

```
SELECT id, email FROM users WHERE email ILIKE '%@x.com' ORDER BY id DESC LIMIT 20;
SELECT status, COUNT(*), SUM(total) FROM orders GROUP BY status HAVING COUNT(*) > 5;
INSERT INTO users(email, name) VALUES ('a@x.com', 'A') RETURNING id;
UPDATE orders SET status = 'paid' WHERE id = $1;
DELETE FROM orders WHERE status = 'abandoned' AND created_at < now() - interval '30 days';
```

Table 94 — Query clauses in order

Clause	Job	Example
WHERE	Filter rows	WHERE total > 1000

Clause	Job	Example
ORDER BY	Sort (<code>DESC</code> = biggest first)	<code>ORDER BY created_at DESC</code>
LIMIT/OFFSET	Page results	<code>LIMIT 20 OFFSET 40</code> (page 3)
GROUP BY + aggregates	Summarize groups	<code>COUNT(*)</code> , <code>SUM(total)</code> , <code>AVG(total)</code>
HAVING	Filter GROUPS (WHERE can't)	<code>HAVING COUNT(*) > 5</code>

JOINS visualized (the fear-killer)

INNER: only matching pairs (users WITH orders) [overlap only]
 LEFT: ALL users + their orders or NULL (find inactive!) .. [left circle all]
 salaries per user: LEFT JOIN + GROUP BY user.

Figure 20 — JOINS as Venn diagrams (users ↔ orders on user_id)

CODE · JOINS you will actually write

```
-- Every user with their order count (including 0-order users!)
SELECT u.name, COUNT(o.id) AS orders, COALESCE(SUM(o.total), 0) AS spent
FROM users u LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name ORDER BY spent DESC;
```

Safe Node access (parameters, never strings!)

CODE · node-postgres with \$1 parameters (injection-proof)

```
import pg from "pg";
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
const byEmail = async email => {
  const { rows } = await pool.query("SELECT * FROM users WHERE email = $1", [email]);
  return rows[0] || null; // $1 = value sent SEPARATELY – never in the SQL text ✓
};
// NEVER: `... WHERE email = '${email}'` – `` OR '1'='1' steals your whole table! (Ch 45)
```

KEY TAKEAWAY — KEY TAKEAWAY

SQL + constraints = data that CANNOT be invalid (uniques, FKs, checks enforced by the engine). Query with JOINS/GROUP BY for answers, paginate everything, and pass values as PARAMETERS — string-built SQL is how databases get stolen.

Quick check — `LEFT JOIN users→orders` shows...

1. Only users with orders
2. ALL users, NULLs where no orders
3. Only orders
4. Duplicates

Answer: B • **LEFT keeps every left row; unmatched rights become NULL — perfect for "inactive users" reports.**

Exercise 39 — Mini shop reports

Schema: products(id, name, price), sales(id, product_id FK, qty, at). Queries: (1) revenue per product, (2) unsold products, (3) top day by revenue. All parameterized from Node.

SOLUTION

(1) `LEFT JOIN + SUM(price*qty) GROUP BY product`. (2) same + `HAVING COUNT(sales.id)=0` (or `WHERE sales.id IS NULL`). (3) `GROUP BY date_trunc('day', at) ORDER BY SUM DESC LIMIT 1`.
Reports = JOIN + GROUP + aggregates, always.

CHAPTER 43

MongoDB and Mongoose — Flexible Document Data

Advanced

~ 110 min

Collections, schemas, queries, relations and when NoSQL wins.

IN THIS CHAPTER

- Documents vs rows
- Mongoose schemas + validation
- CRUD + queries
- Embed vs reference
- SQL vs NoSQL decision guide

Angela \$MongoDB

Elzero mongo

Tutorialspoint Mongo

Documents: JSON that persists

SQL posts | comments | tags + joins to assemble ONE page
MONGO .. { title, body, tags: [...], comments: [{by, text, at}] } – page-ready!
Rule of thumb: read-together → store-together (embed); independent lifecycles → reference.

Figure 21 — Same blog post: SQL (3 tables + JOINS) vs Mongo (1 document)

CODE · Mongoose schema = flexible structure + validation (npm i mongoose)

```
import mongoose from "mongoose";
const postSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true, maxlength: 140 },
  slug: { type: String, required: true, unique: true, index: true },
  tags: { type: [String], default: [] },
  author: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  comments: [{ by: String, text: { type: String, maxlength: 1000 }, at: { type: Date,
default: Date.now } } ]
}, { timestamps: true }); // createdAt + updatedAt free ✓
export const Post = mongoose.model("Post", postSchema);
```

CRUD + queries

CODE · Daily Mongoose operations

```
await Post.create({ title: "Hi", slug: "hi", author: userId });
await Post.find({ tags: "js" }).sort({ createdAt: -1 }).limit(20).populate("author", "name");
await Post.updateOne({ slug }, { $set: { title } });
await Post.deleteOne({ slug });
// Operators: { price: { $gte: 100, $lt: 500 } }, { $or: [{a}, {b}] }
```

Table 95 — Mongo operators (the working set)

Operator	Job
<code>\$gte \$lte \$in</code>	Ranges and lists
<code>\$or \$and</code>	Logic combos
<code>\$set \$inc \$push \$pull</code>	Surgical updates (no full rewrite)
<code>text index</code>	<code>\$text: {\$search}</code> full-text search

Embed vs reference (the design decision)

Table 96 — Embed or reference?

Embed (nest inside)	Reference (ObjectId + populate)
Read together always (post + comments)	Independent life (users, products)
Bounded growth (dozens, not millions)	Unbounded growth (all orders ever)
No cross-parent queries	Queried across parents

CODE · Reference + populate (order points at user + products)

```
const orderSchema = new mongoose.Schema({
  user: { type: ObjectId, ref: "User", required: true, index: true },
  items: [{ product: { type: ObjectId, ref: "Product" }, qty: { type: Number, min: 1 } }],
  total: { type: Number, min: 0 }
});
await Order.findById(id).populate("user", "name email").populate("items.product", "name price");
```

SQL vs NoSQL (the honest guide)

Table 97 — Decision matrix

Need	Winner	Why
Money, inventory (must never corrupt)	PostgreSQL	ACID transactions + constraints enforce truth
Flexible content, fast iteration	MongoDB	Schema evolves without migrations
Complex relations + reports	PostgreSQL	JOINS + GROUP BY are unmatched
Huge write streams, simple reads	MongoDB	Scales horizontally with ease

INTERVIEW QUESTION — SQL OR NOSQL FOR AN E-COMMERCE SITE?

Expected: "Both, split by job: PostgreSQL for orders/payments/inventory where consistency is money; MongoDB (or similar) for catalogs/reviews/activity feeds where shape varies and reads dominate. Decide per BOUNDED CONTEXT, not per company." Dogmatism either way = junior signal.

Exercise 40 — Catalog + orders hybrid

Design: Mongo Product (variants embedded, reviews embedded capped at 50 + overflow note) + SQL orders (FK to users, totals CHECK ≥ 0). Justify each choice in 3 sentences + write both schemas.

SOLUTION

Products vary wildly (sizes/colors/specs) and render as one page → embed in Mongo. Orders move money and join users/reports → SQL constraints + transactions. Overflow: keep latest 50 reviews embedded + `reviewCount` + separate query for full history (bounded growth rule).

PART

Authentication and Security — Trust, Done Right

Logins that hold up, and defenses against the attacks that never stop.

Ch 44 Authentication — Passwords, Sessions and JWT

Ch 45 Web Security — Attacks and Defenses

| Security is not a feature — it is the floor everything stands on.

CHAPTER 44

Authentication — Passwords, Sessions and JWT

Advanced

~ 130 min

Register/login flows, bcrypt hashing, sessions vs tokens, and protected routes.

IN THIS CHAPTER

- Auth vs authz (know the difference!)
- bcrypt hashing + salts
- Sessions (cookies) vs JWT
- Access + refresh rotation
- Protecting Express routes

Angela \$Auth

Elzero auth

MDN security

Authentication vs authorization (say it right)

Table 98 — The two words interviews test

Word	Question	Example
Authentication (authN)	WHO are you?	Login with email + password
Authorization (authZ)	WHAT may you do?	Only admins delete users

CODE · The golden auth rules (memorize all five)

1. NEVER store plain passwords – bcrypt hashes ONLY (cost factor 12)
2. NEVER roll your own crypto – use bcrypt + jsonwebtoken + platform TLS
3. NEVER put secrets in JWT payloads – payloads are READABLE (signed, not encrypted!)
4. Tokens in HttpOnly cookies (XSS-proof) – NOT localStorage (XSS-readable!)
5. ALWAYS HTTPS in production – tokens over HTTP are postcards, not letters

Passwords with bcrypt (the only way)

CODE · Register + login (npm i bcrypt jsonwebtoken)

```
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";

// REGISTER: hash, never store plain ✓
const hash = await bcrypt.hash(password, 12); // 12 = cost factor (~250ms)
await User.create({ email, passwordHash: hash });

// LOGIN: compare (timing-safe) — same message either way (no user enumeration!)
const user = await User.findOne({ email });
const ok = user && await bcrypt.compare(password, user.passwordHash);
if (!ok) return res.status(401).json({ error: { message: "Invalid email or password" } });
```

THINK OF IT LIKE THIS — HASHING IS A MEAT GRINDER (ONE-WAY)

Beef (password) → mince (hash) is easy; mince → beef is impossible. Salts (bcrypt auto-adds) mean identical passwords get DIFFERENT hashes — rainbow tables (pre-ground beef dictionaries) become useless. Cost factor = grinder slowness: slow for attackers, fine for logins.

Sessions vs JWT (the architecture choice)

Table 99 — Sessions (stateful) vs JWT (stateless)

	Sessions + HttpOnly cookie	JWT access + refresh
Server memory	Stores session IDs (Redis/DB)	Stores nothing (verifies signature)
Revoke	Instant (delete session)	Needs denylist/short expiry
Scale	Shared store required	Any server verifies (secret/public key)
Best for	Monoliths, instant-revoke needs	APIs, mobile, microservices

CODE · JWT pair + rotation (15-min access, 7-day refresh in HttpOnly cookie)

```
const access = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, {
  expiresIn: "15m" });
const refresh = jwt.sign({ sub: user.id, v: user.tokenVersion },
  process.env.REFRESH_SECRET, { expiresIn: "7d" });
res.cookie("refresh", refresh, { httpOnly: true, secure: true, sameSite: "strict",
  maxAge: 7*864e5 });
res.json({ data: { access } }); // access in memory (JS var), refresh in cookie ✓

// Logout EVERYWHERE: bump tokenVersion → all refresh tokens die instantly
```

CODE · Protecting routes (middleware reads access token)

```
function requireAuth(req, res, next) {
  const token = req.headers.authorization?.replace("Bearer ", "");
  if (!token) return next({ status: 401, message: "Login required" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // throws if bad/expired
    next();
  } catch { next({ status: 401, message: "Session expired – please log in" }); }
}

const requireRole = role => (req, res, next) =>
  req.user?.role === role ? next() : next({ status: 403, message: "Admins only" });
app.delete("/api/v1/users/:id", requireAuth, requireRole("admin"), handler);
```

WARNING — JWT PAYLOAD IS PUBLIC

Anyone can base64-decode `{sub, role}` — it proves AUTHENTICITY (signature), not secrecy. Never put passwords, emails-you-hide, or internals inside. Secrets stay server-side, referenced by ID.

INTERVIEW QUESTION — WHERE DO YOU STORE JWTS AND WHY?

Expected: "Access token in memory, refresh token in HttpOnly + Secure + SameSite cookie. Never localStorage — any XSS steals it. Short access expiry limits damage; refresh rotation + token versioning enables instant global logout." Mention CSRF+SameSite interplay (next chapter) = senior.

Quick check — Same login error for bad email AND bad password because...

1. Laziness
2. Prevents user enumeration (attackers can't harvest registered emails)
3. Faster
4. Required by JWT

Answer: B · Distinct messages ("no such user" vs "wrong password") let attackers probe your user list. Uniform 401s close the oracle.

Exercise 41 — Full auth slice

Register (validate + unique email + bcrypt) → login (uniform 401) → `/me` (requireAuth) → admin-only DELETE (requireRole) → logout-all (tokenVersion bump). Test: expired access (401), user hitting admin route (403), refresh after logout-all (dead).

SOLUTION

Covered above end-to-end; add tests for each code path. Verify cookies flags in DevTools Application tab (HttpOnly ✓ Secure ✓ SameSite=Strict ✓). Auth done = boring, uniform, and strict.

CHAPTER 45

Web Security — Attacks and Defenses

Advanced

~ 120 min

XSS, CSRF, injection, and the hardening checklist every API needs.

IN THIS CHAPTER

- Threat mindset (trust boundaries)
- XSS: stored/reflected/DOM + CSP
- CSRF + SameSite
- SQL/NoSQL injection
- Security headers + rate limits + checklist

Angela §Security

MDN security

OWASP Top 10

Think like an attacker (trust boundaries)

```
[browser] --input--> [API] --query--> [DB] --data--> [browser renders]
      ^               ^               ^               ^
      XSS here       validate here   inject here    escape here
RULE: validate at ENTRY, parameterize at QUERY, escape at RENDER. Every layer, every time.
```

Figure 22 — Every arrow crossing is an attack surface

XSS: your site running attacker code

Table 100 — XSS flavors

Type	How	Example
Stored	Malicious post saved, runs for every reader	Comment with <code></code>
Reflected	Payload in URL, server echoes it	Search page echoing <code>?q=<script>...</code>
DOM	Client JS sinks user data into HTML	<code>el.innerHTML = location.hash</code>

CODE • Defenses (layered — use ALL)

```
// 1. Render safely: textContent/createElement for user data (Ch 29!) ✓
// 2. Sanitize rich HTML server-side (allowlist tags/attrs) if HTML input is required
// 3. Content-Security-Policy header: only YOUR scripts run (inline attacker code dies)
app.use(helmet.contentSecurityPolicy({
  directives: { defaultSrc: ["'self'"], scriptSrc: ["'self'"], imgSrc: ["'self'",
"data:"] }
}));
// 4. HttpOnly cookies: stolen XSS still can't read tokens (Ch 44!) ✓
```

CSRF: hijacking the logged-in browser

Victim logged into `bank.com` visits `evil.com`, whose hidden form POSTs to `bank.com/transfer` — the browser ATTACHES cookies automatically. Defense layers:

Table 101 — CSRF defenses

Defense	How
<code>SameSite=Strict/Lax</code> cookies	Browser withholds cookies on cross-site requests (primary defense!)
CSRF tokens	Server-issued per-form secret, verified on mutation
Custom headers + CORS	APIs requiring <code>Authorization</code> /JSON preflight block simple forged forms
GET never mutates	Transfers/deletes are POST/DELETE (browsers prefetch GETs!)

Injection: queries built from strings

CODE · NEVER build queries from strings (both databases!)

```
// SQL — parameterized ✓ vs string ✗:
pool.query("SELECT * FROM users WHERE email = $1", [email]); // ✓ safe
// `... WHERE email = '${email}'` // ✗ ' OR '1'='1 steals all rows!

// Mongo — operators from user input are poison:
User.find({ email: req.body.email }); // body {email: {"$ne": null}} matches ALL!
const email = String(req.body.email || ""); // force scalar ✓
User.find({ email }); // safe
```

Table 102 — Injection family (same disease, many organs)

Target	Poison	Cure
SQL	' OR '1'='1	Parameterized queries (\$1, ?)
NoSQL	{"\$ne": null} objects	Type-check scalars; schema validation
Commands	; rm -rf in filenames	Never shell out with input; allowlist chars
Paths	../../etc/passwd	basename + allowlist + uuid renames (Ch 41)
HTML/JS	<script>, onerror=	Escape output + CSP + textContent

Hardening stack (copy this)

CODE · Production security middleware (npm i helmet express-rate-limit)

```
import helmet from "helmet";
import rateLimit from "express-rate-limit";
app.use(helmet()); // secure headers: HSTS, noSniff, frameguard, CSP...
app.use("/api/v1/auth", rateLimit({ windowMs: 15*60e3, max: 20 })); // brute-force killer
app.use("/api", rateLimit({ windowMs: 15*60e3, max: 300 }));
app.use(express.json({ limit: "100kb" })); // giant-body killer
```

SECURITY SHIP CHECKLIST (OWASP-FLAVORED)

- - bcrypt(12) passwords; uniform login errors; HTTPS everywhere; HttpOnly+Secure+SameSite cookies
- - All inputs validated; all queries parameterized; all user HTML escaped/sanitized + CSP
- - Auth on every private route; role checks server-side (never trust client claims!)
- - Rate limits on auth + API; body caps; security headers via helmet
- - Secrets in env only; `npm audit` clean; errors never leak stacks; logs never include secrets/tokens
- - Backups tested (restore rehearsal!) + dependency updates scheduled

INTERVIEW QUESTION — YOUR TOP 3 WEB VULNERABILITIES?

Expected: "Injection — parameterized queries and input validation; XSS — safe rendering plus CSP plus HttpOnly cookies; broken auth — bcrypt, short-lived tokens, rotation, and rate-limited login. Then CSRF via SameSite + tokens, and security headers/rate limiting as baseline hygiene." Structured + layered = hired.

Exercise 42 — Red-team your API

Attack your Ch 40–41 API: XSS comment, `' OR 1=1` login, `{"$ne":null}` body, 50MB upload, 500 rapid logins, forged admin role in JWT. Every attack must fail CLOSED (4xx, no data, no crash) — document each fix.

SOLUTION

Expected results: CSP+escaping neuters XSS; parameters neutralize injection; type checks kill operator objects; caps reject giants (413); rate limit 429s the flood; signature verification rejects forged roles (401). Failed-closed everywhere = shippable.

PART

React — Modern Front-End

Components, state and hooks: build UIs the industry way.

Ch 46 React — Components, State and Hooks

React is JavaScript with superpowers — your Part 4 mastery pays off here.

CHAPTER 46

React — Components, State and Hooks

Advanced

~ 150 min

Thinking in components, managing state, fetching data and routing — a complete React slice.

IN THIS CHAPTER

- Components + props + JSX rules
- useState + events
- Lists, keys, conditional UI
- useEffect + data fetching
- useRef, routing idea + mini project

Gamal React

Angela SReact

MDN React

Thinking in components (Lego UI)

```
<App> (state lives high, flows down)
|-- <Header cartCount={3} />
|-- <ProductList> -- <ProductCard /> x N (props in, events up)
+-- <Footer />
RULES: one component = one job; data flows DOWN via props; events bubble UP via callbacks.
```

Figure 23 — One page = a tree of reusable pieces

CODE · Your first components (Vite: `npm create vite@latest shop -- --template react`)

```
function Price({ value }) { // props = inputs (read-only!)
  return <strong className="price">${value}</strong>;
}
function ProductCard({ product, onAdd }) {
  return (
    <article className="card">
      <h3>{product.name}</h3>
      <Price value={product.price} />
      <button onClick={() => onAdd(product.id)}>Add to cart</button>
    </article>
  );
}
```

Table 103 — JSX rules (HTML with 3 twists)

Rule	Example	Why
<code>className</code> not <code>class</code>	<code><div className="x"></code>	class is a JS keyword
<code>{}</code> injects JS	<code><h1>{user.name}</h1></code>	Any expression: vars, maps, ternaries
One parent	<code><>...</></code> fragments	Components return ONE root
<code>onClick={fn}</code>	camelCase + function ref	NOT strings: <code>onClick={() => add(id)}</code>
<code>key</code> on lists	<code>key={p.id}</code>	Stable identity for updates (NEVER index if reorderable!)

State: useState + events

CODE · State = data that re-renders when changed

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0); // [value, setter]
  return <button onClick={() => setCount(c => c + 1)}>Clicked {count}</button>;
}
// ARRAY/OBJECT state: replace, never mutate! (Ch 28 immutability pays off)
const [cart, setCart] = useState([]);
const add = id => setCart(prev => [...prev, id]);
const remove = id => setCart(prev => prev.filter(x => x !== id));
```

WARNING — STATE MUTATION = SILENT UI DEATH

`cart.push(id); setCart(cart)` keeps the SAME array reference — React sees "nothing changed" and skips rendering. Always create NEW arrays/objects (`[...prev]`, `filter`, `map`). The #1 React beginner bug, inherited straight from Ch 24's references lesson.

Lists, keys, conditional UI

CODE · The three rendering patterns

```
function ProductList({ items, loading, error, onAdd }) {
  if (loading) return <p>Loading...</p>; // 1. early returns per state (Ch 33!)
  if (error) return <p>Failed: {error} <button>Retry</button></p>;
  if (!items.length) return <p>No products yet.</p>; // 2. empty state ✓
  return (
    <ul>{items.map(p => (
      <li key={p.id}><ProductCard product={p} onAdd={onAdd} /></li> // 3. key = stable id
    ))}</ul>
  );
}
// Inline conditions: {isAdmin && <AdminPanel />} ... {count > 0 ? <Cart/> : <Empty/>}
```

Effects: useEffect + data fetching

CODE · Fetch on mount, cleanup on unmount (Ch 32-33 patterns, React-flavored)

```
import { useState, useEffect } from "react";
function useProducts() {
  const [data, setData] = useState([]);
  const [status, setStatus] = useState("loading");
  useEffect(() => {
    const ctrl = new AbortController(); // cleanup cancels flight ✓
    (async () => {
      try {
        const res = await fetch("https://dummyjson.com/products?limit=12", { signal: ctrl.signal });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setData((await res.json()).products); setStatus("ready");
      } catch (e) { if (e.name !== "AbortError") setStatus("error"); }
    })();
    return () => ctrl.abort(); // unmount/StrictMode safety
  }, []); // [] = run ONCE
  return { data, status };
}
```

Table 104 — useEffect dependency array

Array	Runs	Classic bug
<code>[]</code>	Once on mount	Forgetting needed deps (stale data)
<code>[id]</code>	On mount + when id changes	Missing dep = stale closure!
(no array)	After EVERY render	Accidental infinite fetch loops

useRef + routing idea + cart project

CODE · Refs hold non-render values (timers, DOM nodes, previous values)

```
import { useRef, useEffect } from "react";
function Search({ onSearch }) {
  const inputRef = useRef(null);
  useEffect(() => inputRef.current.focus(), []); // autofocus on mount
  return <input ref={inputRef} onChange={e => onSearch(e.target.value)} />;
}
// Routing (npm i react-router-dom): URL ↔ component map
// <Routes><Route path="/" element={<Shop/>} /><Route path="/cart" element={<Cart/>} /></Routes>
```

CODE · Mini project — shop with cart (state lifted to App)

```
function App() {
  const { data: products, status } = useProducts();
  const [cart, setCart] = useState([]);
  const add = id => setCart(p => [...p, id]);
  const total = cart.reduce((s, id) => s + (products.find(p => p.id === id)?.price || 0), 0);
  return (<
    <Header count={cart.length} total={total} />
    <ProductList items={products} loading={status === "loading"}
      error={status === "error" ? "Load failed" : ""} onAdd={add} />
  </>);
}
```

KEY TAKEAWAY — KEY TAKEAWAY

React = components (UI Lego) + props (down) + state (owned data) + effects (outside world). Lift shared state UP to the lowest common parent; derive (totals, counts) instead of storing; fetch in effects with cleanup. Everything else is practice.

INTERVIEW QUESTION — CONTROLLED VS UNCONTROLLED INPUTS?

Expected: "Controlled: value + onChange in state — React owns the truth (validation, disable logic, instant feedback). Uncontrolled: refs read DOM on demand — simpler for one-shot forms. Default to controlled; reach for refs for focus/file inputs." Then note keys must be stable IDs.

Exercise 43 — Reactify the expense tracker

Rebuild Ch 35's tracker in React: `useState` items + `localStorage` effect (load once, save on change) + derived totals + category bars. Same architecture triangle — compare line counts with the vanilla version.

SOLUTION

```
const [items, setItems] = useState(() => load("exp", [])); useEffect(() => save("exp", items), [items]);
```

— persistence becomes 2 lines. Totals via `reduce` during render (derived, never stored).

~40% less DOM code than vanilla: React owns rendering, you own data.

Beyond `useState`: Zustand + Redux Toolkit (global state grows up)

Context broadcasts to every consumer on every change — fine for themes, painful for carts updated 60×/second. Dedicated stores update **ONLY** the components that read the changed slice.

CODE · Zustand (npm i zustand) — global state in 10 lines

```
import { create } from "zustand";
export const useCart = create(set => ({
  items: [],
  add: id => set(s => ({ items: [...s.items, id] })),
  clear: () => set({ items: [] })
}));
// Any component, zero providers:
const items = useCart(s => s.items); // re-renders ONLY when items change ✓
const add = useCart(s => s.add);
```

Table 105 — `useState` vs Context vs Zustand vs Redux Toolkit

Need	Pick	Why
Local (form, toggle)	<code>useState/</code> <code>useReducer</code>	Simplest, co-located
Slow + global (theme, user)	Context	Built-in, no deps
Fast + global (cart, player)	Zustand	

Need	Pick	Why
		Sliced subscriptions, tiny API
Huge teams + time-travel	Redux Toolkit	DevTools, conventions, middleware
<pre> ```jsx:Redux Toolkit slice (the 80% pattern: createSlice + configureStore) // store/cartSlice.js import { createSlice } from "@reduxjs/toolkit"; const slice = createSlice({ name: "cart", initialState: [], reducers: { add: (s, a) => { s.push(a.payload); }, clear: () => [] } }); export const { add, clear } = slice.actions; export default slice.reducer; // Component: const items = useSelector(s => s.cart); const d = useDispatch(); d(add(id)); ``` </pre>		
::: interview Context vs Redux? Expected: "Context is a broadcast pipe, not a state manager — every update re-renders all consumers. I use Context for slow global state, Zustand for fast slices, Redux Toolkit when the team needs DevTools conventions and middleware. I can show the re-render difference in DevTools Profiler." Trade-offs + proof = senior.		

React performance checklist (measure, then memo)

THE 7-STEP REACT SPEED PASS (IN ORDER!)

- - Profiler FIRST: record the interaction, fix the widest bars (no blind memo!)
- - Keys stable (ids!) + lists windowed (virtualize 500+ rows)
- - State pushed DOWN (a typing input shouldn't re-render the page!)
- - Expensive derived data in useMemo; callbacks in useCallback ONLY when passed to memo'd children
- - Routes lazy (`lazy(() => import())` + Suspense) — ship 30KB, not 300KB
- - Images: next/image or sized AVIF + lazy (Ch 61!)
- - Context split (Auth/Cart/Theme separate) + values memoized

CODE · Lazy routes (the biggest single win, 3 lines)

```
import { lazy, Suspense } from "react";
const Admin = lazy(() => import("./Admin.js"));
<Suspense fallback=<p>Loading...</p>><Admin /></Suspense>
```

Exercise 44 — Profile + fix the shop

Record adding-to-cart in React DevTools Profiler: find wasted renders, push state down, split the cart context, lazy-load /admin, and re-record. Target: interaction <100ms + bundle -40%. Screenshot both flame graphs — performance storytelling (Ch 61 + Ch 73 interviews).

SOLUTION

Typical kills: search input re-rendering 500 cards (push query state into the list component + memo rows), monolith context (split!), admin in main bundle (lazy → -120KB). Before/after flame graphs side by side = interview gold.

PART

Capstone — Ship, Show, Get Hired

Turn knowledge into proof: portfolio projects, interviews and your next year.

Ch 47 Capstone — Portfolio, Deployment and Interviews

| Finished is better than perfect — shipped is better than finished.

CHAPTER 47

Capstone — Portfolio, Deployment and Interviews

All levels

~ 120 min

Three hireable projects, full-stack deployment, CV/portfolio polish and interview mastery.

IN THIS CHAPTER

- The 3 projects that get interviews
- Full-stack deploy (front + API + DB)
- Portfolio + CV that recruiters read
- Technical interview playbook
- Your next 12 months

Capstone

Career

Angela \$Deploy

The 3 projects that get interviews

Table 106 — One project per proof (quality > quantity, always)

Project	Proves	Must include
1. Responsive business site (HTML/CSS/JS)	Design + responsive + a11y	Lighthouse 95+, 360px perfect, contact form, deployed
2. API-powered app (JS + public API)	Async + state + UX	4 states, search/debounce, localStorage, error-proof
3. Full-stack app (React + Express + DB + auth)	End-to-end ownership	Login, CRUD, validation, secured + deployed

KEY TAKEAWAY — KEY TAKEAWAY

Recruiters spend ~30 seconds per portfolio: LIVE link first, 3 GIFs/screenshots, 1-line "what + stack" each, GitHub with clean commits. No live link = no proof = no interview. Deploy everything (Ch 37 + below).

Full-stack deployment map

```
[Vercel/Netlify: React front] --HTTPS--> [Render/Railway: Express API]
      |                                     |-- [Neon/Supabase: PostgreSQL]
      +-- env: VITE_API_URL                 +-- env: DATABASE_URL, JWT_SECRET (dashboard!)
      CORS locked to front domain · DB backups ON · logs checked after EVERY deploy.
```

Figure 24 — Production topology (free-tier friendly)

CODE · Deploy sequence (do it deliberately)

1. API first: push → host builds (npm ci) → set env vars → verify /health + 1 route
2. Database: create → migrate schema → verify connection string works from API
3. Front: set VITE_API_URL to live API → deploy → click EVERYTHING on phone+desktop
4. Harden: CORS origins, rate limits, helmet, error shapes, backup schedule
5. Document: README (setup, env list WITHOUT values, scripts, architecture diagram)

CV + portfolio that recruiters read

Table 107 — CV rules (one page, junior)

Do	Don't
"Built X serving Y, tech Z" bullets with NUMBERS	"Responsible for various tasks"
3 links: portfolio, GitHub, LinkedIn	4+ pages, photos, tables, graphics
Skills grouped: Languages / Front / Back / DB / Tools	40-skill keyword soup
English + Arabic versions (Egypt market)	Typos (run spellcheck twice!)

Technical interview playbook

Table 108 — The 5 rounds and how this book prepared you

Round	They test	Your weapon (chapter)
Recruiter call	Communication + basics	Roadmap story (Front) + 3 projects narrative
Live HTML/CSS	Semantic + responsive fast	Parts 2-3 + datasheets; think out loud!
JS coding	Arrays, closures, async, fetch	Ch 26-33 patterns; state approach before code
System/API design	REST + DB + auth choices	Parts 6-8; draw boxes, justify trade-offs
Behavioral	Teamwork, conflicts, learning	STAR stories: Situation→Task→Action→Result

CODE · Live-coding survival template (say it, then write it)

```
// 1. Restate + examples: "So f([1,2,2,3]) → [1,2,3]? Empty → []?"
// 2. Brute force first (correct > clever), then optimize + name complexity
const unique = arr => [...new Set(arr)]; // O(n) time, O(n) space
// 3. Test edge cases OUT LOUD: [], [1,1,1], negatives, huge input
```

INTERVIEW QUESTION — WHY SHOULD WE HIRE YOU AS A JUNIOR?

Expected: "I ship complete, tested features; fundamentals-first so I debug systematically; I document and ask precise questions with reproductions. My portfolio proves it — pick any project and I'll walk you through its hardest bug." Confidence + evidence, never arrogance.

Your next 12 months

Table 109 — After this book (in order!)

Quarter	Focus	Goal
Q1	Depth: TypeScript + testing (Vitest/Jest)	Type-safe tested features
Q2	Breadth: Next.js or advanced React + CI/CD	Production-grade apps
Q3	Craft: accessibility audits + performance budgets	Lighthouse-100 portfolio

Quarter	Focus	Goal
Q4	People: open source + writing + mentoring	Visible expertise, interviews inbound

Exercise 45 — The final boss: ship project #3

Spec: authenticated full-stack app (idea: book-club tracker, gym logger, or expense teams).
 Requirements: Ch 47 tables + Ch 45 checklist + README + live demo video (2 min).
 Deadline: 30 days. Then apply to 5 junior roles with the link. GO.

SOLUTION

There is no solution — there is YOUR shipped link. Grading rubric: works on phone, survives abuse (Ch 45 ex), code you'd show proudly, README a stranger can follow. Ship it, then tell the author — this book's last page is your first day as a developer.

Launch week: the shipping sprint (7 days, 1 hired-ready portfolio)

Table 110 — Day by day (2 hours each — calendar it!)

Day	Mission	Done when
1	Pick 3 projects (Ch 73 gallery)	READMEs drafted, weakest cut
2	README surgery	Live link + screenshot + features + hardest-bug story each
3	Portfolio site live	10-second test passed by a FRIEND (Ch 74!)
4	Lighthouse + a11y pass	90+ all + keyboard-only flows work
5	GitHub + LinkedIn polish	Pinned repos, headline, first build-in-public post
6	Mock interviews ×2	60-second project stories fluent (Ch 63–65 banks)
7	Apply to 5 roles / send 5 proposals	Tracker sheet started (every no = lesson)

PRO TIP — DONE BEATS PERFECT — SHIPPED BEATS DONE

Launch week ends with links, not plans. A live imperfect project + honest README outperforms a "perfect" localhost forever. Employers click links; they cannot click intentions.

Open source: your first contribution (faster than you think)

Table 111 — The first-PR ladder (each rung < 2 hours)

Rung	Action	Why it matters
1	Fix a TYPO in docs (any big repo!)	Learn fork → branch → PR with zero risk
2	Answer one issue with a reproduction	Maintainers LOVE repros (debugging skill proof!)
3	Good-first-issue: small bug + test	Real code merged = public proof of teamwork
4	Add a missing test to a lib you use	Tests are welcomed everywhere, judged kindly
5	Your own tiny package (useDebounceValue!)	Publishing teaches semver + READMEs + CI
<pre>```text:PR message that gets merged</pre>		
Title: fix: search ignores Arabic diacritics (closes #123)		
Body: WHAT (one line) + repro steps + before/after + test added.		
Tone: grateful, brief, patient. Maintainers are volunteers — kindness merges PRs.		
```		

Rung	Action	Why it matters
## The 12-month radar (what to learn after this book)		
::: table Year one, quarter by quarter		
Quarter	Focus	Unlocks
Q1	TypeScript strict + testing depth (Ch 52-54)	Team-ready code, fewer bugs
Q2	Next.js + Postgres depth (Ch 62, 57)	SEO apps, real data jobs
Q3	System design basics (caching, queues, load)	Mid-level interviews
Q4	One specialty: mobile (React Native) / AI APIs / DevOps	Differentiation + pay rise
Always	Ship monthly + write monthly (one post per project!)	Visibility compounds (Ch 74)

**KEY TAKEAWAY — KEY TAKEAWAY**

This book ends; learning compounds. Radar rule: depth before breadth (master one stack!), fundamentals before hype (HTTP beats frameworks), proof before claims (ship monthly). In 12 months, BE the developer this book described — then teach someone else.



## PART

# Deep Dives I — HTML & CSS Beyond the Core

SVG, selectors, backgrounds and frameworks: the professional's visual arsenal.

---

**Ch 48** SVG — Infinitely Sharp Graphics

**Ch 49** Advanced Selectors — :has, :is, :where and Pseudo Power

**Ch 50** Backgrounds, Gradients and Masks Deep-Dive

**Ch 51** Bootstrap and Tailwind Crash Course

| Depth here is what separates "can build" from "can build beautifully."

CHAPTER 48

# SVG — Infinitely Sharp Graphics

Advanced

~ 100 min

Icons, logos and illustrations that stay razor-sharp on every screen.

IN THIS CHAPTER

- SVG vs canvas vs img
- Shapes, paths, groups
- Styling + `currentColor` icons
- Accessible SVGs
- Icon system project

Elzero SVG

Gamal icons

MDN SVG

## Why SVG wins for UI graphics

Table 112 — Image tech decision

Need	Winner	Why
Logos/icons/diagrams	SVG	Vector = sharp at any size, tiny, CSS-styleable
Photos	WebP/AVIF img	Pixels compress photos best
Games/pixel effects	canvas + JS	Per-pixel control

CODE · SVG anatomy (shapes + paths + text)

```
<svg viewBox="0 0 100 100" width="120" role="img" aria-labelledby="t">
 <title id="t">Star badge</title>
 <circle cx="50" cy="50" r="45" fill="#eef2ff" stroke="#1e3a8a" stroke-width="4"/>
 <path d="M50 25l7 14 15 2-11 11 3 15-14-7-14 7 3-15-11-11 15-2z" fill="#c9a227"/>
 <text x="50" y="88" text-anchor="middle" font-size="10">PRO</text>
</svg>
```

Table 113 — Path mini-language (d attribute)

Command	Meaning
M x y	Move pen (start)
L x y / H x / V y	Line to (x,y) / horizontal / vertical
C ... / Q ...	Cubic / quadratic curves
A ...	Arcs (circles, donuts)
Z	Close path
Lowercase (m l c)	RELATIVE coordinates (pen-relative)

## currentColor icon system (the pro pattern)

### CODE · One sprite, infinite themed icons

```
<svg style="display:none">
 <symbol id="i-cart" viewBox="0 0 24 24"><path
d="M3 3h2l2.4 12.4A2 2 0 0 0 9.4 17H18a2 2 0 0 0 2-1.6L21.6 8H6" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round"/></symbol>
 <symbol id="i-x" viewBox="0 0 24 24"><path d="M6 6l12 12M18 6L6 18"
stroke="currentColor" stroke-width="2" stroke-linecap="round"/></symbol>
</svg>
<button class="icon-btn"><svg width="20" height="20"><use href="#i-cart"/></svg> Cart</
button>
```

### CODE · Icons inherit text color automatically

```
.icon-btn { color: navy; } .icon-btn:hover { color: gold; } /* icons follow! */
```

### KEY TAKEAWAY — KEY TAKEAWAY

`currentColor` + `<symbol>` / `<use>` sprite = one HTTP request, infinitely scalable, theme-aware icons colored by plain CSS `color`. This is how every design system ships icons.

## Accessible + responsive SVG rules

### SVG SHIP CHECKLIST

- - Decorative: `aria-hidden="true"` (no title!) ... Meaningful: `role="img"` + `<title>`
- - `viewBox` ALWAYS (scaling!); width/height or CSS for layout size
- - Text as `<text>` (selectable/translatable) or convert to paths for logos
- - Long animations: honor `prefers-reduced-motion`

### Quick check — Best logo format?

1. PNG 4x
2. SVG
3. JPG
4. GIF

**Answer: B · Vector sharpness + tiny size + recolor via CSS. PNG only for photographic marks.**

### Exercise 46 — Build a 5-icon set

Search, cart, user, sun, moon as symbols; theme via color; toggle button swaps sun/moon; decorative instances `aria-hidden`, meaningful ones titled.

#### SOLUTION

Sprite with 5 symbols (24×24 grid, `stroke=currentColor`, round caps);  
`.theme-dark{color:gold}` recolors all; toggle swaps `<use href>`; `aria-hidden` on duplicates.  
Icon system done in ~40 lines.

## CHAPTER 49

# Advanced Selectors — :has, :is, :where and Pseudo Power

Advanced

~ 90 min

Select anything: relational selectors, zero-specificity helpers and generated content.

## IN THIS CHAPTER

- :has() relational styling
- :is/:where specificity control
- Advanced :nth patterns
- ::marker/::selection/::first-letter
- attr() + counters

Elzero advanced

MDN selectors

W3Schools

## :has() — style parents by children (the revolution)

## CODE · Parent-aware styling (no JS!)

```
/* Cards containing images get padding-free tops */
.card:has(> img) { padding-top: 0; }
/* Forms flag errors upward */
form:has(input:invalid) .submit-hint { display: block; }
/* Nav highlights when dropdown open */
nav:has(.dropdown:hover) { background: navy; }
/* Quantity queries: grids adapt to count */
.grid:has(> :nth-child(5)) { grid-template-columns: repeat(3, 1fr); }
```

## WARNING — :HAS() PERFORMANCE

:has() can be expensive over huge trees — scope it (`.card:has(...)` not `*:has(...)`), and never chain multiple :has() on hot paths like table rows.

## :is() vs :where() (specificity weapons)

### CODE · Same matching, different scores

```
/* :is() takes the STRONGEST argument's specificity */
:is(h1, .title, #hero) { ... } /* scores 1-0-0 (the ID!) – dangerous */
/* :where() scores ZERO always – perfect for resets/defaults */
:where(h1, h2, p, ul) { margin: 0; } /* 0-0-0: any class overrides ✓ */
/* Forgiving parsing: unknown selectors inside don't kill the rule */
:is(.a, :bogus, .b) { ... } /* works! plain lists would die entirely */
```

## nth patterns + generated content

### CODE · Position patterns professionals use

```
tbody tr:nth-child(even) { background: #f6f8fc; } /* zebra */
.gallery > :nth-child(3n) { margin-right: 0; } /* every 3rd */
li:nth-child(n+4) { ... } /* 4th onward */ li:nth-child(-n+3) { ... } /* first 3 */
li:only-child { ... } /* lonely items */ :empty { ... } /* truly empty */

::marker { color: gold; font-size: 1.2em; }
::selection { background: #c9a227; color: #111; }
.article::first-letter { font-size: 3em; float: left; line-height: 1; }
a[href^="http"]::after { content: " (" attr(href) ")"; font-size: .8em; } /* print URLs!
*/
```

Table 114 — ::before/::after rules

Rule	Why
Needs <code>content</code> (even <code>""</code> )	No content = no box rendered
Decorative ONLY	Screen readers may skip/announce oddly — never vital info
<code>attr()</code> reads attributes	Tooltips, print URLs, counters

**Quick check — Utility `.hidden` must always win:**

1. !important
2. :where()
3. Later layer + single class
4. Inline styles

**Answer: C · Cascade layers (Ch 29-notes) or disciplined order beat specificity wars; !important is the last resort.**

**Exercise 47 — Pure-CSS accordion states**

Details/summary (Ch 27-ref) + `details[open]` styling + `summary::marker` custom icon + `:has()` to highlight sections containing open items. Zero JS.

**SOLUTION**

```
details[open]{border-color:navy} summary{cursor:pointer;list-style:none} summary::before{content:"▶ "} de
```

— state styling with zero JS.

## CHAPTER 50

# Backgrounds, Gradients and Masks Deep-Dive

Advanced

~ 80 min

Layered art direction with pure CSS: stacks, patterns, blend modes and reveals.

## IN THIS CHAPTER

- Multi-layer stacks
- Gradient types + hard stops
- Pattern gallery (no images!)
- blend modes
- Masks + text clips

Gamal effects

Elzero backgrounds

MDN

## Layering (paint order decoded)

CODE · First listed = TOP layer. Commas stack; / sets size per layer.

```
.card-art {
 background:
 radial-gradient(circle at 80% 10%, rgb(201 162 39 / .35), transparent 40%),
 linear-gradient(135deg, #0b1026, #1e3a8a 60%, #2743a0),
 url(noise.svg);
 background-blend-mode: screen, normal, overlay;
}
```

Table 115 — Gradient types

Type	Syntax idea	Best for
linear	<code>linear-gradient(135deg, a, b)</code>	Heroes, buttons, overlays
radial	<code>radial-gradient(circle at 20% 30%, a, transparent 60%)</code>	Glows, spotlights



Type	Syntax idea	Best for
conic	<code>conic-gradient(from 0deg, a, b, a)</code>	Pie charts, color wheels, borders
repeating-*	<code>repeating-linear-gradient(45deg, a 0 8px, b 8px 16px)</code>	Stripes, patterns
Hard stops	Same position twice ( <code>gold 0 12px, navy 12px</code> )	Crisp edges, flags

## Pattern gallery (zero image files!)

### CODE · Dots, grid, stripes — pure gradients

```
.dots { background-image: radial-gradient(#c9a227 1.5px, transparent 1.6px); background-size: 22px 22px; }
.grid-bg { background-image: linear-gradient(#e2e8f0 1px, transparent 1px), linear-gradient(90deg, #e2e8f0 1px, transparent 1px); background-size: 28px 28px; }
.stripes { background: repeating-linear-gradient(-45deg, #0b1026 0 14px, #131c4d 14px 28px); }
```

## Masks, clips and blend reveals

### CODE · Visibility art direction

```
.hero-title {
 background: linear-gradient(90deg, #c9a227, #fff, #c9a227);
 -webkit-background-clip: text; background-clip: text; color: transparent;
}
.fade-list { mask-image: linear-gradient(black 75%, transparent); } /* soft list end */
.img-duo { filter: grayscale(1) contrast(1.1); mix-blend-mode: multiply; } /* print duotone */
```

### KEY TAKEAWAY — KEY TAKEAWAY

Backgrounds are a STACK (comma layers), gradients are IMAGES (anywhere url() works), masks control VISIBILITY, blend modes control MIXING. Compose the four and "CSS art" stops being magic.

**Exercise 48 — Hero art direction, no images**

Navy gradient + gold radial glow + dot pattern + noise-feel stripes at 4% + gradient-clipped headline + masked fade into next section. All CSS, <30 lines.

**SOLUTION**

Stack 3 backgrounds (glow radial top, dots mid, base linear); headline `background-clip:text`; section `mask-image: linear-gradient(black 85%, transparent)` for the melt. Screenshot it — portfolio-worthy CSS art.

## CHAPTER 51

# Bootstrap and Tailwind Crash Course

Intermediate+

~ 100 min

Utility frameworks: when to use them, how they think, and building the same card both ways.

## IN THIS CHAPTER

- Framework thinking (components vs utilities)
- Bootstrap grid + components
- Tailwind utilities + config
- Same UI both ways
- Custom CSS vs framework decision

[Angela §Bootstrap](#)[Gamal bootstrap](#)[Docs](#)

## Two philosophies

**Table 116 — Components (Bootstrap) vs Utilities (Tailwind)**

	Bootstrap	Tailwind
Idea	Prebuilt components (btn, card, modal)	Tiny utilities composed in HTML
HTML	<code>&lt;button class="btn btn-primary"&gt;</code>	<code>&lt;button class="rounded bg-blue-600 px-4 py-2 text-white hover:"</code>
Speed	Instant standard UI	Instant CUSTOM UI (after learning curve)
Weight	Whole library (tree-shakeable)	Purged to used-only (tiny!)
Escape	Override CSS (specificity fights)	Arbitrary values <code>w-[347px]</code> + config

**CODE · Bootstrap in 60 seconds (CDN + grid + card)**

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<div class="container py-5">
 <div class="row g-4">
 <div class="col-12 col-md-6 col-lg-4">
 <div class="card h-100"><div class="card-body">
 <h5 class="card-title">Beans</h5>
 <button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#m">Buy</
button>
 </div></div>
 </div>
 </div>
</div>
```

**CODE · Tailwind same card (CDN play + utilities)**

```
<script src="https://cdn.tailwindcss.com"></script>
<div class="mx-auto max-w-6xl px-4 py-12">
 <div class="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
 <div class="rounded-2xl border border-slate-200 bg-white p-6 shadow-sm transition
hover:-translate-y-1 hover:shadow-lg">
 <h3 class="text-lg font-bold text-slate-900">Beans</h3>
 <button class="mt-4 rounded-full bg-blue-600 px-5 py-2 text-white hover:bg-blue-700
focus-visible:outline focus-visible:outline-2">Buy</button>
 </div>
 </div>
</div>
```

**Table 117 — Decision matrix (honest)**

Situation	Choice
Admin panels, MVPs, hackathons	Bootstrap (speed!)
Custom brand design systems	Tailwind (control!)
Learning CSS deeply	Neither first — this book's Parts 2-3, THEN frameworks
Tiny landing page	Hand CSS (frameworks = overhead)

**WARNING — CDN LINKS ARE FOR LEARNING**

Play CDNs block rendering + ship dev warnings. Production = npm install + build pipeline (Vite plugin / PostCSS) with purging. (Setup-free rule respected: concepts here, commands in Ch 37/56 contexts.)

**Quick check — `md:grid-cols-2` means...**

1. 2 cols on phones
2. 2 cols  $\geq 768\text{px}$
3. 2 rows always
4. Grid gap 2

**Answer: B** · Tailwind breakpoints are mobile-first min-width: base = phone, md: =  $\geq 768\text{px}$ . Same philosophy as Ch 19.

**Exercise 49 — Framework duel**

Build the pricing section BOTH ways (Bootstrap + Tailwind), then a third time in hand CSS. Compare: lines, control, load size. Keep all three — interviewers LOVE this comparison story.

**SOLUTION**

Expect: Bootstrap fewest lines + most generic; Tailwind most classes + most custom + smallest prod build; hand CSS full control + most time. Verdict skill: choosing per project, not per hype.

## PART

# Deep Dives II — JavaScript Beyond the Core

Collections, patterns, tests and types: write JS that survives teams and years.

---

**Ch 52** Modern JavaScript Deep — Map, Set, Iterators and More

**Ch 53** Design Patterns — Module, Factory, Observer, Singleton

**Ch 54** Testing JavaScript — Vitest From Zero

**Ch 55** TypeScript Essentials — JavaScript That Checks Itself

| Senior JS is not more syntax — it is fewer bugs by construction.

## CHAPTER 52

# Modern JavaScript Deep — Map, Set, Iterators and More

Advanced

~ 100 min

The ES6+ tools seniors reach for daily: proper collections, lazy sequences and clean async iteration.

## IN THIS CHAPTER

- Map vs Object, Set vs Array
- WeakMap/WeakSet caches
- Iterators + generators
- for-await + async generators
- Real-world combos

Elzero advanced

MDN

Gamal JS

## Map vs Object, Set vs Array (choose right)

Table 118 — The decision tables

Need	Use	Why
String keys + JSON + literals	Object <code>{}</code>	Ergonomics, JSON-native
ANY keys (objects, functions) + order + size	Map	<code>map.size</code> , insertion order, no prototype traps
Unique values / dedupe / membership	Set	$O(1)$ <code>has()</code> , auto-unique

**CODE · Collections doing real jobs**

```
// 1. Dedupe + count in two lines
const tags = ["js", "css", "js", "html"];
const unique = [...new Set(tags)]; // ["js","css","html"]
// 2. Frequency map
const freq = new Map();
for (const t of tags) freq.set(t, (freq.get(t) || 0) + 1);
// 3. Object-keyed cache (impossible with {})
const prices = new Map();
prices.set(productObj, 1299); // keys can be OBJECTS ✓
// 4. WeakMap private-ish + GC-friendly metadata
const meta = new WeakMap();
meta.set(user, { lastSeen: Date.now() }); // vanishes when user GC'd ✓
```

**KEY TAKEAWAY — KEY TAKEAWAY**

Object for RECORDS (known string keys, JSON). Map for DICTIONARIES (dynamic/any keys, frequent add/delete). Set for UNIQUENESS. WeakMap/WeakSet for metadata that must not leak memory.

## Generators: lazy power

**CODE · Sequences without arrays (infinite + paged data)**

```
// Infinite IDs, zero memory
function* ids(prefix = "u_") { let n = 1; while (true) yield `${prefix}${n++}`; }
const gen = ids(); gen.next().value; // "u_1" (pauses until next(!))

// Paginate an API lazily — consumers just loop
async function* allPages(fetchPage) {
 let page = 1;
 while (true) {
 const { items, hasMore } = await fetchPage(page++);
 yield* items; // delegate each item
 if (!hasMore) return;
 }
}
for await (const user of allPages(p => api(`/users?page=${p}`))) { paint(user); }
```



**THINK OF IT LIKE THIS — GENERATORS ARE PAUSE BUTTONS**

A normal function runs to the end. A generator **PAUSES** at every `yield` and resumes on `next()` — producing values on demand. Infinite sequences, huge files, paged APIs: all become simple loops.

**Quick check —** `[...new Set([1,1,2])]` =

1. `[1,1,2]`
2. `[1,2]`
3. `Set{1,2}`
4. Error

**Answer: B** · Set drops duplicates; spread converts back to array. The one-line dedupe.

**Exercise 50 — LRU cache with Map**

`LRU(limit)` with get/set; Map insertion order = age; get refreshes (delete+set); set evicts oldest when full. O(1) everything.

**SOLUTION**

```
class LRU{constructor(n){this.n=n;this.m=new Map()}get(k){if(!this.m.has(k))return;const v=this.m.get(k);
```

— Map order does the bookkeeping.

## CHAPTER 53

# Design Patterns — Module, Factory, Observer, Singleton

Advanced

~ 110 min

Four battle-tested solutions to recurring problems — with honest "when NOT to use" notes.

## IN THIS CHAPTER

- Module (encapsulation)
- Factory (configurable creation)
- Observer (events at scale)
- Singleton (one shared thing)
- Anti-pattern warnings

[Elzero patterns](#)[MDN](#)[Interview](#)

## 1. Module (ESM + closures)

CODE · Private-by-default features (Ch 26 closures + Ch 28 modules, combined)

```
// cart.js – state NOBODY can corrupt, API everyone uses
const items = [];
export const add = p => items.push(p);
export const total = () => items.reduce((s, p) => s + p.price, 0);
export const list = () => [...items]; // copy out – never the live array!
```

## 2. Factory (one maker, many configs)

CODE · Configured creators beat option-soup constructors

```
const makeApi = base => (path, opts) =>
 fetch(base + path, { headers: { "Content-Type": "application/json" }, ...opts })
 .then(r => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return r.json(); });
const shopApi = makeApi("https://shop.example/api");
const authApi = makeApi("https://auth.example/");
await shopApi("/products"); // preconfigured, tiny call sites ✓
```

## 3. Observer (pub/sub at scale)

CODE · Tiny event bus — decoupled features (cart ↔ badge ↔ toast)

```
const bus = new Map();
export const on = (ev, fn) => { (bus.get(ev) ?? bus.set(ev, new
Set()).get(ev)).add(fn); };
export const emit = (ev, data) => bus.get(ev)?.forEach(fn => fn(data));
// cart.js: emit("cart:add", item) ... badge.js: on("cart:add", updateCount)
```

## 4. Singleton (exactly one)

CODE · One DB pool / one config / one logger — module instances ARE singletons

```
// db.js
import pg from "pg";
export const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
// Every importer shares THE pool (modules evaluate once) ✓ — no class ceremony needed.
```

Table 119 — Pattern honesty table (when NOT to)

Pattern	Skip when
Module	Single-use script (functions + file scope suffice)
Factory	One configuration exists (YAGNI!)
Observer	Two modules + one event (direct call is clearer)
Singleton	Tests need isolation (inject instead) or state should be per-request

**INTERVIEW QUESTION — SINGLETON IN NODE.JS?**

Expected: "ES modules evaluate once and export live bindings — so an exported instance IS the singleton. For per-request state (tests, tenants) I export a factory instead." Practical + honest beats textbook recitation.

**Exercise 51 — Theme bus + store**

Mini-store: `createStore(init)` with `get/set/subscribe` (observer); `theme` + `cart` as two stores; `header/cart/toast` subscribe. No framework, ~30 lines, React's `useSyncExternalStore`-compatible shape.

**SOLUTION**

```
const createStore=v=>{const s=new Set();return{get:()=>v,set:n=>{v=n;s.forEach(f=>f(v))},sub:f=>(s.add(f))}
```

— `get/set/sub` triple. Unsubscribe fn returned = leak-free effects.

## CHAPTER 54

# Testing JavaScript — Vitest From Zero

Advanced

~ 110 min

Prove code works: unit tests, mocks, and testing async + DOM without fear.

## IN THIS CHAPTER

- Why tests (the bug-cost curve)
- First test in 5 minutes
- Assertions that matter
- Mocking fetch/time
- Testing DOM + async

Testing

Elzero testing

Interview

## The bug-cost curve (why bother)

```
dev writes test $1 (seconds)
code review finds $10 (minutes)
QA finds $100 (hours)
production incident $10,000+ (outage + trust)
Tests move discovery LEFT – cheapest possible moment.
```

Figure 25 — Cost of a bug by discovery phase

CODE · Your first test (npm i -D vitest; script: "test": "vitest")

```
// math.js
export const add = (a, b) => a + b;
// math.test.js
import { describe, it, expect } from "vitest";
import { add } from "../math.js";
describe("add", () => {
 it("adds positives", () => expect(add(2, 3)).toBe(5));
 it("handles negatives", () => expect(add(-2, -3)).toBe(-5));
});
```

**Table 120 — Assertions that matter (90% of tests)**

Matcher	Checks
<code>toBe</code>	Primitives ( <code>Object.is</code> )
<code>toEqual</code>	Deep object/array equality
<code>toContain</code> / <code>toMatch</code>	Substring/regex/array membership
<code>toThrowError</code>	<code>expect(() =&gt; fn()).toThrow("msg")</code>
<code>resolves/rejects</code>	<code>await expect(p).resolves.toBe(x)</code>

## Mocking: fetch, time, randomness

### CODE · Control the uncontrollable

```
import { vi, it, expect, beforeEach } from "vitest";
// 1. Mock fetch (never hit real APIs in unit tests!)
global.fetch = vi.fn(async () => ({ ok: true, json: async () => [{ id: 1 }] }));
// 2. Mock time (debounce/timers without waiting!)
vi.useFakeTimers();
debounced("x"); vi.advanceTimersByTime(300); expect(fetch).toHaveBeenCalledTimes(1);
// 3. Mock randomness/UUIDs (deterministic snapshots)
vi.spyOn(crypto, "randomUUID").mockReturnValue("fixed-id");
```

### CODE · Testing DOM + async (happy-dom + Testing-Library style)

```
// npm i -D happy-dom @testing-library/dom
import { getByRole, fireEvent } from "@testing-library/dom";
document.body.innerHTML = `<form><input name="email"><button>Go</button></form>`;
fireEvent.input(screen.getByRole("textbox"), { target: { value: "a@x.com" } });
await fireEvent.click(screen.getByRole("button"));
expect(document.querySelector(".done")).toBeTruthy();
```

### KEY TAKEAWAY — KEY TAKEAWAY

Test BEHAVIOR ("submitting valid form shows success"), not implementation (not "calls function X"). Pyramid: many fast unit tests, fewer integration, rare E2E. Untested code = legacy code the day after you write it.

**Quick check — Best assertion for `{a:1}` vs `{a:1}` ?**

1. `toBe`
2. `toEqual`
3. `toContain`
4. `toThrow`

**Answer: B** · `toBe` compares identity (fails for two objects); `toEqual` compares deep structure.

**Exercise 52 — Test the cart module**

Cover Ch-pattern cart: add/total/list-copy; fetch-mocked checkout (success + 500 + offline); debounce search (fake timers). 100% of cart.js lines.

**SOLUTION**

Unit: pure fns (add/total/immutability of list()). Integration: mocked-fetch checkout incl. `.ok=false` + network throw. Timers: `advanceTimersByTime` proves single-call debounce. Coverage flag: `vitest --coverage`.

## CHAPTER 55

# TypeScript Essentials — JavaScript That Checks Itself

Advanced

~ 120 min

Types for safety: annotate the hot spots, infer the rest, and catch bugs before runtime.

## IN THIS CHAPTER

- Why types (the refactor test)
- Basic + literal + union types
- Interfaces vs type
- Generics in 10 minutes
- Adopt gradually (allowJs → strict)

TypeScript

Interview

MDN

## The refactor test (why types win)

Rename `user.name` → `user.fullName` across 50 files: JS = find-and-hope (miss one = production crash); TS = compiler lists EVERY spot before you run anything. Types are executable documentation + a tireless reviewer.

## CODE · Types pay rent immediately

```
type ID = string; // aliases name concepts
interface User { id: ID; name: string; age?: number } // ? = optional
function greet(u: User): string { return `Hi ${u.name}`; }
const isAdmin = (u: User): u is Admin => u.role === "admin"; // type GUARD ✓
let id: ID = "u_1"; // let n: number = "x"; // ✗ compile ERROR (not runtime surprise!)
```

Table 121 — The 80/20 type toolkit



Tool	Example	Use	
<code>string/number/boolean</code>	<code>let n: number</code>	Hot-spot annotations	
<code>? optional</code>	<code>age?: number</code>	Partial data, options bags	
Unions	<code>`"light" \</code>	<code>"dark"``</code>	Fixed choices (themes, roles, states!)
Literals	<code>`method: "GET" \</code>	<code>"POST"``</code>	API contracts
<code>unknown</code> + narrowing	<code>if (typeof x === "string")</code>	SAFE any (validate outside data)	
<code>never</code>	exhaustive switch check	Reducers that can't forget cases	

**CODE · Generics: one function, every type (10 minutes)**

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }
first([1, 2]); // T = number (inferred!)
first(["a"]); // T = string
// Constraints keep power + safety:
function longest<T extends { length: number }>(a: T, b: T): T {
 return a.length >= b.length ? a : b;
}
```

## Interfaces vs type + gradual adoption

Table 122 — interface vs type

	interface	type
Extends/merges	<code>extends</code> + declaration merging (libs!)	Intersections <code>&amp;</code> , no merging
Unions/tuples/fns	Awkward	Natural home
Rule of thumb	Object SHAPES (public APIs)	Unions, helpers, everything else

**CODE · Adopt without rewriting (the safe ramp)**

1. `npm i -D typescript; npx tsc --init (allowJs: true, checkJs: false)`
2. Rename ONE hot file `.js` → `.ts`; fix ITS errors only
3. Enable strict gradually: `strictNullChecks` first (null bugs die!)
4. Type boundaries (API I/O with `zod/valibot`) before internals

**INTERVIEW QUESTION — ANY VS UNKNOWN?**

Expected: "any disables checking (contagious, banned except escape hatches). unknown forces narrowing before use — the safe way to type outside data like API responses and `JSON.parse`." Mention `eslint @typescript-eslint/no-explicit-any`.

**Exercise 53 — Type the Notes API client**

Types for `Note`/`ApiError` + typed `api()` helper (method literals, generic response) + exhaustive status handler (`never-check`) + `unknown-narrowed` `JSON` parse. Zero any.

**SOLUTION**

```
type Note={id:string;text:string;done:boolean}; async function api<T>(p:string,o?:{method?:"GET"|"POST"|\n+ const j:unknown=await r.json() narrowed by guards. Client bugs become compile errors.
```

## PART

# Deep Dives III — Backend & Data Beyond the Core

Terminal fluency, templates, realtime, modeling and shipping real servers.

---

**Ch 56** Command Line, Vite and Code Quality

**Ch 57** Templates, Static Files and Realtime Intro

**Ch 58** Data Modeling, Indexes and Transactions

**Ch 59** Deploying Full-Stack Apps — API + DB + Domain

| Backend depth is quiet: fewer outages, faster queries, calmer on-call.

## CHAPTER 56

# Command Line, Vite and Code Quality

Intermediate+

~ 100 min

Terminal confidence, instant dev servers, and automated quality gates.

## IN THIS CHAPTER

- Terminal survival (20 commands)
- Files + pipes + search
- Vite dev/build/preview
- ESLint + Prettier + hooks
- Scripts that run the project

Elzero terminal

Vite

Quality

## Terminal survival (the daily 20)

Table 123 — Commands that run your day

Job	Command
Where/ what	<code>pwd</code> · <code>ls -la</code> · <code>cd dir</code> / <code>cd ..</code>
Files	<code>touch f</code> · <code>mkdir d</code> · <code>cp a b</code> · <code>mv a b</code> · <code>rm f</code> / <code>rm -rf d</code> (careful!)
Read/ search	<code>cat f</code> · <code>less f</code> · <code>grep -rn "todo" .</code> · <code>find . -name "*.js"</code>
Help	<code>cmd --help</code> · <code>man cmd</code>
History/ shortcuts	

Job	Command
	<code>↑</code> rerun · <code>tab</code> complete · <code>ctrl+c</code> stop · <code>ctrl+r</code> search history
Chain	<code>a &amp;&amp; b</code> (b if a ok) · <code>`a \</code> <code>\ b</code> (b if a fails) · <code>a</code> <code>\</code> <code>b`</code> (pipe output→input)

#### CODE · Pipes: small tools, superpowers combined

```

grep -rn "TODO" src | head -20 # first 20 todos with file:line
cat big.log | grep ERROR | wc -l # count errors in one breath
ls | grep ".test.js" # find test files now

```

#### WARNING — RM -RF IS FOREVER (NO RECYCLE BIN!)

`rm -rf node_modules` (safe, reinstallable) vs `rm -rf /` (career-ending). Pause before Enter on ANY `rm -rf`; tab-complete paths; never run pasted commands you can't explain line by line.

## Vite: instant dev, optimized build

Table 124 — Vite's three commands (concepts — Ch 37 has npm basics)

Command	Does
<code>npm run dev</code>	Lightning dev server + hot reload (edits appear instantly, state kept!)
<code>npm run build</code>	Minified, split, hashed production bundle in <code>dist/</code>
<code>npm run preview</code>	Serves <code>dist/</code> locally (test prod build BEFORE deploying!)

#### CODE · vite.config.js — the only config most apps need

```

import { defineConfig } from "vite";
export default defineConfig({
 server: { port: 5173, proxy: { "/api": "http://localhost:5000" } }, // dev: same-origin API ✓
 build: { outDir: "dist", sourcemap: true }
});

```

**KEY TAKEAWAY — KEY TAKEAWAY**

Dev proxy ( `/api` → `localhost:5000` ) kills CORS pain in development: browser sees ONE origin. Production solves it differently (same-domain deploy or locked CORS, Ch 40/45).

## ESLint + Prettier + hooks (robots enforce taste)

**CODE · The quality stack (concepts)**

```
ESLint finds BUGS (unused vars, ==, missing deps) – "is it correct?"
Prettier .. formats CODE (quotes, width) – "is it pretty?" (zero debates!)
Hooks husky + lint-staged run both on EVERY commit – "never commit sins"
```

**CODE · package.json scripts — the project's control panel**

```
{ "scripts": {
 "dev": "vite", "build": "vite build", "preview": "vite preview",
 "lint": "eslint .", "format": "prettier --write .", "test": "vitest run" } }
```

**Quick check — `a && b` runs b when...**

1. Always
2. Only if a succeeded
3. Only if a failed
4. Never

**Answer: B · `&&` chains success paths ( `build && deploy` );**

**Exercise 54 — Green-field setup drill**

On a practice repo: dev/build/preview all green, lint+format clean, pre-commit hook blocking a deliberate `==` sin, README documenting every script. Screenshot the green terminal — process proof.

**SOLUTION**

Deliberately commit `if (x == 1)` → hook fails with the rule name → fix to `===` → commit passes. You just felt quality gates WORKING. (Exact configs evolve — your Ch 37 + docs skills carry you.)

## CHAPTER 57

# Templates, Static Files and Realtime Intro

Advanced

~ 100 min

Server-rendered pages, smart static serving, and live updates with SSE + WebSockets.

## IN THIS CHAPTER

- EJS templates (dynamic HTML)
- Static caching done right
- SSE (server → browser stream)
- WebSockets (two-way live)
- When realtime (and when NOT)

Angela §EJS

Elzero backend

MDN

## EJS: dynamic HTML without a framework

CODE · views/post.ejs — HTML with &lt;%= %&gt; injections (auto-ESCAPED ✓)

```
<h1><%= post.title %></h1>
<p>By <%= post.author.name %> · <%= post.date %></p>
<%= post.comments.forEach(c => { %><%= c.by %>: <%= c.text %><%= %> } %>
```

CODE · Express + EJS wiring (npm i ejs)

```
app.set("view engine", "ejs");
app.get("/posts/:slug", async (req, res, next) => {
 const post = await Post.findOne({ slug: req.params.slug });
 if (!post) return next({ status: 404, message: "Missing post" });
 res.render("post", { post }); // välj → HTML (<%= %> escapes XSS ✓)
});
```

Table 125 — EJS tags



Tag	Job
<code>&lt;%= v %&gt;</code>	Print ESCAPED (user data — default!)
<code>&lt;%- html %&gt;</code>	Print RAW (your sanitized HTML only!)
<code>&lt;% code %&gt;</code>	Logic (loops, ifs — keep tiny!)
<code>&lt;## c %&gt;</code>	Comment ... <code>&lt;%- include("x") %&gt;</code> partials

## Static files + caching that respects updates

CODE · Cache far, invalidate by filename (hashed builds do this free)

```
app.use(express.static("public", { maxAge: "1y", immutable: true })); // app.alb2.js ✓
// HTML itself: NEVER long-cache (it points at hashed assets!)
app.get("/", (req, res) => res.set("Cache-Control", "no-cache").sendFile("index.html"));
```

## Realtime: SSE vs WebSockets

Table 126 — Push tech decision

Need	Use	Why
Server → browser stream (scores, progress, feeds)	SSE (EventSource)	HTTP-based, auto-reconnect, trivial
Two-way live (chat, games, collab)	WebSockets (ws lib)	Full-duplex, low latency
Rare updates	Polling (fetch every N s)	Simple > clever at low frequency

**CODE · SSE in 15 lines (server + client)**

```
// server: ordered stream over plain HTTP
app.get("/api/progress/:job", (req, res) => {
 res.writeHead(200, { "Content-Type": "text/event-stream", "Cache-Control": "no-cache",
 Connection: "keep-alive" });
 const t = setInterval(() => res.write(`data: ${JSON.stringify({ pct: pct() })}\n\n`),
 500);
 req.on("close", () => clearInterval(t));
});
// client: auto-reconnecting stream
const es = new EventSource("/api/progress/7");
es.onmessage = e => paint(JSON.parse(e.data));
```

**CODE · WebSocket echo (npm i ws) — the two-way hello**

```
import { WebSocketServer } from "ws";
const wss = new WebSocketServer({ port: 8080 });
wss.on("connection", ws => {
 ws.on("message", m => ws.send(`echo: ${m}`)); // broadcast: loop wss.clients!
});
```

**WARNING — REALTIME HONESTY**

Sockets add stateful servers (sticky sessions, horizontal-scale pain). Default to fetch/SSE; pay for WebSockets only when two-way + low-latency is the FEATURE (chat, games, live collab).

**Exercise 55 — Live order tracker**

SSE streams order status (received → cooking → on-way → delivered) with progress bar + auto-reconnect resume (Last-Event-ID) + graceful polling fallback if SSE blocked.

**SOLUTION**

Server emits `{stage, at}` events with `id:` lines; client EventSource renders timeline; `es.onerror` → fetch-poll fallback. Same UX three ways — resilience engineering in miniature.

## CHAPTER 58

# Data Modeling, Indexes and Transactions

Advanced

~ 110 min

Design data that stays fast and true: relationships, index strategy and ACID guarantees.

## IN THIS CHAPTER

- ER thinking (1-1/1-N/N-N)
- Normalization vs denormalization
- Index strategy + EXPLAIN
- Transactions (all-or-nothing)
- N+1 and how to kill it

Elzero DB

Angela \$DB

Tutorialspoint

## Relationships (say the sentence!)

Table 127 — Relationship patterns

Say	Model (SQL)	Model (Mongo)
User HAS MANY orders	orders.user_id FK	Order.user ObjectId ref
Order HAS MANY products (and reverse!)	order_items JOIN table (N-N)	items[] embedded refs
User HAS ONE profile	profiles.user_id UNIQUE FK	embed in User

**CODE · N-N done right (order_items join table)**

```
CREATE TABLE order_items (
 order_id INT REFERENCES orders(id) ON DELETE CASCADE,
 product_id INT REFERENCES products(id),
 qty INT CHECK (qty > 0),
 PRIMARY KEY (order_id, product_id)
);
```

## Normalize vs denormalize (the eternal trade)

Table 128 — The trade, honestly

Normalize (split, JOIN)	Denormalize (duplicate, embed)
✓ One truth (update once)	✓ Reads need zero JOINS (fast pages)
✓ No update anomalies	✓ Matches read shapes (feeds, carts)
✗ JOINS on hot reads	✗ Copies drift (sync/cron/eventual)
Rule: money/identity normalized; feeds/caches denormalized	Same rule, other side

## Indexes + EXPLAIN (speed with proof)

**CODE · Index the WHERE/JOIN/ORDER paths (measure, don't guess!)**

```
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at DESC);
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 7 ORDER BY created_at DESC LIMIT 20;
-- Seq Scan = table crawl (bad at scale!) ... Index Scan = seek (good!)
```

Table 129 — Index wisdom

Rule	Why
Index FKs + WHERE + ORDER BY paths	JOINS/sorts/filters seek instead of crawl

Rule	Why
Compound order matters (user, date)	Serves user queries + user+date, NOT date-alone
Too many indexes = slow writes	Each INSERT updates every index — index reads you HAVE
Partial/expression for hot slices	WHERE active partial, lower(email) unique (citext!)

## Transactions + N+1

**CODE · Money moves atomically (all-or-nothing, concurrent-safe)**

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT; -- or ROLLBACK on ANY failure (app code must!)
```

**CODE · N+1: 1 query + N follow-ups (the perf killer) → 1 JOIN instead**

```
// × 101 queries for 100 orders:
for (const o of orders) o.user = await User.findById(o.user_id);
// ✓ ONE query:
SELECT o.*, u.name FROM orders o JOIN users u ON u.id = o.user_id;
// Mongo: .populate() batches (2 queries, not N+1) ✓
```

### INTERVIEW QUESTION — HOW DO YOU FIND SLOW QUERIES?

Expected: "EXPLAIN ANALYZE in staging with production-like data; log slow queries (log_min_duration_statement); fix with indexes/JOIN rewrites/pagination; re-measure. Never index blindly — every index taxes writes." Measurement-first = senior.

**Exercise 56 — Model the book club**

Books, members, loans (who-has-what + due), reviews. SQL schema + constraints + 3 indexes justified + 2 reports (overdue loans, top books) + loan/return as a transaction. EXPLAIN both reports.

**SOLUTION**

loans(member FK, book FK, due, returned_at); UNIQUE active loan per book (partial index WHERE returned_at IS NULL!); reports via JOIN+GROUP; checkout = BEGIN ... INSERT loan ... UPDATE book stock ... COMMIT. Constraints + partial indexes carry the integrity.

## CHAPTER 59

# Deploying Full-Stack Apps — API + DB + Domain

Advanced

~ 110 min

Ship the whole system: hosted API, managed database, env discipline and smoke tests.

## IN THIS CHAPTER

- Production topology recap
- Managed Postgres + migrations
- Env vars + secrets discipline
- Health checks + logs
- Smoke-test + rollback plan

Angela \$Deploy

DevOps

Elzero deploy

## Topology + managed database

```
[Static front: Vercel/Netlify] --HTTPS--> [API: Render/Railway/Fly]
 |-- [Managed PG: Neon/Supabase]
 +-- env: DATABASE_URL · JWT_SECRET · API_TOKEN
Backups ON · statement_timeout set · pooled connections (PgBouncer/Supabase pooler).
```

Figure 26 — Free-tier production shape (Ch 47, hardened)

**CODE · Migrations (schema as versioned code — never click-ops in prod!)**

```
// 001_init.sql (committed, reviewed, applied by CI in order)
CREATE TABLE users (...); CREATE TABLE orders (...);
// migrate up on deploy; DOWN scripts for rollback rehearsal
```

## Env discipline + health/logs

Table 130 — Env rules (prod-grade)

Rule	Practice
Twelve-factor config	EVERYTHING environment-specific in env (URLs, keys, flags)
Never in repo	<code>.env</code> gitignored; <code>.env.example</code> documents NAMES only
Per-environment	dev/staging/prod dashboards hold real values (never shared!)
Rotate + scope	Least-privilege DB users; rotate on leak/offboarding

**CODE · Boot validation (crash LOUD on missing config — not at 3 AM)**

```
for (const k of ["DATABASE_URL", "JWT_SECRET"]) {
 if (!process.env[k]) { console.error(`Missing env: ${k}`); process.exit(1); }
}
app.get("/health", async (req, res) => {
 await pool.query("SELECT 1"); // DB reachable?
 res.json({ ok: true, uptime: process.uptime() });
});
```

## Smoke tests + rollback

**DEPLOY-DAY CHECKLIST**

- - Staging MIRRORED prod (migrations rehearsed there first!)
- - `npm audit` clean; `npm ci` (lockfile!) builds; `/health` green post-deploy
- - Smoke script: register → login → CRUD → logout (5 min, every deploy)
- - Logs/metrics watched 30 min; error-rate alert armed
- - Rollback ready: previous build one-click + DB backup timestamp noted
- - Status page/changelog updated; team knows the deploy window

**WARNING — DEPLOY FRIDAY 6 PM? NO.**

Ship when the team is present + traffic is low + rollback is rehearsed. Heroes deploy Friday night; seniors deploy Tuesday morning and sleep well.



**Exercise 57 — Ship the book-club API**

Managed PG + migrations + seed script + env validation + /health + smoke script + rollback rehearsal on staging. Document the runbook (30 lines) a teammate could follow blindfolded.

**SOLUTION**

Runbook = topology diagram + env list + migrate/smoke/rollback commands + alert links.  
Done = teammate deploys without calling you. That's operational maturity, proven.

## PART

# Deep Dives IV — React Pro & Performance

Routing, shared state, forms — plus speed as a feature.

---

**Ch 60** React Router, Context and Forms

**Ch 61** Web Performance — Core Web Vitals and Speed Budgets

**Ch 62** Next.js First Look — React With Superpowers

| Fast is a feature: measure it, budget it, ship it.

## CHAPTER 60

# React Router, Context and Forms

Advanced

~ 120 min

Multi-page feel, app-wide state without prop-drilling, and forms that validate beautifully.

## IN THIS CHAPTER

- Router (routes, params, nested)
- Context (theme/auth/cart)
- Controlled forms + validation
- Data loading patterns
- Auth-gated routes

Gamal React

Angela \$React

Docs

## Router: URLs for every view (npm i react-router-dom)

## CODE · Routes + params + nested layouts

```
import { BrowserRouter, Routes, Route, Link, useParams, Outlet } from "react-router-dom";
function Layout() { return (<<nav><Link to="/">Shop</Link> <Link to="/cart">Cart</Link></nav><Outlet /></>); }
function Product() { const { id } = useParams(); const { data } = useFetch(`/products/${id}`); ... }
<BrowserRouter>
 <Routes>
 <Route element={<Layout />}>
 <Route index element={<Shop />} />
 <Route path="product/:id" element={<Product />} />
 <Route path="cart" element={<Cart />} />
 <Route path="*" element={<NotFound />} />
 </Route>
 </Routes>
</BrowserRouter>
```

Table 131 — Router essentials

Tool	Job
<code>&lt;Link&gt;</code>	Client-side nav (no reload!) vs <code>&lt;a&gt;</code> (full reload)
<code>useParams</code>	URL variables ( <code>:id</code> )
<code>useSearchParams</code>	<code>?q=&amp;page=</code> state in the URL (shareable filters!)
<code>Outlet</code>	Nested route renders here (layouts!)
<code>useNavigate</code>	Redirect after actions (login → dashboard)

## Context: shared state without drilling

### CODE · Theme + cart stores via Context (small-app state, zero libs)

```
import { createContext, useContext, useState, useMemo } from "react";
const CartCtx = createContext(null);
export const useCart = () => useContext(CartCtx); // custom hook = clean API ✓
export function CartProvider({ children }) {
 const [items, setItems] = useState([]);
 const value = useMemo(() => ({
 items,
 add: id => setItems(p => [...p, id]),
 count: items.length
 })), [items]); // memo = no useless re-renders
 return <CartCtx.Provider value={value}>{children}</CartCtx.Provider>;
}
// App: <CartProvider><Router.../></CartProvider> ... any child: const { count, add } = useCart();
```

### WARNING — CONTEXT ≠ STATE MANAGER

Context broadcasts; it doesn't optimize. High-frequency updates (typing, drags) re-render ALL consumers. Split contexts (AuthCtx/CartCtx/ThemeCtx), memo values, and graduate to Zustand/Redux when updates fly.

## Forms: controlled + validated + accessible

### CODE · The professional form pattern (state + errors + aria)

```
function Checkout() {
 const [form, setForm] = useState({ email: "", city: "" });
 const [errors, setErrors] = useState({});
 const set = k => e => setForm(f => ({ ...f, [k]: e.target.value }));
 const submit = e => {
 e.preventDefault();
 const errs = {};
 if (!/^[@\s]+\.[^\s]+\.[^\s]+$/i.test(form.email)) errs.email = "Valid email
required";
 setErrors(errs);
 if (Object.keys(errs).length) return;
 placeOrder(form);
 };
 return (
 <form onSubmit={submit} noValidate>
 <label htmlFor="em">Email</label>
 <input id="em" value={form.email} onChange={set("email")} aria-invalid={!!
errors.email} aria-describedby="em-err" />
 {errors.email && <p id="em-err" role="alert">{errors.email}</p>}
 <button>Pay</button>
 </form>);
}
```

### KEY TAKEAWAY — KEY TAKEAWAY

Router owns URLS (shareable app state), Context owns SLOW shared state (auth/theme/cart), component state owns the rest (forms, toggles). Validate on submit + blur, announce with role=alert, and keep inputs controlled.

### Exercise 58 — Protected checkout flow

/cart (public) → /checkout (requires login: redirect with state.from) → /done (requires completed order). Persist cart to localStorage; restore on boot; clear after order. Deep-linking /checkout logged-out lands back post-login.

### SOLUTION

`<RequireAuth><Checkout></RequireAuth>` wrapper (Navigate to /login state={{from}}); login navigates back to from||"/"; order context guards /done. Auth + router + persistence composed — the real SPA triangle.

## CHAPTER 61

# Web Performance — Core Web Vitals and Speed Budgets

Advanced

~ 110 min

Load fast, stay smooth: measure, budget, and fix the metrics Google ranks you on.

## IN THIS CHAPTER

- LCP/INP/CLS (the three vials)
- Measuring (Lighthouse + field)
- Image + font + JS diets
- Caching + CDN strategy
- Perf budgets that stick

Performance

MDN perf

Gamal

## The three Core Web Vitals

Table 132 — What Google measures (and users feel)

Metric	Good	Means	Top fixes
LCP (Largest Paint)	$\leq 2.5s$	Hero visible fast	Preload hero img, WebP/AVIF, no render-blocking JS, SSR hero
INP (Interaction delay)	$\leq 200ms$	Clicks respond NOW	Code-split, debounce, workers, fewer listeners
CLS (Layout Shift)	$\leq 0.1$	Nothing jumps around	width/height on media, reserve ad slots, no late injects

**CODE · Hero-speed pattern (LCP killers, slain)**

```
<link rel="preload" as="image" href="hero.avif" fetchpriority="high">

<script src="app.js" defer></script> <!-- never block the hero! -->
```

## Measure like a lab + field scientist

**Table 133 — Lab (reproducible) vs Field (real users)**

Tool	Use
Lighthouse (DevTools)	Lab score + FIX LIST per metric (start here!)
WebPageTest	Filmstrip + waterfall (see WHAT blocks WHAT)
PageSpeed Insights	Lab + FIELD (CrUX real-user data!)
<code>web-vitals</code> lib	Your own RUM: send LCP/INP/CLS to analytics
DevTools Performance	Record interactions, find long tasks (>50ms = INP poison)

**TRY IT YOURSELF — THE 10-MINUTE AUDIT (DO IT ON YOUR SITE NOW)**

Lighthouse → Performance → expand the TOP opportunity → fix exactly that → re-run.  
Repeat 3×. Typical wins: hero preload (−1s LCP), image dimensions (−0.3 CLS), deferred chat widget (−400ms INP). Measured progress beats blind optimization.

## Diets: images, fonts, JavaScript

**Table 134 — The three diets**

Asset	Rules
Images	AVIF/WebP + sizes/srcset + lazy below fold + hero preload + dimensions ALWAYS
Fonts	woff2 + font-display:swap + preload ONE weight + system fallback stack
JS	Route-split (import()) + defer + tree-shake + audit deps (bundlephobia!) + workers for heavy

**CODE · Route-level splitting (ship 30KB now, rest on demand)**

```
const Admin = lazy(() => import("./Admin.js")); // React.lazy + <Suspense>
const chart = await import("./charts.js"); // vanilla: load when tab opens
```

## Caching + budgets

**Table 135 — Cache strategy (Ch 41, sharpened)**

Asset	Cache
Hashed ( <code>app.a1b2.js</code> )	<code>max-age=1y, immutable</code>
HTML shell	<code>no-cache</code> (revalidate — points at hashed assets)
API GETs	Short <code>max-age</code> + ETag/304 (fresh-ish + cheap)

### PERFORMANCE BUDGET (EXAMPLE, ENFORCE IN CI!)

- - LCP  $\leq$  2.5s · INP  $\leq$  200ms · CLS  $\leq$  0.1 (field, p75)
- - JS  $\leq$  170KB gzip home · Images  $\leq$  800KB total · Fonts  $\leq$  2 families/3 weights
- - Lighthouse perf  $\geq$  90 · Zero render-blocking scripts · Every img sized

### Quick check — CLS 0.4 likely cause?

1. Slow API
2. Unsized images/ads injected late
3. Big JS
4. HTTP/1.1

**Answer: B · Layout shift = space unreserved. Dimensions + reserved slots + skeletons fix it.**



**Exercise 59 — Budget your portfolio**

Measure field-ish (throttled Moto G4 preset), set budgets above, fix to green, add Lighthouse CI (fail <90). Screenshot before/after — performance storytelling for interviews.

**SOLUTION**

Typical journey: red (LCP 5s, CLS 0.5) → green in ~2 hours via hero preload + sizes + defer + AVIF. The waterfall BEFORE/AFTER is portfolio gold.

## CHAPTER 62

# Next.js First Look — React With Superpowers

Advanced

~ 100 min

Routing, server components, data fetching and SEO — the React meta-framework in one chapter.

## IN THIS CHAPTER

- App router (folders = routes)
- Server vs Client components
- Data fetching + caching
- Metadata/SEO + images
- When Next (and when Vite-SPA)

React

Docs

Deploy

## Folders ARE routes (App Router)

```
app/page.js → /
app/shop/page.js → /shop
app/product/[id]/page.js → /product/42 (params!)
app/layout.js → wraps ALL (nav + footer once!)
app/loading.js → instant skeleton ... app/error.js → friendly crash UI
```

Figure 27 — app/ directory → URLs (convention over config)

**CODE · Server Component by default (fetch ON the server, ship HTML)**

```
// app/product/[id]/page.js – runs on SERVER (DB/APIs direct, zero client JS!)
export async function generateMetadata({ params }) {
 const p = await getProduct(params.id);
 return { title: `${p.name} – Shop` }; // per-page SEO free ✓
}
export default async function Product({ params }) {
 const p = await getProduct(params.id); // direct DB call – no fetch dance!
 return (<article><h1>{p.name}</h1><AddButton id={p.id} /></article>);
}
// AddButton.js – "use client" ONLY for interactivity (islands, not oceans!)
"use client";
export function AddButton({ id }) { /* useState + onClick here */ }
```

**Table 136 — Server vs Client components**

Server (default)	Client ("use client")
Fetch/DB direct, secrets safe, SEO HTML	useState/effects/events/listeners
Zero JS shipped	JS shipped (keep SMALL islands!)
Cannot: hooks, window, onClick	Can: everything interactive
Rule: server until it MUST interact	Push "use client" DOWN to leaves

## Images + decision matrix

**CODE · next/image (AVIF + sizes + lazy, automatic)**

```
import Image from "next/image";
<Image src="/hero.jpg" alt="..." width={1200} height={600} priority /> // hero: priority!
```

**Table 137 — Next.js vs Vite-SPA (honest)**

Need	Pick
SEO pages, blog, shop, marketing	Next.js (SSR/SSG HTML)
Dashboards behind login, SPAs	Vite-SPA (simpler ops)
Mixed	Next.js (SSR pages + client islands)

Need	Pick
Learning React deeply first	Vite (Ch 46!) → Next after

### Exercise 60 — Migrate the shop to Next

App-router shop: SSR product pages (SEO titles), client cart island (context), next/image gallery, loading/error states, metadata per route. Compare Lighthouse + view-source HTML vs your SPA version.

#### SOLUTION

View-source now shows REAL product HTML (SEO ✓ no-JS ✓); cart stays a tiny client island; images auto-AVIF. Same React skills, production superpowers. Deploy to Vercel in one push.

## PART

# Interview & Exam Prep — Prove Your Skills

90 interview questions with model answers plus practice exams.

---

**Ch 63** Interview Bank I — HTML & CSS (30 Questions)

**Ch 64** Interview Bank II — JavaScript (35 Questions)

**Ch 65** Interview Bank III — Backend, DB, Auth & React (25 Questions)

| Interviews test communication + fundamentals. This part trains both.

## CHAPTER 63

# Interview Bank I — HTML & CSS (30 Questions)

All levels

~ 120 min

Model answers for the HTML/CSS questions employers actually ask.

## IN THIS CHAPTER

- HTML semantics + forms + a11y (10)
- CSS cascade/box/units (10)
- Layout + responsive + motion (10)
- How to answer (STAR + think-aloud)

Interview

Career

## Answering technique (read first!)

## KEY TAKEAWAY — KEY TAKEAWAY

Structure EVERY answer: **Claim** (one sentence) → **Explain** (why/how) → **Example** (code or story) → **Edge** (gotcha + fix). "Specificity scores ID-class-element; an ID beats any classes; I keep selectors flat single-class; I debug in DevTools where losers are crossed out." 30 seconds, hired energy.

## HTML (1-10)

## INTERVIEW QUESTION — 1. BLOCK VS INLINE?

Block (div/p/h1) takes full width and stacks; inline (span/a) flows in text and ignores width/height. inline-block flows inline AND accepts sizes. I center images with display:block + margin auto.

**INTERVIEW QUESTION — 2. SEMANTIC HTML — WHY BOTHER?**

Meaning for Google + screen readers + maintainability: header/nav/main/article/section/aside/footer form landmarks ("skip to content" works). Test: article = shareable alone; section = themed + heading; else div.

**INTERVIEW QUESTION — 3. LABEL + INPUT DONE RIGHT?**

`<label for="id">` matches `<input id>` + `name` for submission; clicking focuses; readers announce. Placeholders are NOT labels. Groups use fieldset + legend.

**INTERVIEW QUESTION — 4. HOW DO YOU MAKE IMAGES FAST + ACCESSIBLE?**

SVG icons, WebP/AVIF photos with picture fallbacks, width+height (CLS!), lazy below fold, hero fetchpriority high, descriptive alt (or alt="" decorative).

**INTERVIEW QUESTION — 5. LOCAL VS SESSION STORAGE VS COOKIES?**

localStorage ~5MB forever (prefs); sessionStorage per-tab (wizards); cookies 4KB sent every request (auth via HttpOnly). Never secrets in localStorage (XSS-readable).

**INTERVIEW QUESTION — 6. WHAT IS THE VIEWPORT META AND WHY?**

`<meta name="viewport" content="width=device-width, initial-scale=1">` makes mobiles use real width — without it, responsive CSS never triggers and pages render tiny.

**INTERVIEW QUESTION — 7. GET VS POST FORMS?**

GET appends to URL (shareable, bookmarkable, length-limited, never secrets); POST hides in body (creates/updates, file uploads, CSRF-protected). Search = GET; register = POST.

**INTERVIEW QUESTION — 8. HOW DO YOU AUDIT ACCESSIBILITY?**

Keyboard-only run (Tab/Enter/focus visible), Lighthouse a11y 95+, contrast 4.5:1, labels/alt/landmarks, aria-current/labels, captions. Fix list before ship.

**INTERVIEW QUESTION — 9. DEFER VS ASYNC SCRIPTS?**

Both download in parallel; defer runs IN ORDER after parsing (default choice); async runs WHEN READY (orderless — analytics/ads). Neither blocks rendering like plain scripts.

**INTERVIEW QUESTION — 10. HOW DO YOU SEO A PAGE?**

Unique title + honest description, semantic headings, fast mobile-friendly (vitals!), descriptive alt/filenames, OG tags, canonical, sitemap + HTTPS. Content quality beats tricks.

## CSS core (11-20)

**INTERVIEW QUESTION — 11. SPECIFICITY BATTLE — WHO WINS?**

Score ID-class-element left-to-right: #nav(1-0-0) beats .a.b.c(0-3-0). I write flat single classes so ORDER decides; !important only for utilities; DevTools shows losers crossed out.

**INTERVIEW QUESTION — 12. CONTENT-BOX VS BORDER-BOX?**

content-box: width = content only (padding/border ADD → 344px surprise). border-box: width = total (sane). Universal `, ::before, ::after{box-sizing:border-box}` opens every stylesheet I write.

**INTERVIEW QUESTION — 13. FONT-SIZE VS FONT-WEIGHT?**

Size = spatial scale (rem/px/vw); weight = stroke thickness (100-900). Big-light vs small-bold prove independence. rem type (user-scalable) + loaded real weights (no faux bold).

**INTERVIEW QUESTION — 14. WIDTH VS MAX-WIDTH? HEIGHT VS MIN-HEIGHT?**

Fixed width overflows phones → width:100% + max-width caps. Fixed height spills wrapped text → min-height elastic. Containers fluid, media aspect-ratio locked.



**INTERVIEW QUESTION — 15. REM VS EM VS PX?**

rem = root (type/spacing, accessible); em = parent-context (text-hugging padding); px = hairlines/borders. Body in rem respects browser font settings — legal-grade a11y.

**INTERVIEW QUESTION — 16. WHY IS MY MARGIN NOT WORKING? (COLLAPSING!)**

Vertical margins merge (max wins), parent/child merge through, empty elements collapse. Fixes: one-direction margins, parent padding, or gap in flex/grid (never collapses).

**INTERVIEW QUESTION — 17. POSITION ABSOLUTE WENT TO THE PAGE CORNER — WHY?**

Absolute anchors to nearest POSITIONED ancestor; all-static parents → viewport root. Fix: position:relative on the intended parent. Bonus: fixed = viewport; sticky = threshold + scroll parent.

**INTERVIEW QUESTION — 18. Z-INDEX IGNORED — DEBUG STEPS?**

z-index needs positioned/flex/grid items; then check stacking-context traps (parent opacity<1, transform, filter, z-index). Fix structure, not bigger numbers; use a --z-* scale.

**INTERVIEW QUESTION — 19. LINE-HEIGHT PX VS UNITLESS?**

Unitless (1.6) multiplies each element's own size — headings/paragraphs both perfect. px inherits rigidly — 40px headings crushed into 24px lines. Unitless always.

**INTERVIEW QUESTION — 20. HOW DO YOU THEME LIGHT/DARK?**

CSS variables for ALL colors + [data-theme="dark"] overrides + JS toggle persisted to localStorage + prefers-color-scheme default. No hardcoded colors anywhere (the toggle test finds them).

## Layout + responsive + motion (21-30)

### INTERVIEW QUESTION — 21. FLEXBOX VS GRID?

Flexbox = 1D (row OR column): navs, rows, centering. Grid = 2D (rows AND columns): scaffolding, galleries. Compose: grid page + flex components. justify=main/row axis, align=cross/column.

### INTERVIEW QUESTION — 22. CENTER A DIV — ALL THE WAYS?

Flex (justify+align center), grid (place-items center), absolute (50% + translate(-50%,-50%)), margin auto (known width), text-align (inline). Modern default: flex/grid one-liners.

### INTERVIEW QUESTION — 23. EXPLAIN REPEAT(AUTO-FIT, MINMAX(260PX, 1FR)).

"As many  $\geq 260\text{px}$  tracks as fit; stretch to share leftovers; drop columns as space shrinks." Responsive galleries, zero media queries. auto-fit collapses empties (vs auto-fill).

### INTERVIEW QUESTION — 24. MOBILE-FIRST WORKFLOW?

Base CSS = phone; min-width layers enhance upward. Breakpoints from CONTENT pain (resize till ugly), not devices. One codebase, fast core for all.

### INTERVIEW QUESTION — 25. CLAMP() FOR FLUID TYPE?

clamp(MIN, fluid-vw, MAX): floor + growth + ceiling. h1 clamp(1.75rem, 1rem+4vw, 3.5rem) scales smoothly with zero breakpoints and never explodes.

### INTERVIEW QUESTION — 26. 60FPS ANIMATION RULE?

Animate transform + opacity ONLY (GPU compositor). width/top/margin trigger layout+paint per frame (jank). translate instead of top; scale instead of width. Plus prefers-reduced-motion fallback.

**INTERVIEW QUESTION — 27. RESPONSIVE IMAGES STRATEGY?**

picture + WebP/AVIF sources + img fallback, srcset/sizes, lazy below fold, hero preload + fetchpriority, width/height anti-CLS. Art direction via media sources.

**INTERVIEW QUESTION — 28. BEM — SELL IT IN 20 SECONDS?**

Block__Element--Modifier: flat single classes → zero specificity wars, readable HTML, greppable, scales to 10k lines. .card__price--sale tells everything.

**INTERVIEW QUESTION — 29. PAGE HAS HORIZONTAL SCROLL ON MOBILE — FIX?**

Find overflow (DevTools, scroll the tree): fixed widths → max-width; long words → overflow-wrap; wide tables → scroll wrapper; negative margins; 100vw + scrollbar. Test 360px.

**INTERVIEW QUESTION — 30. HOW DO YOU ORGANIZE A BIG STYLESHEET?**

Tokens (variables) → reset → base/typography → layout → components (BEM) → utilities; or layers (@layer reset,base,components,utilities). One pattern, enforced by lint.

## CHAPTER 64

# Interview Bank II — JavaScript (35 Questions)

All levels

~ 150 min

The JS questions that decide offers: scope, async, DOM and architecture.

## IN THIS CHAPTER

- Language core (12)
- Functions/scope/this (8)
- Async + APIs (8)
- DOM + browser (7)

Interview

Career

## Language core (1-12)

### INTERVIEW QUESTION — 1. LET VS CONST VS VAR?

const default (binding locked), let for reassignment, var retired (function scope + silent redeclare + undefined hoisting). Block scope + TDZ for both modern ones.

### INTERVIEW QUESTION — 2. PRIMITIVES VS REFERENCES?

Primitives copy VALUES (independent); objects copy POINTERS (shared). const locks binding, not contents. Spread = shallow; structuredClone = deep.

### INTERVIEW QUESTION — 3. == VS ===?

=== always (type + value). == coerces (0=="", ""==0 chaos). Falsy eight: false, 0, -0, 0n, "", null, undefined, NaN.

**INTERVIEW QUESTION — 4. || VS ?? VS ?. ?**

|| falls back on ANY falsy (kills 0/""); ?? only null/undefined (safe defaults); ?. dives safely (undefined, no crash). Combo: user?.city ?? "Unknown".

**INTERVIEW QUESTION — 5. FOR...OF VS FOR...IN?**

of = iterable VALUES (arrays ✓); in = object KEYS (+ inherited! never for arrays). Classic for = index control; while = unknown iterations.

**INTERVIEW QUESTION — 6. MAP VS FOREACH?**

map BUILDS a new array (same length, chainable); forEach DOES effects (returns undefined). Building with forEach+push is junior style.

**INTERVIEW QUESTION — 7. REDUCE IN ONE BREATH?**

Combines all into one: reduce((acc,x)=>..., INIT). Always pass INIT (empty arrays throw without). Totals, groups, histograms.

**INTERVIEW QUESTION — 8. SORT() SURPRISE?**

Default is ALPHABETIC ([10,9,80]→[10,80,9]!) and IN-PLACE. Numbers need (a,b)=>a-b; copy first ([...a].sort / toSorted) unless mutation intended.

**INTERVIEW QUESTION — 9. DESTRUCTURING + SPREAD DEPTH?**

Unpack/merge in one gesture — but SHALLOW (nested shared). Deep = structuredClone. Rest collects (...args); spread expands f(...arr).

**INTERVIEW QUESTION — 10. TEMPLATE LITERALS — WHY DEFAULT?**

``` injection + multiline + tagged templates (safe SQL/HTML builders) beat concatenation in readability and safety.

INTERVIEW QUESTION — 11. MAP VS OBJECT? SET SUPERPOWER?

Map: any keys + size + order + no prototype traps (dictionaries). Set: auto-unique + $O(1)$ has (dedupe: [...new Set(a)]). Objects: records + JSON.

INTERVIEW QUESTION — 12. SHALLOW VS DEEP COPY — FULL ANSWER?

Spread/slice top-level only (nested shared — the trap). structuredClone deep (no functions/DOM). JSON trick legacy (loses Dates/functions). Never mutate unowned inputs.

Functions/scope/this (13-20)

INTERVIEW QUESTION — 13. WHAT IS A CLOSURE? REAL USE?

Function + captured lexical scope = remembers birthplace vars after outer returns. Uses: private state (counters/vaults), factories, handlers remembering data. Demo createCounter on request.

INTERVIEW QUESTION — 14. THIS IN 30 SECONDS?

HOW called: obj.m()→obj; f()→undefined; new→instance; call/apply/bind→forced; arrows→inherit enclosing (no own this). Arrows for callbacks, methods for objects.

INTERVIEW QUESTION — 15. ARROW VS FUNCTION?

Arrows: short + lexical this + no arguments/prototype (not constructible). Functions: own this + hoisted declarations. Callbacks→arrows; methods/constructors→function/class.

INTERVIEW QUESTION — 16. HOISTING TRUTH?

function declarations hoist WHOLE (callable early); var hoists undefined (bug factory); let/const sit in TDZ (ReferenceError before line). Write declarations top-down anyway.

INTERVIEW QUESTION — 17. DEBOUNCE VS THROTTLE?

Debounce: run AFTER quiet (search input). Throttle: run at STEADY rate (scroll/resize). Both cap storms; debounce waits, throttle paces.

INTERVIEW QUESTION — 18. PURE FUNCTIONS — WHY?

Same input→same output, no side effects = testable, cacheable, parallelizable. Impure (random/date/mutate) isolated at edges. map/filter/reduce love purity.

INTERVIEW QUESTION — 19. REST VS SPREAD VS ARGUMENTS?

...args gathers (rest, real array, modern); f(...a) expands (spread); arguments is legacy array-like (no map, banned in arrows — use rest).

INTERVIEW QUESTION — 20. IIFE / MODULES FOR PRIVACY?

Old: IIFE ((()=>{...}))() hides scope. Modern: ESM (export only the API) + closures. Modules evaluate once = singleton by default.

Async + APIs (21-28)

INTERVIEW QUESTION — 21. EVENT LOOP IN 45 SECONDS?

Sync runs on stack; microtasks (promises) drain FULLY; render; ONE macrotask (timeout/IO); repeat. Promise.then beats setTimeout(...,0) always. Blocking sync starves render.

INTERVIEW QUESTION — 22. PROMISE STATES + COMBINATORS?

pending→fulfilled/rejected (settled once). all (need all, fail-fast), allSettled (partial OK), race (first finish), any (first success). Parallel independent work with all.

INTERVIEW QUESTION — 23. FETCH GOTCHAS?

Resolves on 404/500 (check res.ok!), rejects only network-down. Needs timeout (AbortSignal), JSON try/catch, auth headers, 4 UI states (loading/empty/error/ready).

INTERVIEW QUESTION — 24. ASYNC/AWAIT RULES?

await in async only; every await can throw → try/catch/finally; parallel via Promise.all (not sequential loops); always timeout network; rethrow or handle (no silent catch{ }).

INTERVIEW QUESTION — 25. HOW DO YOU HANDLE API ERRORS IN UI?

State machine: loading skeleton → empty / error+retry / ready. Never blank screens; toasts for mutations; optimistic UI with rollback for speed + honesty.

INTERVIEW QUESTION — 26. CALLBACK HELL → MODERN CHAIN?

Nested callbacks → promise chains (.then) → async/await (sync-looking). Errors bubble to one catch; finally cleans up. Flat > nested, always.

INTERVIEW QUESTION — 27. ABORTCONTROLLER — WHY?

Cancels in-flight fetch + abortable listeners + timeouts (AbortSignal.timeout). Prevents race conditions (old search overwriting new) and leaks on unmount.

INTERVIEW QUESTION — 28. MICROTASK STARVATION?

Infinite promise chains (recursive .then) block render/macrotasks forever — frozen UI with "working" code. Fix: yield to macrotasks (setTimeout/RAF chunks) for huge jobs.

DOM + browser (29-35)

INTERVIEW QUESTION — 29. TEXTCONTENT VS INNERHTML?

textContent = safe TEXT (user data ✓); innerHTML parses HTML (XSS with untrusted input!). Build user content via createElement/textContent; innerHTML only for own static templates.

INTERVIEW QUESTION — 30. EVENT DELEGATION — SELL IT?

One parent listener + `e.target.closest()` handles infinite/dynamic children: constant memory, zero re-binding, new rows free. Bubbling is the mechanism; `closest` finds the actor.

INTERVIEW QUESTION — 31. TARGET VS CURRENTTARGET?

`target` = deepest clicked element; `currentTarget` = listener's element. Delegation lives on their difference (button vs list). Mixing them breaks handlers.

INTERVIEW QUESTION — 32. EFFICIENT BIG-LIST RENDERING?

DocumentFragment batch (one reflow), `innerHTML`-once for trusted templates, virtualize 10k+ rows, `content-visibility:auto`, passive scroll listeners, `requestAnimationFrame` writes.

INTERVIEW QUESTION — 33. FORMDATA + VALIDATION COMBO?

`new FormData(form)` reads all named controls; Constraint API (`checkValidity`/`reportValidity`/`setCustomValidity`) for UX; server revalidates everything (browser checks are bypassable).

INTERVIEW QUESTION — 34. LOCALSTORAGE CORRUPTED — APP MUST SURVIVE?

`JSON.parse` in try/catch with defaults (`load(k, fallback)`); schema-check shapes; version keys (v1:) for migrations. Boot never crashes on bad data.

INTERVIEW QUESTION — 35. XSS IN 30 SECONDS?

Attacker JS via unsafe sinks (`innerHTML`/`location`/`eval`). Defenses: `textContent`/`createElement`, `escape` + `sanitize`, CSP headers, `HttpOnly` cookies. Stored > reflected > DOM flavors.

CHAPTER 65

Interview Bank III — Backend, DB, Auth & React (25 Questions)

All levels

~ 120 min

Server-side and React model answers: APIs, data, security and components.

IN THIS CHAPTER

- Node/Express/REST (8)
- SQL/NoSQL/modeling (6)
- Auth/security (6)
- React (5)

Interview

Career

Node/Express/REST (1-8)

INTERVIEW QUESTION — 1. NODE EVENT LOOP — WHY SINGLE-THREADED SCALES?

One thread takes requests; libuv pool does I/O async; callbacks resume. Thousands concurrent, zero thread overhead. Rule: NEVER block (no sync I/O/crypto in handlers).

INTERVIEW QUESTION — 2. DESIGN POST /ORDERS (REST)?

Noun URL + version (/api/v1/orders), validate body → 400 {errors}, create → 201 + resource, auth → 401/403, errors one shape {error:{message}}, idempotency key for retries.

INTERVIEW QUESTION — 3. MIDDLEWARE ORDER MATTERS — PROVE IT?

cors → json → logger → auth → routes → 404 → error-last. json after routes = body undefined; error handler not last = crashes escape; auth placement scopes protection.

INTERVIEW QUESTION — 4. CENTRAL ERROR HANDLING?

next(err) → 4-arg middleware (last!): log full, send {status, generic-or-known message}, never stacks to clients, separate 4xx (client) vs 5xx (bugs + alerts).

INTERVIEW QUESTION — 5. PAGINATION THAT SCALES?

limit/offset + caps (max 100) + meta {page,pages,total}; deep pages → keyset (WHERE id > last, ORDER BY id LIMIT n). Never unbounded lists.

INTERVIEW QUESTION — 6. FILE UPLOADS SAFELY?

Size caps, mime+extension allowlist, uuid renames (path traversal!), outside webroot/object storage, image re-encode (strip metadata), async scan. Serve with safe Content-Type.

INTERVIEW QUESTION — 7. CORS ERROR — WHAT'S HAPPENING?

Browser blocks cross-origin reads without server permission. Fix SERVER-side: exact origins (never \* with credentials), methods/headers allowlist, preflight handling. Dev: Vite proxy.

INTERVIEW QUESTION — 8. HOW DO YOU STRUCTURE AN EXPRESS APP?

routes (validate+call) → services (logic) → data layer (DB); middleware (auth/errors/log); config from env; versioned paths; tests per layer. Thin routes, fat services.

SQL/NoSQL/modeling (9-14)

INTERVIEW QUESTION — 9. SQL VS NOSQL — DECIDE LIKE A SENIOR?

Money/relations/reports → SQL (ACID+JOINS+constraints). Flexible/reads/shapeshift → NoSQL (embed read-together). Split per bounded context, not per company. Both is normal.

INTERVIEW QUESTION — 10. INNER VS LEFT JOIN?

INNER = matched pairs only; LEFT = all left + NULLs (find inactive users!). Draw the Venn; default to LEFT for reports including zeros.

INTERVIEW QUESTION — 11. N+1 AND THE FIX?

1 query + N follow-ups (loop-fetching relations) = latency death. Fix: JOIN (SQL) / populate+lean (Mongo) / dataloaders (GraphQL). Prove with query logs before/after.

INTERVIEW QUESTION — 12. EMBED VS REFERENCE IN MONGO?

Embed: read-together + bounded (post+comments). Reference: independent/unbounded/shared (users, products). Overflow pattern for grown arrays (latest N + count).

INTERVIEW QUESTION — 13. TRANSACTIONS — WHEN?

Multi-write integrity (transfer, checkout): BEGIN...COMMIT, ROLLBACK on any failure, balance checks INSIDE the txn (race-safe). Money without txns = eventually wrong.

INTERVIEW QUESTION — 14. INDEX STRATEGY IN 30 SECONDS?

Index WHERE/JOIN/ORDER paths (compound order matters!), EXPLAIN ANALYZE to prove, partial/expression for hot slices. Every index taxes writes — index reads you HAVE.

Auth/security (15-20)

INTERVIEW QUESTION — 15. BCrypt — WHY AND HOW?

Adaptive one-way hash + auto-salt; cost 12 (~250ms); compare() timing-safe. Same 401 message either way (no user enumeration). Never plain/MD5/SHA-direct.

INTERVIEW QUESTION — 16. SESSIONS VS JWT?

Sessions: server store + instant revoke (monoliths). JWT: stateless verify + short access + rotating refresh in HttpOnly cookie (APIs/mobile). Payload is PUBLIC (signed, not secret).

INTERVIEW QUESTION — 17. WHERE DO TOKENS LIVE?

Access in memory, refresh in HttpOnly+Secure+SameSite cookie. Never localStorage (XSS steals all). Short expiry + rotation + versioned global logout.

INTERVIEW QUESTION — 18. TOP 3 VULNERABILITIES + FIXES?

Injection (parameterize + validate), XSS (safe render + CSP + HttpOnly), broken auth (bcrypt + short tokens + rate limits). Then CSRF (SameSite+tokens) + headers/rate hygiene.

INTERVIEW QUESTION — 19. CSRF DEFENSE STACK?

SameSite cookies (primary) + per-form tokens + custom-header/API preflight + GET-never-mutates. Explain the evil.com-auto-posting-to-bank.com story.

INTERVIEW QUESTION — 20. RATE LIMITING — WHERE AND HOW?

Auth routes strict (20/15min), APIs generous (300/15min), 429 + Retry-After, keyed by IP+user, Redis store in prod. Brute-force dies here.

React (21-25)

INTERVIEW QUESTION — 21. PROPS VS STATE?

Props flow DOWN read-only (parent owns); state is owned + setState re-renders. Lift shared state to lowest common parent; derive (totals) instead of storing.

INTERVIEW QUESTION — 22. WHY KEYS? WHY NOT INDEX?

Keys = stable identity across renders (preserve state/focus, minimal DOM ops). Index breaks on reorder/filter (wrong state attached). Use ids, always.

INTERVIEW QUESTION — 23. USEEFFECT DEPS ARRAY?

[] once; [x] on change; missing = stale closures; absent = every render (loop risk).
Cleanup (abort/unsubscribe) prevents leaks + StrictMode double-runs.

INTERVIEW QUESTION — 24. CONTROLLED VS UNCONTROLLED?

Controlled: value+onChange in state (validate/disable/instant UX — default).
Uncontrolled: refs read on demand (simple one-shots, file inputs, focus).

INTERVIEW QUESTION — 25. MUTATING STATE DIRECTLY — WHAT BREAKS?

Same reference = React skips render (silent stale UI). Always new objects ([...p], filter, map). Immutability enables cheap change detection + time-travel.

PART

Exams & Code Vault — Test and Copy

Timed practice exams with full solutions plus copy-paste production snippets.

Ch 66 Graded Exam I — HTML & CSS (25 Marks)

Ch 67 Graded Exam II — JavaScript (25 Marks)

Ch 68 Graded Exam III — Backend, DB, Auth & React (25 Marks)

| Exam rule: closed book first, solutions after, re-take in a week.

CHAPTER 66

Graded Exam I — HTML & CSS (25 Marks)

Beginner-Intermediate

~ 90 min

Closed-book first. Mark honestly. Revisit weak chapters, re-take in 7 days.

IN THIS CHAPTER

- Section A: quick answers (10)
- Section B: code reading (8)
- Section C: build tasks (7)
- Full solutions + marking guide

Exam

Certificate

Section A — quick answers (10 × 1 mark)

Quick check — A1. Which element for the main article?

1. div
2. article
3. section
4. span

Answer: B ·

Quick check — A2. `box-sizing: border-box` means...

1. Padding adds outside
2. Width includes padding+border
3. Margins merge
4. Height fixed

Answer: B ·**Quick check — A3.** `#a.b` vs `.x.y.z` — winner?

1. `#a.b`
2. `.x.y.z`
3. Tie → later wins
4. Neither

Answer: A ·**Quick check — A4.** `repeat(auto-fit, minmax(200px, 1fr))` is...

1. Fixed 3 cols
2. Responsive auto columns
3. Flex row
4. Error

Answer: B ·**Quick check — A5.** `em` vs `rem`?

1. Same
2. `em=root`
3. `rem=root`, `em=context`
4. Both px-locked

Answer: C ·

Quick check — A6. 60fps-safe properties?

1. width/top
2. transform/opacity
3. margin/padding
4. font-size

Answer: B ·

Quick check — A7. `clamp(1rem, 2vw+1rem, 2rem)` ...

1. Breakpoints
2. Fluid bounded size
3. Grid tracks
4. Animation

Answer: B ·

Quick check — A8. Alt for decorative image?

1. Skip attr
2. `alt=""`
3. `alt="img"`
4. `role=img`

Answer: B ·

Quick check — A9. Sticky needs...

1. z-index only
2. threshold + scrollable ancestor
3. fixed parent
4. JS

Answer: B ·

Quick check — A10. prefers-reduced-motion honors...

1. Print
2. Vestibular sensitivity
3. Dark mode
4. Touch

Answer: B ·

Section B — read the code (4 × 2 marks)

CODE · /\* B1-B2 (2 marks each): predict \*/

```
.box { width: 300px; padding: 20px; border: 5px solid; margin: 10px; box-sizing: content-box; }  
/* B1: rendered OUTER width? */ /* B2: with border-box instead? */
```

CODE · /\* B3-B4 \*/

```
.card { display: flex; gap: 1rem; } .card img { flex-shrink: 0; width: 120px; }  
/* B3: why does the image keep 120px in tight rows? */ /* B4: replace gap with margins —  
what breaks? */
```

Section C — build (2 + 2 + 3 marks)

Exercise 61 — C1 (2 marks). Accessible login form

Email + password + remember + submit; labels, names, required, autocomplete, error region.

SOLUTION

form>label+input#email(type=email,required,autocomplete=email,name=email) +
password(current-password,minlength) + checkbox remember + button; div role=alert for
errors; novalidate only if JS-handled (still validate!).

Exercise 62 — C2 (2 marks). Holy-grail, no media queries

Header/footer + sidebar/main/aside collapsing gracefully on phones.

SOLUTION

grid: "h h h" "s m a" "f f f" / 200px 1fr 200px → wrap in auto-fit alternative OR minmax(0,1fr) + order; simplest pass: flex-wrap columns with flex:1 1 200px main:3 1 300px.

Exercise 63 — C3 (3 marks). Responsive card gallery

Auto-fit minmax cards, aspect-ratio images, hover lift, dark theme via variables.

SOLUTION

grid auto-fit minmax(240px,1fr); img aspect-ratio:16/10 object-fit:cover w/h set; :hover translateY(-4px)+shadow; colors 100% variables + [data-theme=dark]; reduced-motion kills lift.

Marking guide

Table 138 — Grade yourself

| Score | Level | Action |
|-------|--------------|---|
| 22-25 | Exam-ready ✓ | Take Exam II; build portfolio project |
| 17-21 | Solid | Re-read chapters of missed marks; re-take in 3 days |
| 11-16 | Gaps | Re-do Parts 2-3 exercises; re-take in 7 days |
| 0-10 | Restart path | Read Parts 1-3 with the three-pass method (Ch 1) |

| Score | Level | Action |
|--|-------|--------|
| B1=370px (300+40+10; margins extra!) ... B2=300px ...
B3=shrink:0 exempts compression ... B4=margins double
at wraps + collapse bugs | | |

CHAPTER 67

Graded Exam II — JavaScript (25 Marks)

Intermediate

~ 90 min

Predict outputs, debug snippets, and write functions from scratch.

IN THIS CHAPTER

- Section A: predict output (10)
- Section B: debug (8)
- Section C: write code (7)
- Solutions + marking guide

Exam

Certificate

Section A — predict (10 × 1 mark)

CODE · /\* A1-A5: write the output \*/

```
console.log(typeof null);           // A1
console.log(0.1 + 0.2 === 0.3);     // A2
console.log([... "hello"].length);  // A3
console.log([1, 2, 3].map(n => n * 2)); // A4
let a = { n: 1 }; let b = a; b.n = 9; console.log(a.n); // A5
```

CODE · /\* A6-A10 \*/

```
console.log("5" - 2, "5" + 2);      // A6
console.log([10, 9, 80].sort());    // A7
const p = new Promise(r => r(1)); p.then(v => console.log("p", v)); console.log("s"); //
A8 order?
console.log(null ?? "d", 0 || "d", 0 ?? "d"); // A9
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0); // A10
```

Section B — debug (4 × 2 marks)

CODE · /\* B1: crashes on empty array — fix + explain \*/

```
const avg = nums => nums.reduce((s, n) => s + n) / nums.length;
// B2: copy trap — caller's object mutates!
const rename = (u, name) => { u.name = name; return u; };
// B3: sequential slowness — 3 × 800ms. Parallelize.
const a = await get("/a"); const b = await get("/b"); const c = await get("/c");
// B4: listener leak — every render adds another!
useEffect(() => { window.addEventListener("resize", onR); });
```

Section C — write (2 + 2 + 3 marks)

Exercise 64 — C1 (2). groupBy(items, keyFn)

Return {key: [items]}. Empty-safe.

SOLUTION

```
const groupBy=(a,f)=>a.reduce((o,x)=>{const k=f(x);(o[k]?=[]).push(x);return o},{}) — init  
{ } handles empty ✓.
```

Exercise 65 — C2 (2). debounce(fn, ms)

Trailing-edge with cancel().

SOLUTION

```
const debounce=(f,ms)=>{let t;const w(...a)=>{clearTimeout(t);t=setTimeout(()=>f(...a),ms)};w.cancel=()=>{clearTimeout(t);t=null};return w}
```

Exercise 66 — C3 (3). `fetchJson(url, {timeout, retries})`

ok-check + timeout + 1 retry on 5xx/network + typed error.

SOLUTION

Loop attempts: `AbortSignal.timeout`; `!ok` → throw status (retry iff ≥ 500); network throws → retry once; final throw `Error( GET ${url}: ${msg} )`. Always 4-state UI around it.

Table 139 — Marking guide

| Item | Answer |
|--|---|
| A1–A5 | "object" (historic bug!) · false (0.30000000000000004) · 5 · [2,4,6] · 9 (shared ref!) |
| A6–A10 | 3, "52" · [10,80,9] (alpha!) · "s" then "p 1" · "d","d",0 · 3,3,3 (var shared!) |
| B1–B4 | reduce INIT 0 + length guard · spread/structuredClone before edit · Promise.all · return cleanup removeEventListener + [deps] |
| 22–25 exam-ready · 17–21 solid · 11–16 gaps (Parts 4–6) · 0–10 restart Parts 4–5 | |

CHAPTER 68

Graded Exam III — Backend, DB, Auth & React (25 Marks)

Advanced

~ 110 min

Design APIs, write SQL, reason about auth, and debug React.

IN THIS CHAPTER

- Section A: design decisions (10)
- Section B: write SQL/JS (8)
- Section C: architecture mini-task (7)
- Solutions + marking guide

Exam

Certificate

Section A — decide (10 × 1 mark)

Quick check — A1. Create order →

1. GET /orders
2. POST /api/v1/orders
3. POST /createOrder
4. PUT /orders

Answer: B ·

Quick check — A2. Missing token →

1. 404
2. 401
3. 403
4. 500

Answer: B ·**Quick check — A3. LEFT JOIN users-orders finds...**

1. Buyers only
2. All users + NULLs
3. Orders only
4. Duplicates

Answer: B ·**Quick check — A4. Password storage?**

1. SHA-256
2. AES
3. bcrypt + salt
4. Base64

Answer: C ·**Quick check — A5. JWT payload is...**

1. Encrypted
2. Public (signed)
3. Server-only
4. Hashed

Answer: B ·

Quick check — A6. Tokens live...

1. localStorage
2. Memory + HttpOnly cookie
3. URL
4. CSS var

Answer: B ·

Quick check — A7. N+1 fix?

1. More queries
2. JOIN/populate
3. Bigger server
4. Cache all

Answer: B ·

Quick check — A8. List keys should be...

1. Index
2. Random
3. Stable ids
4. None

Answer: C ·

Quick check — A9. 429 means...

1. Auth fail
2. Rate limited
3. Gone
4. Timeout

Answer: B ·

Quick check — A10. useEffect [] runs...

1. Every render
2. Once
3. Never
4. On unmount

Answer: B ·

Section B — write (4 × 2 marks)

Exercise 67 — B1. SQL: top 5 customers by spend**SOLUTION**

```
SELECT u.name, SUM(o.total) s FROM users u JOIN orders o ON o.user_id=u.id GROUP BY u.id ORDER BY s DESC
```

Exercise 68 — B2. Express: validated POST /notes**SOLUTION**

Router + 400-shape validation + 201 + next(err); auth before route; error middleware last.
(Ch 39 pattern verbatim.)

Exercise 69 — B3. React: fetch-on-mount with cleanup**SOLUTION**

```
useEffect(()=>{const c=new AbortController();(async()=>{setS("loading");try{setD(await getJSON(u,c.signal
```

Exercise 70 — B4. bcrypt register/login pair**SOLUTION**

Register: validate → exists?409 → hash(12) → save(id only back). Login: find → compare → same-401 → issue tokens. (Ch 44.)

Section C — architecture (7 marks)**Exercise 71 — C1 (7). Design "CampBook" backend (campsite reviews)**

Resources + routes + schemas (SQL or Mongo, justified!) + auth + 2 indexes + 1 transaction + caching + rate limits + deploy sketch.

SOLUTION

Model answer: users/camps/reviews; REST /api/v1/\*; SQL (relations+reports: avg ratings, bookings integrity) with camps(user FK), reviews(user+camp FK, UNIQUE pair!); bcrypt+JWT; idx reviews(camp,created)+users(email); booking txn; GET cache 60s; auth RL 20/15m; Render+Neon+Vercel topology; smoke+rollback. Marks: nouns 1, verbs/codes 1, schema+justified 1, constraints/idx 1, auth/sec 1, txn 1, ops 1.

Table 140 — Final grading (all three exams)

| Total /75 | Verdict |
|-----------|--|
| 66–75 | Job-ready: apply + ship capstone (Part 11) + interview banks |
| 51–65 | Strong junior: close gaps via study plan, re-take weak exam |
| 36–50 | Learning: guided re-read of failed parts, then re-take |
| 0–35 | Foundations: Parts 1–4 slowly with three-pass method |

| Total /75 | Verdict |
|---|---------|
| Celebrate honestly-earned scores — then BUILD (employers hire proof). | |

Speed drills — 10-minute blitzes (answers at the end of the drill)

Quick check — D1. `box-sizing` default?

1. border-box
2. content-box
3. padding-box
4. none

Answer: B ·

Quick check — D2. `#id` vs `.a.b.c` ?

1. ID wins
2. Classes win
3. Tie
4. Error

Answer: A ·

Quick check — D3. `typeof null` ?

1. null
2. object
3. undefined
4. number

Answer: B ·

Quick check — D4. `[10,9,80].sort()` →

1. [9,10,80]
2. [10,80,9]
3. [80,10,9]
4. Error

Answer: B ·

Quick check — D5. `0 ?? "d"` →

1. "d"
2. 0
3. null
4. Error

Answer: B ·

Quick check — D6. Missing token →

1. 404
2. 401
3. 403
4. 500

Answer: B ·

Quick check — D7. POST /orders creates →

1. 200
2. 201
3. 204
4. 302

Answer: B ·

Quick check — D8. JWT payload is...

1. Encrypted
2. Public signed
3. Server-only
4. Hashed

Answer: B ·**Quick check — D9. N+1 fix?**

1. More queries
2. JOIN/populate
3. Bigger server
4. Cache all

Answer: B ·**Quick check — D10. List keys?**

1. Index
2. Random
3. Stable ids
4. None

Answer: C ·**Exercise 72 — D11 (blitz build, 10 min). Mini card grid**

Auto-fit cards from a JS array (map → join), one button per card (delegation!), click → toast the title. No frameworks.

SOLUTION

```
grid.innerHTML = items.map(i => <article class="card"><h3>${i.t}</h3><button data-id="${i.id}">Hi</button></article> ).join("") + delegated click → toast. (Escape titles in prod — Ch 29!)
```


PART

Code Vault — Copy-Paste Production Snippets

60 battle-tested snippets: CSS, JS, Express, SQL and React.

Ch 69 Snippet Vault I — CSS (20 Snippets)

Ch 70 Snippet Vault II — JavaScript (20 Snippets)

Ch 71 Snippet Vault III — Backend, SQL & React (20 Snippets)

| All snippets are starter-safe: paste, rename, ship. Adapt — never blindly trust.

CHAPTER 69

Snippet Vault I — CSS (20 Snippets)

All levels

~ 60 min

The CSS you will paste in every project: resets, layouts, components and effects.

IN THIS CHAPTER

- Reset + tokens + type (1-5)
- Layout one-liners (6-10)
- Components (11-15)
- Effects + utilities (16-20)

Snippets

Copy-paste

CODE · 1. Modern reset (every project starts here)

```
*, *::before, *::after { box-sizing: border-box; }
* { margin: 0; }
body { line-height: 1.6; font-family: system-ui, sans-serif; }
img, svg, video { display: block; max-width: 100%; }
input, button, textarea, select { font: inherit; }
```

CODE · 2. Theme tokens (light + dark)

```
:root { --bg: #fff; --ink: #0f172a; --brand: #1e3a8a; --gold: #c9a227; --radius: 12px; }
[data-theme="dark"] { --bg: #0b1026; --ink: #e8ecf8; --brand: #7c93ff; }
body { background: var(--bg); color: var(--ink); }
```

CODE · 3. Fluid type scale

```
body { font-size: clamp(1rem, .95rem + .25vw, 1.125rem); }
h1 { font-size: clamp(2rem, 1.2rem + 4vw, 3.5rem); line-height: 1.1; }
h2 { font-size: clamp(1.5rem, 1.1rem + 2vw, 2.25rem); }
```

CODE · 4. Container + section rhythm

```
.wrap { width: min(1100px, 100% - 2rem); margin-inline: auto; }
section { padding-block: clamp(2.5rem, 6vw, 5rem); }
.stack > * + * { margin-top: var(--gap, 1rem); }
```

CODE · 5. Focus-visible + reduced motion

```
:focus-visible { outline: 3px solid var(--gold); outline-offset: 2px; }
@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after { animation-duration: .01ms !important; transition-
duration: .01ms !important; }
}
```

CODE · 6. Perfect centering (3 flavors)

```
.center-flex { display: flex; align-items: center; justify-content: center; }
.center-grid { display: grid; place-items: center; }
.center-abs { position: absolute; inset: 50% auto auto 50%; translate: -50% -50%; }
```

CODE · 7. Auto gallery (zero queries)

```
.gallery { display: grid; gap: 1rem; grid-template-columns: repeat(auto-fit,
minmax(240px, 1fr)); }
.gallery img { aspect-ratio: 4/3; object-fit: cover; width: 100%; }
```

CODE · 8. Holy grail (modern)

```
.page { display: grid; gap: 1rem; grid-template: "h h" auto "s m" 1fr "f f" auto / 220px
1fr; min-height: 100vh; }
@media (max-width: 700px) { .page { grid-template: "h" "m" "s" "f" / 1fr; } }
```

CODE · 9. Sidebar + fluid main

```
.split { display: flex; flex-wrap: wrap; gap: 1rem; }
.split > :first-child { flex: 1 1 220px; } .split > :last-child { flex: 3 1 320px; }
```

CODE · 10. Sticky table headers + scroll shadows

```
.table-wrap { overflow: auto; max-height: 60vh; }  
thead th { position: sticky; top: 0; background: var(--bg); box-shadow: 0 1px 0 #ddd; }
```

CODE · 11. Buttons (primary/ghost/danger)

```
.btn { display: inline-flex; align-items: center; gap: .5rem; padding: .6rem 1.2rem;  
border-radius: 999px; border: 1px solid transparent; font-weight: 600; cursor: pointer; }  
.btn-primary { background: var(--brand); color: #fff; }  
.btn-ghost { border-color: currentColor; background: transparent; }  
.btn-danger { background: #b91c1c; color: #fff; }  
.btn:disabled { opacity: .5; cursor: not-allowed; }
```

CODE · 12. Card + lift

```
.card { background: var(--bg); border: 1px solid #e2e8f0; border-radius: var(--radius);  
padding: 1.25rem; box-shadow: 0 1px 2px rgb(0 0 0 / .06); transition: translate .2s, box-  
shadow .2s; }  
.card:hover { translate: 0 -4px; box-shadow: 0 12px 24px rgb(0 0 0 / .12); }
```

CODE · 13. Form controls (consistent)

```
.field { display: grid; gap: .35rem; margin-bottom: 1rem; }  
.input, select, textarea { padding: .6rem .8rem; border: 1px solid #cbd5e1; border-  
radius: 8px; background: #fff; }  
.input[aria-invalid="true"] { border-color: #b91c1c; box-shadow: 0 0 0 3px rgb(185 28  
28 / .15); }
```

CODE · 14. Badge + avatar + divider

```
.badge { display: inline-block; padding: .15rem .6rem; border-radius: 999px; background:  
#eef2ff; font-size: .8rem; font-weight: 600; }  
.avatar { width: 40px; height: 40px; border-radius: 50%; object-fit: cover; }  
.hr { border: 0; border-top: 1px solid #e2e8f0; margin-block: 1.5rem; }
```

CODE · 15. Navbar (sticky + mobile row)

```
.nav { position: sticky; top: 0; z-index: 50; display: flex; align-items: center; gap:  
1rem; padding: .75rem 1rem; background: color-mix(in srgb, var(--bg) 85%, transparent);  
backdrop-filter: blur(8px); }
```

CODE · 16. Skeleton shimmer

```
.skeleton { border-radius: 8px; background: linear-gradient(90deg, #e5e7eb 25%, #f3f4f6 50%, #e5e7eb 75%); background-size: 200% 100%; animation: shimmer 1.2s infinite; min-height: 1rem; }  
@keyframes shimmer { to { background-position: -200% 0; } }
```

CODE · 17. Truncate (1 + N lines)

```
.cut-1 { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }  
.cut-3 { display: -webkit-box; -webkit-line-clamp: 3; -webkit-box-orient: vertical; overflow: hidden; }
```

CODE · 18. Visually-hidden (a11y)

```
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path: inset(50%); white-space: nowrap; }
```

CODE · 19. Print stylesheet (invoices!)

```
@media print {  
  .nav, .no-print { display: none !important; }  
  body { font-size: 11pt; color: #000; }  
  a[href^="http"]::after { content: " (" attr(href) ")"; }  
}
```

CODE · 20. Scrollbar + selection polish

```
::selection { background: var(--gold); color: #111; }  
* { scrollbar-width: thin; scrollbar-color: var(--brand) transparent; }
```

CHAPTER 70

Snippet Vault II — JavaScript (20 Snippets)

All levels

~ 60 min

DOM, fetch, forms, storage and async helpers — the JS paste-board.

IN THIS CHAPTER

- DOM helpers (21-26)
- Fetch + async (27-31)
- Forms + validation (32-35)
- Storage + utils (36-40)

Snippets

Copy-paste

CODE · 21. \$ helpers (query shortcuts)

```
const $ = (s, r = document) => r.querySelector(s);
const $$ = (s, r = document) => [...r.querySelectorAll(s)];
```

CODE · 22. el() builder (safe DOM, no innerHTML)

```
const el = (tag, attrs = {}, ...kids) => {
  const n = document.createElement(tag);
  for (const [k, v] of Object.entries(attrs)) k === "class" ? n.className = v :
n.setAttribute(k, v);
  n.append(...kids);
  return n;
};
list.append(el("li", { class: "row" }, document.createTextNode(userInput))); // XSS-safe
✓
```

CODE · 23. Delegated actions (one listener, infinite buttons)

```
document.addEventListener("click", e => {
  const b = e.target.closest("[data-act]");
  if (!b) return;
  ({ del: () => b.closest("li").remove(), done: () =>
  b.closest("li").classList.toggle("done") })[b.dataset.act]?.();
});
```

CODE · 24. Modal (accessible, 20 lines)

```
const openModal = dlg => { dlg.showModal();
dlg.querySelector("[autofocus],button,input)?.focus(); };
document.addEventListener("click", e => {
  if (e.target.matches("[data-close]")) e.target.closest("dialog").close();
});
document.addEventListener("keydown", e => { if (e.key === "Escape") $
("dialog[open]")?.close(); });
```

CODE · 25. Tabs (ARIA-correct)

```
$$('[role="tab"]').forEach(t => t.addEventListener("click", () => {
  $$('[role="tab"]').forEach(x => { x.setAttribute("aria-selected", x === t); $
(x.getAttribute("aria-controls")).hidden = x !== t; });
}));
```

CODE · 26. Toast notifications

```
const toast = (msg, type = "ok") => {
  const t = el("div", { class: `toast ${type}`, role: "status" },
document.createTextNode(msg));
  $("#toasts").append(t); setTimeout(() => t.remove(), 3500);
};
```

CODE · 27. fetchJson (timeouts + ok-check + retries)

```

async function fetchJson(url, { timeout = 8000, retries = 1, ...opts } = {}) {
  for (let i = 0; i <= retries; i++) {
    try {
      const r = await fetch(url, { ...opts, signal: AbortSignal.timeout(timeout) });
      if (!r.ok) { if (r.status < 500 || i === retries) throw new Error(`HTTP ${r.status}`);
    }; continue; }
    return await r.json();
  } catch (e) { if (i === retries) throw e; }
}

```

CODE · 28. debounce + throttle

```

const debounce = (f, ms = 300) => { let t; return (...a) => { clearTimeout(t); t =
setTimeout(() => f(...a), ms); } };
const throttle = (f, ms = 200) => { let last = 0; return (...a) => { const n =
Date.now(); if (n - last >= ms) { last = n; f(...a); } }; };

```

CODE · 29. Async state machine (loading/empty/error/ready)

```

const setUI = (state, data) => ({ loading: showSkeleton, empty: showEmpty, error: e =>
showError(e, retry), ready: paint }[state]?.(data));

```

CODE · 30. Parallel + partial-OK patterns

```

const [users, products] = await Promise.all([get("/users"), get("/products")]);
const results = await Promise.allSettled(jobs.map(run)); // some may fail ✓

```

CODE · 31. Lazy-load on scroll (IntersectionObserver)

```

const io = new IntersectionObserver(es => es.forEach(e => {
  if (!e.isIntersecting) return; e.target.src = e.target.dataset.src;
  io.unobserve(e.target);
}), { rootMargin: "200px" });
$$("img[data-src]").forEach(img => io.observe(img));

```

CODE · 32. Serialize form → object

```

const formData = form => Object.fromEntries(new FormData(form).entries());

```


CODE · 33. Validate + announce errors

```
const fail = (input, msg) => { input.setAttribute("aria-invalid", "true"); $
("#err").textContent = msg; input.focus(); };
```

CODE · 34. Live search filter

```
$("#q").addEventListener("input", debounce(e => {
  const q = e.target.value.trim().toLowerCase();
  $("#rows li").forEach(li => li.hidden = !li.textContent.toLowerCase().includes(q));
}, 200));
```

CODE · 35. Confirm dangerous actions

```
form.addEventListener("submit", e => { if (!confirm("Delete forever?"))
e.preventDefault(); });
```

CODE · 36. Safe storage (never crashes boot)

```
const store = {
  load: (k, fb) => { try { const v = JSON.parse(localStorage.getItem(k)); return v ??
fb; } catch { return fb; } },
  save: (k, v) => { try { localStorage.setItem(k, JSON.stringify(v)); } catch {} }
};
```

CODE · 37. fmt: money + dates + relative (Intl)

```
const egp = new Intl.NumberFormat("ar-EG", { style: "currency", currency: "EGP" });
const ago = new Intl.RelativeTimeFormat("en", { numeric: "auto" });
egp.format(1299); ago.format(-2, "day"); // "2" · "٠.٢٩٩٠٠ days ago"
```

CODE · 38. uuid + random helpers

```
const uid = () => crypto.randomUUID();
const pick = a => a[Math.floor(Math.random() * a.length)];
const shuffle = a => a.map(v => [Math.random(), v]).sort((x, y) => x[0] - y[0]).map([,
v]) => v);
```

CODE · 39. copy() deep + download()

```
const deep = structuredClone;
const download = (name, text) => { const a = el("a", { href: URL.createObjectURL(new
Blob([text])), download: name }); a.click(); };
```

CODE · 40. onReady + once helpers

```
const ready = fn => document.readyState !== "loading" ? fn() :
document.addEventListener("DOMContentLoaded", fn, { once: true });
const once = fn => { let done = false; return (...a) => done ? undefined : (done = true,
fn(...a)); };
```

CHAPTER 71

Snippet Vault III — Backend, SQL & React (20 Snippets)

Intermediate+

~ 60 min

Server, database and React pastes: middleware, queries, hooks and patterns.

IN THIS CHAPTER

- Express (41–47)
- SQL + Mongo (48–53)
- React (54–60)

Snippets

Copy-paste

CODE · 41. Express boot (secure baseline)

```
import express from "express"; import helmet from "helmet"; import cors from "cors";
const app = express();
app.use(helmet()); app.use(cors({ origin: process.env.FRONT_URL }));
app.use(express.json({ limit: "100kb" }));
app.get("/health", (req, res) => res.json({ ok: true }));
```

CODE · 42. asyncHandler (no try/catch spam)

```
const asyncH = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
app.get("/notes", asyncH(async (req, res) => res.json(await Note.find())));
```

CODE · 43. Central error middleware (LAST!)

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: { message: err.status ? err.message :
  "Server error", code: err.code } });
});
```

CODE · 44. requireAuth (JWT guard)

```
import jwt from "jsonwebtoken";
const requireAuth = (req, res, next) => {
  const t = req.headers.authorization?.split(" ")[1];
  if (!t) return res.status(401).json({ error: { message: "Login required" } });
  try { req.user = jwt.verify(t, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ error: { message: "Session expired" } }); }
};
```

CODE · 45. Validate body (400-shape)

```
// with express-validator:
body("email").isEmail().normalizeEmail(), body("password").isLength({ min: 8 }),
(req, res, next) => { const e = validationResult(req); if (!e.isEmpty()) return
res.status(400).json({ error: { message: "Invalid input", details: e.array() } });
next(); }
```

CODE · 46. Rate limit (auth routes)

```
import { rateLimit } from "express-rate-limit";
app.use("/auth", rateLimit({ windowMs: 15 * 60e3, limit: 20, standardHeaders: true }));
```

CODE · 47. Pagination helper

```
const page = (q, def = 20, max = 100) => ({ limit: Math.min(+q.limit || def, max),
offset: (+q.page - 1 || 0) * Math.min(+q.limit || def, max) });
```

CODE · 48. Users + orders + index (init migration)

```
CREATE TABLE users (id SERIAL PRIMARY KEY, email CITEXT UNIQUE NOT NULL, hash TEXT NOT
NULL, created_at TIMESTAMPTZ DEFAULT now());
CREATE TABLE orders (id SERIAL PRIMARY KEY, user_id INT REFERENCES users(id) ON DELETE
CASCADE, total INT NOT NULL CHECK (total >= 0), created_at TIMESTAMPTZ DEFAULT now());
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at DESC);
```

CODE · 49. Reports (top customers + daily revenue)

```
SELECT u.email, SUM(o.total) s FROM users u JOIN orders o ON o.user_id = u.id GROUP BY
u.id ORDER BY s DESC LIMIT 5;
SELECT date_trunc('day', created_at) d, SUM(total) FROM orders GROUP BY 1 ORDER BY 1;
```

CODE · 50. Safe transfer (transaction)

```
BEGIN; UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100;
UPDATE accounts SET balance = balance + 100 WHERE id = 2; COMMIT;
```

CODE · 51. Mongoose model (validators + lean)

```
const noteSchema = new Schema({ text: { type: String, required: true, trim: true,
maxLength: 500 }, done: Boolean }, { timestamps: true });
const notes = await Note.find({ done: false }).select("text").lean(); // plain objects,
fast ✓
```

CODE · 52. Mongo reports (aggregate avg)

```
await Review.aggregate([ { $group: { _id: "$camp", avg: { $avg: "$stars" }, n: { $sum:
1 } } }, { $sort: { avg: -1 } } ]]);
```

CODE · 53. Seed script (idempotent)

```
for (const u of seedUsers) await User.updateOne({ email: u.email }, { $setOnInsert: u },
{ upsert: true });
```

CODE · 54. useFetch (loading/empty/error/ready + abort)

```
function useFetch(url) {
  const [state, setState] = useState({ status: "loading" });
  useEffect(() => { const c = new AbortController();
    (async () => { try { const r = await fetch(url, { signal: c.signal });
      if (!r.ok) throw new Error(r.status); const data = await r.json();
      setState({ status: data.length ? "ready" : "empty", data }); }
    catch (e) { if (e.name !== "AbortError") setState({ status: "error", error: e }); } })
    ();
    return () => c.abort(); }, [url]);
  return state;
}
```

CODE · 55. useLocalStorage (persistent state)

```
function useLocalStorage(key, init) {
  const [v, setV] = useState(() => store.load(key, init));
  useEffect(() => store.save(key, v), [key, v]);
  return [v, setV];
}
```

CODE · 56. useDebounceValue (search inputs)

```
function useDebounceValue(v, ms = 300) { const [d, setD] = useState(v);
  useEffect(() => { const t = setTimeout(() => setD(v), ms); return () =>
    clearTimeout(t); }, [v, ms]);
  return d; }
```

CODE · 57. Modal + focus trap lite

```
function Modal({ onClose, children }) {
  useEffect(() => { const h = e => e.key === "Escape" && onClose();
    addEventListener("keydown", h); return () => removeEventListener("keydown", h); },
    [onClose]);
  return createPortal(<div className="overlay" onClick={onClose}><div role="dialog"
    aria-modal onClick={e => e.stopPropagation()}>{children}</div></div>, document.body);
}
```

CODE · 58. Protected route (RequireAuth)

```
const RequireAuth = ({ children }) => { const { user } = useAuth(); const loc =
  useLocation();
  return user ? children : <Navigate to="/login" state={{ from: loc.pathname }} replace /
>; };
```

CODE · 59. Optimistic toggle (like button)

```
const toggle = async () => { setLiked(!liked); try { await api(`/posts/${id}/like`, {
  method: liked ? "DELETE" : "POST" }); } catch { setLiked(liked); toast("Failed -
  restored"); } };
```

CODE · 60. Error boundary (class, 15 lines)

```
class Boundary extends Component { state = {};  
  static getDerivedStateFromError(error) { return { error }; }  
  render() { return this.state.error ? <p role="alert">Something broke. <button  
onClick={() => location.reload()}>Reload</button></p> : this.props.children; } }
```

Vault IV — full-stack mini-recipes (61-68, complete + runnable)

CODE · 61. Contact form → email API (validated + rate-limited)

```
// POST /api/v1/contact — validate, then queue (log here, Resend/SendGrid in prod)  
app.post("/api/v1/contact", contactLimiter,  
  body("email").isEmail().normalizeEmail(), body("message").trim().isLength({ min: 10,  
max: 2000 })),  
  asyncH(async (req, res) => {  
    const e = validationResult(req);  
    if (!e.isEmpty()) return res.status(400).json({ error: { message: "Invalid input",  
details: e.array() } });  
    await queue.add("email", { to: "hello@site.com", ...req.body }); // BullMQ/DB queue  
    res.status(202).json({ ok: true });  
  }));
```

CODE · 62. Search API (q + filters + keyset page)

```
// GET /api/v1/books?q=&author=&after=<id>&limit=20 — fast at ANY depth  
r.get("/", asyncH(async (req, res) => {  
  const { q = "", author = "", after = "", limit = 20 } = req.query;  
  let rows = await store.all("books");  
  if (q) rows = rows.filter(b => (b.title +  
b.author).toLowerCase().includes(q.toLowerCase()));  
  if (author) rows = rows.filter(b => b.author === author);  
  rows.sort((a, b) => a.id.localeCompare(b.id));  
  const i = after ? rows.findIndex(b => b.id === after) + 1 : 0;  
  const data = rows.slice(i, i + Math.min(+limit, 50));  
  res.json({ data, next: data.length ? data[data.length - 1].id : null });  
}));
```

CODE · 63. Paginated list component (React + keyset "Load more")

```
function Books() {
  const [items, setItems] = useState([]); const [next, setNext] = useState(null);
  const [status, setStatus] = useState("loading");
  const load = async (after = "") => {
    setStatus("loading");
    try {
      const r = await fetch(`/api/v1/books?after=${after}&limit=20`).then(x => x.json());
      setItems(p => after ? [...p, ...r.data] : r.data); setNext(r.next);
    } catch { setStatus("error"); }
  };
  useEffect(() => { load(); }, []);
  if (status === "loading" && !items.length) return <p>Loading...</p>;
  if (status === "error") return <p role="alert">Failed. <button onClick={() => load()}>Retry</button></p>;
  return (
    <>
      <ul>{items.map(b => <li key={b.id}>{b.title}</li>)}</ul>
      {next && <button onClick={() => load(next)}>Load more</button>}</>
    </>
  );
}
```

CODE · 64. Avatar upload (client preview + server re-encode)

```
// client: instant preview, then upload
input.addEventListener("change", () => {
  const f = input.files[0];
  if (!f?.type.startsWith("image/") || f.size > 2 * 1024 * 1024) return toast("Image <2MB only");
  preview.src = URL.createObjectURL(f);
  const fd = new FormData(); fd.append("avatar", f);
  fetch("/api/v1/me/avatar", { method: "POST", body: fd }).then(() => toast("Saved!"));
});

// server: multer + sharp re-encode (strips EXIF/malware!) + uuid name
import sharp from "sharp";
app.post("/api/v1/me/avatar", requireAuth, upload.single("avatar"), asyncH(async (req, res) => {
  const name = `${crypto.randomUUID()}.webp`;
  await sharp(req.file.path).resize(256, 256).webp().toFile(`public/avatars/${name}`);
  res.status(201).json({ url: `/avatars/${name}` });
}));
```


CODE · 65. Password reset (token, hashed + expiring)

```
// POST /forgot → email link with ONE-TIME token (store HASH, never raw!)
const raw = crypto.randomBytes(32).toString("hex");
await store.insert("resets", { hash: sha256(raw), userId, exp: Date.now() + 15 * 60e3 });
await sendMail(user.email, `https://app.com/reset?token=${raw}`);
// POST /reset {token, password} → verify hash + expiry, set bcrypt hash, DELETE token
```

CODE · 66. Idempotent checkout (double-clicks charge ONCE)

```
// client sends Idempotency-Key: crypto.randomUUID() per ATTEMPT (reuse on retry!)
app.post("/api/v1/orders", requireAuth, asyncH(async (req, res) => {
  const key = req.headers["idempotency-key"];
  if (!key) return res.status(400).json({ error: { message: "Idempotency-Key required" } });
  if (await store.find("orders", o => o.key === key && o.userId === req.userId))
    return res.json(await store.find("orders", o => o.key === key)); // replay SAME result
  const order = await chargeAndCreate(req.userId, req.body, key);
  res.status(201).json(order);
}));
```

CODE · 67. Soft delete + restore (never lose data)

```
// DELETE sets deletedAt; queries exclude; /restore revives; cron purges after 30 days
r.delete("/:id", requireAuth, asyncH(async (req, res) => {
  const n = await store.update("notes", x => x.id === req.params.id && x.userId === req.userId, { deletedAt: new Date().toISOString() });
  if (!n) return res.status(404).json({ error: { message: "Not found" } });
  res.sendStatus(204);
}));
r.post("/:id/restore", requireAuth, asyncH(async (req, res) =>
  res.json(await store.update("notes", x => x.id === req.params.id, { deletedAt: null })))));
```

CODE · 68. Health + readiness (load-balancer grade)

```
// /health = alive ... /ready = can serve (DB ok?) — orchestrators NEED both
app.get("/health", (req, res) => res.json({ ok: true }));
app.get("/ready", asyncH(async (req, res) => {
  await pool.query("SELECT 1");
  res.json({ ok: true, db: "up", uptime: process.uptime() });
}));
```

CODE · 69. CSV export (stream 1M rows, memory flat)

```
// GET /api/v1/orders.csv — stream, never build the string!
app.get("/api/v1/orders.csv", requireAuth, asyncH(async (req, res) => {
  res.set({ "Content-Type": "text/csv", "Content-Disposition": "attachment;
filename=orders.csv" });
  res.write("id,total,created\n");
  for await (const o of orderStream(req.userId)) res.write(`${o.id},${o.total},${
o.created}\n`);
  res.end();
}));
```

CODE · 70. Nightly purge (cron + idempotent handler)

```
// cron: 0 3 * * * node src/jobs/purge.js — delete soft-deleted >30d, in batches
import { store } from "../store.js";
const cutoff = Date.now() - 30 * 864e5;
for (const c of ["notes", "reviews"]) {
  const rows = await store.all(c);
  const victims = rows.filter(r => r.deletedAt && Date.parse(r.deletedAt) < cutoff);
  for (const v of victims) await store.remove(c, x => x.id === v.id);
  console.log(`purged ${victims.length} from ${c}`);
}
```

CODE · 71. OAuth login sketch (GitHub, 20 lines of concepts)

```
// 1. Link: https://github.com/login/oauth/authorize?client_id=..&scope=read:user
// 2. GitHub → GET /auth/github?code=.. — exchange code for token (server-to-server!)
// 3. Fetch profile with token → find-or-create user → issue YOUR jwt (same as login!)
// NEVER trust profile email blindly: verify + link to existing accounts carefully.
app.get("/auth/github", asyncH(async (req, res) => {
  const t = await exchangeCode(req.query.code); // POST github.com/login/oauth/
access_token
  const gh = await fetch("https://api.github.com/user", { headers: { Authorization:
`Bearer ${t}` } }).then(r => r.json());
  const user = await findOrCreateByGithub(gh);
  res.json({ access: jwt.sign({ sub: user.id }, SECRET, { expiresIn: "15m" }), user });
}));
```

CODE · 72. Sitemap + robots (SEO discovery, generated)

```
// GET /sitemap.xml — build from DB (cache 1h!); robots.txt points at it
app.get("/sitemap.xml", asyncH(async (req, res) => {
  const books = await store.all("books");
  res.type("xml").send(`<?xml version="1.0"?><urlset xmlns="http://www.sitemaps.org/
schemas/sitemap/0.9">` +
    books.map(b => `<url><loc>https://bookhub.dev/books/${b.id}</loc></url>`).join("") +
    `</urlset>`);
}));
// public/robots.txt:  User-agent: *\nAllow: /\nSitemap: https://bookhub.dev/sitemap.xml
```

PART

Projects & Career — Build and Earn

Ten portfolio projects plus the ship-and-earn playbook.

Ch 72 Skill Ladders — 40 Micro-Projects by Level

Ch 73 Project Gallery — 10 Portfolio Builds With Specs

Ch 74 Ship & Earn — Freelancing and Your First Job

| Employers hire proof. This part manufactures it.

CHAPTER 72

Skill Ladders — 40 Micro-Projects by Level

Beginner-Advanced

~ Ongoing

Tiny builds that compound: finish one rung daily and watch skills stack.

IN THIS CHAPTER

- HTML/CSS ladder (1-12)
- JavaScript ladder (13-26)
- Backend/React ladder (27-40)
- Demo-day rules

Projects

Portfolio

KEY TAKEAWAY — KEY TAKEAWAY

One rung daily beats weekend marathons: 40 days × 45 minutes = a portfolio + reflexes. Each rung ships to GitHub with a README (what/why/what-I-learned). Demo-day rule: if you can't demo it in 60 seconds, it's not done.

Table 141 — HTML/CSS ladder (rungs 1-12)

| # | Build | Proves |
|---|--------------------------------------|----------------------------|
| 1 | Tribute page (semantic + fluid type) | Semantics, clamp() |
| 2 | Recipe card (box model drill) | Padding/border/width truth |
| 3 | Pricing table (3 tiers, featured) | Flex/grid, BEM |
| 4 | Navbar + hero + footer landing | Layout composition |
| 5 | Photo gallery (auto-fit, no queries) | Responsive grid |
| 6 | Dark/light themed blog post | Variables, toggle |

| # | Build | Proves |
|----|---|----------------------------------|
| 7 | Animated button set (6 variants) | Transitions, reduced-motion |
| 8 | Login form (accessible) | Labels, validation UX |
| 9 | Dashboard skeleton (holy grail) | Grid areas |
| 10 | Pure-CSS accordion/tabs | :has, details, generated content |
| 11 | Framework duel: pricing in Bootstrap + Tailwind | Framework literacy |
| 12 | Print-perfect invoice | Print CSS, attr() URLs |

Table 142 — JavaScript ladder (rungs 13-26)

| # | Build | Proves |
|----|---------------------------------------|---------------------------|
| 13 | Counter + vault (closures demo) | Scope, privacy |
| 14 | Todo (add/done/delete, localStorage) | CRUD, storage, delegation |
| 15 | Quiz app (score, timer, retry) | State, timers |
| 16 | Weather card (fetch + 4 states) | Async UI, errors |
| 17 | Notes (search, pin, export .txt) | Filter, download |
| 18 | Cart drawer (totals, persist) | Derived state |
| 19 | Debounced live search (500 rows) | Perf, debounce |
| 20 | Countdown + timezone display | Dates, Intl |
| 21 | Kanban (drag + drop, persist) | DnD API, modeling |
| 22 | Expense chart (canvas bars) | Canvas, reduce |
| 23 | Infinite-scroll gallery (observer) | IntersectionObserver |
| 24 | Offline-first notes (IndexedDB) | IDB, sync flags |
| 25 | Mini event-bus + store (no framework) | Observer pattern |
| 26 | Tested utils (Vitest 100%) | Unit tests, mocks |

Table 143 — Backend/React ladder (rungs 27-40)

| # | Build | Proves |
|----|---|----------------------|
| 27 | Notes API (Express + validate + errors) | REST, middleware |
| 28 | API + Postgres (migrate + seed) | SQL, pooling |
| 29 | Auth (register/login/JWT/refresh) | bcrypt, tokens |
| 30 | File upload + gallery | Multer, safe serving |
| 31 | SSE live tracker | Realtime, streams |
| 32 | Blog (EJS + SQLite) | Templates, CRUD |
| 33 | Camp API + aggregate reports | Modeling, GROUP BY |
| 34 | Rate-limited + helmet-hardened API | Security headers |
| 35 | React: shop (router + context + forms) | SPA triangle |
| 36 | React: checkout (protected + persist) | Auth flows |
| 37 | Next.js: SSR product pages | SSR, SEO |
| 38 | Full-stack deploy (API+DB+domain) | Ops, runbook |
| 39 | Lighthouse-90 portfolio | Perf budgets, CI |
| 40 | Capstone: pick ONE gallery project | End-to-end ownership |

Exercise 73 — Demo-day README template

Every rung ships with: title + live link + 3-line what/why + hardest-bug-I-fixed + what-I'd-do-with-more-time. Recruiters read READMEs more than code — write them like stories, not logs.

SOLUTION

Example line: "Hardest bug: dates shifted a day — stored local strings instead of UTC ISO. Fixed with toISOString + Intl display (Ch 28). Lesson: store UTC, display local." Bug stories = senior signals.

CHAPTER 73

Project Gallery — 10 Portfolio Builds With Specs

Intermediate-Advanced

~ 8 weeks

Full specifications: features, data models, milestones and stretch goals.

IN THIS CHAPTER

- Frontend builds (1-4)
- Full-stack builds (5-8)
- Advanced builds (9-10)
- Presentation + interview storytelling

Portfolio

Projects

Frontend builds

Table 144 — 1. Recipe finder (API + filters)

| Feature | Spec |
|--------------------------------|---|
| Search + cuisine/ diet filters | Debounced, URL-synced (?q=&diet=) |
| Favorites | localStorage, heart toggle, filter view |
| 4-state UI | Skeleton/empty/error+retry/ready everywhere |
| A11y | Labels, focus-visible, alt, 95+ Lighthouse a11y |
| Stretch | Meal planner week-view + shopping list export |

Table 145 — 2. Dev-jobs board (data-rich UI)

| Feature | Spec |
|-----------------|--|
| 500 seeded jobs | Filter (role/level/remote/salary) + sort |

| Feature | Spec |
|--------------------------|--|
| Saved + applied tracking | Persisted, badge counts |
| Perf | Virtualized list, <200ms filter (measure!) |
| Stretch | Email-alert mock + JSON import/export |

Table 146 — 3. Finance dashboard (charts + tables)

| Feature | Spec |
|---------------------|---|
| CSV import → charts | Canvas bars/lines, monthly rollups |
| Budgets + alerts | Progress rings, overspend warnings |
| Tables | Sortable, sticky headers, CSV export back |
| Stretch | Dark theme + print monthly report |

Table 147 — 4. Awwwards-style landing (motion + art)

| Feature | Spec |
|--------------------|---|
| Hero art | Layered gradients + SVG + clipped headline (Ch 18-deep) |
| Scroll reveals | IntersectionObserver, staggered, reduced-motion safe |
| Marquee + counters | CSS keyframes + JS count-up on view |
| Stretch | Theme switcher (3 palettes) + 90+ perf WITH all effects |

Full-stack builds

Table 148 — 5. CampBook (reviews platform — the capstone classic)

| Layer | Spec |
|-------|---|
| API | Users/camps/reviews REST, validated, paginated |
| Auth | bcrypt + JWT access/refresh, ownership guards |
| DB | SQL: FKs, UNIQUE(user,camp) review rule, avg-rating reports |

| Layer | Spec |
|---------|---|
| Front | Search + map pins + photos + star input |
| Ops | Helmet/CORS/rate-limit + deploy + smoke tests |
| Stretch | SSE new-review feed + moderation queue |

Table 149 — 6. Realtime chat (rooms + DMs)

| Layer | Spec |
|---------|---|
| Sockets | ws rooms, typing indicators, presence dots |
| History | Paged fetch, infinite scroll-up |
| Auth | JWT handshake, per-room membership |
| Safety | Rate-limit, profanity filter, block/mute |
| Stretch | File drops + emoji reactions + E2E note (honest crypto limits!) |

Table 150 — 7. E-commerce lite (cart → checkout → orders)

| Layer | Spec |
|----------|---|
| Catalog | Filters, search, product pages (SSR for SEO!) |
| Cart | Persist, quantities, promo codes |
| Checkout | Validated forms, mock payment, idempotency keys |
| Orders | History, status SSE tracker, invoice print |
| Stretch | Admin: inventory + refunds + sales charts |

Table 151 — 8. Booking system (availability truth)

| Layer | Spec |
|----------|---|
| Calendar | Real availability, timezone-correct (Intl!) |
| Holds | 10-min seat holds (expiry workers) |
| Payments | Mock gateway + webhook idempotency |

| Layer | Spec |
|-----------|---|
| Integrity | Overlap constraints IN the DB (not hope!) |
| Stretch | Cancellation policies + refund ledger + reminders |

Advanced builds

Table 152 — 9. Collaborative notes (CRDT-flavored)

| Feature | Spec |
|--------------|--|
| Live editing | WebSocket sync, presence cursors |
| Offline | IndexedDB + queued ops + conflict resolution (last-write + UI honesty) |
| History | Version list + restore |
| Stretch | Comments + share links with roles |

Table 153 — 10. Analytics micro-SaaS (the earn demo)

| Feature | Spec |
|---------------|---|
| Snippet | <script> tracker (1KB, privacy-first, no cookies) |
| Dashboard | Realtime visitors, top pages, referrers (SSE) |
| Billing | Mock plans + usage meters + upgrade walls |
| Multi-tenancy | Sites isolated by API key + RLS/ownership |
| Stretch | Funnel view + weekly email digest (mock) |

INTERVIEW QUESTION — TELL ME ABOUT A PROJECT. (60-SECOND STAR)

Expected shape: "CampBook — review platform for campsites (Situation). I owned API+DB+deploy (Task). REST + Postgres with UNIQUE review rule, JWT auth, Render+Neon ship with smoke tests (Action). 95 Lighthouse, zero-downtime deploys, demo at URL (Result)." Numbers + links + ONE hard bug story = memorable.

Exercise 74 — Ship gallery build #5

Follow the CampBook spec end-to-end in 2 weeks (milestones: API → auth → front → ops). Demo video 90 seconds. README with metrics (Lighthouse, test %, uptime). This ONE project answers 80% of junior interview questions.

SOLUTION

Week 1: API+DB+auth+seed+tests. Week 2: front+deploy+perf+video. Done-checklist: live URL, green /health, README metrics, 90s demo, rollback rehearsed. Ship it — done beats perfect.

CHAPTER 74

Ship & Earn — Freelancing and Your First Job

All levels

~ Career

Turn skills into income: profiles, pricing, proposals, CVs and offer negotiation.

IN THIS CHAPTER

- Portfolio that converts
- Freelance profiles + pricing
- Proposals that win
- CV + interviews + negotiation
- First-90-days success

Career

Freelance

Portfolio that converts (the 10-second test)

THE 10-SECOND HOMEPAGE TEST (A STRANGER MUST ANSWER ALL)

- - WHO (name + "Full-Stack Web Developer") ... WHAT (3 best projects, live links!) ... PROOF (metrics: users, Lighthouse, uptime) ... CONTACT (email + GitHub + LinkedIn, one click)
- - 3 projects > 10 tutorials ... Every project: live URL + code + README story + demo video
- - About: photo + stack + what-you're-learning + location/timezone + languages (EN/AR!)
- - No lorem, no dead links, no "under construction" — ever

Freelance: profiles + pricing

Table 154 — Profiles that get clicks (Upwork/Mostaql/Khamsat + LinkedIn)

| Element | Formula |
|---------------|---|
| Title | Outcome ("I build fast booking sites that fill calendars") not stack-list |
| Photo + video | Real face + 60s intro (EN or AR — clients buy trust) |
| First 2 lines | Niche + proof ("12 restaurant sites, 4.9★, Arabic/English") |
| Portfolio | 3 niche-relevant builds (not 20 random) |
| Reviews | Over-deliver first 3 gigs (speed + communication = 5★ flywheel) |

Table 155 — Pricing (EGP-aware starter ladder)

| Stage | Rate logic |
|--------------|---|
| First 3 gigs | Fixed, small, fast (reviews > revenue — but NEVER free for strangers) |
| With 5★ × 5 | Raise 30-50%; quote ranges (good/better/best packages) |
| Established | Value-price (booking site = X reservations/month → price from value) |
| Always | 50% upfront, milestones, scope-in-writing, late-fee clause, kill-fee |

Proposals + CV + interviews

CODE · Proposal anatomy (wins against 50 bids)

1. First line: THEIR problem restated ("Your menu PDF loses mobile orders...")
2. Plan: 3 milestones with dates (shows you scoped it!)
3. Proof: ONE similar build + metric ("...+38% calls in 60 days")
4. Price + timeline + what-you-need-from-them (decisive, easy yes)
5. Question: one sharp clarifier (engagement + expertise signal)

Table 156 — CV (one page, junior) + interview close

| CV block | Rule |
|-------------|--|
| Header | Name, title, city, phone, email, GitHub/LinkedIn/portfolio links |
| Projects ×3 | Name + link + stack + ONE metric each (bullets, verbs!) |
| Skills | Grouped (Front/Back/Data/Tools) — honest levels only |

| CV block | Rule |
|---------------------|---|
| Education/
certs | Brief; courses mapped (Appendix B) if no degree |
| Interview
close | "What does success look like in 90 days?" + "What's the team's biggest challenge?" (shows ownership thinking) |

Table 157 — Offer negotiation (juniors CAN negotiate)

| Move | Script |
|---------------|---|
| Anchor | "Based on X + market, I'm targeting Y-Z..." (researched range!) |
| Beyond salary | Learning budget, mentorship, remote days, review-at-6-months |
| Never | Lie about offers, accept on the spot, decide by salary alone |
| Always | Written offer, probation terms, overtime/on-call expectations clear |

KEY TAKEAWAY — KEY TAKEAWAY

Income = proof × visibility × trust. Proof: 3 shipped builds with metrics. Visibility: portfolio + GitHub green + LinkedIn posts (build in public!). Trust: fast replies, scoped promises, delivered demos. Do all three for 90 days — opportunities compound.

Exercise 75 — The 90-day earn plan

Days 1-30: finish capstone + portfolio + 3 READMEs + LinkedIn (post weekly builds). Days 31-60: 5 tailored proposals/week + 2 mock interviews/week + contribute 1 OSS fix. Days 61-90: close first gig OR 10 real interviews; log every no + lesson. Review weekly; adjust, never stop.

SOLUTION

Success metric: not "a job by day 90" but "10 real conversations + 1 paid gig or final-round". Conversations compound into offers. The log (proposals sent, replies, lessons) IS the system — systems beat motivation.

PART

Extended Datasheets — The Complete Lookup

Every tag, property and method that matters — with an example for each.

Ch 75 HTML Tags A-M — Complete Reference

Ch 76 HTML Tags N-Z + Global Attributes

Ch 77 CSS Properties A-M — Complete Reference

Ch 78 CSS Properties N-Z + Selectors Master List

Ch 79 JavaScript Language A-Z — Complete Reference

Ch 80 Browser + Async APIs A-Z — Complete Reference

When you forget syntax at 2 AM, this part answers in seconds.

CHAPTER 75

HTML Tags A-M — Complete Reference

Reference

~ 90 min

Every essential HTML tag A through M with purpose, example and pro note.

IN THIS CHAPTER

- Document + metadata tags
- Text + semantic tags
- Media + embedded tags
- Form tags A-M
- Interactive tags

Datasheet

W3Schools

MDN

a (anchor) LINKS

All attributes: href (target/anchor/mailto:/tel:) · download (save as) · target (\_self/\_blank/\_parent/\_top/frameName) · rel (noopener/noreferrer/no-follow/prev/next/author/help/license) · hreflang (linked-page language) · referrerpolicy (no-referrer/origin/strict-origin-when-cross-origin) · ping (tracking URLs).

```
<a href="https://x.com/g.pdf" download="guide.pdf" target="_blank" rel="noopener" hreflang="en">Get guide</a>
```

key: target=\_blank ALWAYS pairs with rel=noopener (security!). Empty href="#"? Use a button.

abbr TEXT

Abbreviation with full expansion in title (tooltip + screen readers).

```
<abbr title="Cascading Style Sheets">CSS</abbr>  
key: First-use expansion aids both users and SEO.
```

address SEMANTIC

Contact info for the article/page author — not any postal address.

```
<address>By <a href="mailto:a@x.com">Sara</a>, Cairo</address>  
key: Browsers italicize it; style as needed.
```

area + map MEDIA

Shapes: rect (x1,y1,x2,y2) · circle (x,y,r) · poly (x1,y1,...) · default (whole). Area attrs: alt (REQUIRED) · coords · href · target · download · rel · hreflang · referrerpolicy · shape. Map needs name + img usemap="#name".

```
<map name="m"><area shape="circle"  
coords="90,60,25" href="/lab" alt="Lab"></map>  
key: Every area needs alt — keyboard users tab through them. SVG links usually better today.
```

article LANDMARK

Self-contained shareable piece: post, card, comment, product.

```
<article><h2>Post</h2><p>...</p></article>  
key: Passes the "share alone" test (Ch 8).
```

aside LANDMARK

Tangential content: sidebar, related links, callouts.

```
<aside><h2>Related</h2><ul>...</ul></aside>
```

key: Complements main – never the core content.

audio MEDIA

Attrs: controls · autoplay (muted usually required!) · loop · muted · preload (none/metadata/auto) · src · crossorigin (anonymous/use-credentials). Multi-source fallbacks + text fallback inside.

```
<audio controls preload="metadata" loop><source src="e.mp3" type="audio/mpeg"><source  
src="e.ogg" type="audio/ogg">Transcript...</audio>
```

key: Autoplay with sound is blocked everywhere – muted + user gesture, or controls.

b / i (avoid) LEGACY

Pure visual bold/italic with NO meaning. Prefer strong/em.

```
<!-- <b>bold</b> <i>italic</i> – works but meaningless -->
```

key: Meaning first: strong/em; style with CSS.

base METADATA

One per page, inside head: href (base for ALL relative URLs) + target (\_blank default for all links). Powerful and dangerous.

```
<base href="https://x.com/docs/" target="_blank">
```

key: Breaks #anchors if href set carelessly – prefer absolute URLs; rarely needed.

bdi / bdo I18N

Isolate (bdi) or override (bdo dir=rtl) text direction in mixed-language content.

```
<p><bdi>سأ رة</bdi> scored 95</p><p><bdo dir="rtl">3 + 2</bdo></p>  
key: Essential for Arabic + English mixes.
```

blockquote + cite TEXT

blockquote: long quote + cite attr (source URL, invisible metadata). cite tag: TITLE of a work (book/song/article). footer inside for attribution line.

```
<blockquote cite="https://x.com/essay"><p>...</p><footer>— <cite>On Craft</cite></  
footer></blockquote>  
key: cite = work title, never a bare person's name. q = short inline quotes.
```

body DOCUMENT

All visible content. Scripts + footer live at its end.

```
<body>...<script src="app.js" defer></script></body>  
key: One body; id="top" enables back-to-top links.
```

br TEXT

Line break. For poems/addresses ONLY — never for spacing.

```
<p>12 Nile St.<br>Cairo, Egypt</p>  
key: Spacing = CSS margins, always.
```

button FORMS

type: submit (FORM DEFAULT!) · reset · button. All attrs: disabled · form (attach outside!) · formaction/formenctype/formmethod/formnovalidate/formtarget (per-button submit overrides!) · name/value (sent!) · popovertarget/popoveraction · autofocus.

```
<button type="submit" form="login" formaction="/login?fast=1" name="go" value="1">Log in</button>
```

key: Submit buttons can live OUTSIDE the form via form="id". Inside forms, type defaults to submit – the classic accidental-submit bug.

canvas GRAPHICS

Bitmap surface: width/height ATTRS set the drawing buffer (CSS only scales!).
getContext("2d") draws; fallback content inside for no-JS. HiDPI: scale by devicePixelRatio.

```
<canvas id="c" width="600" height="400">Sales chart (needs JavaScript).</canvas>
```

key: Set width/height ATTRIBUTES (not just CSS) or drawings blur. SVG = vector/sharp; canvas = pixels/fast.

caption TABLES

Table title, announced FIRST by screen readers. First child of table.

```
<table><caption>2026 prices</caption>...</table>
```

key: Every data table deserves one.

cite TEXT

Title of a cited work (book, article, song).

```
<p>As <cite>Clean Code</cite> says...</p>
```

key: Italic by default; style freely.

code TEXT

Inline code fragment. Block code = pre + code.

```
<p>Run <code>npm start</code> now.</p>  
key: Meaning (code), not monospace styling alone.
```

col / colgroup TABLES

colgroup groups columns; col styles ONE column (span=N covers N). Only border/background/width/visibility apply to columns! Cells still need th/td.

```
<colgroup><col span="2" class="num"><col class="name"></colgroup>  
key: Column styling is limited by design – backgrounds/widths yes, padding/text no.
```

data TEXT

Human-readable text + machine-readable value attr (product IDs, scores, timestamps for JS/sort).

```
<data value="SKU-42">Nile Backpack</data> · <data value="2026-09-07">today</data>  
key: JS reads .value – cleaner than parsing text or hiding data-* on random spans.
```

datalist FORMS

Combobox: free typing + suggestions. input list="id" binds; option value = suggestion. NO styling of the popup (browser-owned!).

```
<label>City <input list="cities" name="city"></label><datalist id="cities"><option  
value="Cairo"><option value="Giza"></datalist>  
key: Suggestions only – users can still type anything. Validate server-side!
```

dd / dl / dt **LISTS**

Description list: term (dt) + details (dd). Glossaries, FAQs, specs.

```
<dl><dt>HTML</dt><dd>Page structure.</dd></dl>  
key: Multiple dd per dt allowed.
```

del / ins **TEXT**

Edit tracking: cite (reason URL) + datetime (ISO timestamp) on BOTH. del renders strike, ins underline — style freely.

```
<p>Price: <del datetime="2026-01-01">200</del> <ins datetime="2026-02-01">150 EGP</ins></p>  
key: Changelogs, legal diffs, sale prices — edits with machine-readable WHEN + WHY.
```

details / summary **INTERACTIVE**

Native accordion: open attr = expanded. name attr groups EXCLUSIVE accordions (one open at a time!). summary = always-visible toggle (first child).

```
<details name="faq" open><summary>Shipping?</summary><p>48 hours.</p></details>  
key: Same name="faq" on siblings = browser-managed exclusive accordion, zero JS.
```

dfn **TEXT**

First definition of a term (glossary style).

```
<p><dfn>API</dfn> is a contract between programs.</p>  
key: Pairs with abbr/title for full semantics.
```

dialog INTERACTIVE

Attrs: open (non-modal visible). Methods: showModal() (top layer + Esc + focus trap + ::backdrop!) · show() (modeless) · close(). Events: close/cancel. Forms with method="dialog" close + returnValue.

```
<dialog id="d"><form method="dialog"><button value="yes">Yes</button></form></dialog>
```

key: showModal() > CSS modals: focus trap, Esc, top layer free. Style ::backdrop for the dim.

div GENERIC

Meaningless block box for layout/styling/JS hooks.

```
<div class="card">...</div>
```

key: Check semantic options first (Ch 8).

em TEXT

Stressed emphasis — screen readers stress it audibly.

```
<p>Save <em>before</em> closing.</p>
```

key: Meaning, not italics (that's CSS).

embed / object EMBEDDED

embed (void, no fallback): src · type · width/height. object (fallback body!): data · type · width/height · form · name · usemap. Children of object can be param tags.

```
<object data="menu.pdf" type="application/pdf" width="600" height="400"><p><a href="menu.pdf">Download menu</a></p></object>
```

key: object wins over embed (fallback content!). PDFs/video usually better as links or <video>.

fieldset / legend FORMS

Attrs: disabled (kills whole group at once!) · form (attach outside!) · name. legend = group caption (FIRST child, required for meaning).

```
<fieldset name="addr" disabled><legend>Address (locked)</legend>...</fieldset>
```

key: disabled on fieldset disables + unsubsmts EVERYTHING inside – one-attribute permission switch.

figcaption / figure MEDIA

Media + its caption as one movable unit.

```
<figure><figcaption>Fig 1.</figcaption></figure>
```

key: Caption can be first or last child.

footer LANDMARK

Closing info of page OR section (copyright, links, author).

```
<footer><p>© 2026 Roastly</p></footer>
```

key: Every article can have its own footer.

form FORMS

Attrs: action (URL, default = same page) · method (get/post/dialog) · enctype (urlencoded/multipart for FILES/text-plain) · target (\_self/\_blank/frameName) · autocomplete (on/off) · novalidate (skip native checks) · accept-charset · rel · name.

```
<form action="/upload" method="post" enctype="multipart/form-data" autocomplete="on">...</form>
```

key: Files REQUIRE post + multipart. GET = search/shareable URLs; POST = create/change. Never nest forms.

h1–h6 TEXT

ALL SIX levels: h1 (ONE per page = page topic!) → h2 (major sections) → h3 (subsections) → h4/h5/h6 (deep nesting). Never skip (h1→h3) or pick for size (CSS owns looks!).

```
<h1>Guide</h1><h2>Setup</h2><h3>Windows</h3><h4>ARM notes</h4><h5>Troubleshoot</h5><h6>Footnote</h6>
```

key: Outline test: reading ONLY headings must summarize the page. Skipped levels confuse screen readers + SEO.

head DOCUMENT

Metadata container: title, meta, links. Never visible.

```
<head><title>T</title><link rel="stylesheet" href="a.css"></head>
```

key: CSS here; JS deferred or at body end.

header LANDMARK

Intro block of page/section/article (logo, nav, title).

```
<header><h1>Shop</h1><nav>...</nav></header>
```

key: NOT head (metadata) – header is visible.

hgroup SEMANTIC

Groups h1 + subtitle/tagline as one heading unit.

```
<hgroup><h1>Roastly</h1><p>Fresh beans daily</p></hgroup>
```

key: Keeps outline clean with multi-line titles.

hr TEXT

Thematic break between sections (rendered as a line).

```
<p>...part one...</p><hr><p>...part two...</p>
```

key: Meaning (topic shift), not decoration.

html DOCUMENT

Root: lang (REQUIRED: en/ar/...) · dir (ltr/rtl/auto) · xmlns (XHTML only, skip). Everything nests inside; head + body only children.

```
<html lang="ar" dir="rtl">
```

key: lang drives screen-reader voice + spellcheck + SEO. Wrong/missing lang = mispronounced Arabic.

iframe EMBEDDED

Attrs: src · srcdoc (inline HTML!) · title (REQUIRED for a11y!) · width/height · loading (lazy!) · sandbox (allow-scripts/allow-same-origin/allow-forms/... — empty = strictest) · allow (camera/microphone/fullscreen/...) · referrerpolicy · csp · allowfullscreen.

```
<iframe src="https://maps..." title="Shop map" loading="lazy" sandbox="allow-scripts"
referrerpolicy="no-referrer"></iframe>
```

key: Third-party embeds are trust decisions: sandbox + lazy + title always. Videos: facade pattern (Ch 61).

img MEDIA

Attrs: src · alt (REQUIRED: describe, or "" decorative) · width/height (ANTI-CLS!) · srcset/sizes (responsive!) · loading (eager/lazy) · decoding (async!) · fetchpriority (high/low) · crossorigin · referrerpolicy · ismap/usemap.

```

```

key: width+height + srcset/sizes + lazy-below-fold + async decode = the complete fast-image recipe.

input types FORMS

ALL 22 TYPES — text·search·url·tel·email·password (text family) · number·range (numeric + min/max/step) · date·month·week·time·datetime-local (pickers!) · checkbox·radio (checked/value) · file (accept/multiple/capture!) · color · hidden · image·submit·reset·button (action trio). Unknown type → text (safe!).

```
<input type="email" required> <input type="date" min="2026-01-01"> <input type="range"
min="0" max="100"> <input type="file" accept="image/*" multiple>
```

key: Right type = right mobile keyboard + free validation. See next card for the 25+ shared attributes.

input attributes (all 25+) FORMS

Shared by ALL types: name · value · placeholder · required · readonly (submits!) vs disabled (doesn't!) · minlength/maxlength · size · autocomplete (email/tel/cc-... tokens!) · autofocus · list (datalist!) · pattern (regex!) · title (pattern hint!) · multiple · accept (file!) · capture (camera!) · checked · min/max/step (numeric/date!) · height/width (image!) · src/alt (image!) · form (outside!) · formaction/formenctype/formmethod/formnovalidate/formtarget (submitter overrides!) · dirname · popoverTarget · spellcheck.

```
<input type="text" name="coupon" pattern="[A-Z0-9]{6}" title="6 uppercase letters/digits" required autocomplete="off" minlength="6" maxlength="6">
```

key: pattern + title + required = declarative validation with custom hints. readonly submits, disabled doesn't!

ins TEXT

Inserted text (pairs with del for edits).

```
<ins datetime="2026-05-01">New clause</ins>
```

key: Underlined by default.

kbd TEXT

Keyboard input names. Style as key caps.

```
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd></p>
```

key: Nested kbd = chord parts.

label FORMS

Attrs: for (matches control id — click focuses!) · form (label outside the form!). Wrap style also works. EVERY control needs one (placeholders are NOT labels!).

```
<label for="em">Email</label><input id="em" type="email"> · <label>Remember <input type="checkbox"></label>
```

key: Click-to-focus area grows 10× with labels — mobile UX + screen readers both demand them.

legend FORMS

Caption of a fieldset group.

```
<fieldset><legend>Payment</legend>...</fieldset>
```

key: First child of fieldset, always.

li LISTS

Direct child of ul/ol/menu ONLY. value attr rennumbers ol items (and followers!). Nested lists go INSIDE an li, never bare.

```
<ol><li>One</li><li value="10">Ten</li><li>Eleven (auto!)</li></ol>
```

key: value=10 rennumbers that item AND continues counting after it — exam/spec numbering trick.

link METADATA

rel values: stylesheet · icon · preload (as=font/script/style/image + crossorigin!) · prefetch · preconnect · dns-prefetch · modulepreload · manifest · canonical · alternate (hreflang feeds!). Attrs: href · as · media (print!) · disabled · integrity · crossorigin · hreflang · sizes · title · fetchpriority.

```
<link rel="stylesheet" href="a.css"><link rel="preload" as="font" href="f.woff2"
crossorigin><link rel="icon" href="f.svg" sizes="any">
```

key: preload = this page needs it NOW; prefetch = next page might. Fonts: preload + crossorigin or they double-download!

main LANDMARK

Page's unique core content. EXACTLY one per page.

```
<main><h1>...</h1>...</main>
```

key: Skip-link target: "Skip to main content".

mark TEXT

Highlighted/relevant text (search hits).

```
<p>Results for <mark>coffee</mark>: 12 found</p>
```

key: Relevance highlight, not decoration.

math (MathML) MATH

Native math formulas (no images/JS!): `math` > `mrow`/`mi`/`mn`/`mo`/`mfrac`/`msqrt`/`msup`. Screen readers speak it; scales infinitely; styleable with CSS.

```
<math><mrow><mi>x</mi><mo>=</mo><mfrac><mn>1</mn><mn>2</mn></mfrac></mrow></math>
```

key: All modern browsers render MathML now – retire formula PNGs forever.

menu LISTS

Toolbar/menu list (ul semantics + toolbar contexts).

```
<menu><li><button>Cut</button></li></menu>
```

key: Rare – ul covers most cases.

meta METADATA

charset (utf-8 FIRST!) · name/content (description/author/viewport/theme-color/robots/generator) · http-equiv/content (refresh/content-security-policy/X-UA-Compatible) · property (Open Graph! NOT name!) · media. Viewport + charset on EVERY page.

```
<meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1"><meta name="description" content="..."><meta property="og:title" content="...">
```

key: OG tags use property= (not name=) – the #1 broken-share-preview bug. description ≈ 155 chars.

meter FORMS

Gauge with MEANING: value · min/max · low/high (thresholds!) · optimum (ideal zone — browsers color!). Label it (aria-label or visible text).

```
<label>Disk <meter value="0.7" min="0" max="1" low="0.6" high="0.9" optimum="0.3">70%</meter></label>
```

key: meter = current level in a range (disk, score). progress = task completion over time. Different jobs!

CHAPTER 76

HTML Tags N-Z + Global Attributes

Reference

~ 90 min

The rest of HTML plus the universal attributes every element accepts.

IN THIS CHAPTER

- Tags N-Z with examples
- Global attributes (id/class/data-\*)
- ARIA attribute map
- Content models cheat
- Obsolete tags to avoid

Datasheet

MDN

W3Schools

nav LANDMARK

Major navigation block. aria-label distinguishes multiple navs.

```
<nav aria-label="Main"><ul><li><a href="/">Home</a></li></ul></nav>
```

key: Main navs only – not every link cluster.

noscript METADATA

Fallback shown when JS is disabled/blocked.

```
<noscript><p>This app needs JavaScript enabled.</p></noscript>
```

key: Graceful message > blank page.

object EMBEDDED

External resource with rich fallback content inside.

```
<object data="a.svg" type="image/svg+xml" width="200"></object>
```

key: Fallback chain: object → img → alt text.

ol LISTS

Ordered list attrs: reversed (count DOWN!) · start (first number) · type (1/a/A/i/I). Items: li value= rennumbers mid-list. Nested ol types cascade (1→a→i...).

```
<ol reversed start="5" type="A"><li>Item E</li><li value="1">Item A</li></ol>
```

key: reversed + start = countdowns/rankings with zero JS. type=i/I gives Roman numerals free.

optgroup / option FORMS

select children: option (value=sent! · selected · disabled · label short-name) · optgroup (label=group name! · disabled kills group). Placeholder = disabled selected empty option.

```
<select><option value="" disabled selected>Pick...</option><optgroup label="EU"><option value="eg">Egypt</option></optgroup></select>
```

key: No value attr? The visible TEXT is submitted. Group labels are not selectable – structure, not data.

output FORMS

Live calculation display: for (space-separated source IDs!) · form · name. Update .value from JS — readers announce changes politely.

```
<form oninput="total.value+=a.value + +b.value"><input id="a" value="2">+<input id="b" value="3">=<output name="total" for="a b">5</output></form>
```

key: for="a b" LINKS sources (plural!) so assistive tech explains the math. .value, not.textContent!

p TEXT

Paragraph. Auto-margins; never nest block elements inside.

```
<p>Free shipping over $50.</p>
```

key: Can't contain div/table/ul — browsers auto-close it!

param MEDIA

Plugin parameters for object: name + value pairs (autoplay flags, quality). Void element, children of object only.

```
<object data="movie.swf"><param name="autoplay" value="true"><param name="quality" value="high"></object>
```

key: Legacy plugin territory — modern embeds rarely need it, but readers must recognize it.

picture / source MEDIA

Art direction + formats: source media (breakpoint!) · srcset · sizes · type (avif/webp order!) + img fallback (REQUIRED last child — carries alt + dimensions!).

```
<picture><source srcset="h.avif" type="image/avif"><source media="(max-width:600px)"
srcset="h-m.webp"></picture>
```

key: Browser picks FIRST matching source. Format order (AVIF→WebP→JPG) + art-direction media = pro images.

pre TEXT

Preformatted text: whitespace preserved. Wrap code blocks.

```
<pre><code>const x = 1;</code></pre>
```

key: Escapes needed: < > & inside.

progress FORMS

Task completion: value/max (omit value = INDETERMINATE spinner!). Always label (visible text or aria-label) — bare bars confuse readers.

```
<label>Upload <progress value="35" max="100">35%</progress></label> · <progress>Loading...
</progress>
```

key: No value = indeterminate (unknown duration). meter = gauge level; progress = journey completion.

q TEXT

Inline quote: browsers add "quotes" (locale-aware!). cite attr = source URL (metadata).

Nested q alternates quote styles automatically.

```
<p>She said <q cite="https://x.com/i">ship <q>early</q>, ship often</q>.</p>
```

key: CSS quotes property + ::before/::after content can restyle/translate the marks per language.

rp / rt / ruby TEXT

East-Asian pronunciation annotations (ruby) with fallback parens (rp).

```
<ruby>漢<rp>(</rp><rt>kan</rt><rp>)</rp></ruby>
```

key: Niche but correct for CJK content.

s TEXT

No-longer-accurate text (not a deletion, just outdated).

```
<p><s>$99</s> now $59</p>
```

key: Outdated (s) vs edited-out (del).

samp TEXT

Sample program output.

```
<p>Terminal shows <samp>Done in 2s</samp></p>
```

key: Output (samp) vs input (kbd) vs code (code).

script **RESOURCES**

type: classic (default) · module (deferred + strict + imports!) · importmap · speculationrules · application/ld+json (SEO data!). Loading: defer (ORDERED, after-parse — DEFAULT!) · async (when-ready, orderless) · nomodule (legacy fallback). Security/perf: src · integrity (SRI hash!) · crossorigin · referrerpolicy · fetchpriority · blocking=render.

```
<script type="module" src="app.js"></script><script nomodule defer src="legacy.js"></script>
```

key: module = deferred automatically + strict + scoped. Third-party scripts: async + integrity + consider Partytown.

search **SEMANTIC**

Search-region landmark (header search, filter zones).

```
<search><form action="/s"><input type="search" name="q"></form></search>
```

key: New element — one landmark per search area.

section **LANDMARK**

Themed group WITH heading (chapters, pricing, reviews).

```
<section aria-labelledby="p"><h2 id="p">Pricing</h2>...</section>
```

key: No heading? It's a div, not a section.

select FORMS

Attrs: name · required · multiple (multi-pick!) · size (visible rows → listbox!) · disabled · autofocus · form (outside forms!) · autocomplete. select.value reads the chosen option directly.

```
<select name="city" required size="4" multiple><option>Cairo</option><option>Giza</option></select>
```

key: size>1 + multiple = listbox (Ctrl-click multi). Mobile renders native wheel pickers automatically.

small TEXT

Fine print: legal, copyrights, disclaimers.

```
<p><small>© 2026 · Terms apply.</small></p>
```

key: Legal smallness, not visual smallness.

slot (web components) COMPONENTS

Insertion point inside template/shadow DOM: name (named slots!) + fallback content inside. Light-DOM children with slot="name" project in.

```
<template id="c"><div class="card"><h3><slot name="title">Untitled</slot></h3><slot>Body...</slot></div></template>
```

key: Named slots + ::slotted() CSS = themeable components. Pairs with template + customElements.define.

source MEDIA

Two jobs: (1) in picture: srcset/sizes/media/type (art direction!). (2) in audio/video: src/type (+codecs="avc1.4d401f" for picky formats). Order: best-first.

```
<video controls><source src="v.webm" type='video/webm; codecs="vp9"'><source  
src="v.mp4" type="video/mp4"></video>  
key: type with codecs lets browsers skip unsupported files WITHOUT downloading –  
bandwidth saved.
```

span GENERIC

Meaningless inline hook for styling/JS inside text.

```
<p>Price: <span class="hl">$59</span></p>  
key: Inline div – check semantic tags first.
```

strong TEXT

Strong importance — announced with emphasis.

```
<p><strong>Warning:</strong> unsaved work!</p>  
key: Importance (strong) vs stress (em).
```

style METADATA

Internal CSS: media attr scopes (print/screen!) · blocking=render · title (alternate stylesheets!). Inline <style> in body = allowed but render-blocking — head or files instead.

```
<style media="print">nav,.ads{display:none} a[href^="http"]::after{content:" ("  
attr(href) ")}</style>  
key: media="print" stylesheets download WITHOUT blocking render – free performance for  
print CSS.
```

sub / sup TEXT

Subscript (H2O) / superscript (2<sup>2</sup>, footnotes).

`<p>H<sub>2</sub>O · x<sup>2</sup></p>`
key: Chemical/math truth, not visual tweaks.

summary INTERACTIVE

Visible heading of details (clickable toggle).

```
<details><summary>More</summary><p>Hidden</p></details>
```

svg (+ core shapes) **GRAPHICS**

Inline vector graphics: viewBox (scaling soul!) · width/height · role/title/desc (a11y!).

Shapes: rect/circle/ellipse/line/polyline/polygon/path (M/L/C/Q/A/Z!) · text · g (groups) · defs/ use (reuse!) · linearGradient/radialGradient. CSS styles fills/strokes; fillColor themes icons.

```
<svg viewBox="0 0 24 24" width="24" role="img"><title>Star</title><path
fill="currentColor" d="M12 212.9 6.6 7.1 7.5 4.4 4.8 1.6 7L12 17.5 5.8 211.6 7L2
9.3 17.1-.7z"/></svg>
```

key: viewBox + currentColor + symbol/use sprite (Ch 48) = the professional icon system.

table (full system) TABLES

FULL TABLE SYSTEM — table (the grid) · caption (FIRST child = announced title!) · colgroup/col (column styling, span=N) · thead (repeats on print!) · tbody (main rows, multiple OK) · tfoot (totals — place AFTER thead in source!) · tr (row) · th (header: scope=col/row/colgroup/rowgroup! · abbr · headers · colspan/rowspan) · td (data: headers links to th id · colspan/rowspan).

```
<table><caption>Q3 sales</caption><thead><tr><th scope="col">City</th><th scope="col">Total</th></tr></thead><tbody><tr><th scope="row">Cairo</th><td>9,410</td></tr></tbody><tfoot><tr><th scope="row">Sum</th><td>9,410</td></tr></tfoot></table>
```

key: scope=col/row is THE accessibility upgrade (readers announce headers per cell). tfoot after thead in source, renders at bottom.

template SCRIPTING

Inert markup: parses, renders NOTHING, scripts inside DON'T run. JS clones .content (DocumentFragment!). shadowrootmode=open/closed = DECLARATIVE Shadow DOM (SSR web components!).

```
<template id="row"><tr><td><slot name="city"></slot></td></tr></template>
```

key: Stamp lists/modals from templates: clone → fill slots → append. Parses once, stamps 1000x.

textarea FORMS

Multiline: rows/cols (visible size!) · name · placeholder · required · minlength/maxlength · readonly vs disabled (readonly SUBMITS!) · wrap (soft/hard) · autocomplete · spellcheck · form (outside!) · dirname (direction companion!). NO value attr — default text goes INSIDE.

```
<textarea name="bio" rows="4" cols="40" maxlength="280" wrap="hard" placeholder="Tell us..."></textarea>
```

key: Default value = element CONTENT (not value=!). wrap=hard inserts real newlines on submit.

tfoot / thead / tbody TABLES

Row groups: thead (column headers — REPEATS on printed pages!) · tbody ×N (grouped bodies, zebra per group!) · tfoot (totals — source order: thead→tfoot→tbody allowed, renders tfoot last).

```
<table><thead>...</thead><tfoot><tr><td>Total</td><td>9,410</td></tr></tfoot><tbody>...</tbody></table>
```

key: Sticky headers = thead th { position: sticky; top: 0 }. Printers repeat thead per page automatically.

th / td TABLES

th: scope (col/row/colgroup/rowgroup!) · abbr (short header for readers) · headers · colspan/rowspan · sorted (aria-sort companion). td: headers (space-separated th ids!) · colspan/rowspan · abbr.

```
<th id="c" scope="col"><abbr title="Quantity">Qty</abbr></th>...<td headers="c">42</td>
```

key: Complex tables: id on th + headers on td = unambiguous cell-header mapping for screen readers.

time TEXT

Machine dates: datetime (ISO: YYYY-MM-DD / full timestamp + timezone!) — content stays human ("tomorrow"). Search engines + calendars parse it.

```
<time datetime="2026-09-07">today</time> · <time datetime="2026-09-07T19:30:00+02:00">7:30 PM</time>
```

key: Always include timezone offsets in timestamps — +02:00 Cairo vs UTC mixups corrupt schedules.

title METADATA

Tab/bookmark/search headline. One per page, unique.

```
<title>Beans – Roastly</title>
```

key: ≤60 chars, keyword + brand.

tr TABLES

Table row container for th/td.

```
<tr><td>A</td><td>B</td></tr>
```

key: Striping via tbody tr:nth-child(even).

track MEDIA

Timed text: kind (subtitles/captions/descriptions/chapters/metadata!) · src (.vtt!) · srclang · label · default (auto-selected). Multiple tracks = language menu free.

```
<track kind="captions" src="en.vtt" srclang="en" label="English" default><track  
kind="subtitles" src="ar.vtt" srclang="ar" label="العربية">
```

key: captions = dialogue+SOUNDS (deaf viewers); subtitles = translation. chapters = navigable timeline!

u TEXT

Unarticulated annotation (misspellings) — NOT links/underlines.

```
<p><u>recieve</u> <span>→ receive</span></p>
```

key: Underlines imply links – avoid for style.

ul **LISTS**

Unordered (bullet) list: menus, features, groups.

```
<ul><li>Beans</li><li>Mugs</li></ul>  
key: Menus ARE lists of links.
```

var **TEXT**

Math/programming variable name.

```
<p><var>x</var> = <var>y</var> + 2</p>  
key: Italic by default.
```

video **MEDIA**

Attrs: src · controls · autoplay (needs muted+playsinline on mobile!) · loop · muted · playsinline (iOS!) · poster (preview frame!) · preload (none/metadata/auto) · width/height · crossorigin · disablepictureinpicture · disableremoteplayback.

```
<video controls width="640" height="360" poster="p.webp" preload="metadata"  
playsinline><source src="v.webm" type="video/webm"></video>  
key: poster + preload=metadata = instant perceived speed. Autoplay: muted + playsinline  
or browsers block it.
```

wbr **TEXT**

Optional word-break opportunity (long URLs, codes).

```
<p>https://x.com/<wbr>very/<wbr>long/<wbr>path</p>  
key: Breaks ONLY if needed – unlike hyphens.
```

Global attributes (work on EVERY element)

Table 158 — Universal attributes

Attribute	Job	Example
<code>id</code>	Unique hook (JS/CSS/anchor)	<code>id="cart"</code> (once per page!)
<code>class</code>	Reusable style/JS hooks	<code>class="btn btn-big"</code>
<code>data-*</code>	Custom data for JS	<code>data-id="42"</code> → <code>el.dataset.id</code>
<code>title</code>	Advisory tooltip	<code>title="Delete item"</code>
<code>hidden</code>	Hide (display:none + skipped)	<code>hidden</code> until JS reveals
<code>tabindex</code>	Keyboard order	<code>0</code> = in order, <code>-1</code> = focusable-by-JS-only
<code>lang</code> / <code>dir</code>	Language/direction override	<code>lang="ar" dir="rtl"</code>
<code>contenteditable</code>	User-editable text	<code>contenteditable="true"</code>
<code>draggable</code>	Native drag source	<code>draggable="true"</code> + DnD events
<code>spellcheck</code>	Spellcheck on/off	<code>spellcheck="false"</code> for code
<code>translate</code>	Machine-translation hint	<code>translate="no"</code> for brand names
<code>accesskey</code>	Keyboard shortcut (Alt+letter — usually SKIP: conflicts!)	<code>accesskey="s"</code>
<code>autocapitalize</code>	Mobile auto-caps (off/none/sentences/words/characters)	names, codes
<code>enterkeyhint</code>	Mobile Enter key label (enter/done/go/next/previous/search/send)	<code>enterkeyhint="search"</code>
<code>inputmode</code>	Mobile keyboard (text/decimal/numeric/tel/search/email/url) WITHOUT changing type!	<code>inputmode="numeric"</code>
<code>inert</code>	Entire subtree unfocusable + hidden from readers (background behind modals!)	<code>inert</code> toggle via JS

Attribute	Job	Example
<code>popover</code>	Native popup (auto/manual) + <code>popovertarget</code> trigger, top layer free	menus, toasts
<code>style</code>	Inline CSS (escape hatch only — files/classes win!)	<code>style="color:red"</code>

Table 159 — ARIA quick map (roles + states)

Attribute	Job	
<code>role</code>	Treat-as (button/dialog/alert) — native elements first!	
<code>aria-label</code>	Invisible accessible name	icon buttons
<code>aria-labelledby</code>	Name from another element's id	sections
<code>aria-describedby</code>	Extra description from id	hints, errors
<code>aria-current</code>	Current item in set	nav "page"
<code>aria-expanded/-controls</code>	Open state + controlled id	menus, accordions
<code>aria-hidden</code>	Hide from readers	decorative duplicates
<code>aria-live</code>	Announce updates	toasts, carts ("polite"/"assertive")

Table 160 — Obsolete — never write these

Tag	Replace with
<code></code> <code><center></code>	CSS (color, text-align)
<code><frame></code> <code><frameset></code>	Layouts, components, iframe (rare)
<code><big></code> <code><strike></code> <code><tt></code>	CSS / del / code
<code><applet></code> <code><bgsound></code>	object/video/audio
<code>align/bgcolor/border</code> attrs	CSS equivalents
<code><acronym></code>	<code><abbr></code> (acronym merged into abbr!)

Tag	Replace with
<code><applet></code> <code><bgsound></code>	object/embed, video/audio (plugins are dead)
<code><basefont></code> <code><big></code> <code><tt></code>	CSS font shorthand
<code><dir></code>	<code></code> (directory list retired)
<code><frame></code> <code><frameset></code> <code><noframes></code>	Layouts/components; iframe only for true embeds
<code><strike></code>	<code></code> (edits) or <code><s></code> (outdated)

CHAPTER 77

CSS Properties A-M — Complete Reference

Reference

~ 120 min

Every working CSS property A through M with syntax and example.

IN THIS CHAPTER

- Layout A-M
- Flexbox + Grid A-M
- Typography A-M
- Backgrounds + borders A-M
- Effects A-M

Datasheet

MDN

W3Schools

accent-color FORMS

Tints native checkboxes/radios/range/progress with brand color.

```
input[type="checkbox"] { accent-color: #2563eb; }  
key: One line rebrands all native controls.
```

align-content / align-items / align-self FLEX/GRID

Cross-axis alignment: tracks (content) / all items / one item.

```
.g { align-content: center; align-items: stretch; } .one { align-self: end; }  
key: content = tracks; items = all; self = one.
```

all **RESET**

Resets every property (unset/initial/inherit) — nuclear option.

```
.reset { all: unset; }  
key: Third-party/widget style isolation.
```

animation + @keyframes **MOTION**

ALL 8 longhands: name · duration · timing-function (ease/linear/cubic-bezier/steps!) · delay (NEGATIVE = mid-way start!) · iteration-count (infinite!) · direction (normal/reverse/alternate) · fill-mode (none/forwards/backwards/both!) · play-state (paused!). Shorthand
ORDER: name duration timing delay count direction fill play. @keyframes from/to/% + multiple names comma-separated.

```
.pop { animation: rise .4s ease-out both; }  
@keyframes rise { from { opacity: 0; transform: translateY(12px); } }  
key: both = no pre/post flash.
```

appearance **FORMS**

none strips native control styling for full custom design.

```
select.custom { appearance: none; background: url(arrow.svg) no-repeat right; }  
key: Rebuild focus/arrow affordances after stripping!
```

aspect-ratio **SIZING**

Width/height lock (16/9, 1, 4/3). Media boxes without hacks.

```
.video { aspect-ratio: 16 / 9; width: 100%; }  
key: Replaces padding-top % hacks forever.
```

backface-visibility TRANSFORM

Shows/hides the BACK of 3D-rotated elements. hidden = card-flip essential (paired with rotateY + preserve-3d).

```
.card { transform-style: preserve-3d; transition: transform .6s; }  
.card.flipped { transform: rotateY(180deg); }  
.face { backface-visibility: hidden; }  
.back { transform: rotateY(180deg); }  
key: Without hidden, the mirrored back shows through – the classic broken-flip bug.
```

backdrop-filter EFFECTS

Blurs/behind-glass effect (frosted navs). Needs translucent bg.

```
.nav { background: rgb(255 255 255 / .7); backdrop-filter: blur(10px); }  
key: GPU-costly – use sparingly on mobile.
```

background-\* BACKGROUND

Longhands: image (url/gradient/none!) · position (x y/keywords/edges!) · size (cover/contain/px/%) · repeat (repeat-x/y/no-repeat/space/round!) · attachment (scroll/fixed/local!) · origin (border/padding/content-box) · clip (same + TEXT!) · color. Shorthand: color image position/size repeat attachment. COMMA = stacked layers (first = top!).

```
.hero { background-image: url(h.webp); background-position: center; background-size: cover; }  
key: Shorthand resets ALL longhands – order care!
```

block-size / inline-size LOGICAL

Logical height/width (RTL/flow aware). Prefer over height/width in components.

```
.card { inline-size: 100%; max-inline-size: 60ch; }  
key: Logical = auto RTL support.
```

border-\* / border-image BORDERS

width (thin/medium/thick/px!) · style (none/hidden/dotted/dashed/solid/double/groove/ridge/inset/outset!) · color · per-side (border-top-...!) · logical (border-inline/block-...!) · radius (+ elliptical 50%/10%!). border-image: source slice (numbers fill!) width outset repeat (stretch/repeat/round/space).

```
.box { border: 2px dashed navy; border-inline-start: 4px solid gold; }  
key: Logical borders (inline-start) = RTL-safe.
```

border-collapse / border-spacing TABLES

collapse (shared single borders — THE professional table look!) vs separate (+ border-spacing: h v gaps!). Pairs with empty-cells: show/hide.

```
table { border-collapse: collapse; width: 100%; }  
th, td { border: 1px solid #cbd5e1; padding: .5rem .75rem; }  
key: collapse first, always — double borders scream amateur.
```

bottom / top / left / right / inset POSITION

Offsets for positioned elements; inset = all-in-one (logical available).

```
.full { position: absolute; inset: 0; }  
key: inset: 0 = stretch to all edges.
```

box-shadow EFFECTS

x y blur spread color (+ inset). Stack comma-separated layers.

```
.card { box-shadow: 0 1px 2px rgb(0 0 0 / .06), 0 8px 24px rgb(0 0 0 / .08); }  
key: Two soft layers > one harsh.
```

box-sizing BOX

border-box = width includes padding+border. Universal first rule.

```
*, *::before, *::after { box-sizing: border-box; }  
key: Non-negotiable in modern CSS.
```

break-\* (inside/before/after) PRINT/MULTI-COL

Controls fragmentation (printing + columns).

```
.card { break-inside: avoid; } h2 { break-after: avoid; }  
key: Print stylesheets + column layouts.
```

caption-side TABLES

Caption above (top) or below (bottom) the table.

```
table.fin { caption-side: bottom; }  
key: Financial tables often caption below.
```

caret-color TEXT

Text-cursor (caret) color in inputs/areas.

```
input { caret-color: #2563eb; }  
key: Brand-detail polish.
```

clear FLOW

Push below floats (left/right/both). Legacy float layouts.

```
.after-float { clear: both; }  
key: Flex/Grid killed floats – legacy only.
```

clip-path EFFECTS

Clips to shapes: circle/polygon/inset/path. Animatable!

```
.badge { clip-path: polygon(0 0, 100% 0, 100% 75%, 50% 100%, 0 75%); }  
key: Non-rectangular UI without images.
```

color / color-scheme COLOR

Text color + light/dark native-widget hint.

```
:root { color-scheme: light dark; } body { color: #1c2333; }  
key: color-scheme styles scrollbars/forms natively.
```

column-\* (multicol) LAYOUT

columns shorthand (width count!) · count · width · gap · rule (width/style/color mini-border!) · span (none/all = breakout headings!) · fill (balance/auto). Companion: break-inside: avoid (keep cards whole!) + orphans/widows.

```
.news { columns: 3 200px; column-gap: 2rem; } .news h2 { column-span: all; }  
key: Text-flow columns (≠ Grid!).
```

contain / content-visibility PERF

contain: none/strict/content/size/layout/style/paint (isolate cost!). content-visibility: visible/hidden/auto (auto = skip off-screen render!). Pair with contain-intrinsic-size (placeholder px!) to stop scrollbar jumps.

```
.feed > * { content-visibility: auto; contain-intrinsic-size: 300px; }  
key: Huge feeds render 10× faster.
```

content (pseudo) GENERATED

Inserts ::before/::after content: text/counters/attr/images.

```
a[href^="http"]::after { content: " ⤴"; } q::before { content: open-quote; }  
key: Decorative only – never meaningful content!
```


counter-\* GENERATED

counter-reset (name start!) · counter-increment (step!) · counter-set · display: counter(name, upper-roman?) / counters(name, ".", decimal) for NESTED (1.2.3!). ::before content glues it in. list-style: none + custom counters = designed lists.

```
ol.toc { counter-reset: ch; } ol.toc li::before { counter-increment: ch; content:
counter(ch) ". "; }
key: Custom list numbering without markup.
```

cursor UI

Values: auto/default · pointer (links!) · text/vertical-text · wait/progress/help · move/grab/grabbing · not-allowed/no-drop · crosshair · zoom-in/zoom-out · none (custom cursors!) · context-menu · cell/copy/alias · col-resize/row-resize + directions (ns/ew/nwse/...) · url(icon.png) fallback.

```
.btn { cursor: pointer; } .drag { cursor: grab; } .off { cursor: not-allowed; }
key: Affordance signal — match the action.
```

direction / unicode-bidi I18N

Text flow ltr/rtl + bidi algorithm control.

```
.ar { direction: rtl; } .mix { unicode-bidi: isolate; }
key: dir attribute usually better than CSS.
```

display LAYOUT

Values: block/inline/inline-block · flow-root (clearfix!) · flex/inline-flex · grid/inline-grid · none (gone!) vs contents (box gone, kids stay!) · table/table-row/table-cell (retro layouts) · list-item · ruby · math. Outer (block/inline) × inner (flow/flex/grid) model.

```
.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path:
inset(50%); }
key: Hide accessibly (above) vs display:none (removes all).
```

empty-cells TABLES

Show/hide borders of empty cells (separate model).

```
table { border-collapse: separate; empty-cells: hide; }
key: Only with border-collapse: separate.
```

field-sizing FORMS

content = inputs/textareas AUTO-GROW to fit text (no JS autosize!). fixed = old behavior. Modern browsers, graceful fallback.

```
textarea { field-sizing: content; min-height: 3lh; max-height: 12lh; }
input.auto { field-sizing: content; min-width: 8ch; }
key: Pair with min/max bounds – unbounded growth breaks layouts.
```

filter EFFECTS

Functions (ORDER matters!): blur(px) · brightness() · contrast() · grayscale() · hue-rotate() · invert() · opacity() · saturate() · sepia() · drop-shadow(x y blur color — follows ALPHA, unlike box-shadow!) · url(svg). backdrop-filter = same, behind the glass.

```
img.dim { filter: brightness(.8) saturate(.9); }  
key: filter creates a stacking context (z-index trap!).
```

flex / flex-\* longhands FLEXBOX

Container: direction (row/column/reverse!) · wrap · flow (both!) · justify/align (see cards).
Item: grow shrink basis. Shorthands: flex:1 = 1 1 0% (EQUAL!) · flex:auto = 1 1 auto · flex:none = 0 0 auto · flex:0 0 200px (fixed!) · order · align-self.

```
.side { flex: 0 0 240px; } .main { flex: 1; } .row { flex-flow: row wrap; }  
key: flex:1 = 1 1 0% (basis ZERO — even split).
```

float LEGACY

Text-wrap floats (left/right). Magazine image wraps only now.

```
img.wrap { float: inline-start; margin: 0 1rem 1rem 0; }  
key: Layout via Flex/Grid; float = text wrap only.
```

font-\* (family/size/weight/style/variant) **TYPOGRAPHY**

family (stacks + "quotes for spaces!") · size (absolute/relative/%) · weight (100–900 + relative bolder/lighter!) · style (normal/italic/oblique 10deg!) · variant (small-caps!) + variant-numeric (tabular-nums for tables!) · variant-ligatures · shorthand STRICT: style variant weight size/line-height family (size+family REQUIRED!).

```
body { font: 400 1rem/1.6 "Segoe UI", Arial, sans-serif; }  
key: Shorthand needs size+family minimum.
```

font-display (@font-face) **FONTS**

Loading strategy: swap (text now!) vs block/fallback/optional.

```
@font-face { font-family: X; src: url(x.woff2); font-display: swap; }  
key: swap = never invisible text.
```

font-stretch / font-synthesis **TYPOGRAPHY**

stretch (condensed ↔ expanded %, variable fonts!) · synthesis (none = BLOCK fake bold/italic — force real weights!). style/synthesis-weight small siblings.

```
.display { font-stretch: 125%; }  
body { font-synthesis: none; } /* never faux-bold my Arabic! */  
key: synthesis:none exposes missing weights — then actually LOAD them (font-display card).
```

gap / row-gap / column-gap FLEX/GRID

Gutters without margin hacks (works flex+grid+multicol).

```
.grid { gap: 1.25rem; } .cols { column-gap: 2rem; }
key: Margins for outer space, gap for inner gutters.
```

grid-\* (template/area/place) GRID

Container: template-columns/rows (px/fr/minmax/repeat!) · template-areas (ASCII art!) · auto-columns/rows · auto-flow (row/column/DENSE!) · gap. Item: column/row (line numbers/names/span N!) · area (named!) · place-self. repeat(auto-fit, minmax()) = queries-free responsive.

```
.p { display: grid; place-items: center; place-content: center; gap: 1rem; }
key: place- = align+justify in one.
```

hanging-punctuation TYPOGRAPHY

Optical punctuation hang outside the margin (quotes/bullets).

```
blockquote { hanging-punctuation: first last; }
key: Print-grade polish (support varies).
```

height / min-\* / max-\* SIZING

Fixed vs elastic bounds (width/height + min/max + logical).

```
.card { min-block-size: 12rem; } img { max-inline-size: 100%; }
key: Elastic bounds > fixed sizes (Ch 14).
```

hyphens **TYPOGRAPHY**

Auto word-breaking with lang-aware hyphens.

```
p { hyphens: auto; } h2 { hyphens: none; }  
key: Needs correct lang attribute to work!
```

image-rendering **IMAGES**

Scaling algorithm: pixelated (retro) vs smooth/auto.

```
.pixel-art { image-rendering: pixelated; }  
key: Pixel art + QR codes must stay crisp.
```

inset (+ logical) **POSITION**

top/right/bottom/left in one (+ inline/block logical).

```
.overlay { position: fixed; inset: 0; }  
key: inset: 0 = full-cover in 2 words.
```

isolation **STACKING**

isolate creates a stacking context (blend-mode control).

```
.blend-zone { isolation: isolate; }  
key: Contains mix-blend-mode/z-index chaos.
```

justify-\* (content/items/self) FLEX/GRID

Main/row-axis alignment: tracks/items/one.

```
.nav { justify-content: space-between; } .one { justify-self: center; }  
key: justify = row axis; align = column axis.
```

left / right (see inset) POSITION

Horizontal offsets for positioned elements.

```
.tip { position: absolute; inset-inline-end: .5rem; }  
key: Logical (inline-end) = RTL-safe.
```

letter-spacing / word-spacing TYPOGRAPHY

Tracking (caps labels) / word gaps.

```
.kicker { letter-spacing: .2em; text-transform: uppercase; }  
key: em-based tracking scales with size.
```

line-clamp TYPOGRAPHY

Clamp text to N lines with ellipsis: line-clamp: 3 (+ classic -webkit-box fallback inside!).

Needs display/box-orient/overflow companions.

```
.cut-3 { display: -webkit-box; -webkit-line-clamp: 3; -webkit-box-orient: vertical;  
overflow: hidden; line-clamp: 3; }  
key: line-clamp is now standard – keep the prefixed trio for older Safaris.
```

line-height **TYPOGRAPHY**

Unitless multiplier (1.6 body, 1.1 display).

```
body { line-height: 1.6; }  
key: Unitless ALWAYS (Ch 14).
```

list-style **LISTS**

type (none/disc/circle/square/decimal/decimal-leading-zero/lower-alpha/upper-roman/cjk + custom @counter-style!) · position (outside/inside!) · image (url bullets!). Shorthand: type position image. ::marker styles bullets/numbers directly.

```
ul.clean { list-style: none; } ul.clean li::before { content: "► "; color: gold; }  
key: Keep semantics (ul) – style markers freely.
```

margin-\* (+ logical) **BOX**

Sides: top/right/bottom/left + logical inline-start/end + block-start/end (RTL-safe!) + margin-trim (eat edge margins!). auto = centering/flex absorption. TRBL clockwise shorthand (1-4 values). Vertical margins COLLAPSE (exact rules Ch 14!).

```
.wrap { margin-inline: auto; max-inline-size: 70ch; }  
key: margin-inline:auto = RTL-safe centering.
```


mask-\* EFFECTS

Longhands: image (alpha/luminance!) · mode (match-source/alpha/luminance!) · size/position/repeat/origin/clip (like background!) · composite (add/subtract/intersect/exclude!) · type. Text masks + gradient fades + image cutouts.

```
.fade-b { mask-image: linear-gradient(black 70%, transparent); }  
key: mask = visibility; clip-path = geometry.
```

max-\* / min-\* SIZING

Elastic bounds (width/height + logical + content keywords).

```
img { max-width: 100%; } .btn { min-inline-size: 10ch; }  
key: min()/max()/clamp() supercharge these.
```

mix-blend-mode EFFECTS

Values: normal · multiply (print ink!) · screen (lighten!) · overlay · darken/lighten · color-dodge/burn · hard-light/soft-light · difference/exclusion · hue/saturation/color/luminosity. Needs isolation management (parent stacking!).

```
.duo { mix-blend-mode: multiply; }  
key: Duotone/print effects; isolate parent.
```

CHAPTER 78

CSS Properties N-Z + Selectors Master List

Reference

~ 120 min

The rest of CSS plus every selector pattern in one list.

IN THIS CHAPTER

- Properties N-Z
- Pseudo-classes complete
- Pseudo-elements
- Combinators + specificity table
- Modern CSS (nesting, layers, container)

Datasheet

MDN

W3Schools

object-fit / object-position MEDIA

How replaced content fills its box: cover/contain/fill + focal point.

```
.avatar { width: 80px; height: 80px; object-fit: cover; object-position: top; }  
key: cover + fixed box = perfect thumbs.
```

offset-\* (motion path) MOTION

Longhands: path (path("M...")/url()/ray()/circle(!) · distance (0-100%!) · rotate (auto! follows direction) · anchor · position. Animate offset-distance in @keyframes = motion along curves.

```
.dot { offset-path: path("M0,0 C100,0 100,100 200,100"); animation: mv 3s linear  
infinite; }  
@keyframes mv { to { offset-distance: 100%; } }  
key: Path choreography without JS.
```

opacity EFFECTS

0-1 transparency (whole subtree!). <1 creates stacking context.

```
.disabled { opacity: .5; pointer-events: none; }  
key: Fades hit children — rgba for backgrounds.
```

order FLEX/GRID

Visual order override (DOM/keyboard order UNCHANGED!).

```
.first-look { order: -1; }  
key: Ally trap: visual ≠ focus order. Use rarely.
```

outline (+ :focus-visible) ALLY

Non-layout focus ring. Style it — never remove bare.

```
:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }  
key: outline:none without replacement = keyboard users lost.
```

overflow / overflow-\* FLOW

visible (default!) · hidden · clip (NO scroll container — cheaper! + overflow-clip-margin!) · scroll (ALWAYS bars) · auto (bars when needed) · per-axis x/y (one non-visible ⇒ other computes to auto!). text-overflow: ellipsis needs hidden + nowrap.

```
.chat { overflow-y: auto; overscroll-behavior: contain; }  
key: auto = scrollbar only when needed.
```

overscroll-behavior SCROLL

Values: auto/contain/none (+ x/y axes!). contain = trap scroll in modals/carousels. none = kill pull-to-refresh + glow. block/inline longhands for direction control.

```
.modal { overscroll-behavior: contain; }  
key: Modals/chat must contain scroll.
```

padding-\* (+ logical) BOX

Sides: top/right/bottom/left + logical inline-start/end + block-start/end. NO auto/negative/collapse (unlike margins!). % = of CONTAINER WIDTH (even vertical!). TRBL shorthand. box-sizing decides in/out of width.

```
.btn { padding-block: .6em; padding-inline: 1.2em; }  
key: em-hugging padding scales with text.
```

perspective / perspective-origin TRANSFORM

3D depth on a PARENT (px = viewer distance!): children with rotateX/Y gain realism. origin moves the vanishing point. Children need transform-style: preserve-3d for true nesting.

```
.scene { perspective: 900px; perspective-origin: 50% 20%; }  
.tilt { transform: rotateX(12deg) rotateY(-8deg); }  
key: perspective on PARENT vs perspective() fn on the element itself – different  
scopes, same math.
```

place-\* (content/items/self) GRID

Shorthands: place-content (align justify!) · place-items · place-self. Single value = BOTH axes (place-items: center!). Grid + flex alignment in one line each.

```
.stage { display: grid; place-content: center; place-items: center; }  
key: Dead-center in 3 declarations.
```

pointer-events INTERACT

none = clicks pass through (overlays); auto restores.

```
.badge-float { pointer-events: none; }  
key: Decorative layers must not eat clicks.
```

position (+ sticky) LAYOUT

static (default, unpositionable) · relative (offset + ANCHOR for absolutes!) · absolute (nearest positioned ancestor!) · fixed (viewport!) · sticky (threshold top/etc + scrolls in parent!). Pairs: inset offsets + z-index. Containing-block gotchas: transform/filter ancestors trap fixed!

```
.bar { position: sticky; top: 0; z-index: 50; }  
key: Sticky needs a threshold (top) + scrolling parent.
```

print (@media print) OUTPUT

Rules: hide nav/ads/buttons · black-on-white · URLs via a[href]::after attr() · break-before/after/inside: avoid (keep blocks whole!) · orphans/widows: 3 · @page { size/margin! } · color-adjust: exact for brand colors.

```
@media print { .nav, .ads { display: none; } a[href]::after { content: " (" attr(href)
)"; } }
key: This book's PDF uses print CSS!
```

print-color-adjust OUTPUT

exact = KEEP brand colors/backgrounds in print (browsers strip them by default!). economy = ink-saving default. Use sparingly (logos/charts only!).

```
@media print {
  .logo, .chart { print-color-adjust: exact; -webkit-print-color-adjust: exact; }
}
key: Users pay for ink — exact only where color CARRIES meaning.
```

quotes GENERATED

Custom quotation marks per language for q/counters.

```
q { quotes: "«" "»" "‹" "›"; }
key: Pair with ::before/::after open-quote.
```

resize UI

User-resizable boxes (both/horizontal/vertical/none).

```
textarea { resize: vertical; } .pane { resize: horizontal; overflow: auto; }
key: Needs overflow ≠ visible to work.
```

rotate / scale / translate TRANSFORM

Individual transform properties (compose with transform!).

```
.card:hover { translate: 0 -4px; scale: 1.02; rotate: -1deg; }
key: Separate props animate independently – cleaner than transform soup.
```

row-gap (see gap) FLEX/GRID

Vertical-only gutters.

```
.rows { row-gap: 2rem; column-gap: 1rem; }
key: Split-axis control.
```

scroll-\* (behavior/margin/padding/snap) SCROLL

behavior: smooth (native animated anchors!) · margin (sticky-header anchor offset!) · padding (snap inset!). Snap: type (x/y/both + mandatory/proximity!) on scroller · align (start/center/end!) · stop (always!) on kids. scrollbar-width/color + ::-webkit-scrollbar-\* styling.

```
html { scroll-behavior: smooth; } section { scroll-margin-top: 80px; }
.snap { scroll-snap-type: x mandatory; } .snap > * { scroll-snap-align: center; }
key: scroll-margin fixes anchor-under-sticky-header!
```

scrollbar-\* (styling) SCROLL

Standard: scrollbar-width (auto/thin/none!) · scrollbar-color (thumb track!) · scrollbar-gutter (stable = no layout shift!). WebKit: ::-webkit-scrollbar/scrollbar-thumb/track/button (full custom!). Never kill scrollbars without an alternative!

```
* { scrollbar-width: thin; scrollbar-color: navy #eef; }
key: Subtle > invisible (usability!).
```

scrollbar-gutter SCROLL

Reserves scrollbar space ALWAYS (stable!) so pages don't JUMP when scrollbars appear/disappear. stable both-edges = symmetric layouts.

```
html { scrollbar-gutter: stable; }  
.modal-open { scrollbar-gutter: stable; overflow: hidden; }  
key: The one-line fix for modal-open layout shift – pair with overflow hidden.
```

shape-outside FLOW

Text wraps non-rectangular floats (circle/polygon/url).

```
img.round { float: left; shape-outside: circle(50%); border-radius: 50%; }  
key: Magazine-grade text wraps.
```

tab-size TEXT

Tab character width in pre/code.

```
pre { tab-size: 2; }  
key: Code readability detail.
```

table-layout TABLES

auto (content-sized) vs fixed (predictable, fast).

```
table { table-layout: fixed; width: 100%; }  
key: fixed + widths = stable data grids.
```


text-\* (align/decoration/transform/overflow) TYPOGRAPHY

align (start/end/center/justify + -last!) · decoration longhands (line/style/color/thickness/offset/skip-ink!) · transform (uppercase/lowercase/capitalize/full-width!) · overflow (clip/ellipsis + nowrap!) · indent · wrap (balance/pretty/stable!) · emphasis (CJK marks!).

```
.cut { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }  
key: Ellipsis needs the nowrap+overflow trio.
```

text-size-adjust TYPOGRAPHY

Stops mobile browsers AUTO-INFLATING small text (100%/none). Set 100% on html for predictable type + keep user zoom working!

```
html { -webkit-text-size-adjust: 100%; text-size-adjust: 100%; }  
key: none harms accessibility (blocks zoom scaling) – 100% preserves intent AND zoom.
```

text-wrap (balance/pretty) TYPOGRAPHY

balance (headlines) / pretty (paragraphs, no orphans).

```
h1, h2 { text-wrap: balance; } p { text-wrap: pretty; }  
key: One line kills widowed headings.
```

top (see inset) POSITION

Vertical offset for positioned/sticky.

```
.stick { position: sticky; top: 1rem; }  
key: Sticky without top = static.
```

touch-action INTERACTION

Declares which touch gestures the browser may steal: auto · none (custom sliders/maps!) · pan-x/pan-y/pinch-zoom (allow SOME!). Mandatory for pointer-event drag UI.

```
.slider { touch-action: pan-y; } /* vertical scroll OK, horizontal = mine */  
.map { touch-action: none; }  
key: Custom drag broken on mobile? 90% chance it's a missing touch-action.
```

transform (+ 3D) TRANSFORM

2D: translate(x,y)/translateX/Y · scale · rotate(deg/turn!) · skew · matrix(). 3D: perspective() · translateZ/rotateX/Y · transform-style: preserve-3d · backface-visibility: hidden (card flips!). ORDER matters (right-to-left math!). GPU = 60fps for transform/opacity ONLY.

```
.tilt { transform: perspective(800px) rotateY(-8deg); transform-style: preserve-3d; }  
key: GPU fast-path with opacity (Ch 20).
```

transform-origin / box TRANSFORM

Pivot point + reference box for transforms.

```
.spin { transform-origin: top left; transform-box: fill-box; }  
key: Origin changes EVERYTHING (doors vs wheels).
```

transition-\* MOTION

Longhands: property (ALL = lazy — list props!) · duration · timing-function (ease/steps/cubic-bezier!) · delay (negative = mid-way!) · behavior: allow-discrete (animate display/none!). Shorthand comma-multiples. :hover on/off asymmetry = different in/out timing.

```
.x { transition: opacity .3s ease, display .3s allow-discrete; }  
key: allow-discrete animates to/from display:none!
```

translate (see rotate) TRANSFORM

Independent translation property.

```
.lift:hover { translate: 0 -6px; }  
key: Composes with rotate/scale cleanly.
```

unicode-bidi (see direction) I18N

Bidi isolation/override for mixed-direction text.

```
.num { unicode-bidi: plaintext; }  
key: Prefer bdi elements + dir attributes.
```

user-select UI

Text selection control (none for UI chrome).

```
.btn { user-select: none; } code { user-select: all; }  
key: Buttons draggable-clean; code one-click copy.
```

vertical-align **INLINE**

Inline: baseline/sub/super/top/bottom/middle/text-top/text-bottom/px/%. Table cells: top/middle/bottom. Middle $\neq$ optical center (x-height based!) — flex/grid for true centering.

```
.icon { vertical-align: -2px; } td { vertical-align: top; }  
key: NOT for block centering (use flex/grid!).
```

visibility **DISPLAY**

visible/hidden/collapse. Hidden keeps SPACE ($\neq$ display:none).

```
.ghost { visibility: hidden; }  
key: Layout-stable hiding + transitionable!
```

white-space **TEXT**

Wrapping control: normal/nowrap/pre/pre-wrap/pre-line/break-spaces.

```
.code { white-space: pre-wrap; } .tag { white-space: nowrap; }  
key: pre-wrap = code that wraps.
```

width / inline-size (see sizing) **SIZING**

Fixed/fluid widths + content keywords (fit-content/min/max).

```
.pop { width: fit-content; max-width: 90vw; }  
key: fit-content = shrink-to-content.
```

will-change **PERF**

Hints the browser to PREPARE a property (transform/opacity/scroll-position/contents!). Add BEFORE animation via JS, REMOVE after — parked will-change eats memory!

```
el.addEventListener("mouseenter", () => el.style.willChange = "transform");  
el.addEventListener("animationend", () => el.style.willChange = "auto");  
key: Last-resort perf tool, not a blanket rule – measure first (Ch 60).
```

word-break / overflow-wrap **TEXT**

Long-word/URL breaking strategies.

```
.chat p { overflow-wrap: anywhere; }  
key: anywhere breaks URLs without breaking layout.
```

writing-mode **I18N**

Horizontal/vertical text flow (vertical-rl for CJK).

```
.rail { writing-mode: vertical-rl; }  
key: Vertical labels/rails done right.
```

z-index (+ stacking) **STACKING**

Depth on positioned/flex/grid items; contexts trap (opacity/transform/filter).

```
.modal { z-index: var(--z-modal); }  
key: Scale variables beat 9999 wars (Ch 16).
```

Selectors master list

Table 161 — Every selector pattern

Pattern	Picks	Example	
<code>*</code>	All	resets only	
<code>E</code>	Type	<code>article</code>	
<code>.c</code>	Class	<code>.btn</code>	
<code>#i</code>	ID	<code>#app</code>	
<code>[a]</code> <code>[a="v"]</code> <code>[a^="v"]</code> <code>[a\$="v"]</code> <code>[a*="v"]</code> <code>[a~="v"]</code> <code>`[a\</code>	<code>= "v"</code>	Attributes (present/exact/ starts/ends/contains/word/ lang)	<code>a[href^="http</code>
<code>E F</code>	Descendant	<code>.card p</code>	
<code>E > F</code>	Child	<code>ul > li</code>	
<code>E + F</code>	Adjacent sibling	<code>h2 + p</code>	
<code>E ~ F</code>	Later siblings	<code>h2 ~ p</code>	
<code>:hover</code> <code>:focus</code> <code>:focus-visible</code> <code>:focus-within</code>	Interaction	<code>a:hover</code>	
<code>:active</code> <code>:visited</code> <code>:link</code> <code>:target</code>	Link states	<code>:target</code> = jumped-to!	
<code>:first-child</code> <code>:last-child</code> <code>:nth-child(2n)</code>	Position	<code>tr:nth-child(even)</code>	
<code>:first-of-type</code> <code>:nth-of-type()</code>	Type position	<code>p:first-of-type</code>	
<code>:not()</code> <code>:is()</code> <code>:where()</code> <code>:has()</code>	Logic (where = 0 specificity!)	<code>li:has(> a)</code>	
<code>:checked</code> <code>:disabled</code> <code>:required</code> <code>:valid</code>	Form states	<code>input:invalid</code>	
<code>::before</code> <code>::after</code>	Generated boxes	icons, quotes	

Pattern	Picks	Example
<code>::first-line ::first-letter</code>	Text parts	drop caps
<code>::marker ::selection ::placeholder</code>	UI parts	<code>::selection{background:gold}</code>
<code>::backdrop ::cue</code>	Modal/ video parts	dialog styling

Modern CSS: nesting, layers, container queries

CODE · Nesting (native, no Sass needed)

```
.card {
  padding: 1rem;
  & h3 { margin: 0; }
  &:hover { box-shadow: var(--shadow); }
  @media (min-width: 700px) { padding: 2rem; }
}
```

CODE · Cascade layers (order beats specificity across layers!)

```
@layer reset, base, components, utilities;
@layer utilities { .hidden { display: none; } } /* always wins */
```

CODE · Container queries (components respond to THEIR box, not viewport!)

```
.cards { container-type: inline-size; }
@container (min-width: 500px) { .card { display: grid; grid-template-columns: 1fr 2fr; } }
```

CHAPTER 79

JavaScript Language A-Z — Complete Reference

Reference

~ 120 min

Every core JS API that matters: strings, arrays, objects, numbers, dates and more.

IN THIS CHAPTER

- String methods
- Array + Object methods
- Number + Math + Date
- Map/Set + JSON + Regex
- Operators + syntax recap

Datasheet

MDN

W3Schools

String: case/trim/search STRINGS

Case: toLowerCase/toUpperCase (+ Locale variants for Turkish/Arabic!). Trim: trim/trimStart/trimEnd. Find: includes/indexOf/lastIndexOf/startsWith/endsWith/charAt-all? (see chars card).

```
q.trim().toLowerCase(); s.includes("x"); s.startsWith("https");  
key: includes() is NaN-proof and readable – default over indexOf !== -1.
```


String: slice/substring/split STRINGS

slice(start,end — NEGATIVES count back!) · substring (swaps args, no negatives — legacy!) · substr (LEGACY, avoid!) · split(sep, limit) · at() for single chars (see chars card).

```
"hello".slice(1, 4); "2026-09-07".split("-"); "a,b".split(",", 1);
key: slice > substring (negatives + no arg-swapping surprises). split("") explodes
emoji — use [...str] or Intl.Segmenter.
```

String: replace/pad/repeat STRINGS

replace(str|re/, fn-or-str — FIRST only!) · replaceAll (all!) · \$&/\$1 replacements in strings · padStart/padEnd · repeat · concat (+ operator fine).

```
"a-b".replaceAll("-", "_"); String(7).padStart(3, "0"); "ab".repeat(3);
key: String pattern replaces FIRST match only — the eternal gotcha. replaceAll or /.../g
for every occurrence.
```

String: chars + unicode (at/charAt/normalize) STRINGS

Single chars: at(-1!)/charAt/charCodeAt (UTF-16!) vs codePointAt/fromCodePoint (FULL unicode — emoji ✓). Build: fromCharCode/fromCodePoint. Compare ends: endsWith/startsWith (position!). Trim edges: trimStart/trimEnd. Unicode: normalize (NFC/NFD — canonically-equal Arabic/diacritics!) · isWellFormed/toWellFormed · String.raw (untouched backslashes!). valueOf/toString.

```
"hi".at(-1); "👋".length; // 1, 2 (surrogate pair!) — use [...s].length
"é".normalize("NFC") === "\u00e9"; // true (composed vs decomposed!)
String.raw`C:\n`; // backslash kept!
key: length LIES on emoji (UTF-16 units!) — spread or Intl.Segmenter counts graphemes.
Normalize before comparing user text.
```

String: match/search + localeCompare STRINGS

match(/re/g — with g: array or null!) · matchAll (ITERATOR of full matches!) · search(/re/ — index or -1) · localeCompare (locale-aware A-Z + numeric: sensitivity!).

```
const m = "a1b2".match(/\d/g); // ["1","2"]
for (const x of "a1b2".matchAll(/(?<l>[a-z])(?<n>\d)/g)) console.log(x.groups);
users.sort((a, b) => a.name.localeCompare(b.name, "ar", { numeric: true }));
key: matchAll > match for groups (match drops them
with /g!). localeCompare numeric:true sorts file10 after file2.
```

Array: create/check (from/of/isArray) ARRAYS

from(iterable/mapFn — NodeLists/strings/arguments!) · fromAsync (async iterables!) · of (no single-number trap!) · isArray (cross-realm safe!) · length tricks (truncate/grow).

```
Array.from({ length: 3 }, (_, i) => i); Array.isArray(x); [...nodeList];
key: Array(3) = 3 HOLES (map skips!); Array.from({length:3}) = 3 undefineds (map
works!). isArray beats instanceof (iframes!).
```

Array: stack/queue (push/pop/shift/unshift) ARRAYS

push/pop (END — fast!) · shift/unshift (START — O(n) reindex!) · lengths returned (push/unshift) vs item (pop/shift). Deques for hot paths.

```
s.push(x); s.pop(); q.unshift(x); q.shift();
key: shift/unshift on huge arrays = slow. push+pop stacks are O(1) amortized.
```

Array: slice/concat/flat/join ARRAYS

slice (copy/extract, negatives!) · concat (merge N arrays!) · flat(depth — Infinity deep!) · join (string glue) · toString (join(",")). ALL non-mutating.

```
a.slice(1, 3); a.concat(b); deep.flat(2); a.join(" · ");  
key: flat(Infinity) deep-flattens. join("") rebuilds strings from char arrays.
```

Array: flatMap + pipelines ARRAYS

flatMap (map + flatten ONE level — expand-or-drop: return [] to DROP!). Chain map→filter→flat (or flatMap) → reduce/sort. toSorted/toReversed/toSpliced/with = NON-mutating twins (see modern card!).

```
sentences.flatMap(s => s.split(" ")); // words, one level  
users.filter(u => u.active).flatMap(u => u.tags ?? []);  
key: flatMap returning [] filters AND maps in one pass — the elegant expand/drop.
```

Array: indexOf/includes/find ARRAYS

indexOf/lastIndexOf (strict ===, fromIndex!) · includes (NaN-SAFE!) · find/findIndex/findLast/findLastIndex (predicates!). Primitives → includes; objects → find.

```
[NaN].includes(NaN); users.find(u => u.id === id) ?? null;  
key: includes handles NaN; indexOf doesn't. findLast scans from the end (new!).
```

Array: every/some + keys/values/entries ARRAYS

every (ALL, vacuous-true on empty!) · some (ANY, false on empty!) · keys/values/entries iterators · forEach (no break! — use for...of/every/some to exit).

```
rows.every(r => r.ok); for (const [i, v] of a.entries()) { }  
key: [].every() === true (vacuous truth!) — guard empty arrays in validations.
```

Array: fill/copyWithin ARRAYS

fill(value, start, end — SAME ref for objects!) · copyWithin(target, start, end — memmove!).
In-place buffer/matrix surgery.

```
Array(5).fill(0); buf.copyWithin(0, 3);  
key: fill({}) shares ONE object across slots — map fresh objects instead.
```

Array: modern (at/toSorted/with/findLast) ARRAYS

at(-1!) · findLast/findLastIndex (search from END!) · lastIndexOf · reduceRight · NON-mutating twins: toSorted/toReversed/toSpliced/with(i, v)! (originals sort/reverse/splice MUTATE!).

```
arr.at(-1); // last, readable!  
const ranked = scores.toSorted((a, b) => b - a); // scores UNTOUCHED ✓  
const fixed = row.with(2, "x"); // copy with index 2 replaced  
key: Default sort() is ALPHA ([10,9,80]→[10,80,9]!) and IN-PLACE — toSorted +  
comparator fixes both.
```

Array: groupBy (modern) ARRAYS

Object.groupBy groups by key function (ES2024).

```
Object.groupBy(items, i => i.cat);  
key: Replaces reduce-to-object boilerplate.
```

Object: keys/values/entries/fromEntries OBJECTS

keys/values/entries (own STRING keys!) · fromEntries (rebuild!) · Invert/filter/map via entries round-trip. Symbol keys skipped (see meta card!).

```
Object.entries(o).map(([k, v]) => [k, v * 2]); Object.fromEntries(pairs);  
key: fromEntries(entries) = object map/filter. Non-enumerable + symbols need the meta card.
```

Object: assign/spread + freeze/seal OBJECTS

Merge/copy + immutability levels.

```
const n = { ...a, ...b }; Object.freeze(CONFIG);  
key: freeze = shallow; deep needs recursion/structuredClone.
```

Object: hasOwn + in + optional chain OBJECTS

Own-vs-inherited checks + safe diving.

```
Object.hasOwn(o, "x"); "toString" in o; o?.a?.b ?? d;  
key: hasOwn beats hasOwnProperty (safe objects).
```

Object: get/set + defineProperty OBJECTS

Computed accessors + fine-grained descriptors.

```
const o = { _n: 1, get n() { return this._n; } };
key: Getters look like props, run like fns.
```

Object: meta (create/proto/descriptors) OBJECTS

create(proto — prototypal inheritance!) · getPrototypeOf/setPrototypeOf (rare!) ·
 getOwnPropertyNames (+ non-enumerable!) · getOwnPropertySymbols ·
 getOwnPropertyDescriptor(s) (writable/enumerable/configurable!) · defineProperties
 (batch!) · preventExtensions/seal/freeze (+ is\* checks!) · is (SameValue: NaN equal, -0 ≠
 +0!).

```
const child = Object.create(proto, { x: { value: 1, writable: true } });
Object.getOwnPropertyNames(child); Object.is(NaN, NaN); // true!
key: for-in/keys SKIP symbols + non-enumerables — the meta API sees EVERYTHING.
```

Number: parse/intl/format NUMBERS

Parse: Number()/parseInt(str, RADIX!)/parseFloat · Checks: isInteger/isSafeInteger/isFinite/
 isNaN (SAFE globals!) vs Number.\* twins · Format: toFixed/toPrecision/toExponential/
 toString(radix!) · EPSILON/MAX\_SAFE\_INTEGER/MIN\_VALUE. Intl: currency/percent/units
 per locale.

```
Number.parseInt("08", 10); (255).toString(16); (1/3).toFixed(2);
(1234.5).toPrecision(6);
Number.isSafeInteger(2 ** 53); // false!
new Intl.NumberFormat("ar-EG", { style: "currency", currency: "EGP" }).format(1299);
key: parseInt NEEDS radix 10 (legacy octal!). 0.1+0.2: compare with EPSILON, display
with toFixed. Intl = real i18n (Arabic-Indic digits free).
```

Math: clamp/random/round **MATH**

Rounding: round/ceil/floor/trunc (toward-zero!) · Sign/abs: abs/sign · Powers/roots: pow/sqrt/cbrt/hypot · Logs/exp: log/log10/log2/exp/expm1 · Trig: sin/cos/tan/asin/acos/atan/atan2 · Extremes: min/max (+ spread!) · Random: random (0-1!) · Constants: PI/E/LN2/SQRT2 · Bitwise: imul/clz32/fround. Crypto randomness = getRandomValues (see card 31!).

```
const clamp = (n, a, b) => Math.min(b, Math.max(a, n));
const d6 = 1 + Math.floor(Math.random() * 6);
Math.hypot(3, 4); // 5 — no overflow!
Math.atan2(dy, dx); // full-circle angle
key: random() is NOT crypto (predictable!). trunc ≠ floor for negatives (-1.5 → -1 vs -2!).
```

Date + Intl dates **DATES**

Construct: now()/parse()/UTC() · now(){ms} · Getters: getFullYear/getMonth(0-11!)/getDate(1-31!)/getDay(0=Sun!)/getHours/Minutes/Seconds/ms/getTime/getTimezoneOffset · UTC twins (getUTCHours...) · Setters: set (+ overflow math!) · setTime · toISOString/JSON (UTC!) vs toLocale (local!). Display: Intl.DateTimeFormat (dateStyle/timeStyle/timeZone!). NEVER hand-format.

```
const d = new Date("2026-09-07T19:30:00+02:00");
d.getFullYear(); d.getMonth(); // 8 = September (0-based!)
d.toISOString(); // UTC storage ✓
new Intl.DateTimeFormat("ar-EG", { dateStyle: "full", timeStyle: "short" }).format(d);
key: Months 0-11, days 1-31, getDay 0=Sunday — the three Date traps. Store UTC ISO; display localized.
```

Map / Set (+ Weak\*) COLLECTIONS

Map: set/get/has/delete/clear/size + keys/values/entries (insertion order!) · Set: add/has/delete + UNION/INTERSECTION (ES2025!): union/intersection/difference/symmetricDifference/isSubsetOf/isSupersetOf/isDisjointFrom · WeakMap/WeakSet: object-keys only, NO size/iteration (GC-friendly!).

```
const m = new Map([[1, "a"]]); const u = [...new Set(arr)];
new Set([1,2]).union(new Set([2,3])); // {1,2,3}
const meta = new WeakMap(); meta.set(user, { lastSeen: Date.now() });
key: Set = dedupe in one line + real set algebra now. Weak* = metadata that must not
leak memory.
```

JSON: stringify/parse + reviver DATA

Serialize + revive (Dates!) with replacer/reviver hooks.

```
JSON.stringify(o, null, 2);
JSON.parse(t, (k, v) => k === "at" ? new Date(v) : v);
key: try/catch every parse of outside text.
```

RegExp: test/exec + groups REGEX

Flags: g/i/m (multiline ^\$!) /s (dotAll!) /u (unicode!) /v (unicode sets!) /y (sticky!) /d (indices!). Methods: test (bool — LASTINDEX STATEFUL with /g!) · exec (match+groups+index!) · String: match/matchAll/search/replace/split. Groups: (...)/named (?<n>)/non-capturing (?:...)/lookahead (?=)/lookbehind (?<=).

```
/^(?<y>\d{4})-(?<m>\d{2})$/d.exec("2026-09").indices;
const re = /a/g; re.test("a"); re.test("a"); // true, FALSE (lastIndex moved!)
key: Global regexes REMEMBER lastIndex — reset it or drop /g for test(). Named groups =
self-documenting extraction.
```


Error types + cause **ERRORS**

Typed throws with chained causes.

```
throw new TypeError("Age must be int", { cause: { input } });  
key: cause preserves the original failure.
```

Proxy / Reflect (power tools) **META**

Intercept get/set/has for validation/logging/reactivity.

```
const safe = new Proxy(obj, { set(t, k, v) { if (k === "age" && v < 0) throw new  
RangeError(); return Reflect.set(t, k, v); } });  
key: Frameworks' reactivity engine (Vue!) in 5 lines.
```

Generators + iterators **ADVANCED**

Lazy sequences with yield; custom [Symbol.iterator].

```
function* range(n) { for (let i = 0; i < n; i++) yield i; }  
key: Infinite/paged streams without arrays.
```

Symbols **ADVANCED**

Unique collision-free keys (protocols, semi-private).

```
const ID = Symbol("id"); obj[ID] = 1;  
key: Symbols skip JSON + for-in (hidden channels).
```

BigInt **NUMBERS**

Arbitrary-precision integers (123n). Money/IDs beyond 2^{53} .

```
const big = 9007199254740993n; // 9007199254740993 exact (Number would lie!)  
key: No mixing with Number — convert explicitly.
```

Intl (locale power) **I18N**

Dates, numbers, lists, plurals, relative time per locale.

```
new Intl.RelativeTimeFormat("ar", { numeric: "auto" }).format(-1, "day");  
key: "أمس" free — never hand-roll locales.
```

Structured clone **DATA**

Deep-copy anything (except functions/DOM).

```
const deep = structuredClone(nested);  
key: The deep-copy answer (Ch 28).
```

Function: call/apply/bind + meta **FUNCTIONS**

call(thisArg, a, b)/apply(thisArg, [args]) (invoke NOW, set this!) · bind (bake this + leading args → NEW fn, later!). Meta: name/length (arity!) · toString (source!). Arrows: NO own this/arguments/prototype — inherit + non-constructible.

```
fn.call(user, 1, 2); fn.apply(user, args); const later = fn.bind(user, 1);  
el.addEventListener("click", handler.bind(this)); // bound once — removeEventListener  
needs SAME ref!  
key: bind returns a NEW function every call — store it or listeners leak/unremovable.
```

Operators master row **SYNTAX**

?? vs ||, ?. chain, ... spread/rest, , + void + typeof + in/instanceof.

```
a ?? b; o?.x; f(...args); "k" in o; x instanceof Date; typeof x;  
key: ?? keeps 0/""; || drops all falsy (Ch 25).
```

Destructuring patterns **SYNTAX**

Nested + rename + defaults + rest + swap + params.

```
const { a: { b = 1 } = {}, ...rest } = o; [x, y] = [y, x];  
key: Defaults need = {} guards at every level.
```

Template tag functions **STRINGS**

Tagged templates: SQL/safe-HTML builders.

```
const sql = (s, ...v) => ({ text: s.join("?"), values: v });  
key: Libraries use tags for injection-safety (Ch 45).
```

CHAPTER 80

Browser + Async APIs A-Z — Complete Reference

Reference

~ 120 min

DOM, events, fetch, storage, timers and observers — the browser API surface that matters.

IN THIS CHAPTER

- Selection + traversal
- Editing + attributes
- Events complete
- Fetch/async/storage
- Observers + misc power APIs

Datasheet

MDN

W3Schools

document: select/traverse DOM

Select: `querySelector(All)` · `getElementById` · `getElementsByClassName/TagName/Name` (LIVE!) · `forms/images/links/embeds` collections · `documentElement/head/body` · `activeElement`. Traverse: `parentElement/children/childNodes` · `first/lastElementChild` · `next/previousElementSibling` · `closest/matches` · `contains/compareDocumentPosition`.

```
el.closest("form"); el.matches(":focus-visible"); el.parentElement.children;
key: closest() + delegation = infinite buttons, one listener. *Element* twins skip text
nodes!
```

NodeList vs HTMLCollection DOM

Static (querySelectorAll, forEach ✓) vs LIVE (getElementsBy\*, auto-updates).

```
[...document.getElementsByTagName("li")].forEach(...); // spread live lists!  
key: Live lists mutate DURING loops – spread first.
```

Geometry: rects + scroll positions DOM

getBoundingClientRect (viewport-relative x/y/width/height!) · offsetTop/Left/Width/Height (layout!) · clientWidth/Height (padding box!) · scrollWidth/Height/Top/Left · window scrollX/scrollY + innerWidth/Height · scrollIntoView({behavior/block/inline}) · scrollTo/scrollBy (smooth!) · elementFromPoint/caretPositionFromPoint.

```
const r = el.getBoundingClientRect(); // viewport coords (scroll-aware math!)  
el.scrollIntoView({ behavior: "smooth", block: "center" });  
scrollTo({ top: 0, behavior: "smooth" });  
key: Rects are VIEWPORT-relative (add scrollY for document coords!). Batch reads then writes (layout thrash!).
```

create/insert/remove/clone DOM

createElement + append/prepend/before/after/replaceWith/remove/clone.

```
box.before(head); item.replaceWith(newer); el.remove();  
key: Modern methods beat appendChild/removeChild.
```

insertAdjacentHTML positions DOM

beforebegin/afterbegin/beforeend/afterend string insertion.

```
list.insertAdjacentHTML("beforeend", `- ${safe}</li>`);  
key: Fastest for YOUR trusted templates (escape user data!).

```

textContent/innerText/innerHTML/value DOM

Safe text / rendered text / parsed HTML / form values.

```
el.textContent = user; input.value; box.innerHTML = ownTemplate;  
key: User data → textContent. Always. (XSS, Ch 29)
```

classList/Dataset/style/attrs DOM

classList: add/remove/toggle(force!)/replace/contains · dataset (data-\* strings!) · style (inline only!) + setProperty(custom props!) · attrs: get/set/has/removeAttribute + getAttributeNames/toggleAttribute/hasAttributes · hidden property · getComputedStyle (FINAL values!) · CSS var read: getComputedStyle(el).getPropertyValue("--x").

```
el.classList.toggle("open", force); el.dataset.id; el.hidden = true;  
el.style.setProperty("--brand", "#c9a227");  
getComputedStyle(el).getPropertyValue("--brand"); // resolved value  
key: style.* reads INLINE only (usually "") — getComputedStyle for truth. Custom props  
via setProperty!
```

matchMedia (JS media queries) BROWSER

CSS breakpoints in JS: matchMedia("(max-width: 700px)").matches + change events (addEventListener!) — responsive BEHAVIOR, not just styles.

```
const mq = matchMedia("(prefers-color-scheme: dark)");  
apply(mq.matches); mq.addEventListener("change", e => apply(e.matches));  
key: Batteries for responsive JS: reduced-motion, orientation, hover-capability,  
resolution!
```

Forms: FormData + validity FORMS

Read all + custom validation messages + report.

```
new FormData(form).get("email"); input.setCustomValidity("..."); form.reportValidity();  
key: "" clears custom errors (Ch 31).
```

Events: listen/emit/options EVENTS

addEventListener {once/passive/capture} + CustomEvent dispatch.

```
el.addEventListener("click", fn, { once: true }); el.dispatchEvent(new  
CustomEvent("saved", { detail: id }));  
key: passive:true for scroll/touch perf.
```

Mouse/Keyboard/Touch events EVENTS

Mouse: click/dblclick/contextmenu/mousedown/up/move/over/out/enter/leave (no-bubble twins!) + wheel. Pointer (UNIFIED!): pointerdown/move/up/cancel/over + pressure/pointerType. Touch (legacy): touchstart/move/end. Keyboard: keydown/keyup (e.key! NOT keyCode) + composition (IME!). Forms: input/change/submit/reset/focus/blur/focusin/focusout (bubble twins!) + invalid. Drag: dragstart/dragover/drop/dragend. Clipboard: copy/cut/paste. Page: load/DOMContentLoaded/error/resize/scroll/beforeunload/online/offline/visibilitychange/hashchange/message.

```
addEventListener("keydown", e => { if (e.key === "Escape") close(); });  
key: Pointer Events replace mouse+touch duality (pressure + type included!). keydown  
repeats; keyup doesn't. focus/blur DON'T bubble – focusin/focusout do.
```

preventDefault/stopPropagation EVENTS

Block default (submit/reload) / stop bubbling upward.

```
form.addEventListener("submit", e => { e.preventDefault(); save(); });
key: preventDefault ≠ stopPropagation – different jobs!
```

Event delegation kit EVENTS

Parent listener + closest + dataset actions (Ch 30 pattern).

```
tbody.addEventListener("click", e => { const b = e.target.closest("[data-act]"); if (!
b) return; acts[b.dataset.act]?.(b); });
key: Constant memory + dynamic rows free.
```

Web Components (define + shadow + slots) COMPONENTS

customElements.define (hyphen REQUIRED!) · observedAttributes/
attributeChangedCallback · connected/disconnectedCallback · attachShadow({mode}) (style
ENCAPSULATION!) · slots (projection!) · ::part::slotted styling · template stamp ·
declarative shadow (shadowrootmode!).

```
class XCard extends HTMLElement {
  static observedAttributes = ["title"];
  connectedCallback() {
    this.attachShadow({ mode: "open" }).innerHTML = `<style>h3{color:navy}</
style><h3><slot name="t">Hi</slot></h3>`;
  }
}
customElements.define("x-card", XCard);
key: Framework-proof components: native, encapsulated, work in React/Vue/vanilla alike.
```


fetch complete NETWORK

Methods/headers/body/ok-check/timeout/abort/parse.

```
const r = await fetch(url, { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(o), signal: AbortSignal.timeout(8000) });
key: Check .ok – fetch resolves on 404/500!
```

URL + URLSearchParams NETWORK

Parse/build URLs + query strings safely (no string concat!).

```
const u = new URL("/s", location.origin); u.searchParams.set("q", q);
u.searchParams.set("page", 2);
key: searchParams encodes automatically.
```

Promises + combinators ASYNC

then/catch/finally + all/allSettled/race/any + resolve/reject.

```
await Promise.all([a(), b()]); await Promise.allSettled(jobs);
key: all = need-everything; allSettled = partial-OK.
```

async/await + generators-async ASYNC

try/catch/finally + for-await streams.

```
for await (const chunk of stream) { ... }
key: for-await consumes async iterables.
```

Timers: timeout/interval/rAF **ASYNC**

setTimeout/debounce + setInterval/heartbeat + requestAnimationFrame.

```
requestAnimationFrame(tick); // 60fps-synced (UI motion!)  
key: rAF for visuals; timers for logic.
```

AbortController **ASYNC**

Cancel fetch/streams/listeners; timeout() shorthand.

```
const c = new AbortController(); el.addEventListener("click", fn, { signal:  
c.signal }); c.abort();  
key: Abortable listeners = leak-free components.
```

Storage: local/session/cookies/IDB **STORAGE**

localStorage/sessionStorage: set/get/remove/clear/length/key(i) (STRINGS + ~5MB + SYNC-blocking!) · storage EVENT (cross-tab sync!) · Cookies: document.cookie string + attrs (Expires/Max-Age/Path/Domain/Secure/HttpOnly/SameSite!) · Partitioned (CHIPS!). Secrets → HttpOnly cookies ONLY.

```
localStorage.setItem("t", v); addEventListener("storage", syncTabs);  
document.cookie = "theme=dark; Max-Age=31536000; Path=/; SameSite=Lax; Secure";  
key: storage event fires in OTHER tabs only (perfect cross-tab sync!). JSON + try/catch every read.
```

IndexedDB (offline apps) **STORAGE**

Transactional object DB for offline-first apps (wrappers: idb lib).

```
import { openDB } from "idb"; const db = await openDB("app", 1, { upgrade(d) {  
  d.createObjectStore("notes", { keyPath: "id" }); } });  
key: Offline carts/drafts that survive everything.
```

IntersectionObserver **OBSERVERS**

Lazy-load/infinite-scroll/reveal-on-scroll, GPU-cheap.

```
new IntersectionObserver(es => es.forEach(e => e.isIntersecting && load(e.target)), {  
  rootMargin: "200px" }).observe(el);  
key: Replaces scroll listeners (jank-free).
```

MutationObserver/ResizeObserver **OBSERVERS**

Watch DOM changes / element size changes.

```
new ResizeObserver(es => layout(es[0].contentRect.width)).observe(panel);  
key: Responsive COMPONENTS (with container queries!).
```

document info + visibility + fullscreen BROWSER

Info: title/referrer/cookie/domain/readyState/characterSet/contentType/lastModified · Visibility: visibilityState (visible/hidden!) + visibilitychange (pause games/timers! — NOT beforeunload!) · Fullscreen: requestFullscreen/exitFullscreen/fullscreenElement + fullscreenchange · PiP: video.requestPictureInPicture.

```
document.addEventListener("visibilitychange", () => document.hidden ? pause() :
resume());
await el.requestFullscreen(); // user-gesture required!
key: visibilitychange = the CORRECT tab-hide signal (timers throttle, rAF stops — save
state!).
```

History/Navigation (SPA) BROWSER

History: pushState/replaceState (state+title+URL!) · back/forward/go · length · state · scrollRestoration (auto/manual!) + popstate event. Hash fallback: hashchange. MODERN: Navigation API (navigation.navigate/navigateerror/currentEntry — intercept + transitions!).

```
history.pushState({ p: "/shop" }, "", "/shop"); addEventListener("popstate", render);
// modern: navigation.addEventListener("navigate", e => e.intercept({ handler:
render }));
key: pushState does NOT fire popstate (only back/forward do!). Routers wrap this (React
Router, Ch 46).
```

Location/Navigator/Clipboard **BROWSER**

Location: href/hostname/host/port/pathname/search/hash/origin + assign/replace (no history!)/reload. Navigator: onLine · language/languages · userAgent/userAgentData (modern!) · hardwareConcurrency · share/canShare (native sheet!) · clipboard (readText/writeText + read/write + permissions!) · mediaDevices · geolocation · permissions.query · serviceWorker · storage.estimate · connection · cookieEnabled.

```
await navigator.clipboard.writeText(code); // HTTPS + focus needed!
if (navigator.canShare?.({ url })) await navigator.share({ title, url });
location.assign("/done"); // vs replace() = no back-button trace
key: Clipboard needs HTTPS + focus + permission. canShare guards the native sheet (mobile gold!).
```

BroadcastChannel + microtasks + idle **ASYNC**

BroadcastChannel (tabs/workers SAME-ORIGIN chat!) · MessageChannel/ports (private pipes!) · queueMicrotask (schedule after current, before render!) · requestIdleCallback/idle deadline (defer NON-urgent!) · scheduler.yield/postTask (modern!). Compare: rAF (visual!) vs timers (logic!).

```
const bc = new BroadcastChannel("cart"); bc.postMessage({ n: 3 }); bc.onmessage = sync;
queueMicrotask(cleanup); requestIdleCallback(prefetch, { timeout: 2000 });
key: Tabs talking via BroadcastChannel = multi-tab carts/auth sync with zero server!
```

Web Workers **PERF**

Background threads for heavy compute (no DOM access).

```
const w = new Worker("calc.js"); w.postMessage(data); w.onmessage = e => paint(e.data);
key: 60fps UI + heavy math, together.
```

Canvas/SVG/WebAudio snippets MEDIA

Draw pixels / scale vectors / synthesize sound.

```
ctx.fillRect(0, 0, 10, 10); new AudioContext().createOscillator();  
key: Canvas = pixels; SVG = shapes; Audio = waves.
```

Geolocation/Notifications DEVICE

Geolocation: `getCurrentPosition/watchPosition/clearWatch` + options (`enableHighAccuracy/timeout/maximumAge!`). Errors: `PERMISSION_DENIED/POSITION_UNAVAILABLE/TIMEOUT`. Notifications: `permission/requestPermission` + `new Notification(title, {body/icon/tag/requireInteraction})` · SW push for background.

```
navigator.geolocation.watchPosition(p => map(p.coords), err, { enableHighAccuracy:  
true, timeout: 8000 });  
if ((await Notification.requestPermission()) === "granted") new Notification("Order  
ready!", { body: "Pickup #42" });  
key: Always handle DENIED gracefully + offer manual fallback (city input!).  
watchPosition drains battery – clearWatch!
```

Modules: import/export/dynamic ORGANIZE

Static deps + lazy `import()` + `import.meta` + top-level `await`.

```
const { heavy } = await import("./heavy.js"); new URL("./a.png", import.meta.url);  
key: Dynamic import = route-level code-splitting.
```

import maps + JSON modules ORGANIZE

Bare-specifier mapping + JSON imports (modern browsers).

```
import data from "./d.json" with { type: "json" };  
key: No bundler needed for learning-sized apps.
```

console + performance APIs DEBUG

Console: log/info/warn/error/table/dir/group/time/assert/trace/count/clear + %c styling! Perf: now() (ms, monotonic!) · mark/measure/getEntriesByType · PerformanceObserver (longtask/largest-contentful-paint!) · memory (Chrome!) · timeOrigin + navigation timing.

```
console.table(rows); console.time("x"); /*...*/ console.timeEnd("x");  
new PerformanceObserver(l => report(l.getEntries())) .observe({ type: "longtask",  
buffered: true });  
key: Measure, don't guess: longtask observer finds INP killers in PRODUCTION (RUM!).
```

Crypto (randomUUID/subtle) SECURITY

UUIDs + hashing/encrypting via SubtleCrypto (HTTPS only).

```
crypto.randomUUID(); await crypto.subtle.digest("SHA-256", bytes);  
key:.randomUUID for IDs; subtle for real crypto.
```

PART

Extended Datasheets III — Backend & Tools Lookup

Node, Express, SQL, Mongo, Git and React hooks: the server-side memory bank.

Ch 81 Node.js + Express A-Z — Complete Reference

Ch 82 SQL + Mongo + Git + React Hooks A-Z — Complete Reference

| Backend answers in seconds: APIs, queries and commands with examples.

CHAPTER 81

Node.js + Express A-Z — Complete Reference

Reference

~ 90 min

Server-side JavaScript and the Express API surface that runs your backends.

IN THIS CHAPTER

- Node core (fs/path/events)
- npm + modules
- Express app + routing
- Request/response + middleware
- Errors + security + testing

Datasheet

MDN

Docs

fs/promises (read/write files) NODE CORE

readFile/writeFile/appendFile · copyFile/cp (recursive!) · mkdir/readdir · rename/unlink/rm · stat/access/exists? (use access!) · mkdtemp · watch (recursive!) · open/filehandles (big files!) · glob (node 22+!). Sync twins BLOCK — servers stay async!

```
import { readFile, writeFile } from "node:fs/promises";
const t = await readFile("data.json", "utf8"); await writeFile("out.txt", t);
key: utf8 = string; no encoding = Buffer.
```

path (join/resolve/dirname) NODE CORE

join (segments!) · resolve (absolute!) · dirname/basename/extname · parse/format (object round-trip!) · relative (from→to!) · normalize · isAbsolute · sep/delimiter · posix/win32 (force flavors!).

```
import path from "node:path";
path.join(dir, "file.txt"); path.extname(f); path.basename(f, ".js");
key: join() > string concat (slashes differ!).
```

\_\_dirname in ESM NODE CORE

ESM has no `__dirname` — rebuild from `import.meta.url`.

```
import { fileURLToPath } from "node:url"; import path from "node:path";
const __dirname = path.dirname(fileURLToPath(import.meta.url));
key: Paste this; every ESM project needs it.
```

events (EventEmitter) NODE CORE

on/once (single!) · off/removeListener/removeAllListeners · emit (sync!) · prependListener · listenerCount · getMaxListeners/setMaxListeners · error EVENT (unhandled = CRASH!) · captureRejections · events.once (promise!).

```
import { EventEmitter } from "node:events";
const bus = new EventEmitter(); bus.on("done", id => log(id)); bus.emit("done", 7);
key: MaxListeners warning = probable leak.
```

process (env/argv/exit) NODE CORE

env/argv (slice(2)!) · argv parsing (util.parseArgs!) · cwd/chdir · exit(code)/exitCode · uptime/memoryUsage/cpuUsage · pid/ppid · signals (SIGTERM/SIGINT handlers!) · stdin/stdout/stderr · hrtime · nextTick · emitWarning.

```
process.env.PORT ?? 5000; process.argv.slice(2); process.exit(1); // 0 ok, 1 fail
key: Validate env at BOOT (Ch 41-snippet).
```

Streams (big data, tiny memory) NODE CORE

Types: Readable/Writable/Duplex/Transform/PassThrough · pipeline (stream/promises — ERRORS + CLEANUP!) · finished · Readable.from (any iterable!) · async iteration (for await!) · setEncoding · Transform (map/filter chunks!) · objectMode · highWaterMark (backpressure!). NEVER .pipe without error handling — pipeline() or death.

```
import { createReadStream } from "node:fs";
createReadStream("big.csv").pipe(res); // memory flat ✓
key: 5GB file + 512MB server = streams or death.
```

Buffer + crypto NODE CORE

Buffer: from/alloc/allocUnsafe/concat/compare/byteLength · encodings (utf8/base64/base64url/hex/latin1!) · read/writeUInt/Double · subarray/slice. Crypto: randomUUID/randomBytes (ONLY secure RNG!) · createHash (sha256!) · createHmac · scrypt/pbkdf2 (PASSWORDS!) · timingSafeEqual (compare secrets!) · subtle (WebCrypto!). NEVER Math.random for secrets/tokens.

```
import crypto from "node:crypto";
crypto.randomUUID(); crypto.randomBytes(32).toString("hex"); // tokens, salts
key: randomBytes for secrets; Math.random NEVER.
```

npm scripts + packages **NPM**

Scripts: dev/build/start/test/lint + pre/post hooks (pretest!) · run env (\$npm\_package\_version!) · npx/dlX (zero-install tools!) · workspaces (monorepos!) · audit/fix (vulns!) · outdated/update · fund · pack/publish (libraries!) · .npmrc (registry/auth) · exec.

```
// npm run dev | npm i express | npm i -D vitest | npm ci (CI: lockfile-exact!)  
key: ci > install in CI/deploy (reproducible!).
```

package.json fields **NPM**

type (module!) · main/exports (subpath map + conditions!) · imports (#aliases!) · bin · files (publish whitelist!) · engines (node>=20!) · scripts · deps/devDeps/peerDeps (plugins!) + optional · browserslist · private (block publish!) · version (semver!) · license.

```
{ "type": "module", "engines": { "node": ">=20" }, "imports": { "#db": "./src/db.js" } }  
key: type:module = modern import/export.
```

dotenv (.env config) **CONFIG**

Local env from .env (gitignored!) + .env.example docs names.

```
import "dotenv/config"; // npm i dotenv  
const url = process.env.DATABASE_URL; if (!url) throw new Error("DATABASE_URL missing");  
key: .env NEVER committed (leaked keys = breach).
```

Express app + listen EXPRESS

app.set (trust proxy! — req.ip behind nginx · json spaces · query parser · case sensitive · etag · x-powered-by off!) · app.get/enable/disable · listen(port/host/backlog/callback!) · server.close (graceful!) · app.use/app.METHOD/app.all/app.route/app.param.

```
import express from "express";
const app = express(); app.use(express.json({ limit: "100kb" }));
app.listen(process.env.PORT ?? 5000, () => console.log("up"));
key: limit kills giant-payload DoS.
```

Routing (verbs + params) EXPRESS

Router (mini-apps!) · route(path).get().post() chains! · param(name, fn — preload/validate!) · mergeParams (nested routers!) · caseSensitive/strict per-router · path patterns (regex/wildcards!) · app.use(mount) + next('route'/'router!').

```
const r = express.Router();
r.get("/", list); r.post("/", create); r.get("/:id", one); r.patch("/:id", edit);
r.delete("/:id", drop);
app.use("/api/v1/notes", r);
key: Nouns + verbs + version (REST, Ch 39).
```

req (params/query/body/headers) EXPRESS

params/query/body · headers/get(field!) · cookies/signedCookies (cookie-parser!) · ip/ips (trust proxy!) · path/hostname/protocol/secure · subdomains · xhr (legacy AJAX flag!) · fresh/stale (cache!) · accepts/types/is (content-negotiation!) · range · route · originalUrl/baseUrl.

```
req.params.id; req.query.page; req.body.text; req.headers.authorization; req.ip;
key: Validate ALL four (Ch 45).
```

res (status/json/send/redirect) EXPRESS

status/json/jsonp/send/end · sendStatus · redirect(status?, url!) · cookie(name, val, {httpOnly/secure/sameSite/maxAge/signed!}) + clearCookie · download/attachment (files!) · sendFile (absolute + options!) · type/format (negotiation!) · links/vary/append/set/get · locals (template data!). ONE response per request!

```
res.status(201).json(note); res.sendStatus(204); res.redirect("/login");
res.download(path);
key: ONE response per request (else crash!).
```

Middleware (use + order) EXPRESS

app/router/use (path scoping!) · next()/next(err)/next('route') · error-first 4-arg (LAST!) · built-ins: static/json ({limit!}/urlencoded({extended!}))/raw/text · third-party: cookieParser/compression/morgan/cors/helmet · async: wrap or Express 5 (auto-catch!).

```
app.use((req, res, next) => { console.log(req.method, req.url); next(); });
key: No next() + no res = HANG forever.
```

express.static (files + SPA) EXPRESS

Options: maxAge/immutable (hashed!) · index (false = off!) · fallthrough (→ next on miss!) · redirect (trailing-slash!) · dotfiles (deny!) · extensions · setHeaders (custom cache!) · SPA fallback AFTER /api (else API 404s serve HTML!).

```
app.use(express.static("public", { maxAge: "1y", immutable: true }));
app.get("*", (req, res) => res.sendFile("index.html", { root: "public" }));
key: Fallback AFTER /api (else API 404s serve HTML!).
```

asyncH (async errors) EXPRESS

Catch rejected promises → next(err). No try/catch spam.

```
const asyncH = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);  
key: Every async handler wrapped (or express 5 auto!).
```

Error middleware (LAST, 4 args) EXPRESS

(err, req, res, next) — log full, send safe shape.

```
app.use((err, req, res, next) => { console.error(err); res.status(err.status || 500).json({ error: { message: err.status ? err.message : "Server error" } }); });  
key: 4 args + LAST position (both required!).
```

helmet + cors + rate-limit SECURITY

Helmet: contentSecurityPolicy (directives!) · crossOriginEmbedder/Opener/ResourcePolicy · hsts (maxAge!) · frameguard · noSniff · referrerPolicy. CORS: origin (exact/fn!) · credentials (needs exact origin!) · methods/allowedHeaders/exposedHeaders/maxAge/preflightContinue/optionsSuccessStatus. Rate-limit: windowMs/limit (per-IP!) · keyGenerator · skip · handler (429 JSON!) · store (Redis prod!).

```
app.use(helmet()); app.use(cors({ origin: process.env.FRONT_URL }));  
app.use("/auth", rateLimit({ windowMs: 15 * 60e3, limit: 20 }));  
key: Exact origins (never * + credentials!).
```

Validation (express-validator) SECURITY

Chains: `body/param/query/cookie/header()` · `validators` (`isEmail/isInt/isUUID/isISO8601/...` 100+!) · `sanitizers` (`trim/escape/normalizeEmail/toInt/toBoolean!`) · `custom` (DB checks!) · `withMessage` · `bail` (stop chain!) · `matchedData` (ONLY validated!) · `validationResult` → 400-shape. Validate + sanitize EVERY input!

```
body("email").isEmail().normalizeEmail(), body("age").isInt({ min: 13 }},
(req, res, next) => { const e = validationResult(req); if (!e.isEmpty()) return
res.status(400).json({ error: { message: "Invalid", details: e.array() } }); next(); }
key: Validate + sanitize (normalize/trim/escape).
```

bcrypt (hash + compare) AUTH

`hash(pw, SALT_ROUNDS=12 — ~250ms, tune yearly!)` · `genSalt` · `compare(pw, hash — timing-safe-ish!)` · `compareSync/hashSync` (BLOCKING — avoid in servers!). Same 401 message either way (no enumeration!). NEVER plain/MD5/SHA-direct (fast = crackable!). Pepper (server-side secret + hash) for extra defense.

```
const hash = await bcrypt.hash(pw, 12);
const ok = await bcrypt.compare(pw, user.hash); // same 401 either way!
key: 12 ≈ 250ms (tune yearly; verify < login UX pain).
```

JWT (sign + verify + guard) AUTH

`sign(payload, secret, {expiresIn/audience/issuer/subject/jwtid!})` · `verify` (algorithms WHITELIST! ["HS256"] — kill none-alg!) · `decode` (UNVERIFIED peek!) · Pattern: short access (15m, memory!) + rotating refresh (HttpOnly cookie!) · `version/jti` revocation (logout-all!) · JWKS (Auth0/Firebase!). Payload PUBLIC (signed, not secret!).

```
jwt.sign({ sub: u.id }, S, { expiresIn: "15m" }); jwt.verify(t, S);
key: Payload PUBLIC (signed, not encrypted!). Secrets in env.
```


multer (uploads) FILES

dest (disk!) vs memoryStorage (buffers!) vs custom diskStorage (destination/filename fns!) · limits (fileSize/fields/files/parts!) · fileFilter (mimetype allowlist!) · single/array/fields/none · req.file/files · errors (LIMIT\_FILE\_SIZE → 413!). Post-process: re-encode images (sharp — strips malware!), scan, uuid names, serve with safe Content-Type.

```
multer({ dest: "tmp/", limits: { fileSize: 5 * 1024 * 1024 }, fileFilter: (r, f, cb) =>
  cb(null, ["image/png", "image/jpeg"].includes(f.mimetype)) });
key: Never trust filename/extension (traversal!).
```

API query design (filter/sort/page/shape) EXPRESS

Conventions clients love: ?q= (search!) · ?cat=a&cat=b (arrays!) · ?min=&max= (ranges!) · ?sort=-created,name · ?page=&limit= (capped!) · ?fields=id,name (sparse!) · ?embed=author (includes!) · 400 on garbage + echo normalized query back. Cursor alternative: ?after=<id>.

```
const { q = "", sort = "-created", page = 1, limit = 20, fields } = req.query;
const dir = sort.startsWith("-") ? -1 : 1;
const rows = await find({ q, sort: { [sort.replace(/^-/ , "")]: dir }, skip: (page - 1)
  * limit, limit: Math.min(limit, 100), fields: fields?.split(",") });
key: Cap limit (max 100!), validate sort keys against a whitelist (injection!), echo
  back for debuggability.
```

Testing APIs (supertest) TESTS

request(app).get/post/put/patch/delete · send/set/query/attach (files!) · expect(status/body/headers!) · agent (COOKIES/sessions!) · lifecycle: beforeAll/afterAll (DB up/down!) · test DB (dedicated + truncate!) · coverage (statements/branches!) · factories/fakers (repeatable data!) · 401/403/404/409 negative tests ALWAYS.

```
import request from "supertest"; import app from "./app.js";
await request(app).post("/notes").send({ text: "hi" }).expect(201);
key: Test codes + shapes + auth-failures (401/403!).
```

Graceful shutdown + readiness EXPRESS

SIGTERM/SIGINT handlers: stop accepting → drain in-flight (timeout!) → close DB pool → exit(0!). Kubernetes: /ready fails during drain (no new traffic!). PM2/systemd send SIGINT; Docker stop = SIGTERM + 10s grace.

```
const server = app.listen(PORT);
const bye = async sig => { console.log(sig); server.close(); await pool.end();
process.exit(0); };
process.on("SIGTERM", bye); process.on("SIGINT", bye);
key: process.exit() mid-request = dropped users + corrupt jobs. Drain, then die.
```

Nodemon + env boot check DX

Auto-restart dev + fail-loud missing config.

```
// "dev": "nodemon src/index.js"
for (const k of ["DATABASE_URL", "JWT_SECRET"]) if (!process.env[k]) {
  console.error("Missing " + k); process.exit(1); }
key: Crash at boot, not at 3 AM.
```

CHAPTER 82

SQL + Mongo + Git + React Hooks A-Z — Complete Reference

Reference

~ 90 min

Queries, documents, version control and hooks — the daily-driver cheat sheets.

IN THIS CHAPTER

- SQL CRUD + JOINS + reports
- Constraints + indexes + transactions
- Mongo CRUD + aggregate
- Git daily + undo
- React hooks A-Z

Datasheet

Tutorialspoint

W3Schools

SQL: SELECT + WHERE + ORDER + LIMIT SQL

SELECT DISTINCT (dedupe!) · aliases (AS!) · WHERE (AND/OR/NOT + precedence!) · comparison (=/</>/!=/BETWEEN/IN/LIKE/IS NULL!) · ORDER BY (col/dir + NULLS FIRST/LAST!) · LIMIT/OFFSET (pages!) · FETCH FIRST (standard!) · keyset (WHERE id > last — SCALES!).

```
SELECT id, email FROM users WHERE active AND age >= 13 ORDER BY created_at DESC LIMIT 20 OFFSET 40;
key: OFFSET pages; keyset (id > last) SCALES.
```

SQL: INSERT + UPDATE + DELETE + RETURNING SQL

INSERT (multi-row VALUES! + SELECT-insert!) · ON CONFLICT (upsert: DO NOTHING/UPDATE!) · UPDATE...FROM (joins!) · DELETE...USING · TRUNCATE (fast + reset identity!) · RETURNING \* (no second query!) · Ownership in EVERY WHERE (multi-tenant!) · affected-row checks (0 = 404!).

```
INSERT INTO notes(user_id, text) VALUES ($1, $2) RETURNING *;
UPDATE notes SET text = $1 WHERE id = $2 AND user_id = $3 RETURNING *;
key: Ownership in EVERY WHERE (multi-tenant!).
```

SQL: JOINS (INNER/LEFT/SELF) SQL

INNER (pairs!) · LEFT/RIGHT (all one side + NULLs!) · FULL (everything!) · CROSS (cartesian — deliberate only!) · SELF (hierarchies: employees m/e!) · USING (shared col!) vs ON · LATERAL (per-row subqueries!) · NATURAL (avoid!). LEFT for reports-including-zeros.

```
SELECT o.*, u.email FROM orders o JOIN users u ON u.id = o.user_id;
SELECT u.email, o.id FROM users u LEFT JOIN orders o ON o.user_id = u.id;
-- inactive = NULL ✓
key: LEFT for reports-including-zeros.
```

SQL: GROUP BY + HAVING + aggregates SQL

COUNT (\* / col / DISTINCT!) · SUM / AVG / MIN / MAX · STRING\_AGG / ARRAY\_AGG / JSON\_AGG (collect!) · FILTER (WHERE cond — conditional agg!) · GROUPING SETS / ROLLUP / CUBE (subtotals!) · HAVING (groups!) vs WHERE (rows!) · every non-agg SELECT col must GROUP (or functionally-dep!).

```
SELECT user_id, COUNT(*), SUM(total) FROM orders GROUP BY user_id HAVING COUNT(*) > 3;
key: WHERE filters ROWS; HAVING filters GROUPS.
```

SQL: Constraints (NOT NULL/UNIQUE/CHECK/FK) SQL

NOT NULL/UNIQUE/PRIMARY KEY (implies both!) · CHECK (ranges/patterns!) · FK: REFERENCES + ON DELETE/UPDATE (CASCADE/RESTRICT/SET NULL/SET DEFAULT!) · DEFERRABLE (defer to COMMIT!) · EXCLUDE (no-overlap bookings!) · named CONSTRAINTs (readable errors!) · NOT VALID (add without scan!) · domains (reusable types!) · DEFERRABLE INITIALLY (FK timing!).

```
email CITEXT UNIQUE NOT NULL, qty INT CHECK (qty > 0),
user_id INT REFERENCES users(id) ON DELETE CASCADE
key: UNIQUE(email) kills race-condition dupes.
```

SQL: Indexes + EXPLAIN SQL

Types: btree (default!) · hash · gin (arrays/JSON/full-text!) · gist (geo/ranges!) · brin (huge ordered!) · Partial (WHERE active!) · Expression (lower(email)!) · Covering (INCLUDE cols!) · Multicol order (leftmost rule!) · UNIQUE (constraint + speed!) · CONCURRENTLY (no lock!) · REINDEX · EXPLAIN (ANALYZE/BUFFERS!) — Index Scan ✓ vs Seq Scan ✗.

```
CREATE INDEX idx_o_user_date ON orders(user_id, created_at DESC);
EXPLAIN ANALYZE SELECT ...; -- Index Scan ✓ vs Seq Scan ✗
key: Compound order matters; partial for hot slices.
```

SQL: Transactions SQL

BEGIN/COMMIT/ROLLBACK · SAVEPOINT/ROLLBACK TO/RELEASE (partial!) · Isolation: READ COMMITTED (default) / REPEATABLE READ / SERIALIZABLE (safest + retries!) · SELECT...FOR UPDATE/SHARE (row locks!) · NOWAIT/SKIP LOCKED (queues!) · advisory locks (pg\_advisory\_xact\_lock!) · single-statement atomicity · app retry on serialization failure!

```
BEGIN; UPDATE a SET b = b - 100 WHERE id = 1 AND b >= 100; UPDATE a SET b = b + 100
WHERE id = 2; COMMIT;
key: Checks INSIDE txn (race-safe!).
```

SQL: string/date/math/conditional functions SQL

String: LENGTH/UPPER/LOWER/TRIM/BOTH/SUBSTRING/POSITION/SPLIT\_PART/
 CONCAT\_WS/FORMAT/LPAD/RPAD/REPLACE/REGEXP\_REPLACE/REGEXP\_MATCHES.
 Date: NOW/CURRENT\_DATE/CURRENT\_TIMESTAMP/EXTRACT/DATE\_TRUNC/AGE/
 INTERVAL arithmetic/TO\_CHAR/TO\_DATE. Math: ROUND/CEIL/FLOOR/ABS/MOD/POWER/
 SQRT/RANDOM/GREATEST/LEAST. Conditional: CASE WHEN/COALESCE/NULLIF/CAST::/
 GREATEST.

```
SELECT CONCAT_WS(' ', first, last), EXTRACT(YEAR FROM hired), AGE(NOW(), hired),
       CASE WHEN total > 1000 THEN 'VIP' ELSE 'std' END, COALESCE(nick, first, 'anon') FROM
users;
key: TO_CHAR(d, 'YYYY-MM-DD') + DATE_TRUNC('month', d) = 90% of reporting. AGE returns
INTERVAL (justify it!).
```

SQL: CTEs + VIEWS + set ops SQL

CTEs: WITH x AS (...) (readable pipelines!) + RECURSIVE (org charts/breadcrumbs!) ·
 VIEWS (saved queries!) + MATERIALIZED (cached + REFRESH CONCURRENTLY!) · Set
 ops: UNION (dedupe!) / UNION ALL (fast!) / INTERSECT / EXCEPT. CTEs optimize as fences
 (PG12+: inlined!).

```
WITH RECURSIVE chain AS (
  SELECT id, boss FROM emp WHERE id = 7
  UNION ALL SELECT e.id, e.boss FROM emp e JOIN chain c ON e.id = c.boss
) SELECT * FROM chain;
key: Recursive CTE = hierarchies in pure SQL (no app loops!). UNION vs UNION ALL =
dedupe cost choice.
```

SQL: window functions (OVER/PARTITION) SQL

ROW\_NUMBER/RANK/DENSE\_RANK (rankings!) · LAG/LEAD (prev/next rows!) · FIRST\_VALUE/LAST\_VALUE · SUM/AVG/COUNT OVER (running totals!) · PARTITION BY (per-group!) · ROWS/RANGE frames (BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW!). No GROUP collapse — every row survives!

```
SELECT name, total,
       RANK() OVER (ORDER BY total DESC) AS r,
       LAG(total) OVER (ORDER BY day) AS prev,
       SUM(total) OVER (PARTITION BY city ORDER BY day) AS running
FROM orders;
```

key: Top-N-per-group + running totals + deltas = the window-function power trio. RANK skips ties; DENSE\_RANK doesn't.

SQL: LIKE/ILIKE + full-text SQL

LIKE/ILIKE (% , \_ + ESCAPE!) · leading % kills btree (pg\_trgm GIN for %x%!) · similarity/% operator + threshold · to\_tsvector/to\_tsquery (dictionaries! english/arabic?) · plainto/phraseto\_tsquery · ts\_rank/ts\_headline · GIN on tsvector · prefix :\* (autocomplete!).

```
SELECT * FROM books WHERE title ILIKE '%egypt%';
```

key: Leading % kills indexes (trigram/GIN for scale).

Mongo: CRUD (find/insert/update/delete) MONGO

find/findOne (filter/projection/sort/limit/skip!) · insertOne/Many (ordered!) · updateOne/Many/replaceOne (upsert!) · findOneAndUpdate/Delete (RETURNING twin!) · deleteOne/Many · bulkWrite (mixed batches!) · countDocuments/estimatedDocumentCount/distinct · .lean() (plain fast objects!).

```
await Note.find({ done: false }).select("text").lean();
await Note.updateOne({ _id: id, user }, { $set: { text } });
```

key: .lean() = plain fast objects.

Mongo: Operators (\$in/\$gt/\$regex/\$or) MONGO

Compare: \$eq/\$ne/\$gt/\$gte/\$lt/\$lte/\$in/\$nin · Logic: \$and/\$or/\$nor/\$not · Existence: \$exists/\$type · Arrays: \$all/\$size/\$elemMatch (match object IN array!) · Eval: \$regex (+\$options i!) / \$expr (field-vs-field!) / \$where (JS — slow!) · Elements: \$ (positional updates!).

```
await P.find({ price: { $gte: 10, $lte: 50 }, cat: { $in: ["a", "b"] }, name: { $regex:
q, $options: "i" } });
key: Index the filtered fields (same rules!).
```

Mongo: Schema (validators + refs) MONGO

Types + required/unique/trim/lowercase/enum/min/max/minlength/maxlength/match/default · timestamps · refs (ObjectId + populate!) · nested/subdocs · virtuals (computed!) · methods/statics · pre/post hooks (hash passwords!) · schema.index() (unique/sparse/partial/TTL!) · strict/discriminators.

```
new Schema({ text: { type: String, required: true, trim: true, maxlength: 500 }, user:
{ type: ObjectId, ref: "User" } }, { timestamps: true });
key: App validates UX; schema enforces TRUTH.
```

Mongo: Aggregate (group/sort/reports) MONGO

Stages: \$match (FIRST!) · \$group (\_id + \$sum/\$avg/\$push/...) · \$sort/\$limit/\$skip · \$project/\$addFields/\$unset · \$unwind (arrays→rows!) · \$lookup (JOIN!) + \$unwind · \$count/\$facet (multi-pipeline!) / \$bucket · \$merge/\$out (write results!). Pipeline = Unix pipes for data.

```
await Review.aggregate([ { $group: { _id: "$camp", avg: { $avg: "$stars" }, n: { $sum:
1 } } }, { $sort: { avg: -1 } } ]]);
key: $match FIRST (shrink before grouping!).
```


Mongo: update operators (\$set/\$inc/\$push...) MONGO

Field: \$set/\$unset/\$rename · Numbers: \$inc/\$mul/\$min/\$max · Arrays: \$push (+\$each/\$slice/\$sort!)/\$pull/\$pullAll/\$pop/\$addToSet/\$position · Current: \$currentTime · Bitwise: \$bit · Upsert pairs (\$setOnInsert!). Array filters (\$[el]!) for surgical updates.

```
await U.updateOne({ _id }, { $inc: { logins: 1 }, $push: { tags: { $each: t, $slice: -10 } }, $setOnInsert: { created: new Date() } }, { upsert: true });
key: $slice:-10 in $push = capped arrays (latest-N feeds!) in ONE atomic op.
arrayFilters update nested matches.
```

Mongo: indexes + explain + TTL MONGO

createIndex ({email:1},{unique:true}) · compound (order = ESR rule!) · text (one per collection!) · 2dsphere (geo!) · partial (partialFilterExpression!) · sparse · TTL (expireAfterSeconds!) · hidden · collation (locale sort!) · explain("executionStats") (WINNING plan!) · listIndexes/dropIndex.

```
await C.createIndex({ user: 1, created: -1 });
await C.createIndex({ at: 1 }, { expireAfterSeconds: 3600 }); // self-cleaning!
await C.find(q).explain("executionStats");
key: ESR: Equality → Sort → Range (compound order!). TTL = sessions/OTPs that delete themselves.
```

Git: daily six GIT

status/add/commit/push/pull/log · switch (branches!) · restore (files!) · commit -a (tracked!) /-m/--amend (fix last!) · push -u (upstream!) · pull --rebase · log --oneline/graph/decorate · show/diff/stash. Small commits + imperative messages ("add", not "added").

```
git status; git add -A; git commit -m "feat: cart totals"; git push; git pull --rebase;
git log --oneline -10;
key: Commit small + messages imperative ("add", not "added").
```

Git: branches + merge vs rebase GIT

branch -c/-d/-D/-a · switch -c · merge (--no-ff/ff-only/squash!) · rebase (--onto/--continue/--abort!) · fetch vs pull · tracking (-u/--set-upstream-to!) · push --force-with-lease (SAFE force!) · workspace (parallel branches!). Rebase PRIVATE, merge PUBLIC — never rewrite shared history!

```
git switch -c feat/auth; git fetch; git rebase origin/main; git switch main; git merge feat/auth;
key: Never rebase SHARED history (rewrites teammates!).
```

Git: undo (restore/reset/revert) GIT

restore (files!) · restore --staged (unstage!) · reset --soft/mixed/hard (private ONLY!) · revert (PUBLIC-safe inverse commit!) · reflog (EVERYTHING recoverable ~90d!) · checkout -- file (legacy!) · clean -fd (untracked!). reset --hard DESTROYS — reflog is the parachute.

```
git restore file; git restore --staged file; git revert <sha>; git reset --soft HEAD~1; // reset --hard = DESTROYS
key: revert for public; reset for private ONLY.
```

Git: stash + .gitignore GIT

stash push -m/-u (untracked!)/-S (staged!) · list/show/apply (keep!) vs pop · branch (rescue!) · drop/clear · .gitignore (node\_modules/.env/dist/\*.log!) · global ignore · .gitkeep · skip-worktree/assume-unchanged (local config!). .env committed = rotate keys NOW.

```
git stash push -m wip; git stash pop;
node_modules/ .env dist/ *.log # .gitignore essentials
key: .env committed = rotate keys NOW.
```

Git: remotes/tags/cherry-pick/reflog **GIT**

Remotes: remote -v/add/show/prune + fetch --prune · Tags: tag v1.2.0 (light!) / -a (annotated!) + push --tags · cherry-pick (graft commits!) · revert vs reset (public/private!) · reflog (time machine!) · bisect (binary-search bugs!) · blame/shortlog · worktree (parallel checkouts!).

```
git tag -a v1.2.0 -m "ship"; git push --tags;
git cherry-pick alb2c3; git bisect start; git reflog;
key: Annotated tags = releases (who/when/message!). reflog rescues "lost" commits for
~90 days.
```

React: modern (use/Action/optimistic) **REACT**

use() (unwrap promises/context in render!) · useActionState (form actions + pending!) · useFormStatus (submit state!) · useOptimistic (instant UI + rollback!) · useTransition (non-blocking updates!) · useDeferredValue (laggy filters!) · useId (a11y ids!) · Suspense (loading boundaries!).

```
const [msg, act, pending] = useActionState(save, null);
const [opt, addOpt] = useOptimistic(list, (s, x) => [...s, x]);
<form action={act}><button disabled={pending}>Save</button></form>
key: Actions + optimistic = forms without fetch boilerplate. pending disables
double-submits free.
```

useState + useEffect **REACT**

Owned state + synced side effects (fetch/subscribe + cleanup!).

```
const [n, setN] = useState(0);
useEffect(() => { const c = new AbortController(); load(c.signal); return () =>
c.abort(); }, [url]);
key: Cleanup = no leaks + no stale sets.
```

useMemo + useCallback + memo REACT

Cache values/fns/components (measure FIRST!).

```
const total = useMemo(() => sum(items), [items]);
const onAdd = useCallback(id => add(id), [add]);
key: Premature memo = complexity without proof.
```

useRef + useId REACT

Mutable box (no re-render!) + DOM handles (focus/measure!) · useId (a11y ids!) ·
forwardRef (pass refs down!) + useImperativeHandle (curated API!) · callback refs (measure
on mount!) · previous-value pattern.

```
const inputRef = useRef(null); inputRef.current?.focus();
const id = useId(); // ally label pairing ✓
key: Refs escape renders (previous values, timers).
```

useReducer + useContext REACT

Complex state machines + slow shared state (auth/theme/cart).

```
const [s, dispatch] = useReducer(reducer, init);
const cart = useContext(CartCtx); // split contexts, memo values!
key: Reducer = predictable transitions + testable logic.
```

Custom hooks (useFetch/useLocalStorage) REACT

Extract reusable logic (Ch 24: useFetch/useDebounceValue!).

```
const { status, data } = useFetch("/api/notes"); // one line, 4 states ✓
key: Custom hooks = shared BEHAVIOR (components share UI).
```

Keys + lists + conditional **REACT**

Stable keys + map + && / ternary (never index!).

```
{items.map(it => <Row key={it.id} row={it} />)}  
{count > 0 ? <List /> : <Empty />}  
key: Keys preserve state/focus across renders.
```

Forms (controlled + action-state) **REACT**

value+onChange + validate + role=alert errors.

```
<input value={email} onChange={e => setEmail(e.target.value)} aria-invalid={!!err} aria-  
describedby="e-err" />  
{err && <p id="e-err" role="alert">{err}</p>}  
key: Controlled default; refs for one-shots/files.
```

PART

Capstone — Build BookHub End to End

A complete guided build: API, database, auth, frontend and deploy.

Ch 83 Capstone Backend — API + Auth + Data (Full Code)

Ch 84 Capstone Frontend — Pages + Style + App (Full Code)

| Read with your editor open: type every file, run every step, ship at the end.

CHAPTER 83

Capstone Backend — API + Auth + Data (Full Code)

Advanced

~ 180 min

Every backend file of BookHub, complete and runnable: store, auth, routes, errors, seed.

IN THIS CHAPTER

- Project shape + store module
- Auth (bcrypt + JWT)
- Books + reviews routes
- Errors + security + seed
- Run + test manually

Capstone

Project

0. Shape + install (one minute)

```
bookhub/  
  package.json ..... type:module + scripts (dev/start/seed)  
  .env.example ..... PORT + JWT_SECRET names (never real values!)  
  src/index.js ..... boot: env check + app + listen  
  src/app.js ..... helmet/cors/json/static/routes/errors  
  src/store.js ..... JSON-file repository (swap for PG via Ch 42!)  
  src/auth.js ..... hash/compare + sign/verify + requireAuth + ownerOnly  
  src/routes/books.js .... CRUD + search + pagination  
  src/routes/reviews.js .. nested under books + UNIQUE(user,book) rule  
  src/seed.js ..... demo books + users (idempotent)
```

Figure 28 — BookHub backend files (type these in order)

CODE · package.json — scripts run everything

```
{
  "type": "module",
  "scripts": {
    "dev": "node --watch src/index.js",
    "start": "node src/index.js",
    "seed": "node src/seed.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.2"
  }
}
```


1. Store (JSON-file repository — zero setup, honest pattern)

CODE · src/store.js — collections with atomic writes (read all, write all)

```
import { readFile, writeFile, mkdir } from "node:fs/promises";
import path from "node:path";
const DIR = path.join(process.cwd(), "data");
await mkdir(DIR, { recursive: true });
const file = c => path.join(DIR, `${c}.json`);
async function read(c) {
  try { return JSON.parse(await readFile(file(c), "utf8")); }
  catch { return []; } // missing/corrupt → empty (boot never dies)
}
async function write(c, rows) {
  const tmp = file(c) + ".tmp";
  await writeFile(tmp, JSON.stringify(rows, null, 2));
  await writeFile(file(c), JSON.stringify(rows, null, 2)); // crash-safe-ish: tmp first
}
export const store = {
  async all(c) { return read(c); },
  async find(c, fn) { return (await read(c)).find(fn); },
  async where(c, fn) { return (await read(c)).filter(fn); },
  async insert(c, row) { const rows = await read(c); rows.push(row); await write(c, rows); return row; },
  async update(c, fn, patch) {
    const rows = await read(c); const i = rows.findIndex(fn);
    if (i === -1) return null; rows[i] = { ...rows[i], ...patch }; await write(c, rows);
    return rows[i];
  },
  async remove(c, fn) { const rows = await read(c); const kept = rows.filter(r => !fn(r)); await write(c, kept); return rows.length - kept.length; }
};
export const uid = () => (crypto.randomUUID ? crypto.randomUUID() : String(Date.now()));
```

NOTE — WHY NOT POSTGRES HERE?

Postgres is the production answer (Ch 42) — but it needs a server. This store keeps the capstone runnable ANYWHERE while teaching the repository pattern: routes call store.\*, so swapping to PG later touches ONE file. Honest tradeoff, stated upfront.

2. Auth (bcrypt + JWT + guards)

CODE · src/auth.js — hash/compare + tokens + middleware (Ch 44, production-shaped)

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { store } from "../store.js";
const SECRET = process.env.JWT_SECRET || "dev-only-secret-change-me";
export const hash = pw => bcrypt.hash(pw, 12);
export const sameUserMessage = "Invalid email or password"; // no enumeration!
export async function register({ email, password, name }) {
  email = String(email).toLowerCase().trim();
  if (!/^[^@\s]+@^[^@\s]+\.[^@\s]+$/.test(email)) throw Object.assign(new Error("Valid email required"), { status: 400 });
  if (String(password).length < 8) throw Object.assign(new Error("Password needs 8+ chars"), { status: 400 });
  if (await store.find("users", u => u.email === email)) throw Object.assign(new Error("Email already registered"), { status: 409 });
  const user = await store.insert("users", { id: crypto.randomUUID(), email, name: String(name || email.split("@")[0]), hash: await hash(password), createdAt: new Date().toISOString() });
  return { id: user.id, email: user.email, name: user.name };
}
export async function login({ email, password }) {
  const user = await store.find("users", u => u.email === String(email).toLowerCase().trim());
  if (!user || !(await bcrypt.compare(String(password), user.hash))) throw Object.assign(new Error(sameUserMessage), { status: 401 });
  const access = jwt.sign({ sub: user.id }, SECRET, { expiresIn: "15m" });
  return { access, user: { id: user.id, email: user.email, name: user.name } };
}
export function requireAuth(req, res, next) {
  const t = req.headers.authorization?.split(" ")[1];
  if (!t) return res.status(401).json({ error: { message: "Login required" } });
  try { req.userId = jwt.verify(t, SECRET).sub; next(); }
  catch { res.status(401).json({ error: { message: "Session expired – login again" } }); }
}
```

3. Routes (books + nested reviews)

CODE · src/routes/books.js — CRUD + search + pagination (Ch 39 pattern, complete)

```
import { Router } from "express";
import { store } from "../store.js";
import { requireAuth } from "../auth.js";
const r = Router();

const asyncH = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
const page = q => ({ limit: Math.min(+q.limit || 12, 50), offset: ((+q.page || 1) - 1) *
Math.min(+q.limit || 12, 50) });

r.get("/", asyncH(async (req, res) => {
  const { limit, offset } = page(req.query);
  const q = String(req.query.q || "").toLowerCase();
  let rows = await store.all("books");
  if (q) rows = rows.filter(b => (b.title + " " + b.author).toLowerCase().includes(q));
  const total = rows.length;
  res.json({ data: rows.slice(offset, offset + limit), meta: { total, page:
(+req.query.page || 1), pages: Math.ceil(total / limit) } });
}));

r.get("/:id", asyncH(async (req, res, next) => {
  const book = await store.find("books", b => b.id === req.params.id);
  if (!book) return next(Object.assign(new Error("Book not found"), { status: 404 }));
  const reviews = await store.where("reviews", x => x.bookId === book.id);
  res.json({ ...book, avg: reviews.length ? +(reviews.reduce((s, x) => s + x.stars, 0) /
reviews.length).toFixed(1) : null, reviewsCount: reviews.length });
}));

r.post("/", requireAuth, asyncH(async (req, res) => {
  const { title, author, year } = req.body;
  if (!title?.trim() || !author?.trim()) return res.status(400).json({ error: { message:
"title and author required" } });
  const book = await store.insert("books", { id: crypto.randomUUID(), title:
title.trim(), author: author.trim(), year: +year || null, addedBy: req.userId, createdAt:
new Date().toISOString() });
  res.status(201).json(book);
}));

export default r;
```

CODE · src/routes/reviews.js — nested + one-review-per-user rule (UNIQUE pair!)

```
import { Router } from "express";
import { store } from "../store.js";
import { requireAuth } from "../auth.js";
const r = Router();
const asyncH = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
r.get("/books/:id/reviews", asyncH(async (req, res) => {
  res.json(await store.where("reviews", x => x.bookId === req.params.id));
}));
r.post("/books/:id/reviews", requireAuth, asyncH(async (req, res) => {
  const book = await store.find("books", b => b.id === req.params.id);
  if (!book) return res.status(404).json({ error: { message: "Book not found" } });
  const stars = +req.body.stars;
  if (!(stars >= 1 && stars <= 5)) return res.status(400).json({ error: { message: "stars must be 1-5" } });
  if (await store.find("reviews", x => x.bookId === book.id && x.userId === req.userId))
    return res.status(409).json({ error: { message: "You already reviewed this book" } });
  const review = await store.insert("reviews", { id: crypto.randomUUID(), bookId:
book.id, userId: req.userId, stars, text: String(req.body.text || "").slice(0, 1000),
createdAt: new Date().toISOString() });
  res.status(201).json(review);
}));
r.delete("/reviews/:id", requireAuth, asyncH(async (req, res) => {
  const n = await store.remove("reviews", x => x.id === req.params.id && x.userId ===
req.userId);
  if (!n) return res.status(404).json({ error: { message: "Review not found (or not yours)" } });
  res.sendStatus(204);
}));
export default r;
```

4. App + boot + seed (wire it all)

CODE · src/app.js — secure baseline + routes + 404 + error-last (order matters!)

```
import express from "express";
import helmet from "helmet";
import cors from "cors";
import books from "../routes/books.js";
import reviews from "../routes/reviews.js";
import { register, login } from "../auth.js";
const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.FRONT_URL?.split(",") || true }));
app.use(express.json({ limit: "100kb" }));
app.get("/health", (req, res) => res.json({ ok: true, uptime: process.uptime() }));
const asyncH = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
app.post("/api/v1/auth/register", asyncH(async (req, res) => res.status(201).json(await
register(req.body))));
app.post("/api/v1/auth/login", asyncH(async (req, res) => res.json(await
login(req.body))));
app.use("/api/v1/books", books);
app.use("/api/v1", reviews);
app.use("/api", (req, res) => res.status(404).json({ error: { message: "Unknown
endpoint" } }));
app.use(express.static("public", { maxAge: "1h" })); // capstone front lives here!
// eslint-disable-next-line no-unused-vars
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: { message: err.status ? err.message :
"Server error" } });
});
export default app;
```

CODE · src/index.js — validate env, then listen (crash loud at boot!)

```
import app from "../app.js";
for (const k of ["JWT_SECRET"]) {
  if (!process.env[k]) console.warn(`[warn] ${k} missing — using dev fallback (set it in
production!)`);
}
const port = process.env.PORT || 5000;
app.listen(port, () => console.log(`BookHub API on http://localhost:${port} — /health to
verify`));
```

CODE · src/seed.js — idempotent demo data (run twice, still clean)

```
import { store } from "./store.js";
import { hash } from "./auth.js";
const books = [
  { title: "The Hobbit", author: "J.R.R. Tolkien", year: 1937 },
  { title: "Dune", author: "Frank Herbert", year: 1965 },
  { title: "The Little Prince", author: "Antoine de Saint-Exupéry", year: 1943 },
  { title: "Clean Code", author: "Robert C. Martin", year: 2008 }
];
for (const b of books) {
  if (!(await store.find("books", x => x.title === b.title)))
    await store.insert("books", { id: crypto.randomUUID(), ...b, addedBy: "seed",
    createdAt: new Date().toISOString() });
}
if (!(await store.find("users", u => u.email === "demo@bookhub.dev")))
  await store.insert("users", { id: crypto.randomUUID(), email: "demo@bookhub.dev", name:
  "Demo", hash: await hash("demo-1234"), createdAt: new Date().toISOString() });
console.log("Seeded. Login: demo@bookhub.dev / demo-1234");
```

BACKEND DONE-CHECKLIST (PROVE IT WITH CURL!)

- - `node src/seed.js` → seeded message ... `npm run dev` → listening
- - `GET /health` → `{ok:true}` ... `GET /api/v1/books` → 4 seeded books
- - `POST register` → 201 ... `POST login` → access token ... `POST book without token` → 401 ✓
- - `POST review stars=9` → 400 ... `second review same book` → 409 ... `DELETE other's review` → 404
- - Unknown route → JSON 404 (not HTML!) ... every error = `{error:{message}}` shape

Exercise 76 — Harden it (stretch, 60 min)

Add `express-rate-limit` on `/auth` (20/15min) + refresh tokens in `HttpOnly` cookies (Ch 44) + `PATCH /books/:id owner-only` + `supertest` suite (201/400/401/404/409 all asserted) + migrate `store.js` to `Postgres` (Ch 42) without touching routes. Each is a resume bullet.

SOLUTION

Rate-limit = 3 lines; refresh = login sets cookie + `/refresh` rotates; owner-check = `book.addedBy === req.userId` else 403; `supertest` covers the checklist above; PG swap = same `store.*` signatures over `pg pool`. Architecture pays off exactly here.

CHAPTER 84

Capstone Frontend — Pages + Style + App (Full Code)

Advanced

~ 180 min

The complete BookHub client: semantic HTML, themed CSS, and a vanilla JS app with auth + 4-state UI.

IN THIS CHAPTER

- index.html (full page)
- styles.css (theme + components)
- app.js (api + auth + render)
- Wire-up + smoke test
- Deploy checklist

Capstone

Project

1. index.html — semantic, accessible, complete

CODE · public/index.html — the whole page (save + serve from Express static!)


```

<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>BookHub – find books worth your time</title>
<meta name="description" content="BookHub: search books, read reviews, share your own. A
full-stack capstone.">
<link rel="stylesheet" href="/styles.css">
</head>
<body>
<a class="skip" href="#books">Skip to books</a>
<header class="nav wrap">
  <a class="brand" href="/"> BookHub</a>
  <form id="search" role="search">
    <label class="visually-hidden" for="q">Search books</label>
    <input id="q" name="q" type="search" placeholder="Search title or author..."
autocomplete="off">
  </form>
  <div id="auth-area"><button id="login-btn" class="btn btn-ghost">Log in</button></div>
</header>
<main class="wrap">
  <section class="hero">
    <h1>Find books worth your time.</h1>
    <p>Real reviews from real readers. No spoilers, no ads, no noise.</p>
    <button id="add-book-btn" class="btn btn-primary" hidden>+ Add a book</button>
  </section>
  <p id="status" role="status" aria-live="polite"></p>
  <section id="books" aria-label="Books"><div id="grid" class="gallery"></div></section>
</main>
<dialog id="auth-dialog" aria-labelledby="auth-title">
  <form method="dialog"><button class="x" aria-label="Close">x</button></form>
  <h2 id="auth-title">Welcome back</h2>
  <form id="auth-form">
    <div class="field"><label for="email">Email</label><input id="email" type="email"
required autocomplete="email"></div>
    <div class="field"><label for="password">Password</label><input id="password"
type="password" required autocomplete="current-password"></div>
    <p id="auth-error" role="alert" hidden></p>
    <button class="btn btn-primary" type="submit">Log in</button>
  </form>
</dialog>
<dialog id="book-dialog" aria-labelledby="book-title">
  <h2 id="book-title">Add a book</h2>
  <form id="book-form" method="dialog">
    <div class="field"><label for="title">Title</label><input id="title" required></div>
    <div class="field"><label for="author">Author</label><input id="author" required></
div>
    <button class="btn btn-primary" type="submit">Save book</button>
  </form>
</dialog>

```

```
<div id="toasts" aria-live="polite"></div>  
<script src="/app.js" defer></script>  
</body>  
</html>
```

2. styles.css — tokens + gallery + dialogs + toasts

CODE · public/styles.css — themeable design system in 90 lines

```
*, *::before, *::after { box-sizing: border-box; } * { margin: 0; }
:root { --bg: #fff; --ink: #0f172a; --muted: #64748b; --brand: #1e3a8a; --gold: #c9a227;
--card: #f8fafc; --line: #e2e8f0; --radius: 14px; }
[data-theme="dark"] { --bg: #0b1026; --ink: #e8ecf8; --muted: #94a3b8; --card: #131c4d; --
line: #26306b; }
body { background: var(--bg); color: var(--ink); font-family: system-ui, sans-serif; line-
height: 1.6; }
.wrap { width: min(1100px, 100% - 2rem); margin-inline: auto; }
.skip { position: absolute; left: -999px; } .skip:focus { left: 1rem; top: 1rem;
background: var(--gold); padding: .5rem 1rem; z-index: 99; }
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-
path: inset(50%); }
.nav { display: flex; align-items: center; gap: 1rem; padding-block: 1rem; position:
sticky; top: 0; background: var(--bg); z-index: 50; }
.brand { font-weight: 800; font-size: 1.25rem; text-decoration: none; color: var(--ink); }
#search { flex: 1; max-width: 420px; } #search input { width: 100%;
padding: .55rem .9rem; border-radius: 999px; border: 1px solid var(--line); background:
var(--card); color: var(--ink); }
.btn { padding: .55rem 1.1rem; border-radius: 999px; border: 1px solid transparent; font-
weight: 600; cursor: pointer; }
.btn-primary { background: var(--brand); color: #fff; } .btn-ghost { border-color: var(--
line); background: transparent; color: var(--ink); }
.hero { padding-block: 2.5rem 1.5rem; } .hero h1 { font-size: clamp(2rem, 1.2rem + 4vw,
3.25rem); line-height: 1.1; }
.hero p { color: var(--muted); margin-block: .5rem 1rem; }
.gallery { display: grid; gap: 1rem; grid-template-columns: repeat(auto-fit,
minmax(250px, 1fr)); padding-bottom: 4rem; }
.card { background: var(--card); border: 1px solid var(--line); border-radius: var(--
radius); padding: 1.25rem; transition: translate .2s; }
.card:hover { translate: 0 -4px; } .card h3 { line-height: 1.25; } .card .by { color:
var(--muted); font-size: .9rem; }
.stars { color: var(--gold); font-weight: 700; }
dialog { border: 1px solid var(--line); border-radius: var(--radius); padding: 1.5rem;
background: var(--bg); color: var(--ink); width: min(420px, 100% - 2rem); }
dialog::backdrop { background: rgb(0 0 0 / .5); }
.field { display: grid; gap: .3rem; margin-bottom: .9rem; }
.field input { padding: .55rem .75rem; border: 1px solid var(--line); border-radius: 8px;
background: var(--card); color: var(--ink); }
#toasts { position: fixed; bottom: 1rem; right: 1rem; display: grid; gap: .5rem; }
.toast { background: var(--ink); color: var(--bg); padding: .6rem 1rem; border-radius:
10px; }
:focus-visible { outline: 3px solid var(--gold); outline-offset: 2px; }
@media (prefers-reduced-motion: reduce) { * { transition-duration: .01ms !important; } }
```

3. app.js — api client + auth + 4-state rendering

CODE · [public/app.js](#) — the whole client (~120 lines, zero dependencies)

```

const $ = (s, r = document) => r.querySelector(s);
const api = async (path, { method = "GET", body, token } = {}) => {
  const r = await fetch(`/api/v1${path}`, {
    method, signal: AbortSignal.timeout(8000),
    headers: { "Content-Type": "application/json", ...(token ? { Authorization: `Bearer ${token}` } : {}) },
    body: body ? JSON.stringify(body) : undefined
  });
  if (!r.ok) throw new Error((await r.json().catch(() => ({}))).error?.message || `HTTP ${r.status}`);
  return r.status === 204 ? null : r.json();
};

const session = {
  get token() { return sessionStorage.getItem("bh_token"); },
  get user() { try { return JSON.parse(sessionStorage.getItem("bh_user")); } catch { return null; } },
  save(t, u) { sessionStorage.setItem("bh_token", t); sessionStorage.setItem("bh_user", JSON.stringify(u)); },
  clear() { sessionStorage.clear(); }
};

const toast = msg => { const t = document.createElement("div"); t.className = "toast";
t.textContent = msg; $("#toasts").append(t); setTimeout(() => t.remove(), 3000); };

const paintAuth = () => {
  const u = session.user;
  $("#auth-area").innerHTML = u ? `<span>Hi, ${u.name}</span> <button id="logout" class="btn btn-ghost">Out</button>` : `<button id="login-btn" class="btn btn-ghost">Log in</button>`;
  $("#add-book-btn").hidden = !u;
  $("#login-btn").addEventListener("click", () => { $("#auth-error").hidden = true; $("#auth-dialog").showModal(); });
  $("#logout").addEventListener("click", () => { session.clear(); paintAuth(); toast("Logged out"); });
};

const card = b => `<article class="card"><h3>${b.title}</h3><p class="by">${b.author}${b.year ? ` · ${b.year}` : ""}</p>
<p class="stars">${b.avg ? `★ ${b.avg} (${b.reviewsCount})` : "★ new — be first!"}</p>
<button class="btn btn-ghost" data-review="${b.id}">Review</button></article>`;

const load = async (q = "") => {
  const grid = $("#grid"), status = $("#status");
  status.textContent = "Loading books..."; grid.innerHTML = "";
  try {
    const { data } = await api(`/books?q=${encodeURIComponent(q)}&limit=24`);
    if (!data.length) { status.textContent = q ? `No books match "${q}"` : "No books yet — add the first!"; return; }
    status.textContent = "";
    const full = await Promise.all(data.map(b => api(`/books/${b.id}`)));
    grid.innerHTML = full.map(card).join("");
  } catch (e) { status.textContent = `Couldn't load books: ${e.message}`; }
};

let debounce;
$("#search").addEventListener("input", e => { clearTimeout(debounce); debounce =

```

```

setTimeout(() => load(e.target.value.trim()), 250); });
$("#auth-form").addEventListener("submit", async e => {
  e.preventDefault();
  try {
    const { access, user } = await api("/auth/login", { method: "POST", body: { email: $
    ("email").value, password: ($("#password").value) } });
    session.save(access, user); $("#auth-dialog").close(); paintAuth(); toast(`Welcome, $
    {user.name}!`);
  } catch (err) { const p = ($("#auth-error"); p.textContent = err.message; p.hidden =
    false; }
});
$("#add-book-btn").addEventListener("click", () => $("#book-dialog").showModal());
$("#book-form").addEventListener("submit", async e => {
  const title = ($("#title").value, author = ($("#author").value);
  try { await api("/books", { method: "POST", token: session.token, body: { title,
    author } }); toast("Book added!"); load($("#q").value.trim()); }
  catch (err) { toast(err.message); }
});
document.addEventListener("click", async e => {
  const b = e.target.closest("[data-review]");
  if (!b || !session.user) return toast(session.user ? "" : "Log in to review!");
  const stars = +prompt("Stars (1-5)?");
  if (!(stars >= 1 && stars <= 5)) return;
  try { await api(`/books/${b.dataset.review}/reviews`, { method: "POST", token:
    session.token, body: { stars, text: prompt("One line?" ) || "" } }); toast("Review
    saved!"); load($("#q").value.trim()); }
  catch (err) { toast(err.message); }
});
paintAuth(); load();

```

WARNING — CAPSTONE HONESTY: WHAT THIS CLIENT SIMPLIFIES

Tokens in sessionStorage (fine for learning; production = HttpOnly refresh per Ch 44), prompt() for reviews (replace with a real dialog!), titles interpolated via innerHTML (escape user titles with textContent in production — Ch 29!). The backend is strict; the client shows where to harden next.

5. Stretch implementations (full code: refresh rotation + SSE feed)

CODE · Refresh rotation (auth.js, add below login) — stolen refresh dies on reuse

```
const refreshes = new Map(); // prod: Redis/DB (multi-server!)
export function issueRefresh(userId) {
  const raw = crypto.randomBytes(48).toString("hex");
  refreshes.set(sha256(raw), { userId, exp: Date.now() + 7 * 864e5 });
  return raw;
}
export function rotateRefresh(raw) {
  const h = sha256(raw); const rec = refreshes.get(h);
  refreshes.delete(h); // ONE-TIME: reuse = theft!
  if (!rec || rec.exp < Date.now()) throw Object.assign(new Error("Login again"), {
    status: 401 });
  return { access: jwt.sign({ sub: rec.userId }, SECRET, { expiresIn: "15m" }), refresh:
    issueRefresh(rec.userId) };
}
// routes: login sets res.cookie("refresh", issueRefresh(id), { httpOnly: true, secure:
true, sameSite: "Strict", maxAge: 7*864e5 })
// POST /auth/refresh reads cookie → rotateRefresh → new pair ... POST /auth/logout clears
+ deletes
```

CODE · SSE new-review feed (server + client, 25 lines total)

```
// server: broadcast channel per book (Map<bookId, Set<res>>)
const feeds = new Map();
app.get("/api/v1/books/:id/feed", (req, res) => {
  res.writeHead(200, { "Content-Type": "text/event-stream", "Cache-Control": "no-cache",
    Connection: "keep-alive" });
  const set = feeds.get(req.params.id) ?? feeds.set(req.params.id, new
    Set()).get(req.params.id);
  set.add(res); req.on("close", () => set.delete(res));
});
const broadcast = (bookId, review) =>
  feeds.get(bookId)?.forEach(r => r.write(`data: ${JSON.stringify(review)}\n\n`));
// call broadcast(book.id, review) right after insert in reviews.js ✓
// client: live reviews prepend (with polling fallback!)
const es = new EventSource(`/api/v1/books/${id}/feed`);
es.onmessage = e => prependReview(JSON.parse(e.data));
es.onerror = () => pollEvery(15000); // SSE blocked? degrade, never die
```

4. Wire-up + ship

CAPSTONE DONE-CHECKLIST (THE WHOLE SYSTEM!)

- - `node src/seed.js` → `npm run dev` → open `http://localhost:5000` → 4 books render
- - Search filters live ... login `demo@bookhub.dev/demo-1234` → "Hi, Demo" + Add appears
- - Add book → appears ... review → stars update ... second review → "already reviewed" toast
- - Logout → Add hides ... kill server → "Couldn't load books" (error state, not blank!)
- - Lighthouse: perf/a11y/SEO all 90+ ... keyboard-only full flow works
- - Deploy: API+static on Render (Ch 59) OR split Vercel+Render ... `/health` green ... smoke re-run on prod URL
- - README: live link + stack + 3 features + hardest bug + screenshots ... demo video 90s

KEY TAKEAWAY — KEY TAKEAWAY

You just typed a COMPLETE full-stack app: REST API, auth, validation, errors, seed, semantic HTML, themed CSS, async client, and a deploy path. Every future project is this shape with different nouns. Congratulations — you are officially a full-stack developer. Now build YOUR nouns.

Exercise 77 — Your nouns (the real graduation)

Rebuild BookHub as YOUR idea (recipes? workouts? job leads?) in 2 weeks: same architecture, your domain, one stretch feature (SSE feed? image uploads? admin?). Deploy + README + video. This is portfolio project #1 — the one interviews orbit around.

SOLUTION

Done = stranger uses it unaided for 2 minutes + README metrics + live URL + you can whiteboard its architecture in 5 minutes (routes → store → client states). Ship it, then start project #2 from the gallery (Ch 73).

PART

Appendices — Datasheets, Glossary and Course Maps

Everything worth memorizing, on fast-lookup pages.

Ch 85 Master Datasheet A — HTTP, Git, SQL and Express

Ch 86 Master Datasheet B — CSS, JavaScript, React and Regex

Ch 87 Master Datasheet C — Terminal, HTTP Deep-Dive, Cron and Meta

Ch 88 Glossary — 166 Terms in English and Arabic

Ch 89 Course Maps — Study With the Best Videos, Guided by This Book

| The part you'll open daily for years.

CHAPTER 85

Master Datasheet A — HTTP, Git, SQL and Express

Reference

~ 45 min

The four lookup tables working developers need weekly.

IN THIS CHAPTER

- HTTP status + methods + headers
- Git command map
- SQL query patterns
- Express + middleware + errors

Datasheet

MDN

W3Schools

A1. HTTP master table

Table 162 — Status codes — complete working set

Code	Name	Send when	Example
200	OK	Read/update succeeded	GET product ✓
201	Created	New resource saved	POST user → returns it
204	No Content	Success, empty body	DELETE done
301	Moved Permanently	URL changed forever	Old slug → new (SEO-safe)
304	Not Modified	Cache still fresh	Saves bandwidth
400	Bad Request	Client data invalid	<code>{errors}</code> object!
401	Unauthorized	Login missing/expired	Uniform message (Ch 44)
403	Forbidden	Logged in, not allowed	User hits admin route
404	Not Found	No such path/row	Consistent error shape

Code	Name	Send when	Example
409	Conflict	Duplicate/state clash	Email already registered
422	Unprocessable	Valid shape, bad semantics	Date range inverted (alt. 400)
429	Too Many Requests	Rate limited	<code>Retry-After</code> header!
500	Server Error	Your bug crashed	Log full, send generic
502/503	Bad Gateway/ Unavailable	Upstream/deploy issues	Retry later messaging

Table 163 — Methods + headers essentials

Topic	Essentials
Methods	GET read · POST create · PUT replace · PATCH update · DELETE remove
Request headers	Content-Type · Authorization: Bearer ... · Accept: application/json
Response headers	Location (redirects) · Retry-After (429) · Cache-Control · Set-Cookie
Safety	GET never mutates · POST creates (not idempotent) · PUT/PATCH/DELETE idempotent

A2. Git command map

Table 164 — Git daily driver set

Job	Command
What's changed?	<code>git status</code> (+ <code>git diff</code> for lines)
Snapshot	<code>git add .</code> → <code>git commit -m "Add cart drawer"</code>
History	<code>git log --oneline -10</code>
New feature universe	<code>git switch -c feature/x</code>

Job	Command
Back to stable	<code>git switch main && git pull</code>
Merge it	<code>git merge feature/x</code> (fix <<<< markers if conflicted)
Publish	<code>git push -u origin main</code> (first) → <code>git push</code>
Fetch teammate work	<code>git pull</code>
Undo (unstaged)	<code>git restore FILE ...</code> (staged) <code>git restore --staged FILE</code>
Stash interruption	<code>git stash ...</code> <code>git stash pop</code>
Erase last commit, keep files	<code>git reset --soft HEAD~1</code>
See who wrote a line	<code>git blame FILE</code>
Tag a release	<code>git tag v1.0 && git push --tags</code>

A3. SQL patterns

Table 165 — Query patterns (PostgreSQL)

Need	Pattern			
Page 3 (20/page)	<code>... ORDER BY id LIMIT 20 OFFSET 40</code>			
Search emails	<code>`WHERE email ILIKE '%'</code>	\$1		'%`
Counts per group	<code>SELECT status, COUNT(*) FROM t GROUP BY status</code>			
Top N per group	Window: <code>ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)</code>			
Users + order counts (incl. 0)	<code>LEFT JOIN ... GROUP BY u.id</code>			
Revenue per day	<code>SELECT date_trunc('day', at), SUM(total) ... GROUP BY 1</code>			
	<code>INSERT ... VALUES (\$1,\$2) RETURNING id</code>			

Need	Pattern
Safe insert returning id	
Upsert (insert or update)	<code>... ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name</code>
FK-safe delete user+orders	<code>user_id REFERENCES users(id) ON DELETE CASCADE</code>

A4. Express essentials

Table 166 — Express + middleware map

Need	Code	
Boot	<code>express()</code> + <code>cors({origin})</code> + <code>express.json({limit:"100kb"})</code>	
Route params	<code>/users/:id</code> → <code>req.params.id</code>	
Query	<code>?page=2</code> → <code>req.query.page</code>	
Guard a route	<code>app.delete("/x", requireAuth, requireRole("admin"), fn)</code>	
Throw API error	<code>next({status: 404, message: "..."})</code> → central handler	
404 unknown API	<code>app.use("/api", (req,res) => res.status(404).json(...))</code> AFTER routes	
Error center	<code>app.use((err, req, res, next) => ...)</code> LAST, 4 args	
Static front	<code>app.use(express.static("public"))</code>	
Env port	<code>`Number(process.env.PORT \</code>	<code>\ 5000)`</code>
Security stack	<code>helmet()</code> + <code>rateLimit({windowMs: 15*60e3, max: 300})</code>	

CHAPTER 86

Master Datasheet B — CSS, JavaScript, React and Regex

Reference

~ 45 min

Front-end lookups: layout, methods, hooks and patterns.

IN THIS CHAPTER

- Flex/Grid one-screen recap
- JS method matrix
- React hooks map
- Regex + JSON + storage recap

Datasheet

MDN

Tutorialspoint

B1. Flexbox + Grid one-screen recap

Table 167 — Flexbox (1D: row OR column)

Level	Properties
Container	<code>display: flex</code> · <code>flex-direction: row\column</code> · <code>justify-content</code> (MAIN) · <code>align-items</code> (CROSS) · <code>gap</code> · <code>flex-wrap: wrap</code>
Item	<code>flex: grow shrink basis</code> · <code>flex: 1</code> eats space · <code>0 0 260px</code> fixed · <code>align-self</code> override · <code>order</code> visual only
Recipes	Center: <code>justify+align center</code> · Navbar: <code>space-between + center</code> · Cards: <code>wrap + 1 1 260px</code>

Table 168 — Grid (2D: rows AND columns)

Level	Properties
Container	<code>display:grid</code> · <code>grid-template-columns</code> · <code>gap</code> · <code>repeat/auto-fit/minmax/fr</code>
Responsive	<code>repeat(auto-fit, minmax(260px, 1fr))</code> — zero media queries
Place	<code>grid-column: 2/4</code> (lines!) · <code>span 2</code> · named <code>grid-template-areas</code>
Align	<code>place-items: center</code> (items) · <code>place-content</code> (tracks)

Table 169 — Motion + responsive 60-second recap

Topic	Rule
Animate	<code>transform</code> + <code>opacity</code> ONLY (GPU); <code>transition 150–300ms</code> ; <code>animation ... both</code>
Responsive	Mobile-first <code>min-width</code> ; <code>clamp()</code> type; <code>auto-fit</code> grids; 360/768/1440 tests
Units	<code>rem</code> sizes · <code>em</code> text-hugging · <code>%</code> fluid · <code>px</code> hairlines · <code>vh</code> heroes

B2. JavaScript method matrix

Table 170 — Arrays (non-mutating default!)

Method	Returns	Use
<code>map</code>	new same-length	Transform each
<code>filter</code>	new 0..n	Keep matches
<code>reduce(..., init)</code>	single value	Totals/groups (init REQUIRED)
<code>find / findIndex</code>	item/–1 or undefined	First match
<code>some / every</code>	boolean	Any? All?
<code>slice / concat / toSorted</code>	new	Safe copies
<code>push/pop/shift/unshift/splice/sort</code>	varies	MUTATE — locals only!

Table 171 — Strings / Objects / Async one-liners

Area	Essentials
String	<code>trim/toLowerCase/includes/slice/split/join/replace</code> (immutable — returns new!)
Object	<code>{...o}</code> spread · <code>{a, b} = o</code> destructure · <code>?.</code> safe dive · <code>??</code> default · <code>structuredClone</code> deep
Async	<code>await</code> in <code>async</code> · <code>try/catch/finally</code> · <code>Promise.all</code> parallel · <code>AbortSignal.timeout(ms)</code>
DOM	<code>querySelector(All)</code> · <code>closest</code> · <code>textContent</code> safe · <code>classList</code> states · <code>delegate</code> clicks

B3. React hooks map

Table 172 — Hooks (rules: top-level only, same order every render!)

Hook	Job	Snippet
<code>useState</code>	Owned data → re-render	<code>const [v, setV] = useState(init)</code> (functional updates!)
<code>useEffect</code>	Outside world (fetch/subscribe)	<code>useEffect(() => {...; return cleanup}, [deps])</code>
<code>useRef</code>	Non-render box (DOM/timer)	<code>const r = useRef(null); <input ref={r}/></code>
<code>useMemo/useCallback</code>	Expensive cache / stable fn	Optimize AFTER measuring, not before
Custom (<code>useX</code>)	Reusable logic	<code>useProducts()</code> returns <code>{data, status}</code>

B4. Regex + JSON + storage recap

Table 173 — Validation + data recap

Need	Pattern	
Username 3-16	<code>/^[a-z0-9_]{3,16}\$/i</code>	
Egypt mobile	<code>^\+(\+20\</code>	<code>0)? 1[0125]\d{8}\$^</code>

Need	Pattern
Slug	<code>/^[a-z0-9]+(?:-[a-z0-9]+)*\$/</code>
Strong-ish pw	<code>/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}\$/</code>
JSON round-trip	<code>JSON.stringify(o)</code> / <code>JSON.parse(t)</code> in try/catch
localStorage obj	<code>setItem(k, JSON.stringify(v))</code> / <code>JSON.parse(getItem(k)) ?? fb</code>
Never in storage	Tokens, passwords, secrets (XSS-readable!) → HttpOnly cookies

CHAPTER 87

Master Datasheet C — Terminal, HTTP Deep-Dive, Cron and Meta

Reference

~ 45 min

The lookup tables for servers, schedules, headers and page metadata.

IN THIS CHAPTER

- Terminal + npm command map
- HTTP headers + caching + CORS
- Status codes complete
- Cron schedules
- Meta/OG/SEO tags

Datasheet

MDN

DevOps

C1. Terminal + npm command map

Table 174 — Files, search, servers (daily drivers)

Job	Command
Navigate/ list	<code>pwd</code> · <code>ls -la</code> · <code>cd -</code> (back!) · <code>cd ~</code>
Make/ move/ delete	<code>mkdir -p a/b</code> · <code>touch f</code> · <code>mv a b</code> · <code>cp -r a b</code> · <code>rm f</code> (<code>-rf</code> = forever!)
Read	<code>cat f</code> · <code>less big.log</code> · <code>head -20</code> · <code>tail -f app.log</code> (follow!)
Search	<code>grep -rn "TODO" src</code> · <code>find . -name "*.test.js"</code> · <code>cmd --help</code>

Job	Command	
Combine	<code>`grep ERROR log \</code>	<code>head . build && deploy . test \</code> <code>echo FAIL`</code>
npm run	<code>dev (watch) · build (dist) ·</code> <code>preview (prod local) · lint ·</code> <code>test</code>	
npm install	<code>npm i p · npm i -D p (dev) ·</code> <code>npm ci (CI-exact!) ·</code> <code>npm outdated</code>	
Processes	<code>`ps aux \</code>	<code>grep node . kill PID .</code> <code>ctrl+c (stop) · lsof -i :5000`</code>

C2. HTTP headers + caching + CORS (server fluency)

Table 175 — Headers you must know

Header	Direction	Meaning
Content-Type	Both	Body format: application/json, text/html, image/avif
Authorization	Req	<code>Bearer <token></code> (JWT) — never in URL!
Cache-Control	Res	<code>max-age=31536000, immutable</code> (hashed) · <code>no-cache</code> (HTML shell)
ETag / If-None-Match	Res/Req	Version fingerprint → 304 Not Modified (bandwidth saver)
Location	Res	Redirect target (3xx)
Set-Cookie	Res	<code>session=...; HttpOnly; Secure; SameSite=Strict</code> (the safe trio!)
CORS trio	Both	Origin → <code>Access-Control-Allow-Origin: https://app.com</code> (exact!) + Methods + Headers
Content-Security-Policy	Res	<code>default-src 'self'</code> (XSS killer — helmet sets it!)
X-Content-Type-Options	Res	<code>nosniff</code> (blocks MIME-sniff attacks)

Table 176 — Caching recipes (copy these)

Asset	Cache-Control
Hashed build (<code>app.a1b2.js</code>)	<code>public, max-age=31536000, immutable</code>
HTML shell	<code>no-cache</code> (revalidate every load!)
API GET (semi-static)	<code>private, max-age=60</code> + ETag
API GET (live)	<code>no-store</code> (never cache sensitive!)
Images (content-hashed CDN)	<code>public, max-age=2592000, immutable</code>

C3. Status codes complete (the full map)

Table 177 — 1xx-5xx with "send when"

Code	Meaning	Send when
200 / 201 / 204	OK / Created / No Content	GET ok / POST made it (+body!) / DELETE done
301 / 302 / 304	Moved / Found / Not Modified	Permanent redirect / login bounce / ETag match (cached!)
400 / 401 / 403 / 404	Bad input / Unauthenticated / Forbidden / Missing	Validation fails / no token / not yours / no such id
409 / 422 / 429	Conflict / Unprocessable / Rate-limited	Duplicate email / semantic errors / too many tries (+Retry-After!)
500 / 502 / 503	Crash / Bad gateway / Unavailable	Bug (log+alert!) / upstream dead / overloaded (Retry-After!)
Rule	4xx = client's fault (their fix) ... 5xx = OUR bug (our pager!) ... never leak stacks	

C4. Cron schedules (automate everything)

Table 178 — Cron in 60 seconds (`\* \* \* \* \*` = min hour day month weekday)

Schedule	Pattern
Every 5 min	<code>/5 *</code> (health pings)
Hourly	<code>0</code> (queue drain)
Daily 2 AM	<code>0 2 *</code> (backups, reports)
Weekly Monday 6 AM	<code>0 6 1</code> (digests)
Monthly 1st midnight	<code>0 0 1</code> (invoices)
Rule	Server cron for simple jobs; queues (BullMQ) for retries/scale; idempotent handlers ALWAYS

C5. Meta/OG/SEO head (paste-ready)

CODE · Every page deserves this head (fill + ship)

```
<title>Page – Site</title>
<meta name="description" content="155 honest chars that earn the click.">
<link rel="canonical" href="https://site.com/page">
<meta property="og:title" content="Page – Site">
<meta property="og:description" content="Share-preview line.">
<meta property="og:image" content="https://site.com/og.png">
<meta property="og:type" content="website">
<meta name="twitter:card" content="summary_large_image">
<meta name="theme-color" content="#0b1026">
<link rel="icon" href="/favicon.svg" type="image/svg+xml">
```

Table 179 — SEO truth table (no tricks, just wins)

Factor	Action
Title + description	Unique per page, honest, keyword-natural
Headings + semantics	One h1, logical h2s, real landmarks (Ch 8)
Speed + mobile	Vitals green (Ch 61!), responsive, HTTPS
Images	Descriptive alt + filenames + AVIF + dimensions

Factor	Action
Sharing	OG tags + og:image 1200×630 (test with card validators!)
Discovery	sitemap.xml + robots.txt + internal links with real anchor text

C6. Linux survival (deploy-day commands)

Table 180 — The deploy-day dozen

Job	Command
SSH in	<code>ssh user@server</code> (keys, never passwords!)
Update + Node	<code>sudo apt update && sudo apt upgrade</code> · <code>node -v</code> (use LTS!)
Process keeper	<code>pm2 start src/index.js --name app</code> · <code>pm2 logs</code> · <code>pm2 restart app</code>
Reverse proxy	<code>nginx proxy_pass http://localhost:5000</code> + <code>certbot --nginx</code> (HTTPS!)
Logs now	<code>pm2 logs --lines 100</code> · <code>tail -f ~/.pm2/logs/app-out.log</code>
Disk/RAM	<code>df -h</code> · <code>free -h</code> · <code>htop</code> (what eats RAM?)
Firewall	<code>sudo ufw allow 22,80,443/tcp</code> · <code>sudo ufw enable</code>
Env on server	<code>/etc/app/.env</code> (0600 perms!) loaded by pm2 ecosystem file
Zero-downtime	<code>pm2 reload app</code> (rolling restart — requests survive!)

C7. package.json scripts cookbook

Table 181 — Scripts every project needs

Script	Command	Job
dev	<code>vite</code> / <code>nodemon src/index.js</code>	Hot-reload development
build	<code>vite build</code>	Production bundle → dist/
preview	<code>vite preview</code>	Test the PROD build locally
start	<code>node src/index.js</code>	Production server entry

Script	Command	Job
lint	<code>eslint .</code>	Find bugs + style sins
format	<code>prettier --write .</code>	Auto-format everything
test	<code>vitest run</code>	Full suite once (CI mode)
test:watch	<code>vitest</code>	Suite on every save
seed	<code>node src/seed.js</code>	Demo data (idempotent!)
migrate	<code>node src/migrate.js</code>	Apply pending migrations
smoke	<code>node scripts/smoke.js</code>	Post-deploy register→CRUD→logout
clean	<code>rm -rf dist node_modules</code>	Nuclear fresh start

CHAPTER 88

Glossary — 166 Terms in English and Arabic

Reference

~ 60 min

Every important word in this book, explained in one line — with Arabic meaning.

IN THIS CHAPTER

- A-Z terms
- Arabic translations
- Chapter pointers inside definitions

Glossary

عربي

2xx/4xx/5xx · عائلات رموز الحالة

Status families: 2xx success, 4xx client error, 5xx server error (Ch 3).

AbortController · إلغاء الطلبات

Cancels fetch/listeners; timeouts + race-free UIs (Ch 32).

Accessibility (a11y) · إمكانية الوصول

Designing so everyone — including disabled users — can use the product (Ch 8).

API · واجهة برمجة التطبيقات

Contract letting programs talk: URLs + methods + data shapes (Ch 3, 39).

ARIA · سمات وصف المكونات لقارئات الشاشة

Attributes describing custom widgets to screen readers; native elements first (Ch 8).

Array · مصفوفة

Ordered list `[1, 2]`; methods `map/filter/reduce` transform it (Ch 27).

Arrow function · دالة سهمية

Short `=>` function inheriting `this`; callbacks' best friend (Ch 26).

async/await · البرمجة اللاتزامنية الحديثة

Clean syntax for promises: write `async` like `sync`, with `try/catch` (Ch 32).

Attribute · خاصية الوسم

Extra tag info: `src`, `href`, `alt`, `required` (Ch 4).

Authentication · التحقق من الهوية

Proving WHO you are: login, passwords, tokens (Ch 44).

Authorization · التحقق من الصلاحيات

Checking WHAT you may do: roles, permissions (Ch 44).

AVIF/WebP · صيغ صور حديثة مضغوطة

Modern tiny image formats with fallbacks via `picture` (Ch 6).

Backend · الواجهة الخلفية

Hidden server side: APIs, logic, databases (Ch 1, 38–44).

Baseline · خط الأساس للتوافقية

Web features safe across modern browsers (MDN badge).

bcrypt · تشفير كلمات المرور

Adaptive password hasher; cost 12; verify with compare (Ch 44).

BEM · منهجية تسمية CSS

Block\_\_Element--Modifier: flat classes, zero specificity wars (Ch 22).

Block element · عنصر كتلي

Full-width stacker: div, p, h1, section (Ch 16).

Boolean · قيمة منطقية

true/false decider; falsy 8 memorize (Ch 25).

Box model · نموذج الصندوق

content + padding + border + margin; border-box everywhere (Ch 13).

Branch · فرع في جيت

Parallel git universe for safe experiments (Ch 37).

Breakpoint · نقطة التحول المتجاوب

Width where layout adapts; content-based, mobile-first (Ch 19).

Browser · المتصفح

Client rendering pages; engine runs HTML/CSS/JS (Ch 1).

Bubbling · صعود الأحداث

Events float target→window; delegation rides it (Ch 30).

Bundler · حازم الوحدات

Builds + splits + minifies JS (Vite/Webpack); tree-shakes dead code (Ch 56).

Callback · دالة رد النداء

Function passed to run later: handlers, then, timers (Ch 26).

Cascade · CSS التالي في

Conflict order: origin → specificity → position (Ch 12).

CDN · شبكة توصيل المحتوى

Global edge servers caching files near users (Ch 1).

Certificate (TLS) · شهادة التشفير

Server ID proving HTTPS authenticity (Ch 1).

CI/CD · التكامل والنشر المستمر

Robots test every push (CI) and deploy green builds (CD) (Ch 38).

Class (CSS) · فئة تنسيق

Reusable style hook `.card`; style with classes ~95% (Ch 11).

Class (JS) · فئة برمجية

Blueprint with constructor/methods; #private fields (Ch 28).

Client · العميل

Asking side: browser/phone needing pages/data (Ch 1).

Closure · الإغلاق والدوال الحافظة

Function + captured scope = memory & privacy (Ch 26).

Commit · حفظ نسخة في جيت

Git snapshot with a headline message (Ch 37).

Component · مكون واجهة

Reusable UI Lego with props in, events up (Ch 46).

Console · وحدة التحكم

F12 command line: log/table/errors/timing (Ch 23).

const/let · إعلان المتغيرات

const default; let for reassignment; var retired (Ch 24).

Container query · استعلام الحاوية

Component responds to ITS box, not viewport (Ch 78).

Contrast · تباين الألوان

Text-vs-bg ratio $\geq 4.5:1$ normal, 3:1 large (Ch 15).

Cookie · ملفات تعريف الارتباط

Small server-set storage; HttpOnly = JS-invisible (Ch 31, 43).

CORS · مشاركة الموارد عبر الأصول

Browser gate for cross-origin API calls; lock origins (Ch 40).

Cron · الجدولة الدورية

\* schedules: backups, digests, pings (Datasheet C).

CRUD · العمليات الأربع

Create/Read/Update/Delete = POST/GET/PATCH/DELETE (Ch 40).

CSP · سياسة أمان المحتوى

Header allowlisting script sources; XSS killer (Ch 45).

CSRF · تزوير طلبات المواقع

Forged cross-site requests riding cookies; SameSite+tokens stop it (Ch 45).

CSS · لغة التنسيق

Style language: selectors + declarations + cascade (Part 3).

Dark mode · الوضع الداكن

Theme via data-attribute variables + stored choice (Ch 21, 31).

Database · قاعدة البيانات

Organized persistence: SQL tables or NoSQL docs (Ch 42-43).

Debounce · تأخير التنفيذ

Run after user STOPS (search boxes); throttle = steady rate (Ch 32).

Destructuring · التفكيك

Unpack `{a} = obj`, `[x] = arr`, params too (Ch 28).

DevTools · أدوات المطور

F12 X-ray: Elements/Console/Network/Sources (Ch 1).

DNS · نظام أسماء النطاقات

Domain→IP phonebook; first step of every visit (Ch 1).

DOM · شجرة عناصر الصفحة

Live object tree of the page; JS edits it (Ch 29).

Element · عنصر

Tag + content + closing = one node (Ch 4).

Endpoint · نقطة نهاية API

One API address: method + path + contract (Ch 40).

Event · حدث

Click/key/submit signal; listen with `addEventListener` (Ch 30).

Event delegation · تفويض الأحداث

One parent listener + `closest()` for infinite children (Ch 30).

Event loop · حلقة الأحداث

Sync → microtasks → render → one macrotask, repeat (Ch 32).

Express · إطار السيرفر

Minimal Node framework: routing + middleware (Ch 40-41).

Fetch · جلب البيانات

Promise-based HTTP in browser/Node; check `res.ok!` (Ch 33).

Flexbox · صندوق المرن

1D layout: container aligns, items grow (Ch 17).

Form · نموذج إدخال

Data collector: action + method + named controls (Ch 7).

Fragment (#) · جزء الصفحة

In-page jump `#pricing`; no server trip (Ch 2).

Frontend · الواجهة الأمامية

Visible browser side: HTML/CSS/JS/React (Parts 2-4, 9).

Function · دالة

Named reusable action with params and return (Ch 26).

Generator · المولدات الكسولة

`function*` + `yield`: lazy pausable sequences (Ch 52).

Git · نظام النسخ

Version time-machine: commits, branches, remotes (Ch 37).

GitHub · منصة استضافة الكود

Online git home: push/PR/issues/actions (Ch 37).

Grid · شبكة CSS

2D layout: tracks/lines/areas; auto-fit magic (Ch 18).

Hashing · التجزئة أحادية الاتجاه

One-way mince (bcrypt); compare, never decode (Ch 44).

Heap/Stack · الذاكرة

Heap holds objects; stack holds calls + primitives (Ch 24).

Hoisting · الرفع

Declarations lift (functions whole; let/const locked in TDZ) (Ch 26).

HTML · لغة الهيكلية

Structure language: elements + meaning (Part 2).

HTTP · بروتوكول الويب

Request/response conversation + methods + codes (Ch 3).

HTTPS · الويب المشفر

HTTP inside TLS encryption + certificates (Ch 1).

Hydration · الترطيب

Client JS attaching to server HTML (Next.js); mismatch = bugs (Ch 62).

ID · معرّف فريد

Unique `#one` hook for JS/anchors; classes for styling (Ch 11).

Idempotency · عدم التكرار

Retries safe: same request twice = one effect (payments!) (Ch 39).

Inheritance (CSS) · الوراثة في CSS

Type props pass down; box props don't (Ch 12).

Injection · الحقن

String-built queries/commands poisoned; parameterize! (Ch 45).

Inline element · عنصر مضمن

Text-flow dweller: span/a/strong; no sizes (Ch 16).

INP/LCP/CLS · مؤشرات السرعة

Interaction delay / hero paint / layout shift: the Core Web Vitals (Ch 61).

IP/Port · عنوان و منفذ الشبكة

Machine number + service number (443 HTTPS) (Ch 1).

JavaScript · جافاسكريبت

Web's language: browser UI + Node servers (Part 4).

JOIN · ربط الجداول

Combine tables: INNER matched, LEFT all+NULLs (Ch 42).

JSON · تنسيق تبادل البيانات

Text data shape: "keys", no trailing commas (Ch 3, 28).

JWT · رمز الدخول الموقّع

Signed token pair: short access + rotating refresh (Ch 44).

Keys (React) · مفاتيح القوائم

Stable list identity `key={id}`; never bare index (Ch 46).

Keyset pagination · ترقيم المفاتيح

`WHERE id > last LIMIT n`: deep pages stay fast (Ch 39).

Label · تسمية الحقول

Accessible control name; for=id link (Ch 7).

Layout · حساب التخطيط

Measuring boxes (x/y/w/h); skip via transform (Ch 1, 20).

Lighthouse · مدقق الأداء

Lab audit: perf/a11y/SEO scores + fix list (Ch 61).

localStorage · تخزين المتصفح

5MB forever strings; JSON + prefs only (Ch 31).

Margin collapse · اندماج الهوامش

Vertical margins merge (max wins); gap avoids (Ch 13).

Media query · استعلام الشاشات

min-width layers for bigger screens (Ch 19).

Method (HTTP) · أفعال الطلبات

Verb: GET/POST/PUT/PATCH/DELETE (Ch 3, 39).

Microtask · المهام العاجلة

VIP queue: promises drain fully before render (Ch 32).

Middleware · الوسائط البرمجية

Express assembly line: use/route/error order (Ch 40).

Module · الوحدات البرمجية

File with export/import; dynamic import lazy-loads (Ch 28).

MongoDB · قاعدة بيانات المستندات

Document DB: embed-together, reference-across (Ch 43).

N+1 · مشكلة الاستعلامات

Loop-fetching relations: 101 queries; fix with JOIN/populate (Ch 42).

Node.js · بيئة تشغيل السيرفر

JS runtime: event loop + libuv + ESM (Ch 39).

npm · مدير الحزم

Package manager: install/run/share code (Ch 37).

Null/undefined · القيم الفارغة

Chosen empty vs accidental empty; ?? handles both (Ch 23, 25).

Object · كائن

Named drawers {k: v} + methods + this (Ch 28).

Opacity · الشفافية

Whole-element fade incl. children; ≠ hiding (Ch 16).

Optimistic UI · الواجهة المتفائلة

Update instantly, rollback on failure: speed + honesty (Ch 32).

Package.json · بطاقة المشروع

Project ID: scripts + deps + type module (Ch 37).

Pagination · ترقيم الصفحات

LIMIT/OFFSET + meta {page, pages, total} (Ch 41-42).

Parameter · معامل الدالة

Named function input (argument = passed value) (Ch 26).

Polyfill · التوافقية الاحتياطية

Backports new APIs to old browsers (core-js); baseline shrinks need.

POST/GET · الإرسال والقراءة

Create-in-body vs read-in-URL; forms + APIs (Ch 3, 7).

Prefetch/Preload · الجلب المسبق

Fetch next-route data / hero assets early (Ch 61).

Primary/Foreign key · المفاتيح الأساسية والأجنبية

Row ID / pointer to another row (Ch 42).

Promise · الوعد البرمجي

Future-value receipt: then/catch/finally + all/race (Ch 32).

Props · خصائص المكونات

Read-only component inputs flowing down (Ch 46).

Prototype · النموذج الأولي

Hidden inheritance link; classes sugar it (Ch 28).

Pseudo-class · الفئات الوهمية

State selector :hover :focus :nth-child (Ch 11).

Query string · معاملات الرابط

URL options ?q=&page= ; validate server-side (Ch 2).

Rate limit · تحديد معدل الطلبات

Max requests/window; 429 + Retry-After (Ch 41, 44).

React · مكتبة الواجهات

Component UI library: props/state/effects (Ch 46).

Regex · التعابير النظامية

Shape patterns /^...\$/ ; anchor + keep simple (Ch 31, 34).

rem/em · وحدات القياس النسبية

rem = root-sized (type!), em = parent/text-hugging (Ch 14).

Render tree · شجرة العرض

DOM + CSSOM minus hidden = paint plan (Ch 1).

REST · معمارية الواجهات

Noun URLs + verb methods + codes + versioning (Ch 40).

Salt · الملح العشوائي

Random bcrypt spice defeating rainbow tables (Ch 44).

Scope · النطاق

Visibility zone: block/function/global + chain (Ch 26).

Script · تحميل السكريبت

JS loader: end-of-body or head+defer (Ch 4, 23).

Semantic HTML · HTML الدلالية في

Meaningful tags: article/section/nav/main (Ch 8).

Semver · ترقيم الإصدارات

MAJOR.MINOR.PATCH: breaking/feature/fix;
lockfiles pin exact (Ch 56).

SEO · تحسين محركات البحث

Search love: titles, semantics, speed, OG tags (Ch 8).

Server · الخادم

Answering machine: listens, computes, responds (Ch 1).

Server Component · مكون الخادم

Renders on server, ships HTML + zero JS; client islands interact (Ch 62).

Session · الجلسة

Stateful login: ID cookie + server store (Ch 44).

Sitemap/robots · خريطة الموقع

Discovery files: URLs for crawlers + crawl rules (Datasheet C).

Source map · خريطة المصدر

Debug minified code against originals (prod: private!).

Specificity · CSS الأسبقية في

0-0-0 score: IDs > classes > elements (Ch 12).

Spread · النشر والتجميع

`...` merge/copy shallow; rest `...args` gathers (Ch 27-28).

SSE · أحداث الخادم

Server-Sent Events: one-way live stream over HTTP (Ch 57).

State (React) · حالة المكون

Owned data via useState; replace, don't mutate (Ch 46).

Status code · رمز الحالة

3-digit outcome narrator: 200/201/400/401/404/500 (Ch 3).

String · النصوص

Quoted text; template ``{}`` for values (Ch 23).

TDZ · منطقة الموت المؤقت

let/const locked zone before declaration line (Ch 24, 26).

Template literal · القوالب النصية

Backtick string with ``{}`` + multiline (Ch 23).

this · السياق الحالي

Call-style context: method object / arrow inherits (Ch 26).

Throttle · تحديد المعدل

At most once per N ms (scroll/resize); debounce waits for quiet (Ch 26).

TLS · طبقة التشفير

Encryption + certificates = the S in HTTPS (Ch 1).

Transform · التحويلات

GPU move/scale/rotate; layout-free motion (Ch 20).

Transition · الانتقالات

Smooth A→B: property duration easing (Ch 20).

Tree-shaking · إزالة الميت

Bundler drops unused exports; ESM + sideEffects:false enable it.

Typeof · فحص الأنواع

Type detective (lies about null: "object!") (Ch 23).

Typography · فن النصوص

Type system: scale + weights + rhythm (Ch 14).

URL · عنوان الصفحة

Full address: protocol domain port path query hash (Ch 2).

useEffect · التأثيرات الجانبية

React outside-world hook + cleanup + deps (Ch 46).

useState · حالة رباكت

React owned-data hook + setter (Ch 46).

Validation · التحقق من البيانات

Shape+rule checks: browser UX + server truth (Ch 7, 40).

Variable · المتغير

Named value jar: const-first discipline (Ch 24).

Viewport · نافذة العرض

Visible window; meta unlocks responsive (Ch 4, 19).

Vitest · إطار الاختبار

Fast Vite-native test runner: unit + mocks + coverage (Ch 53).

Void element · عناصر بلا محتوى

Contentless: img/br/hr/input/meta/link (Ch 4).

Webhook · خطاف الويب

Server-to-server event POST (payments!); verify signatures + idempotency.

WebSocket · القناة الحية

Full-duplex live channel: chat, games, collab (Ch 57).

XSS · حقن السكريبت

Attacker code via unsafe rendering; escape + CSP (Ch 29, 44).

z-index · ترتيب العمق

Depth order for positioned items; contexts trap (Ch 16).

Zod · التحقق من البيانات

Schema validation for API boundaries: parse, don't hope (Ch 54).

Zustand · مخزن الحالة

Tiny global store: sliced subscriptions, zero providers (Ch 46).

CHAPTER 89

Course Maps — Study With the Best Videos, Guided by This Book

Reference

~ 30 min

Exactly where each famous course fits into these chapters — watch in the right order.

IN THIS CHAPTER

- Elzero Web School map
- Abdelrahman Gamal map
- Angela Yu bootcamp map
- W3Schools + Tutorialspoint + MDN map
- How to combine video + book

Roadmap

Video

Docs

How to combine video + book (the method that works)

1. **Read the chapter first** (20 min) — build the mental model with analogies + diagrams.
2. **Watch the mapped videos at 1.25×** — you now understand every word; videos add accent + repetition.
3. **Code the chapter project WITHOUT the video** — struggle is learning; rewatch only stuck parts.
4. **Do book exercises closed-book** — then compare with solutions; log mistakes in a notebook.

Table 182 — Elzero Web School (Osama Elzero) → chapters

Playlist / series	Watch with	Focus while watching
HTML course	Ch 4-8	Semantics + forms depth; replicate every example

Playlist / series	Watch with	Focus while watching
CSS course	Ch 11-20	Selectors → Grid order matches; pause and predict output
JavaScript course	Ch 23-33	Scope/closures/DOM/async; code alongside, then solo
JS challenges	Ch 26-28, 35	Solve BEFORE watching his solution (timer on!)

Table 183 — Abdelrahman Gamal tracks → chapters

Playlist / series	Watch with	Focus while watching
HTML & CSS track	Ch 4-22	Project pacing; steal his design eye, keep book's system
JavaScript track	Ch 23-36	Fundamentals phrasing; cross-check with Ch 36 datasheet
React track	Ch 46	Components → hooks order; rebuild his demos from memory

Table 184 — Angela Yu Bootcamp → chapters

Section	Watch with	Focus while watching
Web basics + HTML/CSS	Ch 1-22	Breadth + fun projects; book adds depth + a11y
JavaScript + DOM + jQuery(skip jQuery!)	Ch 23-31	Logic + DOM; SKIP jQuery (2026: vanilla + React)
Node + Express + APIs	Ch 39-41	Server mental model; add book's validation/errors
Databases + Auth	Ch 42-45	End-to-end flows; harden with book checklists
React + Deployment	Ch 46, 37, 46	Ship + portfolio chapters complete the story

Table 185 — Documentation trio → chapters

Resource	Use for	How
		Search "mdn X" for every new API

Resource	Use for	How
MDN Web Docs	TRUTH: exact syntax, compatibility, deep guides	
W3Schools	FAST try-it editor + simple intro	First 5 minutes with any tag/property
Tutorialspoint	Second phrasing + examples	When one explanation doesn't click, read theirs

KEY TAKEAWAY — KEY TAKEAWAY

Book = map + depth + system. Videos = voice + rhythm + motivation. Docs = truth + updates. Students who combine all three — in THAT order per topic — consistently outperform video-only learners 3-to-1 in interviews. The order is the secret.

Exercise 78 — Build your 16-week wall chart

Copy the study plan (Front) onto paper; add the video rows above per week; tick boxes daily. Photograph week 1 done — momentum is visible progress.

SOLUTION

No code — pure discipline system. Students with visible charts finish 4× more often (same psychology as gym streaks). Start today: tick "How to Use This Book" as DONE.